FULL-STACK MASTERCLASS

Build Viby

The Full-Stack Social Platform Masterclass

From absolute beginner to professional full-stack developer — by building a real, premium social-discovery app with Next.js, TypeScript, Tailwind, Supabase, and Vercel.

36 chapters · 8 sections · beginner → production · bubblegum pink Gen-Z aesthetic

Build Viby

Build Viby is a project-based, beginner-to-professional full-stack course that teaches modern web and product engineering by actually building something real. Instead of disconnected tutorials, you ship **Viby** — a premium, mobile-first social-discovery app — one section at a time. By the final chapter you won't just *understand* Next.js, TypeScript, Tailwind, and Supabase; you'll have engineered, designed, secured, and deployed a production-grade social platform with your own hands, and you'll think like the product builder behind it.

Who this book is for

- Curious beginners who can write a little code and want a single, coherent path from "Hello World" to shipping a real app.
- Frontend developers ready to go full-stack and own the backend, database, and realtime layers.
- Bootcamp grads and self-taught engineers who want professional-grade patterns, not toy examples.
- Founders and product-minded builders who want to turn an idea into a deployed, scalable MVP.

What you will be able to build

- A real, working social platform — profiles, discovery, connections, chat, and feeds.
- Complete full-stack applications, frontend to database, with clean architecture.
- Responsive, modern UIs that feel native on mobile and polished on desktop.
- Realtime systems — live chat, presence, notifications, and instant updates.
- Scalable production apps that are secure, observable, and ready for real users.

The stack you will master

- **Next.js 16** — App Router and React 19 (Server Components, Server Actions, streaming).
- **TypeScript** — end-to-end type safety from the database to the UI.
- **Tailwind CSS v4** — CSS-first theming with `@theme` design tokens (no `tailwind.config.js`).
- **shadcn-style UI primitives** — composable, accessible, fully-owned components.
- **Framer Motion** — fluid micro-interactions and premium motion design.
- **Zustand** — lightweight client state for demo / in-memory mode.
- **Supabase** — Auth, PostgreSQL, Storage, and Realtime via `@supabase/ssr`.
- **Vercel** — zero-config, edge-ready production deployment.

Brand identity: bubblegum pink `#FF6FB5` on clean white, mobile-first, with a premium Gen-Z social aesthetic woven through every screen and component.

How to use this book

This is a **project-based** book — you learn by building, not by reading alone.

- **Code along.** Type the code yourself; don't just skim it. Muscle memory matters.
- **Do the exercises and mini-projects.** Each chapter ends with hands-on work that locks the concept in.
- **Build Viby section by section.** Every chapter adds a real piece of the app, so progress compounds.
- **Choose your mode.** Run in **demo mode** (no keys, in-memory state with Zustand) to learn fast and offline, or **connected mode** (real Supabase Auth, database, storage, and realtime) to build the production app for real. The book guides both paths.

How the chapters are structured

Every chapter follows the same rhythm so you always know where you are:

- **Theory** — the core concept explained clearly, from first principles.
- **In practice** — applying it to real Viby code.
- **Diagram** — a visual model of how the pieces fit together.
- **Best practices** — what professionals do, and the traps they avoid.
- **Exercises** — focused reps to make the concept stick.
- **Mini-project** — a self-contained build that proves you've got it.
- **Takeaways** — the essentials to carry into the next chapter.

Ship something real. Let's build Viby.

Table of Contents

FUNDAMENTALS

01 How the Web Works

02 Frontend, Backend & Where Code Runs

03 React Fundamentals: Components, JSX & Props

04 State, Events & Effects

05 TypeScript for React + Talking to APIs

MODERN FRONTEND

06 Next.js & the App Router

07 Routing, Layouts & Navigation

08 Server vs Client Components & Data

09 Tailwind CSS & Mobile-First Responsive Design

10 Reusable Components, shadcn Patterns & Framer Motion

UI / UX DESIGN

11 Color, Theming & the Viby Design System

12 Typography, Spacing & Layout Systems

13 Design Psychology, Gen-Z & Social App UX

BACKEND DEVELOPMENT

14 Databases & PostgreSQL Data Modeling

15 Supabase Architecture & Setup

16 Authentication, Sessions & Row-Level Security

17 CRUD with Server Components & Server Actions

18 Storage & Realtime Systems

BUILDING VIBY

19 Project Setup & Foundation

20 The Onboarding Flow

21 Profiles & the Identity System

22 Discover & the Matching Algorithm

23 Immersive Profiles & the Connect Flow

24 The Activity Feed

25 The Connections System

26 Realtime Chat

27 Groups & Notifications

ADVANCED CONCEPTS

28 Performance & Optimization

29 Caching, Server Components & Scalable Architecture

30 Security, RLS & Rate Limiting

31 Testing, Debugging & Error Handling

PRODUCTION DEPLOYMENT

32 Environment, Config & Deploying to Vercel

33 Supabase in Production

34 Monitoring, Logging & Analytics

STARTUP & PRODUCT THINKING

35 MVP Strategy & Product Thinking

36 Growth, Retention, Scaling & Engineering Workflows

Chapter 1 — How the Web Works

> Every app you'll ever build — including Viby — is just two computers having a polite, very fast conversation. Learn that conversation, and the whole web stops being magic.

What you'll learn

- What "the internet" actually is, and how it's different from "the web"
- The difference between a **client** and a **server**, and why both matter
- How **IP addresses** and **DNS** turn `viby.app` into a real machine you can reach
- The anatomy of an **HTTP request and response**, including headers and bodies
- What **status codes** mean (200, 301, 404, 500…) and how to read them
- What a **browser** actually does when you press Enter
- The high-level split between **frontend** and **backend**, and where Viby's pieces live

Prerequisites

This is the very first chapter, so you need **nothing** but curiosity and a computer with a browser. We'll set up your tools (Node.js, the editor, the Viby starter) in **Chapter 2 — Your Developer Toolkit**, and you'll write your first real Next.js page in **Chapter 3**. For now, just read, picture the analogies, and try the exercises in your browser.

Theory

The internet vs. the web

People say "the internet" and "the web" as if they're the same thing. They're not.

The **internet** is the *roads* — a global network of cables, fiber, routers, and Wi-Fi that lets any two computers send data to each other. The **web** is one kind of *traffic* that drives on those roads: web pages, apps, and APIs that speak a language called **HTTP**. Email, video calls, and online games also use the internet, but they're different vehicles.

When you build Viby, you're building for the web — but it rides on the internet underneath.

Clients and servers: who asks, who answers

Almost everything on the web is a conversation between two roles:

- A **client** *asks* for something. Your phone's browser opening Viby is a client.
- A **server** *answers*. A computer in a data center that holds Viby's pages and data is a server.

Analogy: think of a restaurant. You (the **client**) read the menu and ask the waiter for a dish. The kitchen (the **server**) prepares it and sends it back. You don't walk into the kitchen — you make a polite *request*, and you

get a *response*. The waiter shuttling between you is the network.

A server isn't a magic super-computer. It's just a normal computer running a program that *waits* for requests and knows how to reply. The same machine can be a client in one moment and a server in another.

IP addresses and DNS: finding the right machine

The internet has billions of devices. How does your request find the *one* server that hosts Viby?

Every device on the internet has an **IP address** — a numeric label like `93.184.216.34` (IPv4) or a longer `2606:2800:220:1:248:1893:25c8:1946` (IPv6). It's the *street address* of a computer.

But nobody wants to type numbers. So we use friendly names like `viby.app`. The system that translates names into IP addresses is **DNS** (Domain Name System).

Analogy: DNS is the *phone book* (or your phone's contacts). You type a person's name; it looks up the actual number to dial. When you visit `viby.app`, your computer quietly asks DNS, "What's the IP address for this name?" DNS replies with a number, and *then* your request can travel to the right machine.

```
sequenceDiagram
    participant You as Browser (client)
    participant DNS as DNS resolver
    participant Server as Viby server

    You->>DNS: What's the IP for viby.app?
    DNS-->>You: It's 93.184.216.34
    You->>Server: GET / (HTTP request to that IP)
    Server-->>You: 200 OK + HTML page
```

HTTP: the language of requests and responses

Once your browser knows the server's address, it speaks **HTTP** (HyperText Transfer Protocol). HTTP is just a strict format for "I'd like this" and "here you go."

Every **request** has:

- A **method** — the *verb*, what you want to do:
- `GET` — fetch something (load a profile page)
- `POST` — create something (sign up a new user)
- `PUT` / `PATCH` — update something (edit your bio)
- `DELETE` — remove something (delete a post)
- A **path** — *which* thing: `/discover`, `/profile/maya`, `/api/posts`
- **Headers** — extra info *about* the request (who you are, what format you accept, your auth token)
- An optional **body** — the data you're sending (the JSON for a new Viby post)

Every **response** has:

- A **status code** — a 3-digit summary of how it went (more below)

- **Headers** — info about the response (content type, caching rules, cookies)
- A **body** — the actual content (HTML, JSON, an image)

Analogy: A request is an addressed envelope. The *method* is what you're asking the recipient to do, the *path* is the apartment number, the *headers* are notes on the envelope ("fragile," "I speak English"), and the *body* is the letter inside.

Status codes: how the server says how it went

Status codes are grouped by their first digit:

- **2xx — success.** `200 OK` (here's your page), `201 Created` (your post was saved).
- **3xx — redirect.** `301 Moved Permanently`, `302 Found` — "go look over there instead." Viby uses these to send signed-out users to the login page.
- **4xx — *you* made a mistake.** `400 Bad Request`, `401 Unauthorized` (you're not logged in), `403 Forbidden` (logged in, but not allowed), `404 Not Found` (no such page).
- **5xx — *the server* made a mistake.** `500 Internal Server Error`, `503 Service Unavailable`.

A quick way to remember: **4xx is on you, 5xx is on the server.**

What a browser actually does

When you type `viby.app` and hit Enter, your browser:

1. **Resolves DNS** — turns the name into an IP address.
2. **Opens a connection** — and (for HTTPS) sets up encryption so nobody can read your data.
3. **Sends an HTTP request** — `GET /` to that IP.
4. **Receives the response** — usually an HTML document.
5. **Parses the HTML** — builds an internal tree of the page (the DOM).
6. **Fetches more resources** — CSS, JavaScript, images, fonts referenced by the HTML, each its own request.
7. **Renders** — paints pixels on the screen, then runs the JavaScript that makes the page interactive.

That whole dance often happens in under a second.

Frontend vs. backend

- The **frontend** is everything that runs *in the browser* — the HTML you see, the Tailwind styles, the React components, the animations. In Viby, this is the pink Discover feed, the profile cards, the buttons.
- The **backend** is everything that runs *on the server* — checking passwords, saving posts to the database, deciding who's allowed to see what. In Viby, this is Supabase (auth, database, storage) plus the server-side code in Next.js.

The beautiful part of our stack: **Next.js lets you write both in one project**, in the same language (TypeScript). Some components render on the server (close to the data), others run in the browser (close to the user).

In practice

Let's make this real with the Viby stack. Don't worry about installing anything yet — read the code and notice the *shape* of request and response.

A Server Component fetching data (runs on the backend)

In Next.js 16's App Router, components are **Server Components by default**. This page runs on the server, talks to Supabase, and sends finished HTML to the browser — the database password never reaches the client.

```tsx
// app/discover/page.tsx
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export default async function DiscoverPage() {
  // In Next.js 16, cookies() is async — await it.
  const cookieStore = await cookies();

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll() {
          // Read-only in a Server Component; writes happen in proxy.ts.
        },
      },
    }
  );

  // This is an HTTP request to Supabase, made from the server.
  const { data: people } = await supabase
    .from("profiles")
    .select("username, hobby")
    .limit(10);

  return (
    <main className="mx-auto max-w-md p-4">
      <h1 className="text-2xl font-bold text-brand">Discover your people</h1>
      <ul className="mt-4 space-y-2">
        {people?.map((p) => (
          <li key={p.username} className="rounded-xl bg-white p-3 shadow-sm">
            <span className="font-semibold">@{p.username}</span> loves {p.hobby}
          </li>
        ))}
      </ul>
    </main>
  );
}
```

Reading an async route param

A dynamic route like `/profile/[username]` receives its params as a **Promise** in Next.js 16. Await them:

```
// app/profile/[username]/page.tsx
export default async function ProfilePage({
  params,
}: {
  params: Promise<{ username: string }>;
}) {
  const { username } = await params; // the path segment becomes data
  return <h1 className="text-brand">@{username}</h1>;
}
```

Tailwind v4 brand tokens (the frontend's look)

No `tailwind.config.js`. In Tailwind CSS v4 you define design tokens right in your CSS with `@theme`. That `text-brand` class above comes from here:

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5; /* Viby bubblegum pink */
  --color-brand-ink: #1a1a1a;
}

body {
  background: white;
  color: var(--color-brand-ink);
}
```

A Client Component that *makes a request* on click

`"use client"` opts a component into running in the browser, where it can use state, gestures, and animation. Here, a button sends an HTTP `POST`.

```tsx
// app/discover/WaveButton.tsx
"use client";

import { useState } from "react";
import { motion } from "framer-motion";

export function WaveButton({ username }: { username: string }) {
  const [sent, setSent] = useState(false);

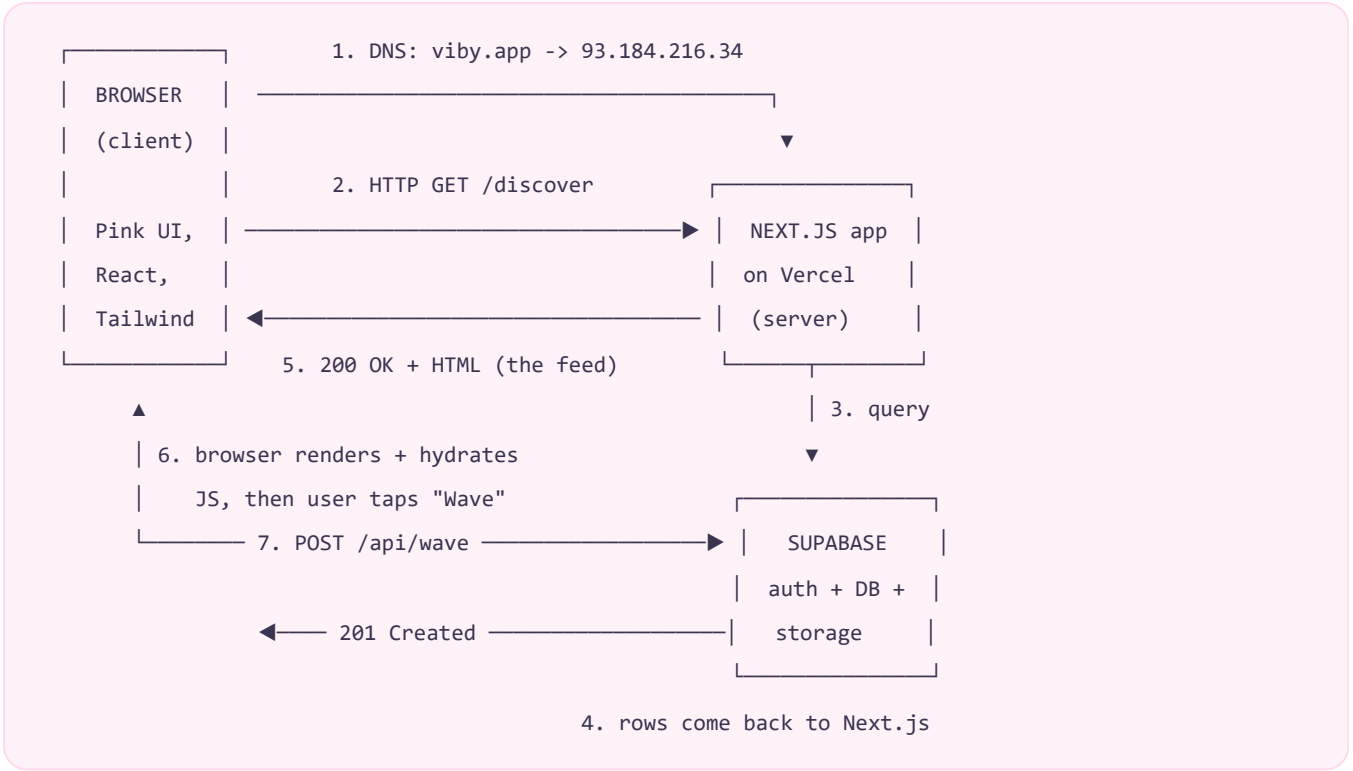
  async function sendWave() {
    // A real HTTP request leaving the browser:
    const res = await fetch("/api/wave", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ to: username }),
    });
    if (res.ok) setSent(true); // res.ok is true for any 2xx status
  }

  return (
    <motion.button
      whileTap={{ scale: 0.92 }}
      onClick={sendWave}
      className="rounded-full bg-brand px-4 py-2 font-semibold text-white"
    >
      {sent ? "Waved 👋" : `Wave at @${username}`}
    </motion.button>
  );
}
```

Notice `res.ok` — that's the browser reading the **status code** for us. It's `true` for 200–299 and `false` for 4xx/5xx.

Architecture / diagram

Here's the full round trip when a signed-in user opens Viby's Discover feed:



The key insight: the **browser never talks to the database directly**. It talks to Next.js; Next.js talks to Supabase. Each arrow is an HTTP request with a method, headers, a body, and a status code coming back.

Best practices & pitfalls

- **Do** read status codes before trusting a response. `fetch` does *not* throw on `404` or `500` — always check `res.ok`.
- **Do** keep secrets on the server. Anything prefixed `NEXT_PUBLIC_` is shipped to the browser; your Supabase **anon** key is fine there, but a service-role key never is.
- **Do** use the right method for the job: `GET` to read, `POST` to create. `GET` requests should never change data.
- **Don't** confuse `401` and `403`. `401` = "we don't know who you are" (log in). `403` = "we know you, you just can't do this."
- **Don't** assume the network is fast or reliable. Requests fail, time out, and arrive out of order. Build for it.
- **Don't** put `"use client"` on everything. It ships more JavaScript and loses server-side data access. Start on the server; reach for the client only when you need interactivity.
- **Pitfall:** forgetting that `cookies()` and `params` are **async** in Next.js 16. Always `await` them.
- **Pitfall:** thinking HTTPS is optional. The `S` (encryption) is why your login password isn't readable by the coffee-shop Wi-Fi. Always use HTTPS in production.

Exercises

1. **Read the verbs.** Open any website, press `F12` to open DevTools, click the **Network** tab, and reload. Find one `GET` request and note its status code. *(Easy)*
2. **Trace a name to a number.** In your terminal, run `nslookup viby.app` (or any site you like). Write down the IP address DNS returns. *(Easy)*
3. **Trigger a 404.** Visit a real site and add `/this-page-does-not-exist` to the URL. Check the Network tab — what status code came back, and what did the body show? *(Medium)*
4. **Inspect headers.** In DevTools, click a single request and find its **Request Headers** and **Response Headers**. Identify the `Content-Type` of the response. *(Medium)*
5. **Map Viby.** On paper, draw the round trip for a user *signing up*: which method, which path, client → Next.js → Supabase, and what status code signals success. *(Harder)*

Mini-project: a request inspector for Viby

Build a tiny page that shows the anatomy of a real HTTP request — a teaching tool you'll reuse to debug Viby later. No database needed.

Create a Server Component route that inspects the request's own headers:

```
// app/inspect/page.tsx
import { headers } from "next/headers";

export default async function InspectPage() {
  const h = await headers(); // async in Next.js 16
  const ua = h.get("user-agent") ?? "unknown";
  const accept = h.get("accept") ?? "unknown";

  return (
    <main className="mx-auto max-w-md p-6">
      <h1 className="text-2xl font-bold text-brand">Request inspector</h1>
      <dl className="mt-4 space-y-3">
        <div className="rounded-xl bg-white p-3 shadow-sm">
          <dt className="text-sm text-gray-500">Your browser (User-Agent)</dt>
          <dd className="break-words font-mono text-sm">{ua}</dd>
        </div>
        <div className="rounded-xl bg-white p-3 shadow-sm">
          <dt className="text-sm text-gray-500">Formats you accept</dt>
          <dd className="break-words font-mono text-sm">{accept}</dd>
        </div>
      </dl>
      <p className="mt-4 text-sm text-gray-600">
        Everything above arrived on the <strong>request headers</strong> — proof
        that your browser tells the server about itself on every visit.
      </p>
    </main>
  );
}
```

When you can run the Viby starter (Chapter 2), open `/inspect`, then watch the same request in the DevTools Network tab. Seeing the *same headers* in your UI and in the browser's tooling is the moment HTTP clicks. **Stretch goal:** add a `WaveButton`-style client component that `fetch`es `/inspect` and prints the returned status code.

Key takeaways

- The **internet** is the roads; the **web** (HTTP) is one kind of traffic on it.
- The web is a **client asking** and a **server answering** — request in, response out.
- **DNS** turns human names (`viby.app`) into machine **IP addresses**.
- Every **HTTP request** has a method, path, headers, and optional body; every **response** has a status code, headers, and body.

- **Status codes:** 2xx success, 3xx redirect, **4xx your fault**, **5xx server's fault**.
- The **frontend** runs in the browser; the **backend** runs on the server. In Viby, Next.js spans both and the browser never touches the database directly.

Up next — Chapter 2: Your Developer Toolkit. We'll install Node.js, your editor, and the Viby starter, then run your first server so all this theory starts breathing on your own machine.

Chapter 2 — Frontend, Backend & Where Code Runs

> Every line of code you write lives *somewhere* — and knowing *where* is the single biggest unlock between "tutorial coder" and "engineer who ships."

What you'll learn

- The mental model of **where code runs**: the browser, the server, and the database — and why it matters.
- The **request lifecycle**: what actually happens between a user tapping a button and seeing a result.
- An overview of **rendering strategies** (SSR, SSG, CSR, streaming) without the jargon fog.
- How **Viby** — our social-discovery app — is split cleanly across these layers in Next.js 16.

Prerequisites

- **Chapter 1** (Welcome to Viby & Your Toolkit): you have Node installed, the Viby repo cloned, and `npm run dev` working.
- Comfort opening a terminal and a code editor. That's genuinely all. We assume *no* prior web theory.

Theory

Let's start with a story, not a spec.

Imagine Viby is a hip neighborhood café where people meet others who share their hobbies. Walk through it with me, because this café *is* the architecture.

The browser is the front counter and dining area. It's everything the customer sees and touches: the menu, the chairs, the music, the smile of the barista. In web terms, the **browser** (Chrome, Safari) renders the buttons, the pink Viby logo, the animations. Code that runs here is the **frontend**. It runs on *the user's own device* — their phone or laptop.

The server is the kitchen. Customers never walk into the kitchen. They place an order at the counter, and the kitchen does the real work: checking the recipe, cooking, plating. In web terms, the **server** is a computer *you control* (in Viby's case, running on Vercel) that does trusted work: checking if someone is logged in, deciding what data they're allowed to see, talking to the database. Code that runs here is the **backend**.

The database is the pantry and the recipe book. It's where all the durable stuff lives: the ingredients, the inventory, the records. Nobody eats raw out of the pantry — the kitchen fetches from it. In Viby, the **database** is Supabase's PostgreSQL: every user profile, every hobby tag, every friend connection is stored there permanently.

Here's the crucial insight beginners miss: **the same JavaScript/TypeScript can run in any of these places, and *where* it runs changes everything.**

- Code in the **browser** is fast to react to taps but **completely untrustworthy** — the user can open dev tools and read or change it. Never put secrets here.
- Code on the **server** is trusted and private. Your database password lives here. The user never sees it.
- Code in the **database** is the source of truth — slow to change, but reliable.

Why "where" matters so much

A beginner writes a "filter profiles by city" feature and thinks: *I'll fetch all profiles into the browser and filter them with JavaScript.* That works for 10 profiles. At 100,000 profiles it ships your entire user table to a stranger's phone — slow, expensive, and a privacy disaster. An engineer asks the *database* to filter, on the *server*, and sends only the 12 results the user needs.

Same logic. Different *where*. Wildly different outcome.

The request lifecycle

When a user opens `viby.app/discover`, here's the real sequence — the heartbeat of every web app:

1. **Browser sends a request.** "GET me the /discover page, please." It includes cookies (which may carry a login session).
2. **The request travels the network** to Vercel, where Viby's server lives.
3. **The server runs your backend code.** It reads the session cookie, asks Supabase "who is this user, and what profiles should they see?", and gets data back.
4. **The server builds a response** — usually HTML, sometimes JSON.
5. **The response travels back** across the network to the browser.
6. **The browser renders** the HTML, then "hydrates" it — wiring up the interactive bits (buttons, animations) with frontend JavaScript.
7. **Later interactions** (liking a profile, opening a chat) may repeat a smaller version of this loop without a full page reload.

Every performance problem, every security bug, every "why is this slow" question maps back to one of these seven steps.

Rendering strategies — the 60-second tour

"Rendering" just means *turning your code into the HTML the user sees*. Where and when that happens is the strategy:

- **CSR (Client-Side Rendering):** The server sends a near-empty page; the browser builds everything with JavaScript. Fast to navigate once loaded, but slow first paint and bad for SEO. The old default.
- **SSR (Server-Side Rendering):** The server builds the full HTML *per request*. Great for personalized, always-fresh pages — like Viby's feed, which differs for every logged-in user.
- **SSG (Static Site Generation):** HTML is built *once at deploy time* and served to everyone. Blazing fast. Perfect for Viby's marketing landing page and `/about`.
- **Streaming:** The server sends HTML *in chunks* as it's ready, so the user sees the shell instantly and the slow parts (a heavy recommendations list) fill in after.

Next.js 16's App Router lets you mix all of these *per route* and even *per component*. You don't pick one for the whole app — you pick the right one for each screen. That's the superpower.

In practice

In Next.js 16's App Router, **every component is a Server Component by default**. That means it runs on the server (the kitchen) — it can talk to the database directly, and its code never ships to the browser. You only opt into the browser with the `"use client"` directive when you need interactivity (clicks, state, animations).

A Server Component: Viby's Discover page

This runs on the server. Notice it `await`s data directly — no loading spinner, no `useEffect`. The user gets fully-formed HTML.

```
// app/discover/page.tsx  (Server Component — runs on the server)
import { createClient } from "@/lib/supabase/server";
import { ProfileCard } from "@/components/profile-card";

export default async function DiscoverPage() {
  const supabase = await createClient();

  // This query runs on the SERVER, talking straight to Supabase.
  // The browser never sees this code or the DB credentials.
  const { data: profiles } = await supabase
    .from("profiles")
    .select("id, display_name, avatar_url, hobbies")
    .limit(12);

  return (
    <main className="mx-auto max-w-2xl p-4">
      <h1 className="text-2xl font-bold text-brand">Discover your people</h1>
      <div className="mt-4 grid grid-cols-2 gap-3">
        {profiles?.map((p) => (
          <ProfileCard key={p.id} profile={p} />
        ))}
      </div>
    </main>
  );
}
```

The `text-brand` class comes from a Tailwind v4 theme token — defined in CSS, no `tailwind.config.js`:

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5;       /* Viby bubblegum pink */
  --color-brand-foreground: #ffffff;
}
```

The Supabase server client (runs on the server)

This is the trusted kitchen helper. It reads cookies using the modern `getAll` / `setAll` handlers from `@supabase/ssr`:

```
// lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies(); // async in Next.js 16

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options)
          );
        },
      },
    }
  );
}
```

A Client Component: the interactive Like button

This needs to run in the *browser* — it responds to taps and animates. So it declares `"use client"` and uses Framer Motion.

```tsx
// components/like-button.tsx
"use client";

import { useState } from "react";
import { motion } from "framer-motion";
import { likeProfile } from "@/app/actions/like";

export function LikeButton({ profileId }: { profileId: string }) {
  const [liked, setLiked] = useState(false);

  return (
    <motion.button
      whileTap={{ scale: 0.85 }}
      onClick={async () => {
        setLiked(true);             // instant UI feedback in the browser
        await likeProfile(profileId); // trusted write happens on the server
      }}
      className="rounded-full px-4 py-2 text-brand"
      aria-pressed={liked}
    >
      {liked ? "✨ Vibing" : "Say hi"}
    </motion.button>
  );
}
```

A Server Action: the trusted write (runs on the server)

The button *called* this, but the code runs back on the server. The `"use server"` directive is the magic bridge — Next.js wires up the network request for you. The database write happens where it's safe.

```
// app/actions/like.ts
"use server";

import { createClient } from "@/lib/supabase/server";
import { revalidatePath } from "next/cache";

export async function likeProfile(profileId: string) {
  const supabase = await createClient();

  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) throw new Error("You must be signed in to vibe.");

  await supabase.from("likes").insert({
    from_user: user.id,
    to_user: profileId,
  });

  revalidatePath("/discover");
}
```

Edge logic with `proxy.ts` (runs before your page, on the server)

Need to guard routes — say, bounce logged-out users away from `/discover`? In Next.js 16 this lives in `proxy.ts` (not the old `middleware.ts`):

```ts
// proxy.ts  (runs on the server, before the request reaches your page)
import { createServerClient } from "@supabase/ssr";
import { NextResponse, type NextRequest } from "next/server";

export async function proxy(request: NextRequest) {
  const response = NextResponse.next();

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value, options }) =>
            response.cookies.set(name, value, options)
          );
        },
      },
    }
  );

  const {
    data: { user },
  } = await supabase.auth.getUser();

  if (!user && request.nextUrl.pathname.startsWith("/discover")) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  return response;
}

export const config = {
  matcher: ["/discover/:path*", "/chat/:path*"],
};
```

Read it top to bottom and you can *feel* the layers: server fetches data, server guards routes, browser handles taps, server handles writes. Every piece in its right home.

Architecture / diagram

Here's the full Viby request lifecycle for a logged-in user opening Discover and tapping "Say hi":

```
sequenceDiagram
    participant B as Browser (frontend)
    participant P as proxy.ts (server)
    participant S as Server Component (server)
    participant DB as Supabase Postgres (database)

    B->>P: GET /discover (with session cookie)
    P->>P: Check auth — user logged in? ✓
    P->>S: Forward request
    S->>DB: SELECT profiles LIMIT 12
    DB-->>S: 12 rows
    S-->>B: Rendered HTML (the page)
    B->>B: Hydrate — wire up Like buttons & motion
    Note over B: User taps "Say hi"
    B->>S: Server Action: likeProfile(id)
    S->>DB: INSERT into likes
    DB-->>S: ok
    S-->>B: revalidate /discover
```

And the layered "where does it live" map:

```
┌──────────────────────────────────────────────────────┐
│  BROWSER (user's phone)          — untrusted, interactive  │
│  "use client" components, Framer Motion, Zustand store    │
└──────────────▲──────────────────────────┬────────────┘
               │ HTML / Server Action calls │
┌──────────────┴──────────────────────────▼────────────┐
│  SERVER (Vercel)                 — trusted, has secrets     │
│  Server Components, Server Actions, proxy.ts, Supabase key │
└──────────────▲──────────────────────────┬────────────┘
               │ SQL queries                │
┌──────────────┴──────────────────────────▼────────────┐
│  DATABASE (Supabase Postgres)    — source of truth          │
│  profiles, likes, hobbies, messages + Row Level Security   │
└──────────────────────────────────────────────────────┘
```

Best practices & pitfalls

Do:

- **Default to Server Components.** Only add `"use client"` when you truly need interactivity, state, or browser-only APIs. This keeps your JavaScript bundle tiny and fast.
- **Fetch data on the server, close to the database.** Filter and limit *in the SQL query*, not in the browser.
- **Keep secrets server-side.** Anything in a Server Component, Server Action, or `proxy.ts` is safe. Anything in a `"use client"` file ships to the user.
- **Use Server Actions for writes** — they're typed, secure, and need no hand-rolled API route.

Don't:

- **Don't put your Supabase *service-role* key in a `NEXT_PUBLIC_` variable.** The `NEXT_PUBLIC_` prefix literally means "ship this to the browser." Only the public anon key belongs there.
- **Don't `useEffect` + fetch for initial page data** when a Server Component can `await` it directly. That's the old CSR habit; it causes loading spinners and slow first paint.
- **Don't trust client-side checks for security.** A disabled button is UX, not protection. Always re-check auth inside the Server Action (note our `getUser()` guard above).
- **Don't reach for `proxy.ts` for heavy work.** It runs on *every* matched request — keep it to fast auth/redirect logic.

Exercises

1. **Trace the layers.** Open `app/discover/page.tsx`. Write a one-sentence comment above each block stating *where* it runs (browser / server / database). Confirm the data fetch is server-side.
2. **Prove the boundary.** Add a `console.log("hello from where?")` to the Discover Server Component and to the `LikeButton` client component. Run `npm run dev`, open the page, and find which log appears in your *terminal* versus your *browser console*. Explain why.
3. **Static vs dynamic.** Create a static `app/about/page.tsx` that returns hard-coded JSX (no data fetch). Compare its behavior to `/discover`. Which is SSG-friendly and why?
4. **Tighten a query.** Modify the Discover query to also filter `.eq("city", "Berlin")`. Notice you changed *where* the filtering happens (database) — never the browser.
5. **(Stretch)** Add a second `matcher` route to `proxy.ts` protecting `/profile/edit`, and verify a logged-out user gets redirected to `/login`.

Mini-project: The "Where Am I Running?" badge

Build a tiny diagnostic that makes the frontend/backend split *visible*.

1. Create a Server Component `app/where/page.tsx` that fetches the current time on the server and renders it, plus a heading "Server rendered at: …".
2. Create a Client Component `components/client-clock.tsx` (`"use client"`) that shows a live ticking clock using `useState` + `useEffect`, labelled "Browser time: …".
3. Render the client clock inside the server page.

```tsx
// app/where/page.tsx (Server Component)
import { ClientClock } from "@/components/client-clock";

export default function WherePage() {
  const serverTime = new Date().toLocaleTimeString();
  return (
    <main className="mx-auto max-w-md p-6 space-y-2">
      <p className="font-semibold text-brand">Server rendered at: {serverTime}</p>
      <ClientClock />
    </main>
  );
}
```

```tsx
// components/client-clock.tsx (Client Component)
"use client";
import { useEffect, useState } from "react";

export function ClientClock() {
  const [now, setNow] = useState<string>("");
  useEffect(() => {
    const id = setInterval(() => setNow(new Date().toLocaleTimeString()), 1000);
    return () => clearInterval(id);
  }, []);
  return <p className="text-gray-500">Browser time (live): {now}</p>;
}
```

What you'll observe: the server line is *frozen* at the moment the page was built on the server. The browser line *ticks* because that code runs live on your device. You've just *seen* the boundary between backend and frontend — the whole chapter in one screen.

Key takeaways

- Code runs in one of three places — **browser (frontend), server (backend), database** — and *where* drives speed, security, and cost.
- The **request lifecycle** is a predictable seven-step loop; every bug and slowdown maps to one step.
- Next.js 16 components are **Server Components by default**; opt into the browser with `"use client"` only for interactivity.
- **Server Actions** (`"use server"`) and `proxy.ts` keep trusted logic and secrets on the server, away from prying eyes.
- Pick rendering strategy **per route**: SSG for marketing, SSR for the personalized feed, streaming for slow sections.

Up next — Chapter 3: HTML, CSS & the Box Model, the Viby Way. Now that you know *where* your UI is built, we'll learn how to actually *build* it — pixel by pixel, in Viby's signature pink.

Chapter 3 — React Fundamentals: Components, JSX & Props

> Every screen in Viby — from a profile card to the whole feed — is just small pieces of UI snapped together like LEGO. Today you learn how to make those pieces.

What you'll learn

- What React actually is, and *why* it beats hand-writing HTML and DOM updates
- JSX: writing UI that looks like HTML but is really JavaScript
- Components as plain functions that return UI
- Props: passing data *into* a component so it can be reused
- Composition: building big screens out of small components
- Rendering lists with `key`, and conditional rendering (show/hide UI)
- Building real Viby UI pieces: a `HobbyTag`, a `ProfileCard`, and a card grid

Prerequisites

- Chapter 1 — your environment and the Viby Next.js 16 project are set up
- Chapter 2 — comfortable with JavaScript basics: variables, functions, arrays, `.map()`, and objects

If `npm run dev` starts the app and you can edit a page, you're ready.

Theory

What is React, and why?

Imagine you're decorating a wall with hundreds of sticky notes. The old way (plain HTML + JavaScript) is like writing every note by hand, then every time one number changes, hunting through the whole wall to find that exact note and rewrite it. Tedious, and easy to update the wrong one.

React is a librarian for your UI. You describe *what the wall should look like* for the current data — "here are the notes" — and React figures out the smallest set of changes needed to make the real wall match. You stop manually touching the page (the "DOM"); you just describe the result, and React keeps the screen in sync with your data.

That single idea — **UI is a function of your data** — is why React scales from a tiny button to an app like Viby with feeds, profiles, and chat.

JSX: HTML that's secretly JavaScript

React lets you write markup *inside* JavaScript using **JSX**:

```
const greeting = <h1>Welcome to Viby</h1>;
```

That looks like HTML, but it isn't — it's JavaScript that React turns into UI. Because it's JavaScript, a few things differ from HTML:

- `class` becomes `className` (because `class` is a reserved word in JS)
- Attributes are camelCase: `onClick`, `tabIndex`
- You drop *values* in with **curly braces** `{ }`:

```
const name = "Aanya";
const tag = <p>Hey {name}, find your people 💖</p>;
```

Anything inside `{ }` is real JavaScript: a variable, `user.name`, `2 + 2`, or a function call. A component must return **one** top-level element; when you need siblings without an extra wrapper, use a **Fragment** `<>...</>`.

Components are just functions

A **component** is a function that returns JSX. That's the whole secret.

```
function HobbyTag() {
  return <span>Climbing</span>;
}
```

Think of a component like a **cookie cutter**. Define the shape once, then stamp out as many cookies as you want. The function is the cutter; each time you write `<HobbyTag />` you stamp one cookie onto the screen. Component names **must start with a capital letter** — that's how React tells your components apart from regular HTML tags like `<span>`.

Props: passing data in

A cookie cutter that only makes "Climbing" cookies is useless. We want *any* hobby. **Props** ("properties") are the inputs you hand a component, exactly like arguments to a function:

```
function HobbyTag({ label }: { label: string }) {
  return <span>{label}</span>;
}

// Used like HTML attributes:
<HobbyTag label="Climbing" />
<HobbyTag label="Lo-fi beats" />
```

Props flow **one direction**: parent → child, like passing a note down a row of desks. A child reads its props but never edits them — props are read-only.

Composition: small pieces, big screens

You don't build a `ProfileCard` as one giant function. You **compose** it: a card *contains* an avatar, a name, and several `HobbyTag`s. Small, focused components snap together into screens — the LEGO model. This keeps each piece simple, testable, and reusable across the app.

Lists and keys

Viby shows *many* profiles. You render a list by mapping an array to components, giving each a stable `key` so React can track items as they're added, removed, or reordered:

```
{hobbies.map((h) => <HobbyTag key={h} label={h} />)}
```

The `key` is React's name tag for each item. Use a stable unique id (a database id is ideal) — **never the array index** if the list can change order, or React may reuse the wrong item.

Conditional rendering

Sometimes UI should appear only *sometimes* — a "verified" badge, an empty state when there are no results. Because JSX is JavaScript, you just use normal logic:

```
{isVerified && <VerifiedBadge />}        // show only if true
{count > 0 ? <Feed /> : <EmptyState />}  // either / or
```

In practice

These are real Viby pieces. In **Next.js 16** with the App Router, components are **Server Components by default** — they render on the server, which is perfect for static, data-driven UI like cards. You only add the `"use client"` directive when a component needs interactivity (clicks, state, animation). Our display components below need none of that, so they stay server components.

First, our Tailwind v4 brand tokens live in CSS via `@theme` — **no** `tailwind.config.js`:

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-bubblegum: #ff6fb5;
  --color-ink: #1a1a1a;
  --radius-card: 1.25rem;
}
```

Now a typed, reusable `HobbyTag`:

```tsx
// components/hobby-tag.tsx
type HobbyTagProps = {
  label: string;
};

export function HobbyTag({ label }: HobbyTagProps) {
  return (
    <span className="rounded-full bg-bubblegum/10 px-3 py-1 text-sm font-medium text-bubblegum">
      {label}
    </span>
  );
}
```

A `ProfileCard` that **composes** `HobbyTag`, renders a **list with keys**, and uses **conditional rendering** for the verified badge:

```tsx
// components/profile-card.tsx
import { HobbyTag } from "./hobby-tag";

export type Profile = {
  id: string;
  name: string;
  city: string;
  isVerified: boolean;
  hobbies: string[];
};

export function ProfileCard({ profile }: { profile: Profile }) {
  return (
    <article className="rounded-card bg-white p-5 shadow-sm ring-1 ring-black/5">
      <header className="flex items-center gap-2">
        <h3 className="text-lg font-bold text-ink">{profile.name}</h3>
        {profile.isVerified && (
          <span aria-label="Verified" className="text-bubblegum">✓</span>
        )}
      </header>

      <p className="text-sm text-ink/60">{profile.city}</p>

      <div className="mt-3 flex flex-wrap gap-2">
        {profile.hobbies.map((hobby) => (
          <HobbyTag key={hobby} label={hobby} />
        ))}
      </div>
    </article>
  );
}
```

A page that maps over data and handles the **empty state** — note the async `searchParams` is awaited, as required in Next.js 16:

```tsx
// app/discover/page.tsx
import { ProfileCard, type Profile } from "@/components/profile-card";

const PROFILES: Profile[] = [
  { id: "u1", name: "Aanya", city: "Mumbai", isVerified: true,
    hobbies: ["Climbing", "Lo-fi beats", "Ceramics"] },
  { id: "u2", name: "Rohan", city: "Pune", isVerified: false,
    hobbies: ["Cycling", "Board games"] },
];

export default async function DiscoverPage(props: {
  searchParams: Promise<{ city?: string }>;
}) {
  const { city } = await props.searchParams;

  const profiles = city
    ? PROFILES.filter((p) => p.city.toLowerCase() === city.toLowerCase())
    : PROFILES;

  return (
    <main className="mx-auto max-w-2xl p-4">
      <h1 className="mb-4 text-2xl font-extrabold text-ink">Discover your people</h1>

      {profiles.length > 0 ? (
        <div className="grid gap-4 sm:grid-cols-2">
          {profiles.map((profile) => (
            <ProfileCard key={profile.id} profile={profile} />
          ))}
        </div>
      ) : (
        <p className="text-ink/60">No one here yet — try a different city 🌎</p>
      )}
    </main>
  );
}
```

Run `npm run dev`, open `/discover`, and you'll see two profile cards. Visit `/discover?city=pune` and the list filters to one — conditional rendering and props doing real work.

Architecture / diagram

How data flows down and UI composes up:

```
flowchart TD
  A["DiscoverPage (Server Component)"] -->|profile prop| B["ProfileCard"]
  A -->|profile prop| C["ProfileCard"]
  B -->|label prop| D["HobbyTag"]
  B -->|label prop| E["HobbyTag"]
  C -->|label prop| F["HobbyTag"]

  subgraph data["Data source"]
    G["PROFILES array"]
  end
  G -. ".map() + key" .-> A
```

Props flow strictly **downward** (parent → child). Each arrow is one prop handed to a child. The page owns the data; children just receive and render it.

Best practices & pitfalls

- **Do** name components with a Capital letter (`ProfileCard`), or React treats them as HTML tags.
- **Do** give every list item a stable `key` (a database `id`). **Don't** use the array index for lists that can reorder, filter, or grow.
- **Do** type your props with a `type` / `interface` — it documents the component and catches mistakes early.
- **Don't** mutate props inside a child. Props are read-only inputs; if data must change, that's *state* (next chapter).
- **Do** keep components small and single-purpose. If a function is hard to name, it's probably doing two jobs — split it.
- **Don't** add `"use client"` to a component that doesn't need interactivity — keep display components as Server Components for speed.
- **Watch out**: `{0 && <X/>}` renders a literal `0` . Guard with a real boolean: `{count > 0 && <X/>}` .
- **Remember** `className` , not `class` ; camelCase event names like `onClick` .

Exercises

1. **Warm-up:** Create a `CityBadge` component that takes a `city` prop and renders it as a small grey pill. Use it inside `ProfileCard` in place of the plain `<p>` .
2. **Props + conditional:** Add an optional `mutualFriends?: number` prop to `ProfileCard` . Show `🤝 {n} mutual` only when `n` is greater than 0.
3. **Lists:** Add a third profile to the `PROFILES` array with four hobbies. Confirm no `key` warning appears in the console, then deliberately remove a `key` to see the warning.
4. **Composition:** Extract the card header (name + verified check) into its own `ProfileHeader` component and compose it inside `ProfileCard` .
5. **Stretch:** Make `HobbyTag` accept an optional `tone?: "pink" | "grey"` prop and switch `className` accordingly using a conditional expression.

Mini-project — Viby "People Near You" strip

Build a horizontally scrolling strip of profile cards for the Viby home screen.

Requirements:

- A `PeopleStrip` component that takes a `profiles: Profile[]` prop.
- Render the cards in a horizontal, scrollable row (Tailwind: `flex gap-4 overflow-x-auto`).
- Reuse the existing `ProfileCard` — no copy-paste.
- Show a heading "People near you" and a count: "12 nearby" using conditional logic for singular/plural ("1 nearby" vs "N nearby").
- If the array is empty, render a friendly empty state instead of the strip.

```tsx
// components/people-strip.tsx
import { ProfileCard, type Profile } from "./profile-card";

export function PeopleStrip({ profiles }: { profiles: Profile[] }) {
  const count = profiles.length;

  return (
    <section className="space-y-3">
      <div className="flex items-baseline justify-between">
        <h2 className="text-xl font-bold text-ink">People near you</h2>
        {count > 0 && (
          <span className="text-sm text-ink/50">
            {count} nearby
          </span>
        )}
      </div>

      {count > 0 ? (
        <div className="flex gap-4 overflow-x-auto pb-2">
          {profiles.map((profile) => (
            <div key={profile.id} className="w-64 shrink-0">
              <ProfileCard profile={profile} />
            </div>
          ))}
        </div>
      ) : (
        <p className="text-ink/60">No one nearby yet — check back soon 💔</p>
      )}
    </section>
  );
}
```

Drop `<PeopleStrip profiles={PROFILES} />` onto your home page and you've shipped a real, reusable Viby feature using nothing but components, props, lists, and conditionals.

Key takeaways

- React keeps the screen in sync with your data: **UI is a function of data**.
- **JSX** is JavaScript that looks like HTML; use `{ }` to drop in values, `className` not `class`.
- **Components are functions** that return JSX; capitalize their names.
- **Props** pass data parent → child and are **read-only**.
- **Compose** small components into big screens; map arrays to lists with a stable `key`.

- Use plain JS (`&&` , ternary) for **conditional rendering**; keep display UI as Server Components.

Up next — Chapter 4: State & Events — making components *remember* and *react* to clicks, so your Viby UI comes alive.

Chapter 4 — State, Events & Effects

> Your Viby screen is about to stop being a poster on a wall and start being a living thing that *remembers, reacts, and breathes.*

What you'll learn

- **State** with `useState` — how a component remembers things (like how many people you've followed)
- **Event handling** — making buttons, taps, and typing actually *do* something
- **Controlled inputs** — owning every keystroke in a text field
- **Lifting state up** — letting two components share one source of truth
- `useEffect` — talking to the outside world, and the bigger lesson: *when NOT to use it*
- **Derived state** — values you calculate instead of store (a beginner superpower)
- **The React render model** — the mental model that makes all of the above click

Prerequisites

You should have finished **Chapter 2 (Components & JSX)** and **Chapter 3 (Props & Composition)**. You need to be comfortable writing a component, passing props into it, and reading them. We'll also lean on the `"use client"` boundary idea from Chapter 3 — in Next.js 16 everything is a **Server Component** until you opt into the browser. State and events live in the browser, so this is our first heavily client-side chapter.

Theory

A component is a snapshot, not a whiteboard

Beginners usually imagine a component as a whiteboard: you draw something, then you walk back up and erase a number and rewrite it. That is **not** how React works, and unlearning the whiteboard is the single most important thing in this chapter.

Instead, think of a React component as a **photographer**. Every time something changes, React doesn't edit the old photo — it asks your component to take a brand-new photo of how things look *right now*. That new snapshot replaces the old one on screen. This taking-a-new-photo is called a **render**.

So the real question becomes: *what makes React take a new photo?* The answer is **state**.

State: the component's memory

State is data that a component remembers between renders, and that — when changed — tells React "take a new photo."

Analogy: state is the **counter on a coffee shop's "now serving" sign**. The number lives somewhere persistent. When the barista bumps it, the sign visibly updates. In React, `useState` is that sign.

```
const [following, setFollowing] = useState(false);
```

Read this left to right:

- `following` — the current value (the photo you're looking at right now)
- `setFollowing` — the **only** legal way to change it ("bump the sign")
- `useState(false)` — the starting value

The golden rule: **never reassign the variable directly** (`following = true` does nothing useful). You call `setFollowing(true)`, which tells React to re-render with the new value. The setter is the doorbell; reassigning is knocking on the window and hoping someone hears.

Events: how the user talks to your app

An **event** is something the user does — a tap, a click, a keystroke, a hover. In React you respond to events with **handlers**: functions you hand to JSX like `onClick={...}`.

Analogy: an event handler is a **doorbell wired to an action**. You don't run the action now; you *connect the wire* and wait. That's why you pass the function (`onClick={handleClick}`) instead of calling it (`onClick={handleClick()}` — wrong, that rings the bell immediately at render time).

Derived state: don't store what you can calculate

Here's a trap every beginner falls into: storing things in state that you could just *compute*.

Analogy: if you know someone's birthday, you don't keep a sticky note with their age and update it every year — you calculate age from the birthday whenever you need it. The birthday is the source of truth; the age is **derived**.

In Viby: if you store a `members` array, the count of members is `members.length`. Don't keep a separate `memberCount` state — it will drift out of sync the moment you forget to update it. **Derived state means: calculate during render, don't store.**

Lifting state up: one source of truth

When two components need the same piece of data, you don't give each its own copy — they'd disagree. You move the state **up** to their closest common parent, and pass it down as props.

Analogy: two roommates shouldn't each keep a private grocery list. You put **one** shared list on the fridge (the parent), and both read and write to it.

`useEffect` : a fire escape, not a front door

`useEffect` lets you run code that reaches **outside** React — fetching data, setting up a subscription, syncing with a non-React system. It runs *after* the render, as a side effect.

The crucial beginner lesson: **most things you reach for `useEffect` for don't need it.** Calculating a value? That's derived state, do it in render. Responding to a click? That's an event handler. `useEffect` is the *fire escape* — there for genuine outside-world syncing (a Supabase Realtime subscription, a browser timer), not the everyday front door.

In practice

Let's build real Viby pieces. These are client components, so they start with `"use client"`.

1. `useState` + an event: the Follow button

```
"use client";

import { useState } from "react";

export function FollowButton({ name }: { name: string }) {
  const [following, setFollowing] = useState(false);

  function toggleFollow() {
    setFollowing((prev) => !prev); // flip based on the latest value
  }

  return (
    <button
      onClick={toggleFollow}
      className={`rounded-full px-5 py-2 text-sm font-semibold transition ${
        following
          ? "bg-white text-pink-600 ring-2 ring-pink-500"
          : "bg-pink-500 text-white hover:bg-pink-600"
      }`}
    >
      {following ? `Following ${name}` : `Follow ${name}`}
    </button>
  );
}
```

Notice `setFollowing((prev) => !prev)`. When your new value depends on the old one, **pass a function**. React hands you the freshest value, which sidesteps a whole class of stale-value bugs.

> **Tailwind v4 note:** there's no `tailwind.config.js`. Your brand pink lives in CSS via `@theme`. In `app/globals.css`:
>
> ```css @import "tailwindcss";
>
> @theme { --color-pink-500: #ff6fb5; / *Viby bubblegum* / --color-pink-600: #ff4fa3; } ```
>
> Now `bg-pink-500` is on-brand everywhere — define the token once, use it forever.

2. Controlled input: the "set your vibe" field

A **controlled input** means React state is the single source of truth for what's in the box. The input shows `value`, and every keystroke calls `onChange` to update that value.

```
"use client";

import { useState } from "react";

export function VibeInput() {
  const [vibe, setVibe] = useState("");

  const MAX = 60;
  const remaining = MAX - vibe.length; // derived — not stored!

  return (
    <div className="space-y-1">
      <input
        type="text"
        value={vibe}
        onChange={(e) => setVibe(e.target.value)}
        maxLength={MAX}
        placeholder="late-night ramen + lo-fi beats"
        className="w-full rounded-xl border border-pink-200 px-4 py-2 outline-none focus:ring-2
focus:ring-pink-500"
      />
      <p className={`text-xs ${remaining < 10 ? "text-pink-600" : "text-gray-400"}`}>
        {remaining} characters left
      </p>
    </div>
  );
}
```

`remaining` is **derived state** in the wild: we never store the count, we compute it from `vibe` on every render. It can never be wrong.

3. Lifting state up: a hobby picker that talks to a preview

The picker and the preview both care about *which hobbies are selected*. So the parent owns the state and passes it down.

```tsx
"use client";

import { useState } from "react";

const HOBBIES = ["Bouldering", "Vinyl", "Matcha", "Film photography", "D&D"];

export function HobbyPicker() {
  // State lives in the PARENT — the single source of truth.
  const [selected, setSelected] = useState<string[]>([]);

  function toggle(hobby: string) {
    setSelected((prev) =>
      prev.includes(hobby)
        ? prev.filter((h) => h !== hobby)
        : [...prev, hobby]
    );
  }

  return (
    <div className="space-y-4">
      <ChipGrid hobbies={HOBBIES} selected={selected} onToggle={toggle} />
      <ProfilePreview selected={selected} />
    </div>
  );
}

function ChipGrid({
  hobbies,
  selected,
  onToggle,
}: {
  hobbies: string[];
  selected: string[];
  onToggle: (hobby: string) => void;
}) {
  return (
    <div className="flex flex-wrap gap-2">
      {hobbies.map((hobby) => {
        const isOn = selected.includes(hobby);
        return (
          <button
            key={hobby}
            onClick={() => onToggle(hobby)}
```

```
            className={`rounded-full px-4 py-1.5 text-sm transition ${
              isOn ? "bg-pink-500 text-white" : "bg-pink-50 text-pink-700"
            }`}
          >
            {hobby}
          </button>
        );
      })}
    </div>
  );
}

function ProfilePreview({ selected }: { selected: string[] }) {
  // Reads the same lifted state via props — always in sync.
  return (
    <p className="text-sm text-gray-600">
      {selected.length === 0
        ? "Pick a few things you're into ✨"
        : `Your people love: ${selected.join(", ")}`}
    </p>
  );
}
```

The child `ChipGrid` doesn't own anything — it reports clicks **up** via `onToggle`, and the parent updates the shared state. Both children re-render from one truth.

4. `useEffect` done right: a live Supabase Realtime subscription

Now a *legitimate* effect. We subscribe to new messages in a Viby chat room. This syncs with an external system (Supabase Realtime), so an effect is correct — and we **clean up** when the component unmounts.

```tsx
"use client";

import { useEffect, useState } from "react";
import { createBrowserClient } from "@supabase/ssr";

type Message = { id: string; body: string };

const supabase = createBrowserClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
);

export function RoomFeed({ roomId }: { roomId: string }) {
  const [messages, setMessages] = useState<Message[]>([]);

  useEffect(() => {
    const channel = supabase
      .channel(`room:${roomId}`)
      .on(
        "postgres_changes",
        { event: "INSERT", schema: "public", table: "messages", filter: `room_id=eq.${roomId}` },
        (payload) => {
          setMessages((prev) => [...prev, payload.new as Message]);
        }
      )
      .subscribe();

    // Cleanup: React runs this when roomId changes or the component unmounts.
    return () => {
      supabase.removeChannel(channel);
    };
  }, [roomId]); // dependency array: re-subscribe only when the room changes

  return (
    <ul className="space-y-2">
      {messages.map((m) => (
        <li key={m.id} className="rounded-lg bg-pink-50 px-3 py-2 text-sm">
          {m.body}
        </li>
      ))}
    </ul>
```

```
  );
}
```

Three things make this a *correct* effect: it touches an external system, it has a **dependency array** (`[roomId]`) so it doesn't re-run on every render, and it **returns a cleanup function** so we don't leak subscriptions.

5. The thing you should NOT use an effect for

Beginners often write this:

```
// 🚫 ANTI-PATTERN — do not do this
const [members, setMembers] = useState<string[]>([]);
const [count, setCount] = useState(0);

useEffect(() => {
  setCount(members.length); // syncing state from state — causes an extra render
}, [members]);
```

Delete all of it. The count is derived:

```
// ✅ Just calculate it during render
const [members, setMembers] = useState<string[]>([]);
const count = members.length;
```

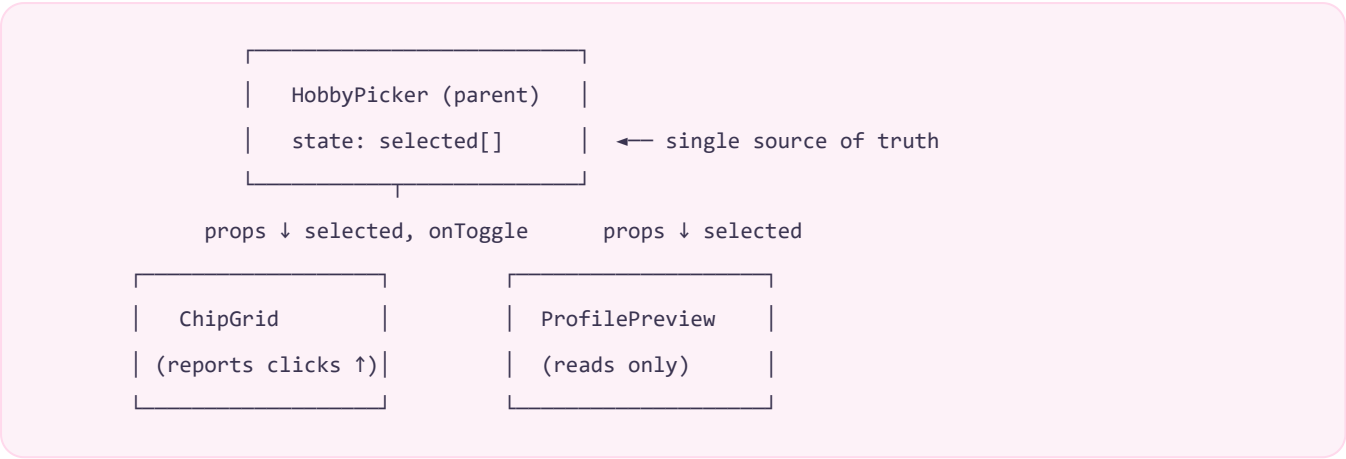
No effect, no second state, no chance of drift, no extra render. If you can compute it from existing props or state, **compute it** — don't mirror it into `useEffect` .

Architecture / diagram

How a click becomes a new photo on screen — the React render loop:

```
flowchart TD
    A[User taps Follow button] --> B[onClick handler runs]
    B --> C["setFollowing(true) — schedules an update"]
    C --> D[React re-renders the component]
    D --> E[Derived values recalculated<br/>e.g. button label, remaining chars]
    E --> F[React compares new snapshot to old]
    F --> G[Only the changed DOM is updated]
    G --> H[useEffect runs AFTER paint<br/>if its dependencies changed]
    H --> A
```

And lifting state up, visually:



Best practices & pitfalls

- **Do** update state with the setter, never by reassigning the variable. `setX(...)`, not `x = ...`.
- **Do** use the updater function (`setX((prev) => ...)`) whenever the next value depends on the previous one.
- **Don't** mutate state directly. `arr.push(x)` then `setArr(arr)` won't re-render reliably — make a *new* array: `setArr([...arr, x])`.
- **Do** prefer **derived state** over storing computed values. If it can be calculated, calculate it.
- **Don't** use `useEffect` to sync one piece of state from another — that's a derived-state smell.
- **Do** pass the handler, not the call: `onClick={handle}` ✅, `onClick={handle()}` 🚫 (the second runs at render).
- **Do** give a `key` to every item in a `.map()` — a stable ID, not the array index when the list reorders.
- **Don't** forget the **cleanup function** in effects that subscribe or set timers — leaks cause ghost updates and warnings.
- **Do** keep the dependency array honest: list every value from the component the effect reads. Lying to it causes stale data.
- **Remember**: `useState` / `useEffect` only work in `"use client"` components. Server Components (the Next.js 16 default) have no state.

Exercises

1. **Warm-up.** Add a `LikeButton` to a Viby post that shows a heart and a count. Tapping toggles liked/unliked and bumps the count up or down. Use one `useState` and the updater function.
2. **Controlled + derived.** Build a `BioField` (textarea) capped at 150 characters. Show a live counter that turns pink when fewer than 20 remain. The counter must be **derived**, not stored.
3. **Lift it up.** Make a `RoomFilter` with a search box and a `RoomList` below it. The typed query lives in the **parent**; the list shows only rooms whose name includes the query (case-insensitive). Filtering is derived state.

4. **Effect with cleanup.** Build an `OnlineDot` that flips between "online" and "away" every 5 seconds using `setInterval` inside `useEffect`. Clear the interval in the cleanup function. (Then ask yourself: could a timer like this ever be derived? Why not?)
5. **Spot the anti-pattern.** Take exercise 2 and *intentionally* rewrite the counter using `useState` + `useEffect`. Notice the extra render and the risk of drift, then revert. Naming the smell is half of avoiding it.

Mini-project: the "Find Your People" onboarding card

Build one polished client component, `FindYourPeople`, combining everything:

- A **controlled** name input and a **controlled** vibe input.
- A row of hobby chips (**lifted state**, multi-select) — reuse the pattern from In Practice #3.
- A live **derived** readiness check: the "Done" button is enabled only when name is non-empty **and** at least 2 hobbies are picked. Compute `const canFinish = name.trim() !== "" && selected.length >= 2;` — no extra state.
- A Framer Motion flourish: animate each selected chip in with `AnimatePresence`, and pop the "Done" button when it becomes enabled.

```
"use client";

import { useState } from "react";
import { motion, AnimatePresence } from "framer-motion";

const HOBBIES = ["Bouldering", "Vinyl", "Matcha", "Film", "D&D", "Running"];

export function FindYourPeople() {
  const [name, setName] = useState("");
  const [selected, setSelected] = useState<string[]>([]);

  const canFinish = name.trim() !== "" && selected.length >= 2; // derived

  function toggle(hobby: string) {
    setSelected((prev) =>
      prev.includes(hobby) ? prev.filter((h) => h !== hobby) : [...prev, hobby]
    );
  }

  return (
    <div className="mx-auto max-w-sm space-y-4 rounded-2xl bg-white p-6 shadow-lg">
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Your name"
        className="w-full rounded-xl border border-pink-200 px-4 py-2 outline-none focus:ring-2
focus:ring-pink-500"
      />

      <div className="flex flex-wrap gap-2">
        {HOBBIES.map((hobby) => {
          const isOn = selected.includes(hobby);
          return (
            <button
              key={hobby}
              onClick={() => toggle(hobby)}
              className={`rounded-full px-3 py-1.5 text-sm transition ${
                isOn ? "bg-pink-500 text-white" : "bg-pink-50 text-pink-700"
              }`}
            >
              {hobby}
            </button>
          );
```

```
        })}
      </div>

      <AnimatePresence>
        {selected.length > 0 && (
          <motion.p
            initial={{ opacity: 0, y: 4 }}
            animate={{ opacity: 1, y: 0 }}
            exit={{ opacity: 0 }}
            className="text-sm text-gray-600"
          >
            Matching you with people who love {selected.join(", ")}
          </motion.p>
        )}
      </AnimatePresence>

      <motion.button
        disabled={!canFinish}
        animate={{ scale: canFinish ? 1.03 : 1 }}
        className="w-full rounded-full bg-pink-500 py-2 font-semibold text-white disabled:opacity-40"
      >
        Find my people
      </motion.button>
    </div>
  );
}
```

Ship it, tap around, and watch how *one* change to state ripples through derived values and animations — with zero `useEffect` needed.

Key takeaways

- A render is React **taking a new photo**; state changes are what trigger it.
- `useState` gives you `[value, setter]` — change state only through the setter, and use the updater function when the next value depends on the old.
- Event handlers are wires you connect (`onClick={fn}`), not actions you run now.
- **Controlled inputs** make React state the single source of truth for form fields.
- **Lift state up** to the closest common parent when components must share it.
- **Derived state** > stored state: if you can calculate it, don't store it.
- `useEffect` is for syncing with the **outside world** (subscriptions, timers) — and it usually needs a dependency array *and* a cleanup function. Most "effects" beginners write should be derived state or event handlers instead.

Up next — Chapter 5: Data Fetching & Server Actions, where we leave in-memory state behind and start reading and writing real Viby data with Supabase, Server Components, and the `"use server"` directive.

Chapter 5 — TypeScript for React + Talking to APIs

> Give your code a memory and a conscience: types catch your mistakes before your users ever do.

What you'll learn

- What TypeScript is and *why* it makes you a faster, calmer developer
- Core building blocks: `type`, `interface`, **union types**, and **generics**
- How to type React **props** and **state** so your components can't be misused
- `fetch`, `async/await`, promises, and JSON — how the browser talks to a server
- How to describe an API response with types and build a small, fully-typed **data layer** for Viby

Prerequisites

- Chapter 2 (JavaScript fundamentals — variables, functions, objects, arrays)
- Chapter 3 (your first React components and JSX)
- Chapter 4 (props, state with `useState`, and the component tree)

You don't need to have written a single line of TypeScript before. We start from zero.

Theory

Why types? The "labeled jars" analogy

Imagine a kitchen where every jar is unlabeled. One looks like sugar but is salt. You only find out when the cake is ruined. JavaScript is that kitchen — a variable can hold a number now and a string five lines later, and nobody warns you until everything's in the oven (production).

TypeScript adds labels to the jars. You say "this jar holds `string`," and if you ever try to pour a `number` in, TypeScript stops you *while you're cooking* — in your editor, with a red squiggle — not after a user has already tasted the result.

TypeScript is just JavaScript with a type-checker bolted on. The browser never runs TypeScript directly; it gets *compiled* to plain JavaScript first. The types are a safety net for **you**, the developer, and they vanish at runtime.

Types and interfaces — describing the shape of things

The smallest labels are the **primitive types**: `string`, `number`, `boolean`.

```
let displayName: string = "Aria";
let age: number = 24;
let isOnline: boolean = true;
```

But Viby is full of *objects* — a user, a hobby, a message. To describe an object's shape, we use a `type` or an `interface`. Think of them as a **form template**: every field is named, and every field has an expected kind of value.

```
type Hobby = {
  id: string;
  label: string;        // e.g. "Bouldering"
  emoji: string;        // e.g. "🧗"
};
```

`interface` does almost the same job and reads nearly identically:

```
interface Profile {
  id: string;
  username: string;
  bio: string;
  hobbies: Hobby[];     // an array of Hobby objects
}
```

Rule of thumb: use `interface` for object shapes (especially things others might extend), and `type` when you need unions, intersections, or to name something that isn't a plain object. In practice they're interchangeable for most day-to-day work — pick one and stay consistent.

Union types — "one of these, exactly"

Sometimes a value is one of a small, fixed set of options. A Viby connection request is `"pending"`, `"accepted"`, or `"declined"` — never `"banana"`. A **union** (the `|` symbol, read as "or") expresses exactly that:

```
type ConnectionStatus = "pending" | "accepted" | "declined";

let status: ConnectionStatus = "pending"; // ✅
// let status: ConnectionStatus = "maybe"; // ❌ red squiggle — not allowed
```

Unions are like a vending machine with three buttons. You can press A, B, or C. There is no button for "whatever I feel like." This single feature eliminates a whole category of typo bugs.

Generics — a type with a blank to fill in

Here's the analogy that makes generics click: a generic is a **shipping box that takes a label**. The box is always the same shape, but you write on the side what's inside — "📦 of shoes," "📦 of books." In TypeScript, that label is written in angle brackets `<T>`, where `T` is a placeholder you fill in later.

You already use generics without knowing it: `Hobby[]` is shorthand for `Array<Hobby>` — "an array **of** Hobby."

Generics let you write *one* reusable function or type that works for *any* content while still keeping it precisely typed:

```
// A box that wraps any successful API result alongside a count.
type ApiList<T> = {
  data: T[];
  count: number;
};

type HobbyList = ApiList<Hobby>;    // data is Hobby[]
type ProfileList = ApiList<Profile>; // data is Profile[]
```

One template, infinite specific shapes. That's the power.

Talking to APIs — the restaurant model

Your React app is the customer. The server (Supabase, in Viby) is the kitchen. Between them, a **waiter** carries your order out and the food back. That waiter is `fetch`.

The catch: the kitchen takes *time*. You don't freeze at the table staring at the wall — you chat, you wait, the food arrives later. In code, that "later" is a **Promise**: an IOU that says "I'll give you a value eventually — either the data, or an error."

`async/await` is the polite way to wait for a Promise. `await` means "pause here until the IOU is fulfilled, then continue with the real value."

```
async function getJoke() {
  const response = await fetch("https://api.example.com/joke");
  const data = await response.json(); // parse the JSON body
  return data;
}
```

JSON (JavaScript Object Notation) is the universal language servers and browsers use to exchange data — basically a JavaScript object written as text. `response.json()` turns that text back into a usable object. Crucially, `fetch` gives back `any` / `unknown` shaped data — TypeScript can't *know* what the server sent, so **we** must describe it. That's the bridge between this chapter's two halves.

In practice

Let's build a small, real **data layer** for Viby: typed models, a typed fetch helper, and a Supabase-backed query — the exact pattern you'll reuse all book long.

1. Define the domain models

Create `lib/types.ts`. This is the single source of truth for what a Viby user looks like.

```
// lib/types.ts
export interface Hobby {
  id: string;
  label: string;
  emoji: string;
}

export type ConnectionStatus = "pending" | "accepted" | "declined";

export interface Profile {
  id: string;
  username: string;
  bio: string;
  avatarUrl: string | null; // null when no photo uploaded yet
  hobbies: Hobby[];
}

// A reusable, generic envelope for any list response.
export interface ApiList<T> {
  data: T[];
  count: number;
}
```

2. A typed `fetch` helper using generics

`fetch` returns untyped data. We wrap it once so every call site gets full type safety for free. The generic `<T>` is the "label on the box."

```
// lib/api.ts
export async function fetchJson<T>(url: string): Promise<T> {
  const res = await fetch(url);

  if (!res.ok) {
    // Always handle the unhappy path — servers fail.
    throw new Error(`Request failed: ${res.status} ${res.statusText}`);
  }

  const json = (await res.json()) as T;
  return json;
}
```

Now calling it tells TypeScript exactly what comes back:

```
import { fetchJson } from "@/lib/api";
import type { ApiList, Profile } from "@/lib/types";

// `discover` is now fully typed as ApiList<Profile> — autocomplete everywhere.
const discover = await fetchJson<ApiList<Profile>>("/api/discover");
console.log(discover.count, discover.data[0].username);
```

3. The real Viby data layer with Supabase (Server Component)

In Next.js 16, components are **Server Components by default** — they can talk to the database directly, no `fetch` round-trip needed. We use `@supabase/ssr` with the `getAll` / `setAll` cookie handlers.

```
// lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies(); // async in Next.js 16

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options)
          );
        },
      },
    }
  );
}
```

Now a typed data-access function. Notice the return type `Promise<Profile[]>` — the contract is explicit:

```
// lib/queries.ts
import { createClient } from "@/lib/supabase/server";
import type { Profile } from "@/lib/types";

export async function getDiscoverProfiles(): Promise<Profile[]> {
  const supabase = await createClient();

  const { data, error } = await supabase
    .from("profiles")
    .select("id, username, bio, avatar_url, hobbies")
    .limit(20);

  if (error) {
    throw new Error(`Failed to load profiles: ${error.message}`);
  }

  // Map the DB's snake_case row to our camelCase Profile model.
  return (data ?? []).map((row) => ({
    id: row.id,
    username: row.username,
    bio: row.bio,
    avatarUrl: row.avatar_url,
    hobbies: row.hobbies ?? [],
  }));
}
```

4. Typing props and state in a component

Here's a `ProfileCard` that consumes our `Profile` type. Props get an `interface`; the Server Component awaits the data layer.

```tsx
// app/discover/page.tsx  (Server Component — no "use client")
import { getDiscoverProfiles } from "@/lib/queries";
import { ProfileCard } from "@/components/profile-card";

export default async function DiscoverPage() {
  const profiles = await getDiscoverProfiles();

  return (
    <main className="mx-auto max-w-md space-y-4 p-4">
      <h1 className="text-2xl font-bold text-brand">Discover your people</h1>
      {profiles.map((profile) => (
        <ProfileCard key={profile.id} profile={profile} />
      ))}
    </main>
  );
}
```

```tsx
// components/profile-card.tsx
"use client";

import { useState } from "react";
import { motion } from "framer-motion";
import type { Profile, ConnectionStatus } from "@/lib/types";

interface ProfileCardProps {
  profile: Profile;
}

export function ProfileCard({ profile }: ProfileCardProps) {
  // State is typed by the union — only valid statuses allowed.
  const [status, setStatus] = useState<ConnectionStatus>("pending");

  return (
    <motion.article
      whileHover={{ scale: 1.02 }}
      whileTap={{ scale: 0.98 }}
      className="rounded-2xl bg-white p-4 shadow-sm ring-1 ring-pink-100"
    >
      <h2 className="text-lg font-semibold">@{profile.username}</h2>
      <p className="mt-1 text-sm text-gray-600">{profile.bio}</p>

      <ul className="mt-3 flex flex-wrap gap-2">
        {profile.hobbies.map((hobby) => (
          <li
            key={hobby.id}
            className="rounded-full bg-pink-50 px-3 py-1 text-sm"
          >
            {hobby.emoji} {hobby.label}
          </li>
        ))}
      </ul>

      <button
        onClick={() => setStatus("accepted")}
        disabled={status === "accepted"}
        className="mt-4 w-full rounded-full bg-brand py-2 font-medium text-white disabled:opacity-50"
      >
        {status === "accepted" ? "Vibing ✓" : "Connect"}
      </button>
    </motion.article>
```

```
  );
}
```

The brand token comes from Tailwind v4's CSS-first config — no `tailwind.config.js`:

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5;
}
```

Architecture / diagram

How a typed request flows through Viby, from page render to database and back — with types guarding every boundary:

```
flowchart TD
    A["DiscoverPage<br/>(Server Component)"] -->|await| B["getDiscoverProfiles()<br/>:
Promise&amp;lt;Profile[]&amp;gt;"]
    B --> C["createClient()<br/>@supabase/ssr"]
    C -->|SQL select| D[("Supabase<br/>PostgreSQL")]
    D -->|snake_case rows| B
    B -->|"map → Profile[]"| A
    A -->|"props: { profile: Profile }"| E["ProfileCard<br/>(Client Component)"]
    E -->|"useState&amp;lt;ConnectionStatus&amp;gt;"| F["UI: Connect / Vibing ✓"]

    style A fill:#ffe4f1,stroke:#ff6fb5
    style E fill:#ffe4f1,stroke:#ff6fb5
    style D fill:#fff,stroke:#999
```

Notice the type annotations sitting on the *edges*: `Promise<Profile[]>` at the data layer, `{ profile: Profile }` at the prop boundary, `ConnectionStatus` at the state. Each is a checkpoint where TypeScript verifies the data is the right shape before it's allowed through.

Best practices & pitfalls

- **Do** define your domain models once in `lib/types.ts` and import them everywhere. One source of truth = no drift.
- **Do** type the return value of every data-layer function (`Promise<Profile[]>`). It documents intent and catches mistakes at the source.

- **Do** handle the error path of every `fetch`/Supabase call. Networks fail; `res.ok` and `error` exist for a reason.
- **Don't** reach for `any`. It silences the type-checker and re-introduces the unlabeled-jar problem. Prefer `unknown` and then narrow.
- **Don't** annotate what TypeScript already infers. `const name = "Aria"` is already `string` — adding `: string` is noise. Let inference do the work; annotate boundaries (props, returns, state) explicitly.
- **Don't** forget `key` on mapped lists, and don't put `"use client"` on a component that doesn't need interactivity — Server Components are the default for a reason.
- **Pitfall:** `await cookies()` and `await params` are *async* in Next.js 16. Forgetting the `await` is a common early bug.

Exercises

1. **Warm-up.** In `lib/types.ts`, add an `interface Message` with `id: string`, `senderId: string`, `body: string`, and `sentAt: string`. Export it.
2. **Unions.** Add a union type `Vibe = "chill" | "adventurous" | "creative" | "social"` and add an optional `vibe?: Vibe` field to `Profile`. Why does the `?` matter?
3. **Generics.** Write a generic function `first<T>(items: T[]): T | undefined` that returns the first element of any array, or `undefined` if it's empty. Test it on a `Profile[]` and a `Hobby[]`.
4. **Async + types.** Using `fetchJson<T>`, write `getProfile(id: string): Promise<Profile>` that calls `/api/profiles/${id}`. Add proper error handling.
5. **Stretch.** Make `ProfileCard` accept an optional `onConnect?: (id: string) => void` prop and call it inside the button's `onClick`. Type the prop precisely.

Mini-project: a typed "Hobby Match" badge

Build a small client component, `HobbyMatch`, that takes the current user's hobbies and another profile, computes how many hobbies they share, and renders an animated match badge.

```tsx
// components/hobby-match.tsx
"use client";

import { motion, AnimatePresence } from "framer-motion";
import type { Hobby, Profile } from "@/lib/types";

interface HobbyMatchProps {
  myHobbies: Hobby[];
  profile: Profile;
}

function countShared(mine: Hobby[], theirs: Hobby[]): number {
  const myIds = new Set(mine.map((h) => h.id));
  return theirs.filter((h) => myIds.has(h.id)).length;
}

export function HobbyMatch({ myHobbies, profile }: HobbyMatchProps) {
  const shared = countShared(myHobbies, profile.hobbies);

  return (
    <AnimatePresence>
      {shared > 0 && (
        <motion.span
          initial={{ scale: 0, opacity: 0 }}
          animate={{ scale: 1, opacity: 1 }}
          exit={{ scale: 0, opacity: 0 }}
          className="inline-block rounded-full bg-brand px-3 py-1 text-sm font-semibold text-white"
        >
          ✨ {shared} shared {shared === 1 ? "hobby" : "hobbies"}
        </motion.span>
      )}
    </AnimatePresence>
  );
}
```

Your task: wire `HobbyMatch` into `ProfileCard`, passing a hard-coded `myHobbies` array for now. Confirm that: - TypeScript refuses to compile if you pass a `string[]` instead of `Hobby[]`. - The badge animates in only when there's at least one shared hobby.

You've just shipped a typed, animated, data-driven feature — the same loop you'll repeat for every Viby screen.

Key takeaways

- TypeScript = JavaScript + labeled jars: errors surface in your editor, not in production.
- `type` / `interface` describe object shapes; **unions** (`|`) lock a value to a fixed set; **generics** (`<T>`) make one template work for any content.
- Annotate the **boundaries** — props, state, and function returns — and let inference handle the rest.
- `fetch` + `async/await` + Promises are how your app talks to a server; **you** must type what comes back, because the server can't.
- A typed data layer (`lib/types.ts` , `lib/queries.ts`) is the backbone of a maintainable app.

Up next — Chapter 6: styling Viby with Tailwind CSS v4, design tokens via `@theme` , and building our first polished, on-brand component library.

Chapter 6 — Next.js & the App Router

> You already know React. This chapter is where React grows up, moves to the city, and gets a real job shipping a production app.

What you'll learn

- Why we reach for a *framework* like Next.js instead of wiring React together by hand
- How the **App Router** turns folders and files into routes — no router config, ever
- The job of each special file: `page`, `layout`, `loading`, `error`, and `route`
- The difference between **Server Components** (the default) and **Client Components** (`"use client"`)
- How async `params`, **Server Actions**, and `proxy.ts` fit together
- How the real **Viby** project is organized so you can navigate it like a local

Prerequisites

- Chapter 3 (HTML/CSS fundamentals) and Chapter 4 (modern JavaScript & TypeScript)
- Chapter 5 (React 19: components, props, hooks, the client/server mental model)
- A working Node.js + pnpm setup from Chapter 2, and the Viby repo cloned locally

---

Theory

Why a framework at all?

Plain React gives you components and a way to render them into a `<div>`. That's it. The moment you ship a *real* product, you discover a long list of things React deliberately does **not** solve:

- Routing (which URL shows which screen?)
- Server rendering for fast first paint and SEO
- Data fetching that doesn't waterfall into a loading-spinner swamp
- Bundling, code-splitting, image optimization, caching
- A safe place to run secret code (API keys, database queries) that never reaches the browser

You *can* assemble all of this yourself with Vite + React Router + a custom Express server + a dozen config files. People did, for years. It's a part-time job to maintain.

A framework is a set of strong, well-tested decisions made for you. Think of plain React as a pile of premium car parts: a great engine, great seats, great wheels. **Next.js is the assembled car** — the parts are bolted together in the right order, the wiring works, and you can just *drive*. You give up a little freedom in exchange for a lot of speed and a lot fewer 2 a.m. bugs.

The big idea: the server is back

For a decade, the trend was "ship everything to the browser." React apps became huge JavaScript bundles that downloaded, then ran, then *finally* fetched data, then *finally* showed pixels.

The App Router flips this. By default, **your components run on the server**, render to HTML, and stream that HTML to the browser instantly. JavaScript is shipped only for the *interactive* parts. These server-by-default components are called **React Server Components (RSC)**.

Analogy: the old way mailed customers a flat-pack and an Allen key — "assemble the page yourself." The App Router ships the furniture **already built**, and only includes a screwdriver for the one drawer that needs to open and close.

Folders are routes

The App Router's central trick: **the file system *is* the router.** A folder inside `app/` becomes a URL segment. Drop a `page.tsx` in it and that URL becomes visitable. No route table, no `<Route path=...>` — the structure on disk *is* the structure of your site.

```
app/discover/page.tsx   →   /discover
app/profile/page.tsx    →   /profile
```

Server vs. Client Components

This is the single most important mental model in modern Next.js:

	Server Component (default)	Client Component (`"use client"`)
Runs where	On the server	In the browser (and once on the server for the initial HTML)
Can do	`await` data, read DB/secrets, big deps	`useState`, `useEffect`, `onClick`, browser APIs
Ships JS to browser	No	Yes
Use for	Pages, layouts, data fetching	Buttons, forms, animations, anything interactive

Rule of thumb: **stay on the server until you can't.** Add `"use client"` only at the leaf where you actually need interactivity (a like button, an animated card), and keep that island as small as possible.

In practice

Let's build the real Viby pieces. We'll assume Next.js 16, React 19, Tailwind v4, and Supabase via `@supabase/ssr`.

A page (Server Component, the default)

No directive needed — this runs on the server and can `await` data directly.

```tsx
// app/discover/page.tsx
import { createClient } from "@/lib/supabase/server";
import { VibeCard } from "@/components/vibe-card";

export default async function DiscoverPage() {
  const supabase = await createClient();
  const { data: people } = await supabase
    .from("profiles")
    .select("id, display_name, avatar_url, hobbies")
    .limit(20);

  return (
    <main className="mx-auto max-w-md px-4 py-6">
      <h1 className="text-2xl font-bold text-brand">Discover your people</h1>
      <div className="mt-4 grid gap-4">
        {people?.map((p) => (
          <VibeCard key={p.id} profile={p} />
        ))}
      </div>
    </main>
  );
}
```

Notice `text-brand` — that's our bubblegum pink, defined once in Tailwind v4's CSS theme (no `tailwind.config.js`):

```css
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5;
  --color-brand-foreground: #ffffff;
  --font-display: "Clash Display", ui-sans-serif, system-ui;
}
```

The Supabase server client (`@supabase/ssr`)

The modern pattern uses `createServerClient` with `getAll` / `setAll` cookie handlers — never the deprecated single-cookie API.

```
// lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (cookiesToSet) => {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            );
          } catch {
            // Called from a Server Component — safe to ignore; proxy refreshes.
          }
        },
      },
    }
  );
}
```

And the browser client for Client Components:

```
// lib/supabase/client.ts
import { createBrowserClient } from "@supabase/ssr";

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
}
```

A layout (shared shell)

Layouts wrap every page beneath them and **persist across navigation** — they don't re-render or unmount when you move between child routes. Perfect for nav bars.

```tsx
// app/layout.tsx — the root layout
import type { Metadata } from "next";
import "./globals.css";

export const metadata: Metadata = {
  title: "Viby — find your people",
  description: "Make friends through shared hobbies and lifestyle.",
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body className="bg-white text-neutral-900 antialiased">{children}</body>
    </html>
  );
}
```

```tsx
// app/(app)/layout.tsx — shared shell for the signed-in experience
import { BottomNav } from "@/components/bottom-nav";

export default function AppLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div className="min-h-dvh pb-16">
      {children}
      <BottomNav />
    </div>
  );
}
```

`(app)` is a **route group** — the parentheses mean "group these routes for layout/organization, but don't add `app` to the URL."

`loading.tsx` — instant skeletons

Drop a `loading.tsx` next to a page and Next.js automatically wraps it in a `<Suspense>` boundary. While the server component awaits data, the user sees this instead of a frozen screen.

```
// app/discover/loading.tsx
export default function Loading() {
  return (
    <div className="mx-auto max-w-md px-4 py-6">
      <div className="h-8 w-48 animate-pulse rounded-full bg-neutral-200" />
      <div className="mt-4 grid gap-4">
        {Array.from({ length: 4 }).map((_, i) => (
          <div key={i} className="h-32 animate-pulse rounded-2xl bg-neutral-100" />
        ))}
      </div>
    </div>
  );
}
```

`error.tsx` — graceful recovery

Error files **must** be Client Components (they receive a `reset` function and need state).

```tsx
// app/discover/error.tsx
"use client";

export default function Error({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  return (
    <div className="mx-auto max-w-md px-4 py-12 text-center">
      <p className="text-lg font-semibold">That vibe didn&apos;t load.</p>
      <p className="mt-1 text-sm text-neutral-500">{error.message}</p>
      <button
        onClick={reset}
        className="mt-4 rounded-full bg-brand px-5 py-2 font-medium text-brand-foreground"
      >
        Try again
      </button>
    </div>
  );
}
```

Dynamic routes & async `params`

In Next.js 16, `params` is a **Promise** — you `await` it (or unwrap it with React's `use()` in a Client Component).

```tsx
// app/profile/[username]/page.tsx
import { createClient } from "@/lib/supabase/server";
import { notFound } from "next/navigation";

export default async function ProfilePage({
  params,
}: {
  params: Promise<{ username: string }>;
}) {
  const { username } = await params;
  const supabase = await createClient();

  const { data: profile } = await supabase
    .from("profiles")
    .select("*")
    .eq("username", username)
    .single();

  if (!profile) notFound();

  return <h1 className="text-2xl font-bold">{profile.display_name}</h1>;
}
```

A Client Component with Framer Motion

Here's a Viby "like" button — interactive, animated, and a small island of client JS.

```tsx
// components/like-button.tsx
"use client";

import { useState } from "react";
import { motion } from "framer-motion";

export function LikeButton({ initialLiked = false }: { initialLiked?: boolean }) {
  const [liked, setLiked] = useState(initialLiked);

  return (
    <motion.button
      whileTap={{ scale: 0.85 }}
      onClick={() => setLiked((v) => !v)}
      aria-pressed={liked}
      className="rounded-full p-2"
    >
      <motion.span
        animate={{ scale: liked ? [1, 1.4, 1] : 1, color: liked ? "#ff6fb5" : "#a3a3a3" }}
        transition={{ duration: 0.25 }}
        className="text-2xl"
      >
        ♥
      </motion.span>
    </motion.button>
  );
}
```

A Server Action (`"use server"`)

Server Actions let a form mutate data **without** writing an API route. Define the function with `"use server"`, pass it straight to a `<form>`.

```tsx
// app/onboarding/page.tsx
import { createClient } from "@/lib/supabase/server";
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";

export default function OnboardingPage() {
  async function saveHobbies(formData: FormData) {
    "use server";
    const hobbies = formData.getAll("hobby").map(String);
    const supabase = await createClient();
    const { data: { user } } = await supabase.auth.getUser();
    if (!user) redirect("/login");

    await supabase.from("profiles").update({ hobbies }).eq("id", user.id);
    revalidatePath("/discover");
    redirect("/discover");
  }

  return (
    <form action={saveHobbies} className="mx-auto max-w-md space-y-4 p-6">
      {["Climbing", "Vinyl", "Ramen", "Film photography"].map((h) => (
        <label key={h} className="flex items-center gap-2">
          <input type="checkbox" name="hobby" value={h} /> {h}
        </label>
      ))}
      <button className="rounded-full bg-brand px-5 py-2 text-brand-foreground">
        Save & continue
      </button>
    </form>
  );
}
```

A Route Handler (`route.ts`)

When you need a raw HTTP endpoint (webhooks, third-party callbacks), use `route.ts` with named exports per method. Viby uses one for the Supabase auth callback:

```
// app/auth/callback/route.ts
import { NextResponse } from "next/server";
import { createClient } from "@/lib/supabase/server";

export async function GET(request: Request) {
  const { searchParams, origin } = new URL(request.url);
  const code = searchParams.get("code");

  if (code) {
    const supabase = await createClient();
    await supabase.auth.exchangeCodeForSession(code);
  }
  return NextResponse.redirect(`${origin}/discover`);
}
```

> A folder can contain **either** a `page.tsx` **or** a `route.ts` for the same segment — not both. Pages render UI; route handlers return data.

Middleware-style logic with `proxy.ts`

For cross-cutting logic that runs *before* a request resolves — refreshing the Supabase session, gating protected routes — Viby uses a root `proxy.ts` (the modern replacement for the old `middleware.ts`).

```ts
// proxy.ts
import { type NextRequest, NextResponse } from "next/server";
import { createServerClient } from "@supabase/ssr";

export async function proxy(request: NextRequest) {
  const response = NextResponse.next({ request });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => request.cookies.getAll(),
        setAll: (cookiesToSet) => {
          cookiesToSet.forEach(({ name, value, options }) =>
            response.cookies.set(name, value, options)
          );
        },
      },
    }
  );

  // Refresh the session so Server Components always see a fresh token.
  const { data: { user } } = await supabase.auth.getUser();

  if (!user && request.nextUrl.pathname.startsWith("/discover")) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  return response;
}

export const config = {
  matcher: ["/((?!_next/static|_next/image|favicon.ico).*)"],
};
```

Architecture / diagram

How a request to `/discover` flows through Viby:

```
flowchart TD
  A["Browser requests /discover"] --> B["proxy.ts: refresh Supabase session, gate route"]
  B --> C["app/layout.tsx (root shell)"]
  C --> D["app/(app)/layout.tsx (bottom nav)"]
  D --> E["app/discover/page.tsx (Server Component)"]
  E -- "awaiting data" --> F["loading.tsx skeleton streams first"]
  E -- "queries" --> G[(Supabase / Postgres)]
  E -- "throws" --> H["error.tsx (Client) with Try again"]
  E --> I["HTML streamed to browser"]
  I --> J["Client islands hydrate: LikeButton, motion cards"]
```

And the folder structure that produces it:

```
app/
├── layout.tsx            → root layout (html/body, metadata)
├── globals.css           → Tailwind v4 @theme tokens
├── page.tsx              → /  (marketing landing)
├── (app)/                → route group (no URL segment)
│   ├── layout.tsx        → signed-in shell + <BottomNav/>
│   ├── discover/
│   │   ├── page.tsx      → /discover
│   │   ├── loading.tsx   → skeleton
│   │   └── error.tsx     → error boundary (Client)
│   └── profile/
│       └── [username]/
│           └── page.tsx  → /profile/:username
├── onboarding/page.tsx   → Server Action form
└── auth/
    └── callback/route.ts → OAuth code exchange
proxy.ts                  → session refresh + route gating
lib/supabase/{server,client}.ts
components/{vibe-card,like-button,bottom-nav}.tsx
```

Best practices & pitfalls

Do

- Keep components on the **server by default**; push `"use client"` to the smallest leaf that needs it.
- Co-locate `loading.tsx` and `error.tsx` with the pages they protect — instant UX wins for free.
- `await params` (and `searchParams`) in Next.js 16 — they're Promises now.
- Use Server Actions for mutations from forms before reaching for a route handler.

- Put your secrets in env vars and only read them inside server code.

Don't

- Don't slap `"use client"` on a `layout.tsx` or `page.tsx` — it forces the whole subtree to ship as client JS.
- Don't import server-only code (the Supabase server client, `next/headers`) into a Client Component. It'll either error or leak.
- Don't create a `tailwind.config.js` — in v4, tokens live in CSS via `@theme`.
- Don't reach for `useEffect` + `fetch` to load page data; `await` it in a Server Component instead.
- Don't name the middleware file `middleware.ts` in this project — Viby standardizes on `proxy.ts`.
- Don't put both `page.tsx` and `route.ts` in one folder.

Exercises

1. **Warm-up.** Create `app/about/page.tsx` so visiting `/about` shows a short "Why Viby exists" blurb. Confirm it inherits the root layout.
2. **Skeletons.** Add a `loading.tsx` to your `/profile/[username]` route that mimics the profile layout with `animate-pulse` placeholders.
3. **Dynamic + not-found.** In the profile page, call `notFound()` when no profile matches, then add a sibling `not-found.tsx` with a friendly "This person ghosted us" message.
4. **Server Action.** Build a `app/feedback/page.tsx` form with a single textarea that writes to a `feedback` table via a `"use server"` action, then `revalidatePath`.
5. **Stretch.** Add a route group `(marketing)` with its own layout (no bottom nav) for `/`, `/about`, and `/pricing`, while keeping `(app)` for the signed-in screens.

Mini-project: the Viby Discover shell

Build a working `/discover` route end-to-end:

1. Create the `(app)` route group with a `layout.tsx` that renders `children` plus a fixed `<BottomNav>` (Discover · Profile).
2. Add `app/(app)/discover/page.tsx` as a Server Component that fetches up to 12 profiles from Supabase and renders them in a responsive grid of `VibeCard`s (avatar, name, top 3 hobbies as pink pills).
3. Add a `loading.tsx` with shimmering skeleton cards.
4. Add an `error.tsx` (Client) with a **Try again** button wired to `reset()`.
5. Make each `VibeCard` include a client `LikeButton` using Framer Motion's `whileTap` and a heart pop animation.
6. Wire `proxy.ts` to redirect signed-out users hitting `/discover` to `/login`.

When you're done you'll have touched **every** special file in the App Router — and it'll look and feel like a real product, not a tutorial.

Key takeaways

- A framework trades a little freedom for assembled, battle-tested defaults — Next.js is "React, with the car already built."
- In the App Router, **folders are routes** and **components are Server Components by default**.
- Master the five special files: `page` (UI), `layout` (persistent shell), `loading` (Suspense skeleton), `error` (Client boundary), `route` (HTTP endpoint).
- Reach for `"use client"` only at small interactive leaves; keep data fetching on the server with `await`.
- Use modern APIs: async `params`, Server Actions (`"use server"`), `@supabase/ssr` with `getAll` / `setAll`, Tailwind v4 `@theme`, and `proxy.ts` instead of `middleware.ts`.

Up next — Chapter 7: Styling with Tailwind CSS v4, where we turn these bare bones into the polished bubblegum-pink Viby aesthetic with design tokens, dark mode, and reusable UI primitives.

Chapter 7 — Routing, Layouts & Navigation

> A great app feels like one room you never leave — you move, things change, but the walls, the lights, and your friends stay put. That's routing done right.

By now Viby has a brand, components, and state. But a social app is a *place* — and places have rooms: a Discover feed, a profile, a chat, a settings drawer. This chapter is where Viby stops being a single page and becomes a navigable product.

What you'll learn

- How the App Router maps **folders to URLs**, and why that's a superpower
- **Nested layouts** that persist across navigation (no flicker, no re-mount)
- **Route groups** `(app)` to share a shell without polluting the URL
- **Dynamic routes** with **async** `params` in Next.js 16 (`await` in Server Components, `use()` in Client Components)
- Client navigation with `<Link>`, programmatic moves with `useRouter`, and `redirect()` on the server
- Building the real **Viby** `(app)` **shell + mobile tab bar** with an animated active indicator

Prerequisites

- **Chapter 3** — Next.js 16 App Router fundamentals (Server vs Client Components)
- **Chapter 4** — Tailwind CSS v4 `@theme` tokens and the Viby brand palette
- **Chapter 5** — shadcn-style UI primitives (`Button` , `cn` helper)
- **Chapter 6** — Framer Motion basics (`motion` , layout animations)

Theory

Folders are your router

In the App Router, **the file system *is* the route table**. There is no `routes.tsx`, no `<Route path=...>` config. You create a folder, drop a `page.tsx` inside, and that folder's path becomes a URL.

```
app/discover/page.tsx   →  /discover
app/profile/page.tsx    →  /profile
```

Think of it like a building. Each **folder is a room**, and `page.tsx` is the furniture *inside* that room — the unique content you came to see. A `layout.tsx` is the room's **architecture**: the walls, the doorway, the lighting that wrap whatever furniture is inside.

Layouts persist; pages swap

Here's the idea that makes the App Router click: **layouts don't re-render when you navigate between their children.** The root layout wraps everything. A nested layout wraps a subtree. When you move from `/discover` to `/profile`, if both live under the same layout, that layout *stays mounted* — only the `page` swaps out.

This is why a music app can keep playing while you browse, and why Viby's tab bar won't flicker or reset as you hop between tabs. The shell is stable; the contents flow through it.

```
RootLayout  (html, body, fonts, theme — never unmounts)
  └ (app) layout  (top bar + bottom tab bar — stays put)
        └ page  (Discover / Profile / Chat — this is what changes)
```

Route groups: structure without URL noise

Sometimes you want files **organized together and sharing a layout**, but you *don't* want that folder name to appear in the URL. Enter the **route group**: a folder wrapped in parentheses, like `(app)`.

`(app)` is invisible to the URL. `app/(app)/discover/page.tsx` is still just `/discover`. We use it to draw a boundary: "everything in here is the logged-in app shell — give it the tab bar." Meanwhile `app/(auth)/login/page.tsx` lives in a *different* group with a clean, chrome-free layout. Same URL space, two completely different shells.

Dynamic routes and the async-params shift

A profile page can't have one folder per user. Instead we use a **dynamic segment**: a folder named `[username]`. Anything matching that slot — `/u/maya`, `/u/devon` — renders the same `page.tsx`, and the matched value arrives as `params`.

The big Next.js 16 change: `params` **is now a Promise.** Routing values resolve asynchronously so the framework can stream and stay fast. In a Server Component you `await` it; in a Client Component you unwrap it with React's `use()` hook. Memorize that fork — it trips up everyone migrating from older Next.

Three ways to move

- `<Link>` — declarative, for things users click. Prefetches in the background; feels instant.
- `useRouter()` — imperative, for code-driven moves ("after this save, go to the new chat"). Client-only.
- `redirect()` — server-side, for guarding routes ("not logged in? bounce them to `/login`"). Throws, so put it before your JSX.

In practice

1. The route map for Viby

```
app/
├─ layout.tsx                # RootLayout: html/body, fonts, theme
├─ page.tsx                  # marketing landing  →  /
├─ (auth)/
│  ├─ layout.tsx             # centered, no tab bar
│  └─ login/page.tsx         # →  /login
└─ (app)/
   ├─ layout.tsx             # APP SHELL: top bar + bottom tab bar
   ├─ discover/page.tsx      # →  /discover
   ├─ chat/page.tsx          # →  /chat
   ├─ profile/page.tsx       # →  /profile   (your own)
   └─ u/[username]/page.tsx  # →  /u/maya    (anyone's)
```

2. The root layout

Defined once. Fonts and the global stylesheet (where our `@theme` tokens live, from Chapter 4) hang here.

```tsx
// app/layout.tsx
import type { Metadata } from "next";
import "./globals.css";

export const metadata: Metadata = {
  title: "Viby — find your people",
  description: "Make friends through shared hobbies and lifestyle.",
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body className="min-h-dvh bg-white text-neutral-900 antialiased">
        {children}
      </body>
    </html>
  );
}
```

3. The `(app)` shell layout

This is the heart of the chapter. One layout, shared by every logged-in screen, holding the persistent tab bar. Because it's a layout, it **never unmounts** as you switch tabs.

```
// app/(app)/layout.tsx
import { redirect } from "next/navigation";
import { createClient } from "@/lib/supabase/server";
import { TabBar } from "@/components/nav/tab-bar";

export default async function AppLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  // Server-side guard: no session → bounce to login.
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) redirect("/login");

  return (
    <div className="mx-auto flex min-h-dvh w-full max-w-md flex-col">
      <header className="sticky top-0 z-10 flex h-14 items-center justify-center border-b border-
neutral-100 bg-white/80 backdrop-blur">
        <span className="text-lg font-extrabold tracking-tight text-brand">
          Viby
        </span>
      </header>

      <main className="flex-1 overflow-y-auto pb-20">{children}</main>

      <TabBar />
    </div>
  );
}
```

Our Supabase server client (introduced in Chapter 6, using `@supabase/ssr`) looks like this — note the `getAll` / `setAll` cookie handlers, the modern API:

```ts
// lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            );
          } catch {
            // Called from a Server Component — safe to ignore;
            // proxy.ts refreshes the session cookie instead.
          }
        },
      },
    }
  );
}
```

4. The animated tab bar

A Client Component (it reads the current path and animates). `usePathname` tells us which tab is active; Framer Motion's shared `layoutId` slides the pink pill between tabs.

```tsx
// components/nav/tab-bar.tsx
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import { motion } from "framer-motion";
import { Compass, MessageCircle, User } from "lucide-react";
import { cn } from "@/lib/utils";

const TABS = [
  { href: "/discover", label: "Discover", Icon: Compass },
  { href: "/chat", label: "Chat", Icon: MessageCircle },
  { href: "/profile", label: "You", Icon: User },
] as const;

export function TabBar() {
  const pathname = usePathname();

  return (
    <nav className="fixed inset-x-0 bottom-0 z-20 mx-auto flex w-full max-w-md items-center justify-around border-t border-neutral-100 bg-white/90 px-2 py-2 backdrop-blur">
      {TABS.map(({ href, label, Icon }) => {
        const active = pathname === href || pathname.startsWith(href + "/");
        return (
          <Link
            key={href}
            href={href}
            className="relative flex flex-1 flex-col items-center gap-0.5 py-1.5 text-xs font-medium"
          >
            {active && (
              <motion.span
                layoutId="tab-pill"
                className="absolute inset-0 rounded-2xl bg-brand/10"
                transition={{ type: "spring", stiffness: 500, damping: 35 }}
              />
            )}
            <Icon
              className={cn(
                "relative size-6 transition-colors",
                active ? "text-brand" : "text-neutral-400"
              )}
            />
            <span
```

```
              className={cn(
                "relative",
                active ? "text-brand" : "text-neutral-500"
              )}
            >
              {label}
            </span>
          </Link>
        );
      })}
    </nav>
  );
}
```

A reminder from Chapter 4 — `text-brand` and `bg-brand/10` come from our token, defined in CSS, **no `tailwind.config.js`**:

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5;
}
```

5. Dynamic profile route — async `params` in a Server Component

```tsx
// app/(app)/u/[username]/page.tsx
import { notFound } from "next/navigation";
import { createClient } from "@/lib/supabase/server";

export default async function ProfilePage({
  params,
}: {
  params: Promise<{ username: string }>;
}) {
  const { username } = await params; // ← await: params is a Promise

  const supabase = await createClient();
  const { data: profile } = await supabase
    .from("profiles")
    .select("username, display_name, bio, vibes")
    .eq("username", username)
    .single();

  if (!profile) notFound();

  return (
    <section className="p-5">
      <h1 className="text-2xl font-extrabold">{profile.display_name}</h1>
      <p className="text-sm text-neutral-500">@{profile.username}</p>
      <p className="mt-3 text-neutral-800">{profile.bio}</p>
      <ul className="mt-4 flex flex-wrap gap-2">
        {profile.vibes?.map((v: string) => (
          <li
            key={v}
            className="rounded-full bg-brand/10 px-3 py-1 text-sm text-brand"
          >
            {v}
          </li>
        ))}
      </ul>
    </section>
  );
}
```

6. Same params, but in a Client Component — `use()`

When a route needs client interactivity and still wants `params`, unwrap the Promise with React's `use()`:

```tsx
// app/(app)/u/[username]/edit/page.tsx
"use client";

import { use, useState } from "react";

export default function EditVibes({
  params,
}: {
  params: Promise<{ username: string }>;
}) {
  const { username } = use(params); // ← use(): unwrap in Client Components
  const [draft, setDraft] = useState("");

  return (
    <form className="p-5">
      <h1 className="font-bold">Editing @{username}</h1>
      <input
        value={draft}
        onChange={(e) => setDraft(e.target.value)}
        className="mt-3 w-full rounded-xl border border-neutral-200 p-2"
        placeholder="Add a vibe…"
      />
    </form>
  );
}
```

7. Programmatic navigation with `useRouter`

Use this when *code* decides to move — e.g. after starting a chat with someone.

```tsx
// components/profile/start-chat-button.tsx
"use client";

import { useRouter } from "next/navigation";
import { useTransition } from "react";
import { Button } from "@/components/ui/button";
import { createChat } from "@/app/(app)/chat/actions";

export function StartChatButton({ targetId }: { targetId: string }) {
  const router = useRouter();
  const [pending, startTransition] = useTransition();

  function handleClick() {
    startTransition(async () => {
      const chatId = await createChat(targetId); // Server Action
      router.push(`/chat/${chatId}`); // jump into the new conversation
    });
  }

  return (
    <Button onClick={handleClick} disabled={pending}>
      {pending ? "Starting…" : "Say hi"}
    </Button>
  );
}
```

The matching Server Action — note the `"use server"` directive:

```
// app/(app)/chat/actions.ts
"use server";

import { createClient } from "@/lib/supabase/server";

export async function createChat(targetId: string): Promise<string> {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();

  const { data, error } = await supabase
    .from("chats")
    .insert({ a: user!.id, b: targetId })
    .select("id")
    .single();

  if (error) throw new Error(error.message);
  return data.id;
}
```

8. Session-refresh logic in `proxy.ts` (not `middleware.ts`)

In Next.js 16, the project-root `proxy.ts` replaces the old `middleware.ts` for request-time logic. We use it to keep the Supabase session cookie fresh on every navigation.

```
// proxy.ts
import { createServerClient } from "@supabase/ssr";
import { NextResponse, type NextRequest } from "next/server";

export async function proxy(request: NextRequest) {
  const response = NextResponse.next({ request });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value, options }) =>
            response.cookies.set(name, value, options)
          );
        },
      },
    }
  );

  await supabase.auth.getUser(); // refreshes the token if expired
  return response;
}

export const config = {
  matcher: ["/((?!_next/static|_next/image|favicon.ico).*)"],
};
```

Architecture / diagram

How a tap on the **Profile** tab flows through the persistent shell:

```
flowchart TD
    A["User taps Profile tab"] --> B["&lt;Link href='/profile'&gt; intercepts<br/>(client-side, no full reload)"]
    B --> C{"Already inside<br/>(app) layout?"}
    C -- "Yes" --> D["(app) layout STAYS mounted<br/>tab bar + header persist"]
    C -- "No (cold load)" --> E["RootLayout → (app) layout render<br/>redirect() guard runs"]
    D --> F["Only page.tsx swaps:<br/>discover/page → profile/page"]
    E --> F
    F --> G["usePathname() updates<br/>Framer Motion slides pink pill"]
```

```
URL space            File system                        Shell applied
─────────            ───────────                        ─────────────
/login        ↔      app/(auth)/login/page.tsx          centered, no tabs
/discover     ↔      app/(app)/discover/page.tsx        app shell + tabs
/u/maya       ↔      app/(app)/u/[username]/page.tsx    app shell + tabs
                            └── (app) and (auth) vanish from the URL
```

Best practices & pitfalls

Do

- Put **shared, stable chrome** (tab bar, header) in a layout — never re-render it per page.
- Guard protected subtrees with `redirect()` in the `(app)/layout.tsx`, *and* keep `proxy.ts` refreshing the session.
- Type dynamic params as `Promise<{ … }>` and `await` (server) or `use()` (client).
- Use `<Link>` for anything clickable — you get prefetching for free.
- Call `notFound()` when a dynamic record doesn't exist; it renders your `not-found.tsx`.

Don't

- Don't reach for `useRouter().push()` where a `<Link>` works — you lose prefetch and accessibility.
- Don't destructure `params` synchronously (`{ params: { username } }`) — that's the old API and now throws.
- Don't create a `middleware.ts` in Next.js 16 — use `proxy.ts`.
- Don't add a `tailwind.config.js` for tokens — define them in `@theme` (Chapter 4).
- Don't call `redirect()` inside a `try/catch` that swallows errors — it works *by* throwing.
- Don't forget `redirect()` returns `never`; place it before any JSX so TypeScript narrows correctly.

Exercises

1. **Settings tab (easy).** Add `app/(app)/settings/page.tsx` and a fourth tab to `TabBar` with a `Settings` icon. Confirm the active pill animates to it.
2. **Loading UI (easy–medium).** Add `app/(app)/u/[username]/loading.tsx` showing a skeleton card while the profile query resolves. Throttle your network and watch it appear.
3. **Not-found (medium).** Create `app/(app)/u/[username]/not-found.tsx` with a friendly "No Viber by that name" message and a `<Link>` back to `/discover`.
4. **Search params (medium).** On `/discover`, read `searchParams` (also a Promise now) to support `/discover?vibe=hiking`, filtering the feed server-side.
5. **Nested layout (hard).** Give `/chat` its own nested `layout.tsx` with a conversations sidebar that persists while `/chat/[chatId]` pages swap inside it.

Mini-project — The Viby app shell

Ship the navigable skeleton of Viby end to end.

Goal: a logged-in user can tap between Discover, Chat, and You; the shell never flickers; tapping a profile card opens `/u/[username]`; visiting any `(app)` route while logged out bounces to `/login`.

Build steps

1. Scaffold the `(app)` and `(auth)` route groups exactly as in the route map above.
2. Implement `app/(app)/layout.tsx` with the Supabase guard + `redirect("/login")` and mount `<TabBar />`.
3. Drop in the animated `TabBar` with the `layoutId="tab-pill"` indicator.
4. Build three stub pages (`discover`, `chat`, `profile`) — even just an `<h1>` each — to prove the shell persists across navigation.
5. Add the dynamic `u/[username]/page.tsx` reading async `params` and querying `profiles`.
6. On `discover`, render a few profile cards as `<Link href={` /u/${p.username} `}>`.
7. Wire `proxy.ts` for session refresh.

Done when: switching tabs animates the pink pill with zero layout flash, the header/tab bar stay mounted (drop a `console.log` in `TabBar` — it should log once, not per navigation), and a logged-out visit to `/discover` lands on `/login`.

Key takeaways

- **Folders are routes; `page.tsx` is content; `layout.tsx` is persistent chrome.**
- **Layouts don't re-mount on navigation** — that's what keeps Viby's shell stable.
- **Route groups `(app)` / `(auth)`** share layouts without touching the URL.
- **`params` is a Promise in Next.js 16** — `await` on the server, `use()` on the client.
- **`<Link>` to click, `useRouter` to move in code, `redirect()` to guard on the server.**
- Use **`proxy.ts`**, not `middleware.ts`, for request-time session work.

Up next — Chapter 8: Data Fetching & Server Components, where those stub pages start pulling real Vibers from Supabase, with caching, streaming, and loading states that feel instant.

Chapter 8 — Server vs Client Components & Data

> The single biggest "aha" of the App Router: most of your UI never has to ship to the browser at all.

What you'll learn

- Why **Server Components are the default** in Next.js 16, and what `"use client"` actually does.
- How to decide — on a per-component basis — whether something runs on the server or in the browser.
- How to **fetch data directly inside Server Components** (no `useEffect`, no loading spinner gymnastics).
- How to **pass data across the server → client boundary**, and the *serialization* rules that govern it.
- How to compose the two worlds: server shells with small interactive client "islands."
- The real performance and security wins — and the traps that bite beginners.

Prerequisites

- Chapter 4 (TypeScript fundamentals) — we lean on typed props.
- Chapter 6 (Next.js App Router, routing & layouts) — you should know what a `page.tsx` and `layout.tsx` are.
- Chapter 7 (Tailwind v4 & our UI primitives) — we'll style with `@theme` tokens.
- A running Viby project with Supabase wired via `@supabase/ssr` (set up at the end of Chapter 6).

Theory

Imagine Viby's **Discover** page: a grid of people near you who share your hobbies. Some of that page is *static structure* — headings, a profile card's name, avatar, and bio, the shared-interest tags. Other parts are *alive* — a "Wave 👋" button that reacts to your tap, a follow toggle, a hover animation.

For years, React shipped **all** of that to the browser as JavaScript. The browser downloaded your components, re-ran them, and re-built the DOM the server had already described. We paid the "JavaScript tax" even on parts that never change after they render.

React Server Components (RSC) split the work. Think of it like a **restaurant kitchen vs. the dining table**:

- The **kitchen (server)** does the heavy prep: it talks to the database, reads secrets, assembles the dish. Customers never see the kitchen, and the kitchen's tools (knives, the safe with the recipes) never leave it.
- The **table (client/browser)** is where interaction happens: you cut your food, react, ask for more. It needs cutlery (JavaScript), but only for the things you actually touch.

In Next.js 16's App Router, **every component is a Server Component by default**. It runs once, on the server, during the request. Its output isn't HTML strings and it isn't "a bundle" — it's a serialized description

of the UI (the *RSC payload*) that React streams to the browser and stitches into the page. That component's *code* never ships to the browser.

When you need real interactivity — state, effects, event handlers, browser APIs — you opt a component into the client with the `"use client"` directive at the top of the file. That marks the boundary: this component, and everything it imports, becomes part of the browser bundle.

The mental model that matters

`"use client"` is not "this code runs only in the browser." It's a **boundary marker**. A client component still pre-renders on the server for the initial HTML (that's SSR), then *hydrates* in the browser so it can become interactive. The directive's real job is: *"from here down, ship the JavaScript and wire up interactivity."*

Two rules follow from this, and they trip up everyone at first:

1. **Server Components can import and render Client Components.** (Kitchen plates the dish, then hands it to the table.)
2. **Client Components cannot import Server Components** — but they *can* receive them as `children` / props. (The table can't reach into the kitchen, but the kitchen can place something on the table.)

The serialization boundary

When a Server Component passes props *into* a Client Component, those props cross the network. They get **serialized** (turned into the RSC payload), shipped, and deserialized in the browser. So props must be serializable: strings, numbers, booleans, `null`, arrays, plain objects, `Date`, `Map`, `Set`, and — importantly — **other Server Components passed as `children`**.

What you **cannot** pass across that boundary: functions (except Server Actions, which we'll meet), class instances, Symbols, database clients, anything with methods. Pass the *data*, not the *machinery*.

In practice

Let's build a slice of Viby's Discover page the right way.

A Server Component that fetches data

No `"use client"`, no `useEffect`, no spinner. We `await` the data right in the component. This file's code never reaches the browser — which is exactly why it's safe to use the server-side Supabase client (with cookies) here.

```tsx
// app/discover/page.tsx  — Server Component (default)
import { createClient } from "@/lib/supabase/server";
import { ProfileCard } from "@/components/profile-card";

export default async function DiscoverPage() {
  const supabase = await createClient();

  const { data: profiles } = await supabase
    .from("profiles")
    .select("id, display_name, bio, avatar_url, interests")
    .limit(24);

  return (
    <main className="mx-auto max-w-5xl px-4 py-8">
      <h1 className="text-2xl font-bold text-ink">Find your people</h1>
      <p className="mt-1 text-muted">People near you who share your vibe.</p>

      <ul className="mt-6 grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3">
        {profiles?.map((p) => (
          <li key={p.id}>
            <ProfileCard profile={p} />
          </li>
        ))}
      </ul>
    </main>
  );
}
```

Here's the server Supabase client it uses — note the `getAll` / `setAll` cookie handlers required by `@supabase/ssr`:

```
// lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies(); // async in Next.js 16

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            );
          } catch {
            // Called from a Server Component render — safe to ignore;
            // cookie writes happen in Server Actions / proxy.ts.
          }
        },
      },
    }
  );
}
```

A mostly-server card with a small client island

`ProfileCard` is a Server Component — it just renders data. The only interactive bit, the **Wave** button, is a separate Client Component. We keep the JavaScript footprint tiny by isolating interactivity.

```tsx
// components/profile-card.tsx  — Server Component
import { WaveButton } from "@/components/wave-button";

type Profile = {
  id: string;
  display_name: string;
  bio: string | null;
  avatar_url: string | null;
  interests: string[];
};

export function ProfileCard({ profile }: { profile: Profile }) {
  return (
    <article className="rounded-2xl border border-black/5 bg-white p-5 shadow-sm">
      <div className="flex items-center gap-3">
        <img
          src={profile.avatar_url ?? "/avatar-fallback.png"}
          alt={profile.display_name}
          className="h-12 w-12 rounded-full object-cover"
        />
        <h2 className="font-semibold text-ink">{profile.display_name}</h2>
      </div>

      {profile.bio && <p className="mt-3 text-sm text-muted">{profile.bio}</p>}

      <ul className="mt-3 flex flex-wrap gap-2">
        {profile.interests.map((tag) => (
          <li
            key={tag}
            className="rounded-full bg-brand/10 px-2.5 py-1 text-xs font-medium text-brand"
          >
            {tag}
          </li>
        ))}
      </ul>

      {/* Pass plain, serializable data across the boundary */}
      <WaveButton profileId={profile.id} name={profile.display_name} />
    </article>
  );
}
```

The Client Component (the island)

`"use client"` sits at the very top. This component owns state, an event handler, and a Framer Motion gesture — none of which can exist in a Server Component.

```
// components/wave-button.tsx
"use client";

import { useState } from "react";
import { motion } from "framer-motion";
import { wave } from "@/app/discover/actions";

export function WaveButton({
  profileId,
  name,
}: {
  profileId: string;
  name: string;
}) {
  const [waved, setWaved] = useState(false);

  async function handleWave() {
    setWaved(true); // optimistic UI
    await wave(profileId); // calls a Server Action
  }

  return (
    <motion.button
      whileTap={{ scale: 0.92 }}
      onClick={handleWave}
      disabled={waved}
      aria-label={`Wave at ${name}`}
      className="mt-4 w-full rounded-xl bg-brand py-2.5 font-semibold text-white disabled:opacity-60"
    >
      {waved ? "Waved 👋" : "Wave 👋"}
    </motion.button>
  );
}
```

The bridge back: a Server Action

How does the client *send data back*? Not by importing the database client (that would leak secrets into the bundle). Instead it calls a **Server Action** — a function marked `"use server"` that runs on the server but can

be called like a normal async function from the client. This is the *one* kind of "function" allowed to cross the boundary.

```
// app/discover/actions.ts
"use server";

import { createClient } from "@/lib/supabase/server";
import { revalidatePath } from "next/cache";

export async function wave(targetProfileId: string) {
  const supabase = await createClient();

  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) throw new Error("Not signed in");

  await supabase.from("waves").insert({
    from_user: user.id,
    to_user: targetProfileId,
  });

  revalidatePath("/discover");
}
```

Passing a Server Component as `children` (composition pattern)

When you *do* need a client wrapper (say, an animated container) around server-rendered content, don't drag the content into the client bundle. Pass it as `children`:

```tsx
// components/reveal.tsx
"use client";
import { motion } from "framer-motion";

export function Reveal({ children }: { children: React.ReactNode }) {
  return (
    <motion.div
      initial={{ opacity: 0, y: 12 }}
      whileInView={{ opacity: 1, y: 0 }}
      viewport={{ once: true }}
    >
      {children}
    </motion.div>
  );
}
```

```tsx
// usage in a Server Component — ProfileCard stays on the server
<Reveal>
  <ProfileCard profile={p} />
</Reveal>
```

`Reveal` is a client island, but `ProfileCard` it wraps is still rendered on the server and slotted in as already-finished UI. Best of both worlds.

Architecture / diagram

```
flowchart TD
  subgraph Server["💻 Server (Next.js + Supabase)"]
    A["DiscoverPage (Server Component)\nawait supabase.from('profiles')"]
    B["ProfileCard (Server Component)\nrenders name, bio, tags"]
    SA["wave() Server Action\n'use server'"]
    DB[("PostgreSQL\nRLS-protected")]
  end

  subgraph Wire["⇅ Network boundary (RSC payload)"]
    P["Serialized props:\nprofileId, name (strings)"]
  end

  subgraph Client["📱 Browser"]
    W["WaveButton (Client Component)\n'use client' — useState, onClick"]
  end

  DB --> A
  A --> B
  B -- "serializable props" --> P
  P --> W
  W -- "calls" --> SA
  SA --> DB
```

The dashed lesson: **data flows down through serialization; actions flow back up through the `"use server"` channel.** Database access never crosses into the browser box.

Best practices & pitfalls

Do

- **Keep `"use client"` as low in the tree as possible.** Push interactivity to leaf "islands" (a button, a toggle), not whole pages.
- **Fetch in Server Components** with `await`. It's simpler, faster, and keeps queries off the network-tab radar.
- **Pass only serializable data** across the boundary — plain objects and primitives.
- **Use Server Actions** (`"use server"`) for mutations instead of building API routes for every form.
- **Put secrets and the service-role Supabase client server-side only.** Anything not prefixed `NEXT_PUBLIC_` stays out of the bundle.

Don't

- **Don't slap `"use client"` at the top of a layout or page "just in case."** It turns the whole subtree into client code and kills the RSC win.

- **Don't `await` data inside a Client Component** — you can't make a client component `async`. Fetch on the server and pass it down, or use a data library / `use()` on a passed-in promise.
- **Don't pass functions, class instances, or a Supabase client as props** to a client component. They aren't serializable. (Server Actions are the only exception.)
- **Don't reach for `useEffect` + `fetch` for initial data.** That's the old SPA pattern; it reintroduces spinners and waterfalls the App Router eliminates.
- **Don't read `params` / `searchParams` synchronously.** In Next.js 16 they're async: `const { id } = await params;` (or `use(params)` in a client component).

Exercises

1. **Spot the boundary.** Open Viby's profile page. List which parts *must* be client (need state/events/browser APIs) and which can stay server. Justify each.
2. **Shrink the bundle.** Find a component in your project that has `"use client"` but contains a large chunk of non-interactive markup. Split the interactive bit into its own island and keep the rest on the server.
3. **Server fetch.** Build `app/profile/[id]/page.tsx` as an async Server Component that reads `id` via `await params` and fetches that profile from Supabase.
4. **Serialization error on purpose.** Try passing a function as a prop from a Server Component to a Client Component. Read the error Next.js gives you, then fix it by converting the function into a Server Action.
5. **Composition challenge (hard).** Wrap a server-rendered list in an `AnimatePresence` client wrapper so items animate on mount — *without* moving the list items into the client bundle.

Mini-project: "Vibe Match" spotlight card

Build a single-card feature for the Discover page that shows one suggested person and lets the user act on it.

Requirements

1. `app/discover/spotlight/page.tsx` — a Server Component that fetches one random profile from Supabase (`profiles` table) and renders it. No client directive in this file.
2. `<MatchActions />` — a Client Component island with two buttons: **"Wave 👋"** and **"Skip"**. Use `useState` to track which action was taken and a Framer Motion `whileTap` gesture for feedback.
3. `waveAction` / `skipAction` — Server Actions (`"use server"`) that record the choice in Supabase and call `revalidatePath("/discover/spotlight")` so the next render shows a fresh face.
4. **Boundary discipline** — the page passes only `{ id, display_name, avatar_url }` (plain serializable props) into `<MatchActions />`. No Supabase client crosses the line.
5. **Polish** — style with your `@theme` tokens (`bg-brand` , `text-ink`), and wrap the card in a `Reveal` client component so it fades in on mount while staying server-rendered inside.

Stretch: add an optimistic "It's a vibe!" toast that appears instantly on Wave, before the Server Action resolves.

You'll know it works when: the page loads with zero loading spinner, the network tab shows no client-side profile fetch, and the buttons still feel snappy.

Key takeaways

- **Server Components are the default**; `"use client"` is an opt-in *boundary*, not a "run in browser only" switch.
- Server Components **fetch data directly** with `await` and keep secrets and DB access off the client.
- Props from server → client **must be serializable**; pass *data*, not *machinery*.
- **Server Components can render Client Components**; clients receive servers only as `children` /props.
- **Server Actions** (`"use server"`) are the sanctioned channel for sending work back to the server.
- Keep client islands **small and low** in the tree to ship less JavaScript and load faster.

Up next — Chapter 9: *Data Fetching, Caching & Streaming* — `loading.tsx` , Suspense boundaries, and how Next.js 16 caches and revalidates your Viby feed so it stays fast *and* fresh.

Chapter 9 — Tailwind CSS & Mobile-First Responsive Design

> Your users meet Viby on a 390px phone screen first — so that's where *we* start too.

What you'll learn

- Why **utility-first CSS** beats hand-rolled stylesheets for product teams shipping fast.
- How **Tailwind CSS v4** works with zero config — design tokens live in your CSS via `@theme`, not a `tailwind.config.js`.
- The **mobile-first** mental model and how Tailwind's responsive breakpoints encode it.
- The **device-frame approach**: designing inside a phone shell so every screen feels native on mobile.
- Building real **Viby layouts** — a discovery feed, profile cards, and a sticky mobile tab bar — that scale gracefully up to desktop.

Prerequisites

- **Chapter 6** (Next.js 16 App Router) — you should be comfortable with Server vs. Client Components and the `app/` directory.
- **Chapter 7** (React 19 fundamentals) — props, composition, `use client`.
- **Chapter 8** (TypeScript for components) — typed props and `tsx` files.
- A running Viby Next.js project with Tailwind v4 installed (we set this up at the end of Chapter 6).

Theory

Why utility-first at all?

Imagine you're decorating a room. The traditional CSS approach is like hiring a carpenter to **build a custom piece of furniture for every wall** — beautiful, but slow, and the moment you want to rearrange, you're back to the workshop. Over months, your `styles.css` grows into a graveyard of `.card-wrapper-v2-final` classes nobody dares delete.

Utility-first is like having a wall of perfectly labelled drawers: `flex`, `gap-4`, `rounded-2xl`, `bg-brand`. You compose furniture *in place*, in your markup, from tiny single-purpose pieces. The genius isn't that you type more classes — it's that you **stop inventing names** and **stop context-switching** between files. The styling lives where the structure lives.

Three concrete wins for a product team:

1. **No naming fatigue.** `mt-4` always means the same thing. You never debate whether it's `.card__title` or `.cardTitle`.

2. **Dead-code-free by construction.** Delete a component and its styles vanish with it — there's no orphaned CSS.
3. **Constrained by design.** You pick from a *scale* (`gap-2` , `gap-4` , `gap-6`), not arbitrary `gap: 13px` . That constraint is what makes a UI look consistent and "designed."

The honest trade-off: your HTML gets longer. We tame that with **components** (extract a `<Button>` once, reuse everywhere) — not by abandoning utilities.

Tailwind v4: the CSS *is* the config

If you've seen older Tailwind tutorials, burn the `tailwind.config.js` from your memory. **Tailwind v4 is CSS-first.** You import the engine and declare your design tokens in a special `@theme` block right in your stylesheet:

```
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5;
}
```

That single line does something magical: it **generates utilities**. Because you named a token `--color-brand` , Tailwind now hands you `bg-brand` , `text-brand` , `border-brand` , `ring-brand` , and friends — automatically. Tokens are the *source*; utilities are the *output*. This is the whole model.

Analogy: `@theme` is the **brand style guide** for the whole app. Designers hand you "our pink is `#FF6FB5` , our base radius is 1rem." You write that down once in CSS, and every utility across the codebase speaks that vocabulary.

Mobile-first: small screen is the default, not an afterthought

Here's the rule that trips up everyone coming from desktop-era CSS:

> **An unprefixed utility applies to *every* screen size. A prefixed utility (`md:` , `lg:`) applies *from that breakpoint up*.**

So `p-4 md:p-8` means: "4 units of padding everywhere, bumping to 8 from the medium breakpoint onward." You **never** write `sm:` to target *only* phones — phones get the unprefixed value. You layer enhancements *upward*.

Why design up instead of down? A phone is the **constrained** environment: one column, thumb-reachable, limited width. If your design survives there, widening it is easy — you have *more* room, not less. Start wide and you'll be forever amputating layouts to fit small screens. Viby is a social app; the overwhelming majority of sessions happen on phones, so mobile-first isn't just tidy — it's where the product actually lives.

Tailwind v4's default breakpoints (defined as `--breakpoint-*` tokens you can override):

Prefix	Min-width	Typical device
(none)	0	phones
`sm:`	40rem	large phones / small tablets
`md:`	48rem	tablets
`lg:`	64rem	laptops
`xl:`	80rem	desktops

In practice

1. The Viby theme

This is the heart of your design system. Create `app/globals.css`:

```
@import "tailwindcss";

@theme {
  /* Brand */
  --color-brand: #ff6fb5;
  --color-brand-soft: #ffe3f1;
  --color-ink: #1b1320;
  --color-muted: #7a7080;

  /* Radii & shadows tuned for a soft, premium feel */
  --radius-card: 1.25rem;
  --shadow-pop: 0 10px 30px -12px rgb(255 111 181 / 0.45);

  /* A friendly display font (loaded via next/font in layout) */
  --font-display: var(--font-grotesk), system-ui, sans-serif;
}
```

Now `bg-brand`, `text-ink`, `rounded-card`, `shadow-pop`, and `font-display` all exist as utilities — typed nowhere, available everywhere.

2. A reusable Button (extract once, reuse forever)

This is how we keep markup short *without* leaving utility-land. It's a Server Component — no interactivity needed, so no `"use client"`.

```
// components/ui/button.tsx
import { type ComponentProps } from "react";

type ButtonProps = ComponentProps<"button"> & {
  variant?: "primary" | "ghost";
};

export function Button({ variant = "primary", className = "", ...props }: ButtonProps) {
  const base =
    "inline-flex h-11 items-center justify-center rounded-full px-6 " +
    "text-sm font-semibold transition active:scale-95 disabled:opacity-50";
  const variants = {
    primary: "bg-brand text-white shadow-pop hover:brightness-105",
    ghost: "bg-brand-soft text-brand hover:bg-brand/15",
  };
  return <button className={`${base} ${variants[variant]} ${className}`} {...props} />;
}
```

> Note the `h-11` (44px) min target — that's the accessible minimum tap size for thumbs. Mobile-first thinking shows up in *sizing*, not just layout.

3. The discovery feed — mobile-first grid

The Viby feed is **one column on phones**, two on tablets, three on laptops. Watch how the breakpoint prefixes layer upward:

```tsx
// app/discover/page.tsx  (Server Component by default)
import { PersonCard } from "@/components/person-card";
import { getDiscoveryProfiles } from "@/lib/profiles";

export default async function DiscoverPage() {
  const profiles = await getDiscoveryProfiles();

  return (
    <main className="mx-auto w-full max-w-5xl px-4 pb-24 pt-6">
      <h1 className="font-display text-2xl font-bold text-ink md:text-3xl">
        Find your people
      </h1>
      <p className="mt-1 text-sm text-muted">
        People near you who love what you love.
      </p>

      <section className="mt-6 grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3">
        {profiles.map((p) => (
          <PersonCard key={p.id} profile={p} />
        ))}
      </section>
    </main>
  );
}
```

`grid-cols-1` is the phone default; `sm:grid-cols-2` and `lg:grid-cols-3` are the upgrades. `max-w-5xl` plus `mx-auto` keeps the feed from sprawling on huge monitors. `pb-24` reserves room for the mobile tab bar we add below.

4. A profile card with a gesture (Client Component)

Cards have a tap animation and an interest-tag row that scrolls horizontally on small screens. Interactivity + Framer Motion means this one needs `"use client"`.

```tsx
// components/person-card.tsx
"use client";

import { motion } from "framer-motion";
import Image from "next/image";

type Profile = {
  id: string;
  name: string;
  age: number;
  city: string;
  avatarUrl: string;
  interests: string[];
};

export function PersonCard({ profile }: { profile: Profile }) {
  return (
    <motion.article
      whileTap={{ scale: 0.98 }}
      className="overflow-hidden rounded-card bg-white shadow-pop ring-1 ring-black/5"
    >
      <div className="relative aspect-4/5 w-full">
        <Image
          src={profile.avatarUrl}
          alt={profile.name}
          fill
          sizes="(max-width: 640px) 100vw, 33vw"
          className="object-cover"
        />
        <div className="absolute inset-x-0 bottom-0 bg-gradient-to-t from-ink/70 to-transparent p-4">
          <p className="text-base font-semibold text-white">
            {profile.name}, {profile.age}
          </p>
          <p className="text-xs text-white/80">{profile.city}</p>
        </div>
      </div>

      {/* Tags scroll sideways on phones, wrap-free, no scrollbar */}
      <ul className="flex gap-2 overflow-x-auto p-3 [scrollbar-width:none]">
        {profile.interests.map((tag) => (
          <li
            key={tag}
            className="shrink-0 rounded-full bg-brand-soft px-3 py-1 text-xs font-medium text-brand"
```

```
            >
              {tag}
            </li>
          ))}
        </ul>
      </motion.article>
    );
}
```

Two mobile-first details worth internalizing: `aspect-4/5` keeps the image a consistent portrait shape at any width (no layout shift), and the `overflow-x-auto` tag row is a classic phone pattern — horizontal scroll where vertical space is precious.

5. The sticky mobile tab bar (and why it hides on desktop)

Phones get a thumb-friendly bottom bar; laptops don't need it. `fixed` + `bottom-0` on mobile, `lg:hidden` to remove it on large screens.

```tsx
// components/mobile-tab-bar.tsx
import Link from "next/link";
import { Compass, MessageCircle, User } from "lucide-react";

const tabs = [
  { href: "/discover", label: "Discover", icon: Compass },
  { href: "/messages", label: "Messages", icon: MessageCircle },
  { href: "/profile", label: "Profile", icon: User },
];

export function MobileTabBar() {
  return (
    <nav
      className="fixed inset-x-0 bottom-0 z-50 flex justify-around border-t border-black/5
                 bg-white/90 pb-[env(safe-area-inset-bottom)] backdrop-blur lg:hidden"
    >
      {tabs.map(({ href, label, icon: Icon }) => (
        <Link
          key={href}
          href={href}
          className="flex flex-1 flex-col items-center gap-1 py-3 text-muted
                     transition active:text-brand"
        >
          <Icon className="size-5" />
          <span className="text-[11px] font-medium">{label}</span>
        </Link>
      ))}
    </nav>
  );
}
```

`pb-[env(safe-area-inset-bottom)]` is a real-world must: it pushes the bar above the iPhone home indicator. This is an *arbitrary value* — square-bracket syntax for the rare case the design scale doesn't cover.

6. The device-frame approach

When building mobile-first, it helps to *develop inside a phone shell* so you're never tempted to design for the full browser width. Wrap your dev preview in a frame:

```
// components/device-frame.tsx
export function DeviceFrame({ children }: { children: React.ReactNode }) {
  return (
    <div className="grid min-h-dvh place-items-center bg-brand-soft p-4 lg:p-8">
      <div
        className="relative h-[844px] w-[390px] overflow-hidden rounded-[2.5rem]
                   border-8 border-ink bg-white shadow-2xl"
      >
        <div className="h-full overflow-y-auto">{children}</div>
      </div>
    </div>
  );
}
```

`390x844` is the iPhone 14 viewport. Building inside this frame keeps your hands honest — but remember it's a **development aid**, not your production layout. The real app uses fluid widths and the breakpoints above; the frame just disciplines your eye.

Architecture / diagram

How a token becomes a responsive, rendered pixel:

```
flowchart TD
    A["@theme in globals.css<br/>--color-brand: #ff6fb5"] --> B["Tailwind v4 engine<br/>generates utilities"]
    B --> C["bg-brand · text-brand · ring-brand · ..."]
    C --> D["JSX className<br/>grid-cols-1 sm:grid-cols-2 lg:grid-cols-3"]
    D --> E{"Viewport width?"}
    E -->|"< 40rem (phone)"| F["1 column · tab bar visible"]
    E -->|"40–64rem (tablet)"| G["2 columns"]
    E -->|"≥ 64rem (laptop)"| H["3 columns · tab bar hidden"]
```

The flow is one-directional: **tokens → utilities → markup → breakpoint-resolved layout**. Change the token, and every screen updates.

Best practices & pitfalls

- **Do** start every layout from the unprefixed (phone) state, then add `md:` / `lg:` enhancements. **Don't** write desktop CSS and patch downward.
- **Do** define design decisions as `@theme` tokens (`--color-brand`, `--radius-card`). **Don't** scatter hex codes and pixel values across components.

- **Don't** reach for arbitrary values like `mt-[13px]` reflexively — they're an escape hatch, not the norm. Stick to the scale; it's what keeps the UI coherent.
- **Do** extract repeated utility clusters into components (`<Button>`, `<PersonCard>`). **Don't** copy-paste 12-class strings — and don't prematurely abstract `@apply` either; component composition is the v4 idiom.
- **Do** size tap targets at `h-11` / `size-11` (44px) minimum and honor `env(safe-area-inset-*)` on phones.
- **Don't** forget `"use client"` is only for interactivity (motion, hooks, events). Keep layout components on the server — Framer Motion and event handlers force client mode, plain markup does not.
- **Do** test at real breakpoints (resize the browser, use device toolbar). The device frame is a guide, not a guarantee.
- **Don't** misuse `sm:` thinking it means "small/phones." It means "small breakpoint **and up**."

Exercises

1. **Token swap.** Add a `--color-brand-deep: #d94f93` token and use it for `:hover` on the primary `Button`. Confirm `hover:bg-brand-deep` works with zero config changes.
2. **Two-up bio.** On the profile page, make the avatar and bio stack vertically on phones but sit side-by-side from `md:` up using `flex-col md:flex-row`.
3. **Responsive type.** Make the `Find your people` heading `text-2xl` on phones, `text-3xl` on `md`, and `text-4xl` on `lg`. Eyeball it in the device toolbar.
4. **Hide & show.** Add a left sidebar nav that is `hidden lg:block`, complementing the mobile tab bar that is `lg:hidden`. Only one should ever be visible.
5. **Safe-area polish.** Add `pt-[env(safe-area-inset-top)]` to a sticky top header and verify it doesn't collide with a notch in the device frame.

Mini-project — the Viby Discover screen

Build a complete, responsive **Discover** screen that ties the chapter together.

Requirements

1. Add the full Viby `@theme` block (Section 1) to `app/globals.css`.
2. Build `PersonCard` (Section 4) and render a feed of at least 6 mock profiles in the responsive grid (Section 3): 1 column on phones, 2 on tablets, 3 on laptops.
3. Add the `MobileTabBar` (Section 5), fixed to the bottom and hidden from `lg:` up. Reserve space with `pb-24` on the feed so cards aren't covered.
4. During development, wrap the page in `DeviceFrame` to verify the phone experience first, then remove it and resize the real browser to confirm it scales to 2 and 3 columns.

Stretch: add a horizontally-scrolling "interest filter" chip row above the feed (sticky on scroll with `sticky top-0`), reusing the tag styling from `PersonCard`.

You'll know you're done when the screen feels like a native app at 390px **and** a polished web app at 1280px — from the *same* markup.

Key takeaways

- **Utility-first** trades long markup for zero naming, zero dead CSS, and built-in design constraints.
- **Tailwind v4 is CSS-first**: declare tokens in `@theme`, and utilities are generated automatically — no `tailwind.config.js`.
- **Mobile-first** means unprefixed = phone; `md:` / `lg:` layer enhancements *upward*. `sm:` means "and up," never "phones only."
- The **device-frame approach** keeps your eye honest during development without dictating production layout.
- Real Viby layouts — responsive grids, gesture cards, safe-area tab bars — all flow from the same tokens.

Up next — Chapter 10: shadcn-style UI Primitives & Framer Motion, where we turn these raw utilities into a polished, animated component library Viby can ship on.

Chapter 10 — Reusable Components, shadcn Patterns & Framer Motion

> One great `<Button />` beats a hundred copy-pasted divs — this is the chapter where Viby stops looking like a tutorial and starts feeling like a real product.

What you'll learn

- How to design a **component API** that's a joy to use: small, predictable, composable.
- How to build **variants** with `cva` (class-variance-authority) and merge classes safely with `cn()`.
- How to compose **primitives** — `Button`, `Chip`, `Avatar` — that the rest of Viby is built on.
- How to add **premium motion** with Framer Motion: `motion` components, `AnimatePresence` for mount/unmount, gestures (`whileHover` / `whileTap`), and `layout` animations.
- When a component should be a **Server Component** and when it earns a `"use client"`.

Prerequisites

- Chapter 6 (Tailwind CSS v4 and the `@theme` tokens that define our bubblegum-pink brand).
- Chapter 7 (TypeScript fundamentals — generics, unions, and `React.ComponentProps`).
- Chapter 9 (the Next.js 16 App Router: Server Components by default, `"use client"` only when needed).

You should have the Viby app running locally with the brand tokens already defined in `globals.css`.

Theory

Why components at all?

Imagine you run a bakery. You *could* hand-pipe every cupcake from scratch, but you'd never ship a hundred a day with a consistent look. Instead you build a **piping nozzle** — one tool, used everywhere, that guarantees the same swirl every time. A reusable component is that nozzle. You design the interface once (the shape of the swirl), and every cupcake in the shop inherits it for free.

A good component has a **narrow, honest API**: the props you can pass are exactly the things that *should* change, and nothing else. If your `Button` accepts twelve boolean flags, callers will combine them in nonsensical ways. If it accepts one `variant` prop with a closed set of values, the design system stays coherent.

The two axes of a component API

Every primitive varies along two axes:

1. **Appearance** — discrete, designer-chosen looks: `primary`, `secondary`, `ghost`, sizes `sm` / `md` / `lg`. These are *enumerable*. You want autocomplete and type errors when someone passes `variant="purple"`.
2. **Behaviour & passthrough** — everything a native element already does: `onClick`, `disabled`, `aria-label`, `type`. You should *not* re-invent these; you forward them.

`cva` handles axis 1. `React.ComponentProps<"button">` handles axis 2.

Why `cva` + `cn()`?

Tailwind is utility classes as strings. Strings are easy to mess up: duplicate classes, conflicting ones (`px-2 px-4` — who wins?), and sprawling ternaries. Two small libraries fix this:

- `cva` (class-variance-authority) turns "variant → class string" into a typed, declarative map. Think of it as a *recipe card*: base ingredients plus a table of variations.
- `cn()` is our tiny helper that runs `clsx` (conditional joining) through `tailwind-merge` (last-conflicting-class-wins). So `cn("px-2", "px-4")` correctly yields `"px-4"`. It's the *referee* that resolves class fights.

Composition over configuration

The deepest idea in this chapter: **build small primitives, then compose them**. A profile card isn't a giant `<ProfileCard configEverything />`. It's an `Avatar`, a few `Chip`s, and a `Button`, arranged together. Small pieces compose into infinite layouts; one giant configurable component composes into nothing.

Why animate?

Motion is *feedback*. When a user taps "Follow" on Viby and the button springs slightly, their brain registers "it worked" before any network call returns. Framer Motion gives us this with almost no code — and crucially, it animates **enter and exit** (plain CSS can't animate an element that's being removed from the DOM). That's what makes a Gen-Z social app feel alive instead of clinical.

In practice

The `cn()` helper

This lives once, in `src/lib/utils.ts`, and is imported everywhere.

```
// src/lib/utils.ts
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

Install the deps:

```
npm install class-variance-authority clsx tailwind-merge framer-motion
```

Brand tokens (recap from Chapter 6)

Our variants reference these Tailwind v4 theme tokens. No `tailwind.config.js` — it's all CSS:

```
/* src/app/globals.css */
@import "tailwindcss";

@theme {
  --color-viby: #ff6fb5;          /* bubblegum pink */
  --color-viby-ink: #1b1320;      /* near-black text */
  --color-viby-soft: #fff0f7;     /* pink tint surface */
  --radius-pill: 9999px;
}
```

These generate utilities like `bg-viby`, `text-viby-ink`, and `rounded-pill` automatically.

The `Button` primitive

A Server Component-safe button (no hooks, no state) using `cva`:

```tsx
// src/components/ui/button.tsx
import { cva, type VariantProps } from "class-variance-authority";
import { cn } from "@/lib/utils";

const buttonVariants = cva(
  // base: applies to every button
  "inline-flex items-center justify-center gap-2 font-medium whitespace-nowrap " +
    "transition-colors focus-visible:outline-none focus-visible:ring-2 " +
    "focus-visible:ring-viby/50 disabled:pointer-events-none disabled:opacity-50",
  {
    variants: {
      variant: {
        primary: "bg-viby text-white hover:bg-viby/90",
        secondary: "bg-viby-soft text-viby-ink hover:bg-viby-soft/70",
        ghost: "bg-transparent text-viby-ink hover:bg-viby-soft",
        outline: "border border-viby text-viby hover:bg-viby-soft",
      },
      size: {
        sm: "h-8 px-3 text-sm rounded-pill",
        md: "h-10 px-5 text-sm rounded-pill",
        lg: "h-12 px-7 text-base rounded-pill",
      },
    },
    defaultVariants: { variant: "primary", size: "md" },
  }
);

export interface ButtonProps
  extends React.ComponentProps<"button">,
    VariantProps<typeof buttonVariants> {}

export function Button({ className, variant, size, ...props }: ButtonProps) {
  return (
    <button
      className={cn(buttonVariants({ variant, size }), className)}
      {...props}
    />
  );
}

export { buttonVariants };
```

Why this is good:

- `VariantProps<typeof buttonVariants>` *derives* the `variant` / `size` types from the recipe — they can never drift out of sync.
- `React.ComponentProps<"button">` forwards `onClick`, `disabled`, `type`, `aria-*` for free.
- `className` is merged **last** via `cn()`, so a caller can override anything without specificity wars.

Usage anywhere, including Server Components:

```
<Button variant="primary" size="lg">Find my people</Button>
<Button variant="ghost" size="sm" aria-label="Skip">Skip</Button>
```

The `Chip` primitive (Viby hobby tags)

Chips are how Viby surfaces shared interests — "Bouldering", "Lo-fi", "Matcha".

```
// src/components/ui/chip.tsx
import { cva, type VariantProps } from "class-variance-authority";
import { cn } from "@/lib/utils";

const chipVariants = cva(
  "inline-flex items-center gap-1 rounded-pill px-3 py-1 text-xs font-medium transition-colors",
  {
    variants: {
      tone: {
        neutral: "bg-viby-soft text-viby-ink",
        active: "bg-viby text-white",
        muted: "bg-zinc-100 text-zinc-500",
      },
    },
    defaultVariants: { tone: "neutral" },
  }
);

export interface ChipProps
  extends React.ComponentProps<"span">,
    VariantProps<typeof chipVariants> {}

export function Chip({ className, tone, ...props }: ChipProps) {
  return <span className={cn(chipVariants({ tone }), className)} {...props} />;
}
```

The `Avatar` primitive (composing primitives)

`Avatar` shows a user photo, falling back to initials. It uses `next/image` and stays a Server Component:

```tsx
// src/components/ui/avatar.tsx
import Image from "next/image";
import { cn } from "@/lib/utils";

interface AvatarProps {
  src?: string | null;
  name: string;
  size?: number; // px
  className?: string;
}

export function Avatar({ src, name, size = 40, className }: AvatarProps) {
  const initials = name
    .split(" ")
    .map((p) => p[0])
    .slice(0, 2)
    .join("")
    .toUpperCase();

  return (
    <span
      className={cn(
        "relative inline-flex shrink-0 items-center justify-center overflow-hidden rounded-pill bg-viby-soft text-viby-ink font-semibold",
        className
      )}
      style={{ width: size, height: size, fontSize: size * 0.4 }}
    >
      {src ? (
        <Image src={src} alt={name} fill sizes={`${size}px`} className="object-cover" />
      ) : (
        initials
      )}
    </span>
  );
}
```

Adding motion: the `FollowButton`

Now the fun part. A button that springs on tap needs client-side JS, so it gets `"use client"`. We compose our existing `Button` styles with a `motion.button`. Notice we reuse `buttonVariants` rather than duplicating classes:

```tsx
// src/components/follow-button.tsx
"use client";

import { motion } from "framer-motion";
import { useState } from "react";
import { cn } from "@/lib/utils";
import { buttonVariants } from "@/components/ui/button";

export function FollowButton({ initial = false }: { initial?: boolean }) {
  const [following, setFollowing] = useState(initial);

  return (
    <motion.button
      whileTap={{ scale: 0.94 }}
      whileHover={{ scale: 1.03 }}
      transition={{ type: "spring", stiffness: 400, damping: 25 }}
      onClick={() => setFollowing((f) => !f)}
      className={cn(
        buttonVariants({ variant: following ? "secondary" : "primary", size: "md" })
      )}
    >
      {following ? "Following" : "Follow"}
    </motion.button>
  );
}
```

`AnimatePresence`: animating things that leave

When a Viby user dismisses a "person you might vibe with" card, we want it to fly out, not vanish. Only `AnimatePresence` can animate exit:

```tsx
// src/components/discovery-deck.tsx
"use client";

import { AnimatePresence, motion } from "framer-motion";
import { useState } from "react";
import { Avatar } from "@/components/ui/avatar";
import { Chip } from "@/components/ui/chip";

interface Person {
  id: string;
  name: string;
  avatar?: string | null;
  hobbies: string[];
}

export function DiscoveryDeck({ people }: { people: Person[] }) {
  const [deck, setDeck] = useState(people);
  const dismiss = (id: string) => setDeck((d) => d.filter((p) => p.id !== id));

  return (
    <ul className="flex flex-col gap-3">
      <AnimatePresence initial={false}>
        {deck.map((person) => (
          <motion.li
            key={person.id}
            layout
            initial={{ opacity: 0, y: 12 }}
            animate={{ opacity: 1, y: 0 }}
            exit={{ opacity: 0, x: 80 }}
            transition={{ type: "spring", stiffness: 300, damping: 30 }}
            className="flex items-center gap-3 rounded-2xl bg-white p-4 shadow-sm"
          >
            <Avatar src={person.avatar} name={person.name} size={48} />
            <div className="min-w-0 flex-1">
              <p className="font-semibold text-viby-ink">{person.name}</p>
              <div className="mt-1 flex flex-wrap gap-1">
                {person.hobbies.map((h) => (
                  <Chip key={h}>{h}</Chip>
                ))}
              </div>
            </div>
            <button
              onClick={() => dismiss(person.id)}
```

```
              aria-label={`Dismiss ${person.name}`}
              className="text-zinc-400 hover:text-viby"
            >
              ×
            </button>
          </motion.li>
        ))}
      </AnimatePresence>
    </ul>
  );
}
```

The magic word is `layout`: when one card exits, the remaining cards **smoothly slide up** to fill the gap, because Framer Motion measures their positions before and after and animates the difference (the FLIP technique). You get this for one keyword.

Wiring it into a Server Component page

The page itself is a Server Component that fetches via Supabase and passes plain data to the client deck:

```tsx
// src/app/discover/page.tsx
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";
import { DiscoveryDeck } from "@/components/discovery-deck";

export default async function DiscoverPage() {
  const cookieStore = await cookies();
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) =>
          toSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options)
          ),
      },
    }
  );

  const { data } = await supabase
    .from("profiles")
    .select("id, name, avatar, hobbies")
    .limit(10);

  return (
    <main className="mx-auto max-w-md p-4">
      <h1 className="mb-4 text-2xl font-bold text-viby-ink">People to vibe with</h1>
      <DiscoveryDeck people={data ?? []} />
    </main>
  );
}
```

Architecture / diagram

```
flowchart TD
    subgraph tokens["Design tokens (globals.css @theme)"]
      T["--color-viby, --radius-pill ..."]
    end
    subgraph recipes["Variant recipes (cva)"]
      BV["buttonVariants"]
      CV["chipVariants"]
    end
    subgraph primitives["Primitives (Server-safe)"]
      B["Button"]
      C["Chip"]
      A["Avatar"]
    end
    subgraph motion["Motion layer ('use client')"]
      FB["FollowButton (motion + buttonVariants)"]
      DD["DiscoveryDeck (AnimatePresence + layout)"]
    end
    subgraph page["Server Component page"]
      P["/discover  -> Supabase fetch"]
    end

    T --> BV --> B
    T --> CV --> C
    B --> FB
    C --> DD
    A --> DD
    P --> DD
```

The flow is one-directional: **tokens → recipes → primitives → motion-wrapped composites → pages**. Each layer only knows about the one below it, which is exactly why the system scales.

Best practices & pitfalls

Do:

- Keep primitives **Server Components** (no `"use client"`) unless they need state, effects, or motion. `Button`, `Chip`, `Avatar` ship zero JS to the browser.
- Always put `className` **last** in `cn()` so callers can override.
- Reuse `buttonVariants` inside motion components instead of re-writing classes — single source of truth.
- Give every `AnimatePresence` child a **stable, unique** `key` (use the DB id, never the array index).
- Use **spring** transitions for interactions (taps, follows) and **tween/ease** for entrances — springs feel physical, which suits gestures.

- Respect `prefers-reduced-motion`: wrap motion with Framer's `<MotionConfig reducedMotion="user">` at the root.

Don't:

- Don't pass arbitrary color props (`color="#abc"`). Enumerate variants instead — it keeps the brand consistent.
- Don't concatenate Tailwind classes with template strings when they might conflict; that's what `cn()` / `tailwind-merge` is for.
- Don't animate `width` / `height` / `top` / `left` directly when you can use `transform` (`x`, `y`, `scale`) — transforms are GPU-cheap and won't trigger layout thrash. Reach for `layout` for size/position changes.
- Don't forget: `AnimatePresence` only animates **direct children** that mount/unmount. Conditionally render the child, not a wrapper around it.
- Don't add `"use client"` to a whole page just to animate one button — push the boundary down to the smallest component.

Exercises

1. **Add an `icon` size to `Button`** — a square `40×40` variant for icon-only buttons, plus an `aria-label` reminder in a JSDoc comment. (Easy)
2. **Build a `Badge` primitive** with `cva` that has `tone` (`new`, `pro`, `beta`) variants, reusing `rounded-pill`. (Easy)
3. **Make `Chip` interactive** — add a `selected` boolean and a tap animation so tapping toggles between `neutral` and `active`. Hint: this one needs `"use client"`. (Medium)
4. **Add a "liked" heart** to each `DiscoveryDeck` card that scales up with a spring `whileTap` and changes color, persisting in local state. (Medium)
5. **Stagger the deck entrance** — use a parent `motion.ul` with `variants` and `staggerChildren` so cards cascade in on first load. (Hard)

Mini-project: The Viby "Vibe Tags" picker

Build a self-contained onboarding step where a new user picks their interests.

Requirements:

- A grid of `Chip`s for ~12 hobbies ("Bouldering", "Lo-fi", "Matcha", "Anime", "Run club", ...).
- Tapping a chip toggles it between `neutral` and `active`, with a `whileTap={{ scale: 0.9 }}` spring (so the `Chip` gets wrapped in a `motion.button`).
- A live count: "3 selected" updating as you tap.
- A `Button variant="primary" size="lg"` labeled "Start vibing", **disabled** until at least 3 are selected.
- When enabled, the button should gently pulse once (animate `scale` from 1 → 1.05 → 1) using `animate` to draw the eye.

Stretch: persist the selection to Supabase with a Server Action (`"use server"`) that writes `hobbies` to the user's `profiles` row, then `redirect("/discover")` .

This wires together everything from the chapter: tokens, `cva` variants, `cn()` , primitive composition, and gesture + state-driven motion.

Key takeaways

- A great component has a **narrow API**: enumerated variants for looks, native prop passthrough for behaviour.
- `cva` maps variants to classes declaratively; `cn()` (clsx + tailwind-merge) resolves conflicts and lets callers override.
- **Compose** small primitives (`Button` , `Chip` , `Avatar`) instead of building giant configurable components.
- Keep primitives **Server Components**; add `"use client"` only at the smallest unit that needs interactivity or motion.
- **Framer Motion** gives you gestures (`whileHover` / `whileTap`), enter/exit animation (`AnimatePresence`), and effortless reflow (`layout`) — the difference between a demo and a premium product.

Up next — Chapter 11: Forms, Validation & Server Actions, where we'll capture real user input with type-safe schemas and write it straight to Supabase.

Chapter 11 — Color, Theming & the Viby Design System

> Color isn't decoration — it's the fastest message your app sends before a single word is read.

What you'll learn

- The working color theory you actually need: hue/saturation/lightness, contrast, and 60-30-10.
- How to turn a brand color (#FF6FB5) into a **token-based palette** that scales across light and dark.
- How to define those tokens natively in **Tailwind CSS v4** `@theme` — no `tailwind.config.js`.
- Building Viby's signature **gradients, glow, and glass** surfaces, accessibly.
- Shipping a **design-token sheet**: a single living page that documents every token in the app.

Prerequisites

- Chapter 4 (Tailwind v4 setup, the `@theme` block, `globals.css`).
- Chapter 7 (shadcn-style UI primitives and the `cn()` helper).
- Chapter 9 (Framer Motion basics — `motion` components and gestures).
- Comfort with Next.js 16 Server vs. Client Components (Chapter 6).

Theory

Color is a system, not a swatch

Beginners pick colors one at a time: "this pink looks nice here." Professionals define a **system** once and let every screen draw from it. Think of it like a kitchen. An amateur grabs whatever spice is closest; a chef builds a *mise en place* — everything measured, labeled, in reach. Our mise en place is a set of **design tokens**: named values (`--color-primary`, `--radius-lg`) that the whole codebase references instead of raw hex.

The payoff: change `--color-primary` in one place and every button, badge, and focus ring updates. Tokens are the difference between "we have a pink" and "we have a design system."

HSL: the only color model that thinks like a designer

Hex (`#FF6FB5`) is how computers store color; **HSL** is how humans reason about it:

- **Hue** (0–360°): the color's identity. Viby's bubblegum pink sits around **332°**.
- **Saturation** (0–100%): how vivid vs. gray.
- **Lightness** (0–100%): how close to white or black.

HSL makes palettes *derivable*. Keep the hue, walk the lightness up and down, and you get a tonal scale (50 → 950) that feels like one family because it *is* one family. That's exactly how Tailwind's built-in scales work, and

how we'll build Viby's.

`#FF6FB5` ≈ `hsl(332, 100%, 72%)`. From that single seed we generate the whole ramp.

Contrast is non-negotiable

A pretty color that nobody can read is a bug. **WCAG** sets minimum contrast ratios: **4.5:1** for normal text, **3:1** for large text and UI components (borders, icons, focus states). Bubblegum pink on white is only ~2:1 — gorgeous for *fills*, illegal for *body text*. So the rule for Viby: pink is a **surface and accent** color; readable text rides on near-black ink (`--color-foreground`) or pure white on pink fills. We'll bake an accessible "on-primary" token in so nobody has to guess.

60-30-10: the proportion that makes brands feel calm

A balanced UI is roughly **60% dominant** (white/neutral background), **30% secondary** (cards, muted surfaces), **10% accent** (the pink — buttons, links, highlights). The mistake new designers make is flooding the screen with brand color; restraint is what makes the 10% *pop*. Viby looks premium precisely because pink is a spark, not a wall.

Light and dark from one source of truth

We never hand-author two palettes. We define **semantic tokens** (`--color-background` , `--color-foreground` , `--color-primary`) and swap their *values* under a `.dark` class. Components only ever speak in semantics — `bg-background` , `text-foreground` — so they're theme-agnostic by construction.

In practice

1. The token sheet, defined in CSS

Tailwind v4 has no JS config. Tokens live in `globals.css` inside `@theme` , and every token there automatically becomes a utility class (`--color-primary` → `bg-primary` , `text-primary` , `border-primary`).

```
/* app/globals.css */
@import "tailwindcss";

/* 1) Raw brand ramp — the immutable source of truth.
   One hue (332°), walked across lightness. */
@theme {
  --color-pink-50:  hsl(330 100% 97%);
  --color-pink-100: hsl(331 100% 93%);
  --color-pink-200: hsl(332 100% 88%);
  --color-pink-300: hsl(332 100% 82%);
  --color-pink-400: hsl(332 100% 77%);
  --color-pink-500: hsl(332 100% 72%); /* #FF6FB5 — the brand */
  --color-pink-600: hsl(332 78%  60%);
  --color-pink-700: hsl(332 65%  48%);
  --color-pink-800: hsl(332 60%  38%);
  --color-pink-900: hsl(332 55%  28%);
  --color-pink-950: hsl(332 60%  16%);

  --color-ink-50:  hsl(320 16% 97%);
  --color-ink-500: hsl(320 8%  46%);
  --color-ink-900: hsl(322 24% 12%);

  /* 2) Semantic tokens — what components actually reference.
     These point AT the ramp, so a rebrand is a one-line change. */
  --color-background: white;
  --color-foreground: hsl(322 24% 12%);
  --color-primary: var(--color-pink-500);
  --color-primary-foreground: white;   /* text/icons ON pink — passes contrast */
  --color-muted: hsl(330 40% 97%);
  --color-muted-foreground: hsl(320 8% 46%);
  --color-card: white;
  --color-border: hsl(330 30% 91%);
  --color-ring: var(--color-pink-500);

  /* 3) Shape + motion tokens belong in the system too. */
  --radius-lg: 1rem;
  --radius-xl: 1.5rem;
  --shadow-glow: 0 0 0 1px hsl(332 100% 72% / 0.25),
                 0 8px 30px -8px hsl(332 100% 72% / 0.45);
}
```

2. Dark mode: same names, new values

We override the semantic layer only — the raw ramp never changes. Use a `@custom-variant` so `dark:` utilities work with a class toggle.

```
/* still in globals.css */
@custom-variant dark (&:where(.dark, .dark *));

.dark {
  --color-background: hsl(322 24% 8%);
  --color-foreground: hsl(330 40% 96%);
  --color-primary: var(--color-pink-400); /* lift lightness so it glows on dark */
  --color-primary-foreground: hsl(322 30% 10%);
  --color-muted: hsl(322 20% 14%);
  --color-muted-foreground: hsl(320 12% 65%);
  --color-card: hsl(322 22% 11%);
  --color-border: hsl(322 18% 20%);
}
```

Because `--color-primary` is referenced by the `bg-primary` utility, the *same component* renders pink-500 in light mode and pink-400 in dark — zero component changes.

3. A token-driven Button (shadcn-style)

```tsx
// components/ui/button.tsx
import { cva, type VariantProps } from "class-variance-authority";
import { cn } from "@/lib/utils";

const button = cva(
  "inline-flex items-center justify-center rounded-lg font-medium " +
    "transition-colors focus-visible:outline-none focus-visible:ring-2 " +
    "focus-visible:ring-ring focus-visible:ring-offset-2 disabled:opacity-50",
  {
    variants: {
      variant: {
        primary:
          "bg-primary text-primary-foreground hover:bg-pink-600 shadow-glow",
        ghost: "text-foreground hover:bg-muted",
        outline: "border border-border text-foreground hover:bg-muted",
      },
      size: { sm: "h-9 px-3 text-sm", md: "h-11 px-5", lg: "h-13 px-7 text-lg" },
    },
    defaultVariants: { variant: "primary", size: "md" },
  },
);

export function Button({
  className,
  variant,
  size,
  ...props
}: React.ComponentProps<"button"> & VariantProps<typeof button>) {
  return <button className={cn(button({ variant, size }), className)} {...props} />;
}
```

Notice: no hex anywhere. The button speaks only tokens, so it's automatically correct in both themes.

4. Gradients, glow & glass — the Viby signature

```tsx
// components/viby-card.tsx  (Server Component — no interactivity needed)
export function VibyCard({ children }: { children: React.ReactNode }) {
  return (
    <div
      className={[
        "rounded-xl p-6",
        // Glass: translucent surface + blur + hairline border
        "bg-card/70 backdrop-blur-xl border border-border/60",
        // Glow: our shadow token
        "shadow-glow",
      ].join(" ")}
    >
      {children}
    </div>
  );
}
```

```css
/* A reusable brand gradient, defined as a token-backed utility */
@layer utilities {
  .bg-viby-gradient {
    background-image: linear-gradient(
      135deg,
      var(--color-pink-400) 0%,
      var(--color-pink-600) 100%
    );
  }
  .text-viby-gradient {
    background: linear-gradient(135deg, var(--color-pink-500), var(--color-pink-700));
    -webkit-background-clip: text;
    background-clip: text;
    color: transparent;
  }
}
```

```tsx
// A hero headline using the gradient text utility
<h1 className="text-4xl font-bold text-viby-gradient">
  Find your people.
</h1>
```

5. A theme toggle that respects the system

The toggle flips a class on `<html>` and persists choice. Reading from `localStorage` and matching system preference means we need a Client Component, but we keep it tiny.

```
// components/theme-toggle.tsx
"use client";
import { useEffect, useState } from "react";
import { motion } from "framer-motion";
import { Button } from "@/components/ui/button";

export function ThemeToggle() {
  const [dark, setDark] = useState(false);

  useEffect(() => {
    const stored = localStorage.getItem("viby-theme");
    const prefersDark = window.matchMedia("(prefers-color-scheme: dark)").matches;
    const isDark = stored ? stored === "dark" : prefersDark;
    setDark(isDark);
    document.documentElement.classList.toggle("dark", isDark);
  }, []);

  function toggle() {
    const next = !dark;
    setDark(next);
    document.documentElement.classList.toggle("dark", next);
    localStorage.setItem("viby-theme", next ? "dark" : "light");
  }

  return (
    <Button variant="ghost" size="sm" onClick={toggle} aria-label="Toggle theme">
      <motion.span key={dark ? "moon" : "sun"} initial={{ rotate: -90, opacity: 0 }}
        animate={{ rotate: 0, opacity: 1 }} transition={{ duration: 0.2 }}>
        {dark ? "🌙" : "🌞"}
      </motion.span>
    </Button>
  );
}
```

To avoid a flash of the wrong theme, set the class **before** React hydrates with a tiny inline script in the root layout's `<head>`:

```
// app/layout.tsx (excerpt)
<script
  dangerouslySetInnerHTML={{
    __html: `(function(){try{var s=localStorage.getItem('viby-theme');
      var d=s?s==='dark':matchMedia('(prefers-color-scheme:dark)').matches;
      document.documentElement.classList.toggle('dark',d);}catch(e){}})();`,
  }}
/>
```

Architecture / diagram

The flow from a single brand hue to a rendered, themeable pixel:

```
flowchart TD
  A["Brand seed<br/>#FF6FB5 = hsl(332 100% 72%)"] --> B["Raw ramp tokens<br/>--color-pink-50 ... 950"]
  B --> C["Semantic tokens<br/>--color-primary, --color-background, ..."]
  C -->|".dark overrides values"| D["Theme layer<br/>light / dark"]
  D --> E["Tailwind utilities<br/>bg-primary, text-foreground"]
  E --> F["Components<br/>Button, VibyCard"]
  F --> G["Rendered UI<br/>accessible in both themes"]
  C -.documents.-> H["/design page<br/>token sheet"]
```

The arrows only ever flow downward: components never reach past the semantic layer to raw hex. That one rule keeps the whole system maintainable.

Best practices & pitfalls

- **Do** reference semantic tokens (`bg-primary`) in components, never raw ramp steps or hex. Raw steps are for *defining* the system, not consuming it.
- **Do** keep one hue family per brand color and derive the ramp by lightness — it guarantees harmony.
- **Don't** put brand pink behind body text. Reserve it for fills, accents, and large display text; check 4.5:1 with a contrast checker.
- **Don't** create a separate dark palette by hand. Override semantic values under `.dark` only.
- **Do** ship a no-flash inline script; a dark-mode flash on load reads as "amateur."
- **Don't** overuse glow/glass. Glass needs a busy/colored backdrop to read; over a flat white it just looks blurry. Follow 60-30-10.
- **Do** include radius, shadow, and spacing in your tokens — a design system is more than color.
- **Don't** animate `box-shadow` directly in hot paths; prefer transitioning a pseudo-element's opacity for glow on hover to stay at 60fps.

Exercises

1. **Derive a step.** Compute `--color-pink-450` between 400 and 500 in HSL and add it to the ramp. Verify the utility `bg-pink-450` works on a `<div>`.
2. **Audit contrast.** Use a checker to test `text-primary` on `bg-background` in both themes. If it fails, adjust which ramp step `--color-primary` references in `.dark` and re-test.
3. **A semantic `success` token.** Add `--color-success` / `--color-success-foreground` (a mint green ~150°), wire a `success` Button variant, and confirm it themes correctly.
4. **Glow on hover, performantly.** Refactor `VibyCard` so the glow appears only on hover by transitioning a pseudo-element's opacity rather than the `box-shadow` property.
5. **System-preference live sync.** Extend `ThemeToggle` so that when the user has *not* made an explicit choice, the app follows OS theme changes in real time via a `matchMedia` `change` listener.

Mini-project — The Viby Design Token Sheet

Build `/design`, a living page that renders every token so the whole team (and your future self) can see the system at a glance.

```tsx
// app/design/page.tsx  — Server Component, zero client JS
const RAMP = [50, 100, 200, 300, 400, 500, 600, 700, 800, 900, 950];
const SEMANTIC = [
  "background", "foreground", "primary", "primary-foreground",
  "muted", "muted-foreground", "card", "border",
] as const;

export default function DesignSystemPage() {
  return (
    <main className="min-h-screen bg-background text-foreground p-8 space-y-12">
      <header>
        <h1 className="text-4xl font-bold text-viby-gradient">Viby Design Tokens</h1>
        <p className="text-muted-foreground">One source of truth for color, shape & glow.</p>
      </header>

      <section>
        <h2 className="text-2xl font-semibold mb-4">Pink ramp</h2>
        <div className="grid grid-cols-11 gap-2">
          {RAMP.map((step) => (
            <div key={step} className="text-center">
              <div className="h-16 rounded-lg border border-border"
                   style={{ background: `var(--color-pink-${step})` }} />
              <span className="text-xs text-muted-foreground">{step}</span>
            </div>
          ))}
        </div>
      </section>

      <section>
        <h2 className="text-2xl font-semibold mb-4">Semantic tokens</h2>
        <div className="grid grid-cols-2 sm:grid-cols-4 gap-4">
          {SEMANTIC.map((name) => (
            <div key={name} className="rounded-lg border border-border overflow-hidden">
              <div className="h-16" style={{ background: `var(--color-${name})` }} />
              <code className="block p-2 text-xs">--color-{name}</code>
            </div>
          ))}
        </div>
      </section>

      <section>
        <h2 className="text-2xl font-semibold mb-4">Surfaces</h2>
        <div className="grid sm:grid-cols-3 gap-4">
```

```
          <div className="rounded-xl p-6 bg-viby-gradient text-white">Gradient</div>
          <div className="rounded-xl p-6 shadow-glow bg-card">Glow</div>
          <div className="rounded-xl p-6 bg-card/70 backdrop-blur-xl border border-border/60">
            Glass
          </div>
        </div>
      </section>
    </main>
  );
}
```

Add a `ThemeToggle` to the page header and watch every swatch re-theme live. This page is your contract: if a new component doesn't match what's here, the component is wrong — not the system.

Key takeaways

- Design with a **system of tokens**, not one-off hex values; change once, update everywhere.
- Reason in **HSL** and derive ramps by walking lightness across a single hue.
- Split tokens into **raw ramp** → **semantic** → **theme overrides**; components consume semantics only.
- Tailwind v4 puts the whole token sheet in **CSS** `@theme` — every token becomes a utility for free.
- Respect **contrast** and **60-30-10**: pink is a spark, not a flood.
- A **token sheet page** keeps the system honest and onboards teammates instantly.

Up next — Chapter 12: Typography & Spacing — A Type Scale That Breathes, where we give Viby a voice to match its color.

Chapter 12 — Typography, Spacing & Layout Systems

> Great design isn't decoration you add at the end — it's a system of decisions you make once, then reuse a thousand times.

You've shipped working screens. Now we make them *feel* like a real product. The difference between "a student project" and "a startup app" is rarely the features — it's the rhythm: consistent type, breathing room, and a hierarchy that guides the eye without a single instruction. This chapter turns Viby's look into a **system**.

What you'll learn

- How to build a **type scale** and pair a display font with a body font that actually works
- **Spacing rhythm** — why a 4px-based scale beats eyeballed margins
- A **radius and elevation system** for cards, sheets, and buttons
- The mechanics of **visual hierarchy**: size, weight, color, and space working together
- How to encode all of it as **Tailwind v4** `@theme` **tokens** and apply them across real Viby screens

Prerequisites

- Chapter 3 (project setup) and Chapter 4 (Tailwind v4 + the `@theme` block in `globals.css`)
- Chapter 8 (shadcn-style UI primitives — `Button`, `Card`)
- Comfort with Server Components vs. `"use client"` (Chapter 6)

Theory

Why a *system* and not just "good taste"

Imagine furnishing a house where every door is a different height and every wall a slightly different white. Nothing is *wrong*, but the whole thing feels off. UI is the same. When one card uses `padding: 14px`, the next uses `16px`, and a third uses `1rem`, your brain registers the inconsistency as "amateur" even if it can't name why.

A design system replaces a thousand ad-hoc choices with a small set of **tokens** — named, reusable values. You decide the spacing scale *once*. After that, you're never asking "16 or 18?"; you're picking from a menu. That's what makes design fast *and* consistent.

Type scales — the musical analogy

A type scale is like a musical scale: a fixed set of notes that sound good together. Instead of random sizes (13px, 15px, 22px, 31px), you generate sizes from a base by multiplying by a **ratio**.

- Base: `16px` (1rem) — your body text.
- Ratio: a common choice is the **Major Third (1.25)** or **Perfect Fourth (1.333)**. Bigger ratio = more dramatic contrast between headings and body.

With a 1.25 ratio from a 16px base: 16 → 20 → 25 → 31 → 39 → 49. Each step is "one note up." Viby is a punchy, mobile-first social app, so we want strong contrast: a big confident display size for moments like the onboarding hero, and a calm, readable body.

Pairing fonts. The reliable formula is *one expressive display face + one neutral, highly-readable body face*. Contrast is the goal — if both fonts are geometric sans-serifs, they fight. For Viby we'll pair **Sora** (a rounded, friendly geometric sans for display — on-brand Gen-Z energy) with **Inter** (the workhorse UI body font, superb at small sizes).

Rhythm and spacing — the 4px grid

Vertical and horizontal rhythm come from a single base unit. The industry standard is a **4px base** (sometimes 8px). Every gap, padding, and margin is a multiple: 4, 8, 12, 16, 24, 32, 48, 64. Why? Multiples of 4 align cleanly on screens, and constraining yourself to the scale eliminates the "is this 15 or 16?" dithering. Tailwind already bakes this in (`p-4` = 16px, `gap-2` = 8px), so we lean on it rather than fighting it.

Think of spacing like the silence in music. The notes (your content) matter, but the rests (your whitespace) are what make it *legible*. Cramped UIs read as stressful; generous spacing reads as premium.

Radius and elevation

Radius sets personality. Sharp corners (0–2px) feel corporate and serious; soft corners (12–24px) feel friendly and modern. Viby is warm and social, so we go generously rounded.

Elevation is how you communicate stacking — what floats above what. In flat design, elevation isn't drop-shadow soup; it's a small, deliberate ramp of shadows (rest → raised → overlay → modal). Each level should feel like a real step up off the page. Like the floors of a building: the lobby, then a few defined storeys — not forty indistinguishable ones.

Visual hierarchy — the four levers

You guide attention with four tools, used together:

1. **Size** — bigger = more important.
2. **Weight** — bolder = louder.
3. **Color/contrast** — high contrast pulls the eye; muted recedes.
4. **Space** — isolation creates emphasis (a lone CTA surrounded by whitespace screams "click me").

The mistake beginners make is shouting with all four at once on everything. Hierarchy is *relative*: something is only "loud" because other things are quiet.

In practice

1. Load the fonts (Next.js 16, `next/font`)

In the App Router, `next/font` self-hosts Google Fonts at build time (zero layout shift, no network request) and exposes them as CSS variables.

```
// app/layout.tsx
import type { Metadata } from "next";
import { Sora, Inter } from "next/font/google";
import "./globals.css";

const sora = Sora({
  subsets: ["latin"],
  variable: "--font-display",
  display: "swap",
  weight: ["600", "700", "800"],
});

const inter = Inter({
  subsets: ["latin"],
  variable: "--font-body",
  display: "swap",
});

export const metadata: Metadata = {
  title: "Viby — find your people",
  description: "Social discovery through shared hobbies and lifestyle.",
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en" className={`${sora.variable} ${inter.variable}`}>
      <body className="bg-background text-foreground antialiased">
        {children}
      </body>
    </html>
  );
}
```

2. Encode the system as Tailwind v4 `@theme` tokens

In Tailwind v4 there is **no** `tailwind.config.js` — design tokens live in CSS inside an `@theme` block. The font variables we just created get wired in here, alongside our type scale, radius, and shadow ramp.

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  /* Brand */
  --color-primary: #ff6fb5;
  --color-primary-foreground: #ffffff;
  --color-background: #ffffff;
  --color-foreground: #18121c;
  --color-muted: #f4f1f4;
  --color-muted-foreground: #6b6470;

  /* Fonts — consume the next/font variables */
  --font-display: var(--font-display), ui-sans-serif, system-ui, sans-serif;
  --font-body: var(--font-body), ui-sans-serif, system-ui, sans-serif;

  /* Type scale — base 16px, ~1.25 ratio (Major Third) */
  --text-xs: 0.75rem;     /* 12 */
  --text-sm: 0.875rem;    /* 14 */
  --text-base: 1rem;      /* 16 */
  --text-lg: 1.25rem;     /* 20 */
  --text-xl: 1.563rem;    /* 25 */
  --text-2xl: 1.953rem;   /* 31 */
  --text-3xl: 2.441rem;   /* 39 */
  --text-4xl: 3.052rem;   /* 49 */

  /* Line-height pairs to each step (set via utilities below) */
  --leading-tight: 1.1;
  --leading-snug: 1.3;
  --leading-relaxed: 1.6;

  /* Radius system — friendly, generous */
  --radius-sm: 0.5rem;    /* 8  - inputs, chips */
  --radius-md: 0.875rem;  /* 14 - buttons */
  --radius-lg: 1.25rem;   /* 20 - cards */
  --radius-xl: 1.75rem;   /* 28 - sheets, modals */
  --radius-full: 9999px;  /* avatars, pills */

  /* Elevation ramp — soft, brand-tinted */
  --shadow-raised: 0 1px 2px rgba(24, 18, 28, 0.06),
                   0 2px 8px rgba(24, 18, 28, 0.05);
  --shadow-overlay: 0 8px 24px rgba(24, 18, 28, 0.10);
  --shadow-modal: 0 24px 64px rgba(24, 18, 28, 0.18);
```

```
  --shadow-glow: 0 8px 28px rgba(255, 111, 181, 0.35);
}

/* Two semantic type roles built on the scale */
@layer components {
  .text-display {
    font-family: var(--font-display);
    font-weight: 800;
    line-height: var(--leading-tight);
    letter-spacing: -0.02em;
  }
  .text-body {
    font-family: var(--font-body);
    line-height: var(--leading-relaxed);
  }
}
```

Because we named tokens like `--color-primary`, `--radius-lg`, and `--shadow-overlay`, Tailwind v4 auto-generates utilities: `bg-primary`, `rounded-lg`, `shadow-overlay`, `text-3xl`, and so on. No config file, no plugin — the CSS *is* the source of truth.

3. A profile card that uses the whole system

Here's a Viby discovery card — the thing you swipe through to find people who share your hobbies. Notice every value is a token: type from the scale, `rounded-lg`, `shadow-overlay`, spacing on the 4px grid.

```tsx
// components/discovery/profile-card.tsx
import Image from "next/image";

type Person = {
  name: string;
  age: number;
  tagline: string;
  hobbies: string[];
  avatarUrl: string;
};

export function ProfileCard({ person }: { person: Person }) {
  return (
    <article className="overflow-hidden rounded-lg bg-background shadow-overlay">
      <div className="relative aspect-[4/5] w-full">
        <Image
          src={person.avatarUrl}
          alt={person.name}
          fill
          className="object-cover"
          sizes="(max-width: 480px) 100vw, 380px"
        />
        {/* gradient scrim so text stays readable over any photo */}
        <div className="absolute inset-x-0 bottom-0 h-2/5 bg-gradient-to-t from-foreground/70 to-
transparent" />
        <div className="absolute inset-x-0 bottom-0 p-5 text-primary-foreground">
          <h2 className="text-display text-2xl">
            {person.name}{" "}
            <span className="font-normal opacity-80">{person.age}</span>
          </h2>
          <p className="text-body mt-1 text-sm opacity-90">{person.tagline}</p>
        </div>
      </div>

      <div className="flex flex-wrap gap-2 p-4">
        {person.hobbies.map((h) => (
          <span
            key={h}
            className="rounded-full bg-muted px-3 py-1 text-sm text-muted-foreground"
          >
            {h}
          </span>
        ))}
```

```
      </div>
    </article>
  );
}
```

Read it top to bottom: the **name** is the loudest thing (display font, `text-2xl`, full white), the **age** recedes (normal weight, reduced opacity), the **tagline** is quieter still (body font, `text-sm`), and hobby chips are calm muted pills. Four levers, one clear path for the eye — and not a single hand-tuned pixel.

4. A reusable elevation primitive

Wrapping elevation in a tiny component keeps the ramp consistent and gives you one place to tune it. Server Component by default — no client JS needed.

```
// components/ui/surface.tsx
import { cn } from "@/lib/utils";

const elevation = {
  flat: "",
  raised: "shadow-raised",
  overlay: "shadow-overlay",
  modal: "shadow-modal",
} as const;

export function Surface({
  level = "raised",
  className,
  ...props
}: React.ComponentProps<"div"> & { level?: keyof typeof elevation }) {
  return (
    <div
      className={cn("rounded-lg bg-background", elevation[level], className)}
      {...props}
    />
  );
}
```

5. Animate elevation on interaction (Framer Motion)

A card that lifts on hover *feels* tactile. Keep the motion subtle — premium, not bouncy-house. This needs the client.

```
// components/discovery/interactive-card.tsx
"use client";

import { motion } from "framer-motion";

export function InteractiveCard({ children }: { children: React.ReactNode }) {
  return (
    <motion.div
      whileHover={{ y: -4 }}
      whileTap={{ scale: 0.98 }}
      transition={{ type: "spring", stiffness: 320, damping: 26 }}
      className="rounded-lg bg-background shadow-raised hover:shadow-overlay"
    >
      {children}
    </motion.div>
  );
}
```

The Tailwind `hover:shadow-overlay` and Framer's `y: -4` work together: the card both *rises* (transform) and *casts a bigger shadow* (the elevation ramp). That's elevation done right.

Architecture / diagram

How the system flows from a few token definitions out to every screen:

```
flowchart TD
    A["next/font<br/>Sora + Inter"] -->|CSS variables| B["@theme tokens<br/>globals.css"]
    B --> C["Auto-generated utilities<br/>text-3xl · rounded-lg · shadow-overlay"]
    C --> D["UI primitives<br/>Button · Card · Surface"]
    D --> E["Screen components<br/>ProfileCard · Onboarding · Feed"]
    E --> F["Viby app — consistent<br/>type · spacing · elevation"]

    subgraph "Single source of truth"
        B
    end
```

And the hierarchy levers, ranked by how strongly each pulls the eye on a Viby card:



Best practices & pitfalls

- **Do** pick a base size + ratio and generate the whole scale. **Don't** add one-off sizes "just this once" — that's how scales rot.
- **Do** pair *one* display font with *one* body font. **Don't** ship three+ font families; it reads as chaos and bloats your font payload.
- **Do** use `font-display: swap` and self-host via `next/font` to kill layout shift (CLS). **Don't** `<link>` to Google Fonts in `<head>` — it's slower and shifts the page.
- **Do** snap every gap/padding to the 4px grid (Tailwind's defaults). **Don't** write arbitrary values like `p-[13px]` unless you have a documented reason.
- **Do** keep a *small* elevation ramp (3–4 levels). **Don't** stack heavy shadows on everything — flat-with-purpose beats shadow soup.
- **Do** tighten `line-height` on big headings (1.1) and loosen it on body (1.5–1.6). **Don't** use one line-height for all sizes; large tight, small loose.
- **Do** check contrast: body text should hit WCAG AA (4.5:1). Bubblegum pink `#FF6FB5` on white is **not** AA for small text — use it for fills/accents, and use `--color-foreground` for reading text.
- **Don't** use color *alone* to convey state (a green vs. red dot is invisible to many users) — pair it with an icon or label.

Exercises

1. **Re-scale.** Regenerate the type scale using a Perfect Fourth ratio (1.333) instead of 1.25. Update the `@theme` tokens and observe how the onboarding hero gets more dramatic. Which ratio suits Viby better, and why?
2. **Spacing audit.** Open any screen you built earlier and find three places using off-grid values (e.g. `mt-[18px]`). Snap them to the nearest scale step and note whether the layout looks more cohesive.
3. **Add a level.** Extend the elevation ramp with a `--shadow-sheet` value for a bottom sheet that sits between `overlay` and `modal`. Add it to the `Surface` primitive's `elevation` map.

4. **Hierarchy challenge.** Take the `ProfileCard` and make the **tagline** the loudest element instead of the name — using only the four levers. Then argue (in a comment) which version serves a *discovery* app better.
5. **Dark mode token swap (stretch).** Add a `@media (prefers-color-scheme: dark)` block that overrides `--color-background`, `--color-foreground`, and the shadow ramp. Confirm every component adapts without touching a single component file.

Mini-project — the Viby Design Tokens reference screen

Build a `/styleguide` route (a Server Component page) that renders your living design system, so the whole team designs from the same source.

Goal: one page that displays:

- The full **type scale** — each step rendered at size with its name and px value.
- The **spacing scale** — labeled bars showing 4 → 64.
- The **radius set** — swatches from `sm` to `full`.
- The **elevation ramp** — four `Surface` cards, one per level.

```tsx
// app/styleguide/page.tsx
import { Surface } from "@/components/ui/surface";

const typeSteps = [
  ["text-4xl", "49px", "Find your people"],
  ["text-3xl", "39px", "Section heading"],
  ["text-2xl", "31px", "Card title"],
  ["text-xl", "25px", "Subhead"],
  ["text-lg", "20px", "Lead paragraph"],
  ["text-base", "16px", "Body — the workhorse"],
  ["text-sm", "14px", "Captions & chips"],
] as const;

const space = [1, 2, 3, 4, 6, 8, 12, 16] as const; // ×4px

export default function StyleguidePage() {
  return (
    <main className="mx-auto max-w-2xl space-y-12 p-6">
      <h1 className="text-display text-4xl text-primary">Viby Design System</h1>

      <section className="space-y-3">
        <h2 className="text-display text-xl">Type scale</h2>
        {typeSteps.map(([cls, px, sample]) => (
          <div key={cls} className="flex items-baseline gap-4">
            <code className="w-44 shrink-0 text-sm text-muted-foreground">
              {cls} · {px}
            </code>
            <span className={`text-display ${cls}`}>{sample}</span>
          </div>
        ))}
      </section>

      <section className="space-y-3">
        <h2 className="text-display text-xl">Spacing (×4px)</h2>
        {space.map((s) => (
          <div key={s} className="flex items-center gap-4">
            <code className="w-16 text-sm text-muted-foreground">{s * 4}px</code>
            <div className="h-3 rounded-sm bg-primary" style={{ width: s * 4 }} />
          </div>
        ))}
      </section>

      <section className="space-y-3">
```

```
        <h2 className="text-display text-xl">Radius</h2>
        <div className="flex flex-wrap gap-4">
          {["sm", "md", "lg", "xl", "full"].map((r) => (
            <div
              key={r}
              className={`grid h-20 w-20 place-items-center bg-muted text-sm text-muted-foreground
rounded-${r}`}
            >
              {r}
            </div>
          ))}
        </div>
      </section>

      <section className="space-y-3">
        <h2 className="text-display text-xl">Elevation</h2>
        <div className="grid grid-cols-2 gap-6">
          {(["raised", "overlay", "modal"] as const).map((level) => (
            <Surface key={level} level={level} className="grid h-24 place-items-center">
              <span className="text-sm text-muted-foreground">{level}</span>
            </Surface>
          ))}
        </div>
      </section>
    </main>
  );
}
```

Ship this once and you'll reference it for the rest of the book. When a teammate asks "what's our card radius?" you point at `/styleguide` instead of arguing in Slack. (One caveat for the radius swatches: because `rounded-${r}` is built dynamically, add those class names to a safelist comment or list them statically if your build tree-shakes unused utilities.)

Key takeaways

- A design system swaps a thousand ad-hoc choices for a **small set of named tokens** — fast *and* consistent.
- Generate your **type scale** from a base size and a ratio; pair **one display + one body** font for contrast.
- Snap all spacing to a **4px grid**; whitespace is a feature, not wasted space.
- Keep a **small radius set and a 3–4 step elevation ramp** — restraint reads as premium.
- Drive hierarchy with **size, weight, contrast, and space** — relatively, never all at once.

- In this stack the system lives in **Tailwind v4** `@theme` **tokens** (no config file), fed by `next/font` variables, and is the single source of truth for every screen.

Up next — Chapter 13: Color, Theming & Dark Mode, where we turn that bubblegum pink into a full, accessible palette and make Viby beautiful in the dark.

Chapter 13 — Design Psychology, Gen-Z & Social App UX

> Great social apps don't *show* you content — they make you *feel* something in under 400ms, and then make you want to come back tomorrow.

What you'll learn

- **Emotional design**: why the first 5 seconds decide whether a user stays, and how to engineer warmth into pixels.
- **Cognitive load**: how to make Viby feel effortless by giving the brain less to do.
- **Social & feedback loops**: the mechanics behind likes, streaks, and "someone viewed your profile" — and how to use them ethically.
- **Onboarding psychology**: progressive disclosure, the goal-gradient effect, and the dreaded empty state.
- **Premium feel**: micro-interactions, motion, and the details that separate a $0 app from a startup-grade one.
- **Dark patterns to avoid**: the manipulative tricks that work short-term and burn trust long-term.
- **Designing an identity system that is NOT a dating clone**: how copy, taxonomy, and affordances steer Viby toward *friendship and discovery*.

Prerequisites

- Chapter 6 (Tailwind CSS v4 with `@theme` tokens) — we'll lean on our design tokens here.
- Chapter 8 (shadcn-style UI primitives) — `Button`, `Card`, `Avatar`.
- Chapter 10 (Framer Motion) — `motion`, `AnimatePresence`, gestures.
- Chapter 11 (Zustand demo state) and Chapter 12 (Supabase Realtime) — for the live feedback loop in the mini-project.

Theory

Emotional design: people decode feeling before function

Don Norman splits design into three levels: **visceral** (gut reaction to look/feel), **behavioral** (how good it is to *use*), and **reflective** (the story you tell yourself after — "I'm the kind of person who uses this"). Most beginners obsess over behavioral and ignore the other two. For a social app, that's fatal: the visceral layer fires in ~50ms, before a single word is read.

Think of your UI like meeting a person. Before they speak, you've judged their posture, their warmth, whether they're *like you*. Viby's bubblegum pink (`#FF6FB5`) on generous white space is a deliberate visceral signal: playful, soft, non-threatening — the opposite of a corporate blue LinkedIn or a sultry red dating app.

Cognitive load: the brain is lazy on purpose

Working memory holds only a handful of items at once. Every choice, every unlabeled icon, every "wait, what does this do?" spends a slice of that budget. **Cognitive load** comes in three flavors:

- **Intrinsic** — inherent difficulty of the task (joining a hobby group).
- **Extraneous** — difficulty *you* added with bad UI (a cluttered screen). This is the one we ruthlessly cut.
- **Germane** — effort that builds the user's mental model (a good onboarding tour).

Analogy: a well-designed screen is a clean kitchen. Everything you need for *this* step is on the counter; everything else is in a drawer. **Hick's Law** says decision time grows with the number of options — so we show one primary action per screen, not five.

Social & feedback loops: the dopamine machine, used kindly

A feedback loop is *action → reward → motivation to repeat*. Social apps run on three engines:

1. **Reciprocity** — you waved, they waved back. Humans are wired to return gestures.
2. **Social proof** — "12 people share your love of bouldering" lowers the fear of joining.
3. **Variable reward** — you don't know *who* will react, so you check. This is the most powerful and the most dangerous lever; we'll use it with restraint.

The ethical line: a loop is healthy when it helps the user reach *their* goal (find friends), and toxic when it exists only to maximize *our* session time.

Onboarding psychology: earn the right to ask

New users have zero investment and infinite exits. Two principles save us:

- **Progressive disclosure** — reveal complexity only as needed. Don't ask for 14 interests up front; ask for 3, celebrate, then offer more.
- **Goal-gradient effect** — motivation increases as a goal feels closer. A progress bar that starts at 20% (not 0%) makes people finish.

Premium feel: the invisible 10%

Premium isn't a gradient — it's *responsiveness*. Buttons that depress, lists that stagger in, optimistic UI that reacts *before* the server replies. The brain reads sub-100ms feedback as "this thing is alive."

NOT a dating clone: identity through constraint

A dating app and a friendship app can look identical in a Figma file — the difference lives in **affordances and language**. Swipe-to-judge-faces says "rate this body." A grid of *shared interests* says "find your people." We design Viby so the unit of attraction is a **shared hobby**, never a face. We'll codify this below.

In practice

1. Encoding emotion as design tokens (Tailwind v4 `@theme`)

Emotion should be *systematized*, not vibes. We define warmth in CSS tokens once (Chapter 6) and reuse everywhere.

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  /* Brand — visceral warmth */
  --color-viby: #ff6fb5;
  --color-viby-soft: #ffe1f0;
  --color-viby-ink: #2a1622;

  /* Friendship-coded accents (NOT romance reds) */
  --color-spark: #ffd166;   /* shared-interest highlight */
  --color-calm: #7ee0c2;    /* "you're matched on a hobby" */

  /* Premium motion */
  --ease-spring: cubic-bezier(0.34, 1.56, 0.64, 1);
  --radius-bubble: 1.25rem;
}
```

2. Low cognitive load: one primary action per screen

A profile card that surfaces *shared interests* (social proof) and exactly **one** clear next step (Hick's Law).

```tsx
// components/discover/PersonCard.tsx  (Server Component by default)
import { Card } from "@/components/ui/card";
import { Avatar } from "@/components/ui/avatar";
import { WaveButton } from "./WaveButton";

type Person = {
  id: string;
  name: string;
  avatarUrl: string;
  sharedInterests: string[]; // the unit of attraction — hobbies, not looks
};

export function PersonCard({ person }: { person: Person }) {
  return (
    <Card className="rounded-[--radius-bubble] p-5 bg-white">
      <div className="flex items-center gap-3">
        <Avatar src={person.avatarUrl} alt={person.name} className="size-14" />
        <div>
          <h3 className="font-semibold text-viby-ink">{person.name}</h3>
          {/* Social proof, not a hot-or-not score */}
          <p className="text-sm text-viby-ink/70">
            {person.sharedInterests.length} shared interests
          </p>
        </div>
      </div>

      <ul className="mt-3 flex flex-wrap gap-2">
        {person.sharedInterests.slice(0, 3).map((tag) => (
          <li
            key={tag}
            className="rounded-full bg-viby-soft px-3 py-1 text-xs font-medium text-viby"
          >
            {tag}
          </li>
        ))}
      </ul>

      {/* Exactly ONE primary action */}
      <WaveButton personId={person.id} className="mt-4 w-full" />
    </Card>
  );
}
```

3. The feedback loop: optimistic "Wave" with a Server Action

A wave is reciprocity-coded (friendly), not a swipe (judgmental). We make it feel *instant* with React 19's `useOptimistic`, then persist via a Server Action.

```
// app/discover/actions.ts
"use server";

import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";
import { revalidatePath } from "next/cache";

export async function sendWave(personId: string) {
  const cookieStore = await cookies(); // async in Next.js 16

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) =>
          toSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options),
          ),
      },
    },
  );

  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) throw new Error("Not authenticated");

  const { error } = await supabase
    .from("waves")
    .insert({ from_user: user.id, to_user: personId });

  if (error) throw error;
  revalidatePath("/discover");
}
```

```tsx
// components/discover/WaveButton.tsx
"use client";

import { useOptimistic, useTransition } from "react";
import { motion } from "framer-motion";
import { Button } from "@/components/ui/button";
import { sendWave } from "@/app/discover/actions";

export function WaveButton({
  personId,
  className,
}: {
  personId: string;
  className?: string;
}) {
  const [waved, setWaved] = useOptimistic(false);
  const [isPending, startTransition] = useTransition();

  return (
    <motion.div whileTap={{ scale: 0.96 }} className={className}>
      <Button
        disabled={waved || isPending}
        className="w-full bg-viby text-white transition-[transform,background-color]"
        onClick={() =>
          startTransition(async () => {
            setWaved(true); // feels instant — premium responsiveness
            await sendWave(personId);
          })
        }
      >
        {waved ? "Wave sent 👋" : "Say hi"}
      </Button>
    </motion.div>
  );
}
```

4. Onboarding: progressive disclosure + goal-gradient

Ask for **3** interests, show a head-start progress bar, celebrate on completion.

```tsx
// components/onboarding/InterestPicker.tsx
"use client";

import { useState } from "react";
import { motion, AnimatePresence } from "framer-motion";

const INTERESTS = ["Bouldering", "Lo-fi", "Film photography", "Ramen", "Indie games", "Plants"];
const GOAL = 3;

export function InterestPicker() {
  const [picked, setPicked] = useState<string[]>([]);
  // Goal-gradient: start the bar at 20%, never empty
  const progress = 20 + (picked.length / GOAL) * 80;
  const done = picked.length >= GOAL;

  function toggle(tag: string) {
    setPicked((p) =>
      p.includes(tag) ? p.filter((t) => t !== tag) : [...p, tag],
    );
  }

  return (
    <div className="space-y-5">
      <div className="h-2 w-full overflow-hidden rounded-full bg-viby-soft">
        <motion.div
          className="h-full bg-viby"
          animate={{ width: `${Math.min(progress, 100)}%` }}
          transition={{ type: "spring", stiffness: 180, damping: 22 }}
        />
      </div>

      <p className="text-sm text-viby-ink/70">Pick {GOAL} things you love</p>

      <div className="flex flex-wrap gap-2">
        {INTERESTS.map((tag) => {
          const active = picked.includes(tag);
          return (
            <motion.button
              key={tag}
              whileTap={{ scale: 0.94 }}
              onClick={() => toggle(tag)}
              className={`rounded-full px-4 py-2 text-sm font-medium transition-colors ${
                active ? "bg-viby text-white" : "bg-viby-soft text-viby"
```

```
              }`}
            >
              {tag}
            </motion.button>
          );
        })}
      </div>

      <AnimatePresence>
        {done && (
          <motion.p
            initial={{ opacity: 0, y: 8 }}
            animate={{ opacity: 1, y: 0 }}
            exit={{ opacity: 0 }}
            className="font-semibold text-viby"
          >
            Nice — we found 24 people who love these too ✨
          </motion.p>
        )}
      </AnimatePresence>
    </div>
  );
}
```

Note the celebration line: it's **social proof delivered at the moment of investment** — the strongest time to reduce drop-off.

Architecture / diagram

The healthy feedback loop, and where each psychological lever attaches:

```
flowchart TD
    A[User opens Discover] --> B[PersonCard: shared interests = social proof]
    B --> C{One clear action: Say hi}
    C -->|whileTap + useOptimistic| D[Instant visual reward < 100ms]
    D --> E[Server Action: insert wave -> Supabase]
    E --> F[Realtime: recipient gets 'someone waved' ping]
    F --> G[Reciprocity: they wave back]
    G --> H[Match on a HOBBY, not a face]
    H --> I[Conversation seeded with shared interest]
    I -->|healthy goal reached| A

    style D fill:#ffe1f0,stroke:#ff6fb5
    style H fill:#7ee0c2,stroke:#2a1622
```

The loop closes on a **goal the user actually wants** (a real conversation), not on raw session time. That single design decision is what keeps Viby out of dark-pattern territory.

Best practices & pitfalls

Do

- Give every action sub-100ms feedback (optimistic UI, `whileTap`). Perceived speed *is* premium.
- Use motion with intent: `--ease-spring` for playful, short durations (150–300ms). Animation should clarify, never delay.
- Anchor matches to **shared interests**, never appearance scores.
- Respect `prefers-reduced-motion` — wrap large animations so accessibility users aren't punished.
- Write copy in a warm, second-person voice ("find your people"), not transactional ("browse users").

Don't

- ❌ **Confirmshaming** ("No thanks, I hate making friends") — manipulative and off-brand.
- ❌ **Forced continuity / roach motel** — easy to subscribe, impossible to leave. Make "delete account" findable.
- ❌ **Infinite variable-reward feeds** with no natural stopping point. Add "you're all caught up" states.
- ❌ **Fake urgency / vanity counters** ("5 people are looking at you NOW!") that aren't true.
- ❌ Swipe-on-faces, hot-or-not scoring, or proximity-first matching — these silently turn a friendship app into a dating clone.
- ❌ Dumping 10 onboarding fields on screen one. Disclose progressively.

Exercises

1. **Warm empty state (easy).** The Discover feed has no people yet. Design and build an empty state with an illustration placeholder, one encouraging line of copy, and a single CTA ("Add interests"). No dead ends.
2. **Reduced-motion guard (medium).** Refactor `InterestPicker` so all Framer Motion animations are disabled when the user has `prefers-reduced-motion: reduce`. Hint: Framer Motion's `useReducedMotion()` hook.
3. **Dark-pattern audit (medium).** Take any onboarding flow from an app on your phone. List 3 dark patterns it uses and rewrite each as an ethical alternative in Viby's voice.
4. **Streak, ethically (hard).** Add a "kept in touch" streak that rewards *replying within 48h* — but design it so missing a day produces gentle encouragement, never guilt or loss-aversion pressure. Justify your copy choices in comments.
5. **Taxonomy redesign (hard).** Replace any one screen's "users" framing with "interests" framing. Show before/after and explain how the affordance shift moves Viby further from a dating clone.

Mini-project: the "Someone waved at you" live ping

Build a realtime feedback loop that closes the reciprocity cycle. When another user sends a wave (via the Server Action above), the recipient sees a warm, dismissible toast — no fake urgency, just genuine social signal.

```tsx
// components/discover/WaveListener.tsx
"use client";

import { useEffect, useState } from "react";
import { createBrowserClient } from "@supabase/ssr";
import { motion, AnimatePresence } from "framer-motion";

export function WaveListener({ userId }: { userId: string }) {
  const [from, setFrom] = useState<string | null>(null);

  useEffect(() => {
    const supabase = createBrowserClient(
      process.env.NEXT_PUBLIC_SUPABASE_URL!,
      process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    );

    const channel = supabase
      .channel("waves-inbox")
      .on(
        "postgres_changes",
        {
          event: "INSERT",
          schema: "public",
          table: "waves",
          filter: `to_user=eq.${userId}`,
        },
        (payload) => setFrom((payload.new as { from_user: string }).from_user),
      )
      .subscribe();

    return () => {
      supabase.removeChannel(channel);
    };
  }, [userId]);

  return (
    <AnimatePresence>
      {from && (
        <motion.div
          initial={{ opacity: 0, y: 24, scale: 0.95 }}
          animate={{ opacity: 1, y: 0, scale: 1 }}
          exit={{ opacity: 0, y: 24 }}
          transition={{ type: "spring", stiffness: 220, damping: 20 }}
```

```
          className="fixed bottom-6 left-1/2 -translate-x-1/2 rounded-[--radius-bubble]
                     bg-white px-5 py-3 shadow-lg ring-1 ring-viby-soft"
        >
          <p className="text-sm font-medium text-viby-ink">
            Someone waved at you 👋{" "}
            <button
              onClick={() => setFrom(null)}
              className="ml-2 font-semibold text-viby"
            >
              Wave back
            </button>
          </p>
        </motion.div>
      )}
    </AnimatePresence>
  );
}
```

Mount it once in your authenticated layout:

```
// app/(app)/layout.tsx  (Server Component)
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";
import { WaveListener } from "@/components/discover/WaveListener";

export default async function AppLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  const cookieStore = await cookies();
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: () => {}, // read-only in a layout
      },
    },
  );

  const {
    data: { user },
  } = await supabase.auth.getUser();

  return (
    <>
      {children}
      {user && <WaveListener userId={user.id} />}
    </>
  );
}
```

Why this is ethical by design: the toast appears *only* on a genuine event, it's instantly dismissible, and the CTA points back to a real human goal (reciprocate, start talking) — not "open the app again." That's the whole philosophy of this chapter compiled into one component.

> Route-protection note: this layout reads auth but doesn't *enforce* it. In Next.js 16 we handle redirect logic in `proxy.ts` (Chapter 9), not a `middleware.ts` file.

Key takeaways

- Users feel before they think — engineer the **visceral** layer (color, space, motion) deliberately.
- Cut **extraneous cognitive load**: one primary action per screen, social proof over raw choice.
- Build **feedback loops around the user's goal** (real connection), not around your session-time metric.
- Onboard with **progressive disclosure** and a **goal-gradient** head start; celebrate investment with social proof.
- Premium = **responsiveness**: optimistic UI and intentful, sub-300ms motion.
- Avoid dark patterns — they win the week and lose the user.
- Keep Viby *not a dating clone* through **affordances and language**: the unit of attraction is a shared hobby, never a face.

Up next — Chapter 14: Component Architecture & Design Systems at Scale, where we turn these psychological principles into a reusable, tokenized component library that stays consistent as Viby grows.

Chapter 14 — Databases & PostgreSQL Data Modeling

> Your app is only as trustworthy as the data model underneath it — so let's design Viby's brain before we wire up its mouth.

Up to now, Viby has lived in memory. Zustand stores held your profiles, your swipes, your matches — and every refresh wiped the world clean. That was perfect for learning the front end. But a real social-discovery platform needs to *remember*: who you are, who you vibed with, who matched back, and what everyone is posting. This chapter is where Viby grows a permanent memory.

What you'll learn

- What a relational database actually is, and why PostgreSQL is the industry default
- Tables, rows, columns, and how to choose good data types
- Primary keys and foreign keys — the glue that makes data *relational*
- The three relationship shapes: one-to-one (1:1), one-to-many (1:N), and many-to-many (N:M)
- Normalization — how to stop your data from rotting
- Indexes — how to make queries fast (and when not to)
- A complete, production-shaped schema for Viby: `profiles`, `swipes`, `matches`, `posts`, `messages`

Prerequisites

- **Chapter 11–13** (Backend foundations): you understand servers, environment variables, and Supabase basics.
- **Chapter 8** (Auth): you know Supabase Auth issues a user in `auth.users` with a UUID `id`. We'll build *on top* of that.
- Comfort reading SQL is *not* assumed — we teach it here.

Theory

Why a relational database?

Imagine Viby's data as a giant shared spreadsheet workbook. Each *sheet* is a **table** (profiles, posts, messages). Each *row* is one record — one person, one post, one message. Each *column* is one attribute with a fixed type — a display name (text), an age (integer), a "created at" timestamp.

A **relational database** is that workbook with superpowers: it enforces *rules* (this column must be a valid email; this post must belong to a real user), it answers questions across sheets instantly ("show me every post by people I matched with this week"), and it guarantees that two requests at the same time don't corrupt each other (**ACID** transactions — Atomicity, Consistency, Isolation, Durability).

We use **PostgreSQL** because it's the open-source gold standard: rock-solid, feature-rich (JSON, full-text search, geospatial), and it's exactly what Supabase runs under the hood. When you write SQL for Supabase, you're writing real Postgres.

Keys: the difference between a list and a graph

A **primary key (PK)** is the unique fingerprint of a row. No two rows in a table may share it. In Viby, a profile's primary key is its `id` (a UUID).

A **foreign key (FK)** is a column that *points* at another table's primary key. A `post.author_id` is a foreign key to `profiles.id`. This is the magic word "relational": foreign keys turn isolated lists into a connected graph, and the database *refuses* to let you create a post whose author doesn't exist (referential integrity).

> Analogy: a primary key is your passport number. A foreign key is your name written on a concert ticket — it only means something because it points back to a real passport.

The three relationship shapes

One-to-one (1:1) — each row in A matches at most one row in B. Example: an `auth.users` record has exactly one `profiles` record. We model this by giving `profiles.id` the *same* value as the user's auth id.

One-to-many (1:N) — one row in A owns many rows in B, but each B belongs to one A. Example: one profile authors many posts. We put the FK on the "many" side: `posts.author_id`.

Many-to-many (N:M) — rows on both sides connect to many on the other. Example: profiles ↔ interests (you have many hobbies; each hobby has many fans). You can't store this with a single FK, so you introduce a **join table** (also called a junction or bridge table) that holds one row per pairing.

Normalization — don't repeat yourself, with data

Normalization is the discipline of storing each fact exactly once. The classic smell is duplication: if you copy a user's display name into every post they write, then a name change means hunting down hundreds of rows — and you'll miss some, leaving Viby showing two names for one person.

The practical rules (first three normal forms, plain-English):

1. **1NF** — one value per cell. No comma-jammed lists like `"music,hiking,art"` in a single column.
2. **2NF** — every column depends on the *whole* primary key (matters for composite keys).
3. **3NF** — non-key columns depend only on the key, not on each other. Don't store `birth_year` *and* `age` — age is derivable.

Normalize first. **Denormalize later, deliberately**, only when measured performance demands it (e.g., a cached `like_count`).

Indexes — the book's table of contents

Without an index, finding "all posts by author X" forces Postgres to scan *every* post (a sequential scan). An **index** is a sorted lookup structure (a B-tree) that turns that scan into a near-instant jump — exactly like the index at the back of a textbook.

The trade-off: indexes cost disk space and slow down writes (every insert must update the index too). So index the columns you **filter, join, or sort on** — and not much else. Foreign keys are almost always worth indexing.

In practice

We'll build Viby's schema in the Supabase SQL Editor (Dashboard → SQL Editor) or as a migration file. Everything below is real Postgres.

1. The `profiles` table (1:1 with auth.users)

```
-- One profile per authenticated user.
create table public.profiles (
  id          uuid primary key references auth.users (id) on delete cascade,
  username    text unique not null,
  display_name text not null,
  bio         text,
  avatar_url  text,
  birth_year  smallint check (birth_year between 1900 and 2025),
  city        text,
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now()
);
```

Notice `id` is both the **primary key** *and* a **foreign key** to `auth.users(id)` — that's the 1:1 link. `on delete cascade` means: if the auth user is deleted, their profile vanishes automatically. `username unique` enforces no duplicate handles. `check` constraints reject bad data at the door.

2. Interests (N:M via a join table)

A profile has many interests; an interest has many fans. That's many-to-many.

```
create table public.interests (
  id    serial primary key,
  slug  text unique not null,    -- e.g. 'bouldering'
  label text not null            -- e.g. 'Bouldering'
);

-- The join table: one row per (profile, interest) pairing.
create table public.profile_interests (
  profile_id  uuid not null references public.profiles (id) on delete cascade,
  interest_id integer not null references public.interests (id) on delete cascade,
  primary key (profile_id, interest_id)   -- composite key prevents duplicates
);
```

The **composite primary key** `(profile_id, interest_id)` guarantees you can't add the same interest to a profile twice — and gives us a built-in index on the pairing.

3. Swipes and matches (the heart of discovery)

A **swipe** is a directed action: I vibed (or passed) on someone. A **match** is the mutual result.

```
create table public.swipes (
  id         uuid primary key default gen_random_uuid(),
  swiper_id  uuid not null references public.profiles (id) on delete cascade,
  swipee_id  uuid not null references public.profiles (id) on delete cascade,
  direction  text not null check (direction in ('vibe', 'pass')),
  created_at timestamptz not null default now(),
  -- you can only swipe a given person once
  unique (swiper_id, swipee_id),
  -- you can't swipe yourself
  check (swiper_id <> swipee_id)
);

create table public.matches (
  id          uuid primary key default gen_random_uuid(),
  profile_a   uuid not null references public.profiles (id) on delete cascade,
  profile_b   uuid not null references public.profiles (id) on delete cascade,
  created_at  timestamptz not null default now(),
  -- store the pair in a canonical order so (a,b) and (b,a) can't both exist
  check (profile_a < profile_b),
  unique (profile_a, profile_b)
);
```

The `profile_a < profile_b` trick is a pro move: by always storing the smaller UUID first, we make a match between two people *unique* regardless of who swiped first.

4. Posts (1:N) and messages (1:N within a match)

```
create table public.posts (
  id         uuid primary key default gen_random_uuid(),
  author_id  uuid not null references public.profiles (id) on delete cascade,
  body       text not null check (char_length(body) <= 2000),
  image_url  text,
  like_count integer not null default 0,  -- deliberate denormalization
  created_at timestamptz not null default now()
);

create table public.messages (
  id          uuid primary key default gen_random_uuid(),
  match_id    uuid not null references public.matches (id) on delete cascade,
  sender_id   uuid not null references public.profiles (id) on delete cascade,
  body        text not null check (char_length(body) between 1 and 4000),
  created_at  timestamptz not null default now()
);
```

Messages belong to a `match` (you can only DM someone you matched with) *and* record who sent them — two foreign keys, both meaningful.

5. Indexes for the queries we'll actually run

```
-- "Show me a feed: recent posts by author."
create index posts_author_created_idx on public.posts (author_id, created_at desc);

-- "Have these two already swiped each other?"
create index swipes_swipee_idx on public.swipes (swipee_id);

-- "Load this match's messages in order."
create index messages_match_created_idx on public.messages (match_id, created_at);
```

These are **composite indexes** ordered to match our access patterns: filter by `match_id`, then sort by `created_at`. Postgres uses the leftmost columns, so order matters.

6. Reading it from Next.js 16 (Server Component + @supabase/ssr)

Now the payoff — querying the schema from a React Server Component. We use the `@supabase/ssr` server client with `getAll` / `setAll` cookie handlers.

```
// app/lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies(); // Next.js 16: cookies() is async

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options)
          );
        },
      },
    }
  );
}
```

```tsx
// app/feed/page.tsx — a Server Component, no "use client" needed
import { createClient } from "@/app/lib/supabase/server";

export default async function FeedPage() {
  const supabase = await createClient();

  // Join posts -> profiles in ONE query using the FK relationship.
  const { data: posts, error } = await supabase
    .from("posts")
    .select(
      `id, body, image_url, like_count, created_at,
       author:profiles ( username, display_name, avatar_url )`
    )
    .order("created_at", { ascending: false })
    .limit(20);

  if (error) throw error;

  return (
    <ul className="mx-auto max-w-md space-y-4 p-4">
      {posts.map((post) => (
        <li key={post.id} className="rounded-2xl bg-white p-4 shadow-sm">
          <p className="font-semibold text-[--color-brand]">
            {post.author.display_name}
          </p>
          <p className="mt-1 text-sm text-gray-700">{post.body}</p>
        </li>
      ))}
    </ul>
  );
}
```

Supabase's `author:profiles ( ... )` syntax follows the `author_id` foreign key automatically — that's relational modeling paying dividends. The brand pink lives in a Tailwind v4 theme token (`@theme { --color-brand: #FF6FB5; }`), referenced as `[--color-brand]` .

7. Creating a swipe with a Server Action

```
// app/actions/swipe.ts
"use server";

import { createClient } from "@/app/lib/supabase/server";
import { revalidatePath } from "next/cache";

export async function recordSwipe(swipeeId: string, direction: "vibe" | "pass") {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) throw new Error("Not authenticated");

  // Insert the swipe (the UNIQUE constraint blocks accidental double-swipes).
  const { error } = await supabase
    .from("swipes")
    .insert({ swiper_id: user.id, swipee_id: swipeeId, direction });

  if (error && error.code !== "23505") throw error; // 23505 = unique violation

  revalidatePath("/discover");
}
```

In production you'd promote the mutual-match check into a Postgres function/trigger so a match is created atomically — but the schema above is what makes that trigger trivial to write.

Architecture / diagram

```
erDiagram
    auth_users ||--|| profiles : "1:1"
    profiles ||--o{ posts : "authors (1:N)"
    profiles ||--o{ swipes : "swiper (1:N)"
    profiles ||--o{ profile_interests : ""
    interests ||--o{ profile_interests : ""
    profiles ||--o{ matches : "in (1:N)"
    matches ||--o{ messages : "contains (1:N)"
    profiles ||--o{ messages : "sends (1:N)"

    profiles {
        uuid id PK_FK
        text username
        text display_name
    }
    swipes {
        uuid id PK
        uuid swiper_id FK
        uuid swipee_id FK
        text direction
    }
    matches {
        uuid id PK
        uuid profile_a FK
        uuid profile_b FK
    }
    posts {
        uuid id PK
        uuid author_id FK
        text body
    }
    messages {
        uuid id PK
        uuid match_id FK
        uuid sender_id FK
    }
```

`profile_interests` is the N:M join table bridging `profiles` and `interests`. Every other arrow is a 1:N relationship anchored by a foreign key.

Best practices & pitfalls

- **Do** put foreign keys on the "many" side and index them. An unindexed FK is the #1 cause of mysterious slow joins.
- **Do** use `timestamptz` (timezone-aware), never naive `timestamp`. Store UTC, format in the UI.
- **Do** use UUIDs (`gen_random_uuid()`) for public-facing IDs so you don't leak row counts or invite enumeration attacks.
- **Do** enforce rules in the database with `check`, `unique`, and `not null`. The DB is your last line of defense; app code has bugs.
- **Don't** store lists in a single column (`"music,art"`). That breaks 1NF and makes filtering miserable — use a join table.
- **Don't** denormalize prematurely. Add a cached `like_count` only after you've measured the read cost.
- **Don't** forget **Row Level Security (RLS)**. In Supabase, a table with no RLS policy is wide open. Every table here needs `alter table ... enable row level security;` plus policies (covered next chapter).
- **Don't** delete rows you might need for audit/abuse review — consider a soft-delete `deleted_at timestamptz` column.

Exercises

1. **Warm-up:** Add a `pronouns text` column and an `is_verified boolean not null default false` column to `profiles`. Write the `alter table` statements.
2. **Relationships:** Model a `post_likes` table so a user can like a post at most once. Which relationship shape is this, and what should the primary key be?
3. **Indexing:** A query filters `messages` by `match_id` and sorts newest-first, but it's slow. Write the exact `create index` that fixes it and explain the column order.
4. **Normalization:** You find `profiles.city` *and* `profiles.country` are repeated thousands of times with typos. Refactor into a `cities` table and an FK. Sketch the new schema.
5. **Challenge:** Write a SQL `check` constraint (or trigger) that prevents a user from messaging in a match they don't belong to. Why is the FK alone insufficient?

Mini-project — "Vibes that matched"

Build a Server Component page at `app/matches/page.tsx` that shows the current user's matches with the *other* person's display name and avatar.

Requirements: 1. Query `matches` where the logged-in user is either `profile_a` or `profile_b` (hint: `.or("profile_a.eq...,profile_b.eq...")`). 2. For each match, resolve the *other* profile (the one that isn't you) and show their `display_name` + `avatar_url`. 3. For each match, fetch the most recent message body as a preview, ordered by `created_at desc`, limited to 1. 4. Render each match as a tappable card (mobile-first, brand-pink accent) linking to `app/messages/[matchId]/page.tsx`. 5. Wrap the list in a Framer Motion `motion.ul` so cards fade-and-rise in with `<AnimatePresence>` as data loads.

Stretch: add an index that makes the "latest message per match" lookup fast, and explain why your composite index helps.

This project forces you to traverse three tables (`matches` → `profiles` → `messages`) through foreign keys — exactly the relational thinking this chapter built.

Key takeaways

- A relational database stores facts as **tables** of typed **rows** and **columns**, enforcing integrity the app can't be trusted to.
- **Primary keys** uniquely identify rows; **foreign keys** connect tables — together they turn lists into a navigable graph.
- The three shapes: **1:1** (shared/foreign key), **1:N** (FK on the many side), **N:M** (a join table).
- **Normalize** to store each fact once; denormalize only with evidence.
- **Indexes** make reads fast at a small write cost — index what you filter, join, and sort on.
- Viby's core is just six tables: `profiles` , `interests` / `profile_interests` , `swipes` , `matches` , `posts` , `messages` .

Up next — Chapter 15: Supabase Row Level Security & Authorization, where we lock these tables down so users can only ever read and write the rows they're allowed to.

Chapter 15 — Supabase Architecture & Setup

> Up to now Viby has been a beautiful app pretending to have a backend. In this chapter we give it a real one — without breaking the demo for a single second.

What you'll learn

- What **Supabase** actually is: managed Postgres + Auth + Storage + Realtime + Edge, and where each piece fits Viby.
- How to **create a Supabase project**, find your URL and keys, and wire them into env vars safely.
- How to use the **SQL editor** and run our `schema.sql` to stand up every table and Row Level Security policy at once.
- The crucial difference between the **browser client and the server client** with `@supabase/ssr`, and why we need both.
- The **connected-vs-demo mode flag** pattern that lets Viby run with zero backend *or* a full one — no code changes in your components.

Prerequisites

- Chapter 6 (Next.js App Router) and Chapter 8 (Server vs Client Components) — we lean hard on the server/client boundary here.
- Chapter 5 (TypeScript for React) — clients and rows are typed.
- Chapter 11 (Color & theming) only insofar as you have a running Viby app you can `npm run dev`.
- A free Supabase account (we'll create one together) and the Viby repo with `@supabase/ssr` already in `package.json`.

Theory

For the first fourteen chapters, Viby's data lived in a **Zustand store seeded from `lib/mock.ts`** — fake profiles, fake matches, fake messages, all in memory. That was deliberate: it let you build every screen, animation, and interaction without first becoming a database administrator. But an in-memory store forgets everything on refresh, can't be shared between two phones, and has no concept of "you." To ship a real social-discovery app, we need a backend that **persists data, knows who's logged in, stores photos, and pushes live updates**.

You could assemble that yourself: a Postgres server, an auth service, an object store like S3, a websocket layer, plus the glue between them. That's weeks of plumbing before you write a single feature. **Supabase** is that whole stack, managed for you, around a real Postgres database.

Think of Supabase as a **fully-furnished apartment versus an empty lot**. The empty lot (raw cloud infrastructure) is yours to build on, but you're pouring the foundation and running the pipes. The furnished

apartment gives you a working kitchen, plumbing, and locks on the door — you move in and start living. The key word is *managed*: someone else handles backups, scaling, and security patches. The other key word is **Postgres** — underneath the convenience, you have a real, standard SQL database you can take anywhere. No lock-in to a proprietary data model.

The five pieces

Supabase bundles five services around your database:

- **Postgres** — the database itself. Tables, relationships, indexes, SQL. This is the source of truth for every Viby profile, swipe, match, and message.
- **Auth** — sign-up, login, sessions, OAuth providers, magic links. Critically, it issues a `user.id` (a UUID) that your tables reference. We covered the *concept* of identity earlier; here it becomes real.
- **Storage** — an object store for files (think avatars and post images). It's S3-style buckets with the same auth rules as your database.
- **Realtime** — a websocket layer that streams database changes to subscribed clients. When someone sends a Viby message, every device in that conversation hears about it instantly.
- **Edge Functions** — serverless functions for logic that shouldn't live in the client (webhooks, payments, AI calls). We won't need them this chapter, but they're there.

The thing that ties Storage, Realtime, and your queries together — and the thing beginners most often skip — is **Row Level Security (RLS)**. RLS lets you write authorization *as SQL policies on the table itself*. "A user can read any profile but only update their own." "Only members of a conversation can read its messages." Once a policy is on, the database enforces it for **every** client, even a leaked anon key. Your security doesn't live in your app code where a bug can bypass it; it lives in the database where it can't.

Why two clients?

Here's the part that confuses everyone coming from older tutorials. Next.js 16 runs your code in **two very different environments**: on the server (Server Components, Server Actions, route handlers, the `proxy.ts` request layer) and in the browser (Client Components). Supabase needs to know *who* the request is for so RLS can do its job — and "who you are" is stored in a cookie.

- On the **server**, there's no `window`, no `localStorage`. The session lives in the request's **cookies**, which Next exposes through `next/headers`. The server client reads and writes those cookies.
- In the **browser**, there's no access to Next's cookie store; the client manages the session through the document's cookies directly.

So `@supabase/ssr` gives us two factory functions — `createBrowserClient` and `createServerClient` — and our job is to wrap each in a tiny helper. Use the wrong one in the wrong place and auth silently breaks: you'll be logged in on the client and anonymous on the server, or vice versa. Match the client to the environment and the session flows seamlessly across both.

In practice

1. Create the project

1. Go to supabase.com, sign in, and click **New project**.
2. Pick an organization, name it `viby`, and set a strong **database password** (save it — it's the Postgres superuser password).
3. Choose a region close to your users.
4. Wait ~2 minutes while it provisions.

Then open **Project Settings** → **API**. You need two values:

- **Project URL** — `https://<your-ref>.supabase.co`
- **anon (public) key** — a JWT, safe to expose to the browser *because RLS protects your data*.

There's also a **service_role key**. It **bypasses RLS** — treat it like a root password. Never put it in a `NEXT_PUBLIC_` variable, never ship it to the browser. We won't use it this chapter.

2. Set environment variables

Create `.env.local` in the Viby root (it's already in `.gitignore` — never commit secrets):

```
# .env.local
NEXT_PUBLIC_SUPABASE_URL=https://your-ref.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6...
```

The `NEXT_PUBLIC_` prefix is a Next.js convention: it inlines the value into the browser bundle. That's exactly what we want for these two — they're meant to be public. Anything *without* the prefix stays server-only. Restart `npm run dev` after editing env files; they're read at boot, not hot-reloaded.

3. Run the schema in the SQL editor

In the dashboard, open the **SQL Editor** (left sidebar). This is a browser-based query console wired straight to your Postgres database — the same place you'll later debug queries and inspect data.

Open `supabase/schema.sql` from the Viby repo, paste the whole thing in, and click **Run**. One click stands up every table, enum, index, RLS policy, the Realtime publication, and a trigger that auto-creates a profile on signup. Here's the shape of it:

```
-- supabase/schema.sql (excerpt)
create table if not exists profiles (
  id            uuid primary key references auth.users on delete cascade,
  username      text unique not null,
  display_name  text not null,
  bio           text,
  avatar_url    text,
  interests     text[] default '{}',   -- denormalized for fast discovery scoring
  city          text,
  onboarded     boolean default false,
  created_at    timestamptz default now()
);

-- Turn RLS ON, then say exactly who can do what:
alter table profiles enable row level security;
create policy "profiles readable" on profiles
  for select to authenticated using (true);
create policy "edit own profile" on profiles
  for update to authenticated using (auth.uid() = id);
```

Two details worth internalizing. First, `id` references `auth.users` — every profile is *owned* by an auth user, and `on delete cascade` means deleting the account cleans up the profile. Second, `auth.uid()` is a Postgres function Supabase provides that returns the current request's user ID; it's how a policy knows "this row is yours."

The schema also auto-creates a profile whenever someone signs up, so your app never has to remember to do it:

```
create or replace function public.handle_new_user()
returns trigger language plpgsql security definer set search_path = public as $$
begin
  insert into public.profiles (id, username, display_name)
  values (new.id, split_part(new.email, '@', 1),
          coalesce(new.raw_user_meta_data->>'name', 'New member'));
  return new;
end $$;

create trigger on_auth_user_created
  after insert on auth.users
  for each row execute function public.handle_new_user();
```

After running it, open the **Table Editor** — you should see `profiles`, `swipes`, `matches`, `posts`, `messages`, `communities`, `events`, and friends. The lot is now furnished.

4. The two clients

The browser client (`lib/supabase/client.ts`) is tiny — it reads the public env vars and lets `@supabase/ssr` manage the session via document cookies:

```
// lib/supabase/client.ts  — used inside "use client" components
import { createBrowserClient } from "@supabase/ssr";

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL ?? "",
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY ?? "",
  );
}
```

The server client (`lib/supabase/server.ts`) is the one to study. In Next.js 16, `cookies()` is **async**, so the factory is `async` and we `await` it. We hand Supabase `getAll` / `setAll` cookie handlers so it can read the session and refresh tokens:

```
// lib/supabase/server.ts  — used in Server Components, Server Actions, route handlers
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL ?? "",
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY ?? "",
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) => {
          try {
            toSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options),
            );
          } catch {
            // Called from a Server Component (read-only cookies) —
            // safe to ignore because proxy.ts refreshes the session.
          }
        },
      },
    },
  );
}
```

That `try/catch` matters: a Server Component **can't** write cookies (the response headers are already streaming). So who keeps the session fresh? Our `proxy.ts` — Next 16's replacement for `middleware.ts`. It runs on every request, touches `getUser()` to refresh the token, and writes the cookie back on the *response*:

```
// proxy.ts  (Next 16 renamed middleware → proxy)
import { createServerClient } from "@supabase/ssr";
import { NextResponse, type NextRequest } from "next/server";

export async function proxy(request: NextRequest) {
  const url = process.env.NEXT_PUBLIC_SUPABASE_URL;
  const key = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;
  if (!url || !key) return NextResponse.next(); // demo mode: do nothing

  let response = NextResponse.next({ request });
  const supabase = createServerClient(url, key, {
    cookies: {
      getAll: () => request.cookies.getAll(),
      setAll: (cookiesToSet) => {
        cookiesToSet.forEach(({ name, value }) => request.cookies.set(name, value));
        response = NextResponse.next({ request });
        cookiesToSet.forEach(({ name, value, options }) =>
          response.cookies.set(name, value, options),
        );
      },
    },
  });

  await supabase.auth.getUser(); // refresh if needed
  return response;
}

export const config = {
  matcher: ["/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*)"],
};
```

5. The connected-vs-demo flag

Here's the pattern that keeps the whole book honest. We never want a missing env var to crash Viby — beginners should be able to clone, `npm run dev`, and click everything. So we have **one source of truth**:

```
// lib/supabase/config.ts
export const hasSupabase =
  !!process.env.NEXT_PUBLIC_SUPABASE_URL &&
  !!process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;
```

Every data function branches on it. If `hasSupabase` is false, fall back to the mock store; otherwise hit Postgres. Because both branches return the **same typed shape**, your components never know or care which

mode they're in:

```
// lib/queries/profiles.ts  (excerpt, simplified)
import { createClient } from "@/lib/supabase/server";
import { hasSupabase } from "@/lib/supabase/config";
import { mockProfiles } from "@/lib/mock";
import type { Profile } from "@/lib/types";

export async function getProfileById(id: string): Promise<Profile | null> {
  if (!hasSupabase) {
    return mockProfiles.find((p) => p.id === id) ?? null;
  }
  const supabase = await createClient();
  const { data } = await supabase
    .from("profiles")
    .select("id, username, display_name, bio, avatar_url, interests, city")
    .eq("id", id)
    .maybeSingle();
  return data ? mapProfileRow(data) : null; // map DB row → app Profile
}
```

The `mapProfileRow` helper converts snake_case DB columns into the camelCase `Profile` type your UI already expects, so swapping from demo to connected mode is invisible above this layer. That's the payoff: the same Viby, two backends.

Architecture / diagram

```
flowchart TD
  subgraph Browser["Browser (Client Components)"]
    CC["use client component"] -->|createBrowserClient| BC["Supabase browser client"]
  end

  subgraph Edge["proxy.ts (every request)"]
    PX["refresh session\nvia getUser()"]
  end

  subgraph Server["Next.js server (RSC / Server Actions)"]
    RSC["Server Component"] -->|createServerClient + cookies| SC["Supabase server client"]
    Q["lib/queries/* checks hasSupabase"]
    RSC --> Q
  end

  Q -->|demo mode| MOCK["Zustand store + lib/mock.ts"]
  BC -->|HTTPS + anon key| SUPA
  SC -->|HTTPS + anon key| SUPA
  PX -->|cookies| SUPA

  subgraph SUPA["Supabase project"]
    AUTH["Auth (user.id)"] --> PG[("Postgres + RLS")]
    STORAGE["Storage buckets"]
    RT["Realtime (websockets)"]
    PG --> RT
  end
```

The anon key is the *same* on both clients; RLS — driven by `auth.uid()` from the session cookie — is what decides what each request may actually see.

Best practices & pitfalls

- **Do** keep the `service_role` key server-only and out of the repo. If it leaks, rotate it immediately — it ignores RLS entirely.
- **Do** turn on RLS for every table that holds user data *and* write policies. RLS with no policies = nobody can read anything; RLS off = everybody can read everything. Both are common first-day mistakes.
- **Do** use the server client for anything that runs on the server (RSC, Server Actions, `proxy.ts`) and the browser client only inside `"use client"`. Mixing them breaks the session silently.
- **Don't** instantiate a single shared client at module top-level for the server — create one per request inside `await createClient()` so cookies are scoped correctly.

- **Don't** forget the `try/catch` in the server client's `setAll`; without `proxy.ts` refreshing sessions, logins will expire and feel "randomly logged out."
- **Don't** assume env vars hot-reload — restart the dev server after editing `.env.local`.
- **Do** keep `hasSupabase` as the *only* place you check for configuration, so demo mode never rots.

Exercises

1. **Smoke test.** Create the project, set `.env.local`, restart dev, and add a temporary Server Component page that calls `await createClient()` then `supabase.from("profiles").select("*")`. Log the result. You should get an empty array (not an error) — proof RLS and the connection both work.
2. **Read your own row.** In the SQL editor, manually insert one row into `profiles` with a real `auth.users` id (create a test user under **Authentication** → **Users** first). Confirm your page now returns it.
3. **Break and fix RLS.** Temporarily `drop policy "profiles readable" on profiles;` and reload the page. Watch the row disappear (RLS denies by default). Re-create the policy and confirm it returns. Internalize *deny-by-default*.
4. **Flag flip.** Comment out the env vars, restart, and confirm Viby still runs entirely on mock data with no crash. Then restore them. The `hasSupabase` branch is doing its job.
5. **(Stretch)** Add a `getCommunities()` query in `lib/queries/communities.ts` following the `profiles.ts` pattern: mock branch when `!hasSupabase`, real `select` otherwise, mapping rows to your app type.

Mini-project: "Backend health" badge

Build a small server-rendered badge that proves, at a glance, which mode Viby is in.

1. Create `components/backend-badge.tsx` as a **Server Component**.
2. Import `hasSupabase`. If false, render a pink `Demo mode` pill.
3. If true, `await createClient()`, run `const { error } = await supabase.from("profiles").select("id").limit(1);` and render `Connected` (green) when there's no error, or `Config error` (red) when there is.
4. Drop `<BackendBadge />` into your root layout's footer.

```
// components/backend-badge.tsx
import { hasSupabase } from "@/lib/supabase/config";
import { createClient } from "@/lib/supabase/server";

export default async function BackendBadge() {
  if (!hasSupabase) {
    return <span className="rounded-full bg-[--color-brand]/10 px-3 py-1 text-xs text-[--color-
brand]">Demo mode</span>;
  }
  const supabase = await createClient();
  const { error } = await supabase.from("profiles").select("id").limit(1);
  const ok = !error;
  return (
    <span className={`rounded-full px-3 py-1 text-xs ${ok ? "bg-green-100 text-green-700" : "bg-red-100
text-red-700"}`}>
      {ok ? "Connected" : "Config error"}
    </span>
  );
}
```

You now have a one-glance answer to "is the backend wired up?" — invaluable as we start migrating screens off mock data.

Key takeaways

- Supabase is **managed Postgres** with Auth, Storage, Realtime, and Edge bolted on — and a real SQL database underneath, so no lock-in.
- **RLS is your authorization layer**: it lives in the database, enforces deny-by-default, and protects you even though the anon key is public.
- Next.js 16 needs **two clients** — `createServerClient` (cookies via `next/headers`, async) for the server, `createBrowserClient` for `"use client"` — and `proxy.ts` keeps the session fresh.
- One `schema.sql` run in the **SQL editor** stands up the entire Viby data model in a single click.
- The `hasSupabase` **flag** lets Viby run demo-only or fully connected with identical component code — that's the pattern we'll lean on for the rest of Section 4.

Up next — Chapter 16: Authentication with Supabase Auth, where we turn `auth.uid()` from an abstraction into a real signed-in human, and finally give Viby members their own identity.

Chapter 16 — Authentication, Sessions & Row-Level Security

> Auth isn't a login screen — it's a chain of trust that runs from the user's browser all the way down to a single `WHERE` clause in your database.

What you'll learn

- How **Google OAuth** and **email OTP** (passwordless one-time codes) actually work, and how to wire both into Viby with `@supabase/ssr`.
- What the **auth callback route** is for, and why OAuth needs a server-side handshake.
- How **session refresh** works across requests using Next.js 16's `proxy.ts` — and why your app silently logs people out without it.
- How to **protect routes** with the `(app)` route-group layout, and how to gate un-onboarded users.
- How **Row-Level Security (RLS)** turns Postgres itself into your authorization layer, so a leaked API key can't dump your users' messages.

Prerequisites

- Chapter 6 (Next.js App Router) and Chapter 7 (routing, layouts, route groups) — you should know what `layout.tsx` and a `(group)` folder do.
- Chapter 8 (Server vs Client Components) — we lean hard on the server/client boundary and Server Actions.
- Chapter 15 (Supabase setup) — you should have a Supabase project and the `lib/supabase/*` clients in place.
- A Viby checkout with `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` set (see `.env.example`). Without them, Viby runs in **demo mode** on Zustand — every snippet below guards on `hasSupabase`.

Theory

The "why" before the "how"

Authentication answers **"who are you?"** Authorization answers **"what are you allowed to touch?"** Beginners conflate them, and that's exactly how data leaks happen: they verify the login, then trust the client to ask only for its own rows. The client is never trustworthy. Anyone can open DevTools, grab your anon key (it's public — it ships to the browser), and hit your database directly.

So Viby splits the job in two:

1. **Sessions** prove identity. A signed token (JWT) lives in an HTTP-only cookie. Every request carries it; the server verifies it.
2. **Row-Level Security** enforces permission *inside Postgres*. Even with a valid token, the database refuses to return rows you don't own — no application code required.

Think of a members-only club. The **session cookie** is your wristband: the bouncer checks it once at the door (and the wristband is hard to forge). **RLS** is the rule that every locker only opens for the person whose name is on it — enforced by the lockers themselves, not by trusting members to be polite. Even if someone sneaks past the door, the lockers stay shut.

Two ways in: OAuth vs. OTP

Viby offers two front doors, and they have fundamentally different shapes:

- **Google OAuth** is a *redirect dance*. We never see the user's Google password. We bounce them to Google, Google authenticates them, and Google bounces them back to us carrying a short-lived `code`. We exchange that `code` for a session on the **server**. That exchange is why we need a dedicated callback route — the code is single-use and must be redeemed server-side so the session cookie gets set on our domain.
- **Email OTP** (one-time passcode) is *passwordless*. The user types their email, Supabase emails a 6-digit code, the user types it back, and we verify it. No passwords to leak, no password reset flow to build. This is the friendliest possible auth for a Gen-Z social app — and it's why Viby leads with "No passwords ever."

Why sessions need *refreshing*

A session token is deliberately short-lived (Supabase access tokens expire in ~1 hour). That limits the blast radius if a token leaks. But you don't want users re-logging-in every hour, so there's also a long-lived **refresh token**. Someone has to trade the expiring access token for a fresh one — quietly, on every request — or your Server Components will start seeing `null` users halfway through a session.

In the browser, the Supabase client does this automatically. But Server Components **can't set cookies** after rendering starts. So we need a piece of code that runs *before* the request reaches your pages, refreshes the token if it's stale, and writes the new cookie back. In Next.js 16 that's `proxy.ts` (the file formerly known as `middleware.ts`).

In practice

1. The two Supabase clients

Everything starts with the right client for the right environment. Viby keeps a tiny `hasSupabase` flag so the app degrades gracefully to demo mode.

```ts
// lib/supabase/config.ts
export const hasSupabase =
  !!process.env.NEXT_PUBLIC_SUPABASE_URL &&
  !!process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;
```

```ts
// lib/supabase/client.ts  — runs in the browser
import { createBrowserClient } from "@supabase/ssr";

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL ?? "",
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY ?? "",
  );
}
```

```ts
// lib/supabase/server.ts  — runs in RSC / route handlers / server actions
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies(); // Next 16: cookies() is async — await it
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL ?? "",
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY ?? "",
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) => {
          try {
            toSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options),
            );
          } catch {
            // Called from a Server Component, where cookies are read-only.
            // Safe to ignore — proxy.ts refreshes the session instead.
          }
        },
      },
    },
  );
}
```

The `getAll` / `setAll` pair is the modern `@supabase/ssr` contract. **Do not** use the old single-cookie `get` / `set` / `remove` handlers — they're removed in current versions and cause subtle session-loss bugs.

2. The login screen — OAuth and OTP

The login page is a Client Component because it owns interactive state (which step, the email, the six code digits). Here are the three real handlers from `app/login/page.tsx`, trimmed to the auth essentials:

```
"use client";
import { createClient } from "@/lib/supabase/client";
import { hasSupabase } from "@/lib/supabase/config";

// Google OAuth — redirect dance. Note redirectTo points at our callback route.
async function signInGoogle() {
  if (!hasSupabase) return finish(); // demo fallback
  const { error } = await createClient().auth.signInWithOAuth({
    provider: "google",
    options: { redirectTo: `${window.location.origin}/auth/callback` },
  });
  if (error) setError(error.message);
}

// Email OTP — step 1: send the code.
async function sendCode() {
  if (!hasSupabase) return setStep("otp");
  const { error } = await createClient().auth.signInWithOtp({
    email,
    options: {
      shouldCreateUser: true, // create the account on first login
      emailRedirectTo: `${window.location.origin}/auth/callback`,
    },
  });
  if (error) setError(error.message);
  else setStep("otp");
}

// Email OTP — step 2: verify the 6-digit code.
async function verify(token: string) {
  if (!hasSupabase) return finish();
  const { error } = await createClient().auth.verifyOtp({
    email,
    token,
    type: "email",
  });
  if (error) return setError(error.message);
  router.push("/discover");
}
```

Two things to notice. OAuth **redirects the whole browser** away to Google, so `signInWithOAuth` never resolves to a logged-in user here — the user comes back through the callback route. OTP, by contrast, completes *in place*: `verifyOtp` sets the session cookie and we navigate to `/discover` ourselves.

3. The auth callback route

When Google (or a clicked magic-link) sends the user back, they land here with a `?code=...`. We exchange it for a session — server-side — then redirect into the app.

```
// app/auth/callback/route.ts
import { NextResponse } from "next/server";
import { createClient } from "@/lib/supabase/server";
import { hasSupabase } from "@/lib/supabase/config";

export async function GET(request: Request) {
  const { searchParams, origin } = new URL(request.url);
  const code = searchParams.get("code");
  const next = searchParams.get("next") ?? "/discover";

  if (code && hasSupabase) {
    const supabase = await createClient();
    await supabase.auth.exchangeCodeForSession(code); // sets the cookie
  }

  return NextResponse.redirect(`${origin}${next}`);
}
```

Because this is a Route Handler (not a Server Component), it's allowed to *write* cookies — so `exchangeCodeForSession` can persist the session. We deliberately don't decide onboarding here; the `(app)` layout owns that, so there's exactly one source of truth.

4. Session refresh with `proxy.ts`

This is the unsung hero. It runs before every matching request, refreshes the token if it's about to expire, and writes the fresh cookie onto the response.

```
// proxy.ts  (project root — Next 16's replacement for middleware.ts)
import { createServerClient } from "@supabase/ssr";
import { NextResponse, type NextRequest } from "next/server";

export async function proxy(request: NextRequest) {
  const url = process.env.NEXT_PUBLIC_SUPABASE_URL;
  const key = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;
  if (!url || !key) return NextResponse.next(); // demo mode: no-op

  let response = NextResponse.next({ request });

  const supabase = createServerClient(url, key, {
    cookies: {
      getAll: () => request.cookies.getAll(),
      setAll: (cookiesToSet) => {
        cookiesToSet.forEach(({ name, value }) =>
          request.cookies.set(name, value),
        );
        response = NextResponse.next({ request });
        cookiesToSet.forEach(({ name, value, options }) =>
          response.cookies.set(name, value, options),
        );
      },
    },
  });

  // Touching getUser() validates the token and refreshes it if needed.
  await supabase.auth.getUser();
  return response;
}

export const config = {
  matcher: [
    // Everything except static assets and images.
    "/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*)",
  ],
};
```

The pattern looks fiddly but it's load-bearing: you must write refreshed cookies onto **both** the incoming `request` (so the rest of this request sees them) **and** the outgoing `response` (so the browser stores them). The `let response` reassignment inside `setAll` is exactly that. Calling `getUser()` is the trigger that performs the refresh — never call `getSession()` here for auth decisions (more on that in pitfalls).

5. Protecting routes with the `(app)` layout

Viby's authenticated screens all live under the `(app)` route group, so a single async layout guards every one of them. It also enforces the **onboarding redirect**: a signed-in user who hasn't finished their profile gets bounced to `/onboarding`.

```
// app/(app)/layout.tsx
import { redirect } from "next/navigation";
import { DeviceFrame } from "@/components/shell/DeviceFrame";
import { TabBar } from "@/components/shell/TabBar";
import { hasSupabase } from "@/lib/supabase/config";
import { createClient } from "@/lib/supabase/server";

export default async function AppLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  if (hasSupabase) {
    const supabase = await createClient();
    const {
      data: { user },
    } = await supabase.auth.getUser();
    if (!user) redirect("/login"); // gate 1: must be signed in

    const { data: profile } = await supabase
      .from("profiles")
      .select("onboarded")
      .eq("id", user.id)
      .maybeSingle();
    if (!profile?.onboarded) redirect("/onboarding"); // gate 2: must be onboarded
  }

  return (
    <DeviceFrame>
      <main className="no-scrollbar relative flex-1 overflow-y-auto overflow-x-hidden">
        {children}
      </main>
      <TabBar />
    </DeviceFrame>
  );
}
```

Two layers of defense here: `proxy.ts` keeps the session fresh, but this layout makes the *authorization decision*. `proxy.ts` alone is not enough protection — it can't reliably redirect for every page, and security checks belong as close to the data as possible.

6. Finishing onboarding with a Server Action

When the user completes onboarding, a Server Action writes their profile and flips `onboarded`. The `"use server"` directive marks the whole module as server-only — it never ships to the browser.

```ts
// app/actions/auth.ts
"use server";
import { redirect } from "next/navigation";
import { createClient } from "@/lib/supabase/server";
import { hasSupabase } from "@/lib/supabase/config";

export async function saveOnboarding(input: OnboardingInput) {
  if (!hasSupabase) return { ok: true as const };

  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) return { ok: false as const, error: "Not signed in" };

  const { error } = await supabase.from("profiles").upsert({
    id: user.id, // RLS requires this to equal auth.uid()
    username: (user.email?.split("@")[0] ?? `viby_${user.id.slice(0, 6)}`).toLowerCase(),
    display_name: input.displayName,
    bio: input.bio,
    interests: input.interests,
    vibe_tags: input.vibeTags,
    city: input.city,
    photos: input.photos,
    onboarded: true,
  });

  return error ? { ok: false as const, error: error.message } : { ok: true as const };
}

export async function signOut() {
  if (hasSupabase) {
    const supabase = await createClient();
    await supabase.auth.signOut();
  }
  redirect("/");
}
```

Notice the action re-derives `user` server-side rather than trusting any id sent from the client. That id then has to match `auth.uid()` for RLS to allow the write — belt and suspenders.

7. Row-Level Security — Postgres is the last line of defense

Here's the part that separates a toy from a shippable product. Enabling RLS on a table means **deny by default**: no row is readable or writable until a policy explicitly allows it. From Viby's `supabase/schema.sql`:

```
-- Lock the tables down first.
alter table profiles enable row level security;
alter table swipes   enable row level security;
alter table messages enable row level security;

-- Profiles: any logged-in user can READ any profile (discovery needs this)…
create policy "profiles readable" on profiles
  for select to authenticated using (true);

-- …but you may only UPDATE / INSERT your own row.
create policy "edit own profile" on profiles
  for update to authenticated using (auth.uid() = id);
create policy "insert own profile" on profiles
  for insert to authenticated with check (auth.uid() = id);

-- Swipes are private to the swiper — read AND write.
create policy "own swipes" on swipes for all to authenticated
  using (auth.uid() = swiper_id) with check (auth.uid() = swiper_id);

-- Messages: only members of the conversation can read it…
create policy "conv messages readable" on messages for select to authenticated using (
  exists (
    select 1 from conversation_members m
    where m.conversation_id = messages.conversation_id
      and m.profile_id = auth.uid()
  )
);
-- …and you can only send as yourself.
create policy "send as self" on messages for insert to authenticated
  with check (auth.uid() = sender_id);
```

`auth.uid()` is the magic: Supabase reads the JWT from the request and exposes the user's id to every policy. `using (...)` controls which rows are **visible** (reads, and the pre-image of updates/deletes); `with check (...)` controls which rows you're **allowed to write**. A `for all` policy with both clauses covers select/insert/update/delete in one shot.

The chat policy is the showpiece: even though `messages` is readable, the subquery means you only ever see messages from conversations *you're a member of*. That rule lives in the database. No leaked anon key, no buggy query, no malicious client can route around it.

Finally, Viby auto-creates a profile row the moment a user signs up, via a trigger:

```
create or replace function public.handle_new_user()
returns trigger language plpgsql security definer set search_path = public as $$
begin
  insert into public.profiles (id, username, display_name)
  values (
    new.id,
    split_part(new.email, '@', 1),
    coalesce(new.raw_user_meta_data->>'name', 'New member')
  );
  return new;
end $$;

create trigger on_auth_user_created
  after insert on auth.users
  for each row execute function public.handle_new_user();
```

`security definer` lets the trigger insert into `profiles` even though the brand-new user can't yet — it runs with the function owner's rights. The `set search_path = public` guard is mandatory here to prevent a known privilege-escalation attack.

Architecture / diagram

```
sequenceDiagram
    participant B as Browser (login page)
    participant G as Google
    participant CB as /auth/callback (route handler)
    participant PX as proxy.ts
    participant L as (app)/layout.tsx
    participant DB as Postgres + RLS

    B->>G: signInWithOAuth → redirect
    G-->>CB: redirect back with ?code
    CB->>CB: exchangeCodeForSession(code) → sets cookie
    CB-->>B: redirect to /discover
    B->>PX: GET /discover (cookie attached)
    PX->>PX: getUser() refreshes token, rewrites cookie
    PX->>L: forward request
    L->>DB: getUser() + select onboarded
    DB-->>L: rows filtered by auth.uid() (RLS)
    alt no user
        L-->>B: redirect /login
    else not onboarded
        L-->>B: redirect /onboarding
    else ok
        L-->>B: render the app
    end
```

Best practices & pitfalls

- **DO** use `supabase.auth.getUser()` for any auth decision on the server. It re-validates the JWT against Supabase. **DON'T** use `getSession()` for security — it trusts the cookie's contents without verification and can be spoofed.
- **DO** keep `proxy.ts` thin and let it call `getUser()`. **DON'T** put your authorization logic *only* in `proxy.ts` — always also guard in the layout/page closest to the data.
- **DO** enable RLS on every table that holds user data and treat the anon key as fully public. **DON'T** ever ship the `service_role` key to the browser — it bypasses RLS entirely.
- **DON'T** forget `with check` on insert/update policies. A `using`-only policy lets a user *change a row's owner to someone else* on update. Always pair them.
- **DO** point both `redirectTo` (OAuth) and `emailRedirectTo` (OTP) at `/auth/callback`, and register that exact URL in Supabase → Auth → URL Configuration. Mismatched redirect URLs are the #1 cause of "it works locally but not on Vercel."
- **DO** return the same generic message for "wrong code" and "unknown email" — don't let attackers enumerate which emails have accounts.

- **DON'T** read or write cookies from a Server Component and expect it to persist — it can't. That's the entire reason `proxy.ts` exists.

Exercises

1. **Sign-out button.** Wire the `signOut` Server Action into a button in `TopBar`. Confirm that after sign-out, visiting `/discover` redirects you to `/login`.
2. **A `next` param.** Make the login flow remember where the user was headed: pass `?next=/communities` through `signInWithOtp`'s `emailRedirectTo`, read it in the callback, and land them there instead of `/discover`.
3. **Resend cooldown.** Add a 30-second cooldown to the OTP "Send code" button so users can't spam Supabase's rate limit. Show a live countdown with Framer Motion.
4. **RLS for comments.** The `comments` table has RLS enabled but no policies yet — so it's currently locked shut. Write `select` (readable to all authenticated), `insert` (`auth.uid() = author_id`), and `delete` (own comments only) policies, mirroring the `posts` policies.
5. **Prove RLS works.** In the Supabase SQL editor, run a `select * from swipes` as a specific user (use `set local role` / a test JWT). Confirm you only see that user's swipes, then try to insert a swipe with someone else's `swiper_id` and watch it get rejected.

Mini-project: a "Who can see this?" debug panel

Build a small `/settings/security` page (inside `(app)`) that makes RLS *visible* to the learner.

- A Server Component reads the current `user` via `createClient()` and shows their id and email.
- Below it, render three live queries and their results: `profiles` (count returned), `swipes` (count — should equal *only this user's* swipes), and `messages` (count — only conversations they belong to).
- Add a deliberately "naughty" Server Action that tries to `update` someone *else's* profile (`.eq("id", SOME_OTHER_ID)`) and display the result. It should silently affect **0 rows** — proof that RLS, not your code, stopped it.
- Style it with the bubblegum-pink card aesthetic and animate each result row in with a Framer Motion `stagger`.

The payoff: a learner sees, in their own UI, that the database refused to leak or mutate data even when the app *asked it to*. That's the moment RLS stops being abstract.

Key takeaways

- Authentication (who you are) and authorization (what you can touch) are **separate jobs** — Viby solves them with sessions and RLS respectively.
- **OAuth** needs a server-side callback route to exchange a `code` for a session; **email OTP** is passwordless and completes in place via `verifyOtp`.

- `proxy.ts` is Next 16's middleware: it refreshes the Supabase session on every request so Server Components always see a valid user.
- The `(app)` layout is your single, server-side authorization checkpoint — and your onboarding gate.
- Always verify identity with `getUser()`, never trust `getSession()` for security, and never ship the `service_role` key.
- **RLS is the real guarantee.** With `using` + `with check` policies and `auth.uid()`, Postgres itself refuses to leak or mutate rows you don't own — even if your app code is buggy or your anon key is public.

Up next — Chapter 17: Building Viby's Data Layer with Supabase Queries & Server Actions, where we turn these locked-down tables into typed, RLS-safe reads and writes that power Discover, the feed, and chat.

Chapter 17 — CRUD with Server Components & Server Actions

> Your data layer is the spine of Viby. In this chapter you'll give it real muscle — reading on the server, mutating with Server Actions, and snapping the UI to instant with optimistic updates.

What you'll learn

- How to **read data** directly in Server Components — no `useEffect`, no client fetch, no loading spinner gymnastics.
- How to **mutate data** safely with `"use server"` Server Actions, and refresh the screen with `revalidatePath`.
- How to make edits feel instant with **optimistic UI** (`useOptimistic` + `useActionState`).
- How to build a **typed query layer** that isolates Supabase from your components, with **row → model mappers** so your app never touches a raw DB row.
- The Viby patterns `getProfileById` and `saveProfile`, end to end.

Prerequisites

- **Chapter 9** — TypeScript types & interfaces (we lean hard on typed models).
- **Chapter 14** — Supabase setup with `@supabase/ssr` (the `createServerClient` / `createBrowserClient` helpers).
- **Chapter 15** — Auth & sessions (we read the logged-in user inside actions).
- **Chapter 16** — Database schema & RLS for the `profiles` table.

Theory

For the whole frontend half of this book, data lived in the browser. A component mounted, fired a `fetch`, flipped a `loading` flag, and eventually rendered. That's the **client-fetch** model, and it has three taxes: a waterfall (HTML arrives, *then* JS, *then* data), a spinner the user stares at, and your database credentials nervously close to the browser.

The App Router flips this. **Server Components render on the server**, where your code is already millimeters from the database. So a component can just... ask for data and `await` it. No state, no effect, no spinner. Think of it like the difference between **mailing a request form and waiting for a reply** (client fetch) versus **walking into the back office where the filing cabinet lives** (server component). One of these is dramatically faster, and the keys to the cabinet never leave the building.

That handles *reading*. But a social app is all about *writing* — editing your profile, posting a vibe, joining a circle. For that, React and Next.js give us **Server Actions**: functions you mark with the `"use server"` directive. They run **only on the server**, but you can call them straight from a `<form>` or a button as if they

were local. Under the hood Next.js wires up a secure POST endpoint for you. The analogy: a Server Action is a **trusted back-office clerk you can summon from the front desk**. The customer (browser) never walks into the back; they hand the clerk a slip, the clerk does the privileged work, and comes back with a result.

After a mutation, the server's cached render of a page is stale. `revalidatePath("/profile")` tells Next.js: "*throw away the cached version of this route and re-render it with fresh data.*" The next request — or the current one, since Next.js streams the fresh tree back — sees the new data automatically.

Finally, **optimistic UI**. Even a fast round-trip is ~200ms, and on mobile that's a perceptible lag. Optimistic UI means we **assume success and update the screen immediately**, then reconcile when the server answers. If the server rejects, React rolls the optimistic state back. It's tipping the waiter before the meal arrives — almost always right, and gracefully reversible when it isn't.

One more principle that separates hobby code from startup-grade code: **never let components talk to Supabase directly.** Put a **query layer** between them. Components ask for a `Profile`; the query layer knows about tables, columns, snake_case, and joins, and hands back a clean typed **model**. The translator between a raw DB row and a model is a **mapper**. This single boundary is what lets you refactor your schema later without touching a single component.

In practice

1. The typed model and the mapper

Database rows are `snake_case` and shaped by storage concerns. Our app wants `camelCase` domain models. Keep them in separate files so the seam is obvious.

```ts
// lib/types/profile.ts

// The domain model — what the rest of Viby sees.
export interface Profile {
  id: string;
  username: string;
  displayName: string;
  bio: string;
  avatarUrl: string | null;
  hobbies: string[];
  city: string | null;
  createdAt: string;
}

// Exactly the row shape returned by Supabase (snake_case, nullable).
export interface ProfileRow {
  id: string;
  username: string;
  display_name: string;
  bio: string | null;
  avatar_url: string | null;
  hobbies: string[] | null;
  city: string | null;
  created_at: string;
}
```

```
// lib/mappers/profile.ts
import type { Profile, ProfileRow } from "@/lib/types/profile";

// row -> model: the ONE place snake_case and nulls get normalized.
export function rowToProfile(row: ProfileRow): Profile {
  return {
    id: row.id,
    username: row.username,
    displayName: row.display_name,
    bio: row.bio ?? "",
    avatarUrl: row.avatar_url,
    hobbies: row.hobbies ?? [],
    city: row.city,
    createdAt: row.created_at,
  };
}

// model (partial) -> row: used when writing.
export function profileToRow(
  input: Pick<Profile, "displayName" | "bio" | "hobbies" | "city">
): Partial<ProfileRow> {
  return {
    display_name: input.displayName,
    bio: input.bio,
    hobbies: input.hobbies,
    city: input.city,
  };
}
```

2. The query layer

This module is the only thing in Viby that imports the Supabase client for `profiles`. It returns models, never rows. (The `createClient` helper is the `createServerClient` wrapper you built in Chapter 14.)

```ts
// lib/queries/profiles.ts
import "server-only"; // hard guarantee: this never ships to the browser
import { createClient } from "@/lib/supabase/server";
import { rowToProfile, profileToRow } from "@/lib/mappers/profile";
import type { Profile, ProfileRow } from "@/lib/types/profile";

export async function getProfileById(id: string): Promise<Profile | null> {
  const supabase = await createClient();
  const { data, error } = await supabase
    .from("profiles")
    .select("id, username, display_name, bio, avatar_url, hobbies, city, created_at")
    .eq("id", id)
    .maybeSingle();

  if (error) throw new Error(`getProfileById failed: ${error.message}`);
  return data ? rowToProfile(data as ProfileRow) : null;
}

export async function saveProfile(
  id: string,
  patch: Pick<Profile, "displayName" | "bio" | "hobbies" | "city">
): Promise<Profile> {
  const supabase = await createClient();
  const { data, error } = await supabase
    .from("profiles")
    .update(profileToRow(patch))
    .eq("id", id)
    .select("id, username, display_name, bio, avatar_url, hobbies, city, created_at")
    .single();

  if (error) throw new Error(`saveProfile failed: ${error.message}`);
  return rowToProfile(data as ProfileRow);
}
```

> Note: `import "server-only"` makes the build **fail loudly** if a Client Component ever imports this file by accident. Cheap insurance.

3. Reading in a Server Component

No `"use client"`, no hooks. In Next.js 16, dynamic route `params` is a Promise — `await` it.

```
// app/profile/[id]/page.tsx
import { notFound } from "next/navigation";
import { getProfileById } from "@/lib/queries/profiles";

export default async function ProfilePage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const profile = await getProfileById(id);

  if (!profile) notFound();

  return (
    <main className="mx-auto max-w-md p-6">
      <h1 className="text-2xl font-bold text-brand">{profile.displayName}</h1>
      <p className="text-sm text-neutral-500">@{profile.username}</p>
      <p className="mt-4 text-neutral-800">{profile.bio}</p>
      <ul className="mt-4 flex flex-wrap gap-2">
        {profile.hobbies.map((h) => (
          <li key={h} className="rounded-full bg-brand/10 px-3 py-1 text-sm text-brand">
            {h}
          </li>
        ))}
      </ul>
    </main>
  );
}
```

The brand color comes from your Tailwind v4 theme tokens (Chapter 7), defined in CSS — no `tailwind.config.js`:

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5;
}
```

4. The Server Action

A `"use server"` file. We re-check auth inside the action — **never trust the client** to tell us who it is — and revalidate after writing.

```
// app/profile/[id]/actions.ts
"use server";

import { revalidatePath } from "next/cache";
import { createClient } from "@/lib/supabase/server";
import { saveProfile } from "@/lib/queries/profiles";

export type SaveState = { ok: boolean; message: string };

export async function saveProfileAction(
  _prev: SaveState,
  formData: FormData
): Promise<SaveState> {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();

  if (!user) return { ok: false, message: "You must be signed in." };

  const displayName = String(formData.get("displayName") ?? "").trim();
  const bio = String(formData.get("bio") ?? "").trim();
  const city = String(formData.get("city") ?? "").trim() || null;
  const hobbies = String(formData.get("hobbies") ?? "")
    .split(",")
    .map((h) => h.trim())
    .filter(Boolean);

  if (displayName.length < 2) {
    return { ok: false, message: "Display name needs at least 2 characters." };
  }

  // RLS still enforces that user.id can only update its own row.
  await saveProfile(user.id, { displayName, bio, hobbies, city });

  revalidatePath(`/profile/${user.id}`);
  return { ok: true, message: "Profile saved." };
}
```

5. The form + optimistic UI (Client Component)

`useActionState` wires the form to the action and exposes pending state. `useOptimistic` paints the new value instantly.

```tsx
// app/profile/[id]/edit-form.tsx
"use client";

import { useActionState, useOptimistic } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { saveProfileAction, type SaveState } from "./actions";
import type { Profile } from "@/lib/types/profile";

const initial: SaveState = { ok: false, message: "" };

export function EditProfileForm({ profile }: { profile: Profile }) {
  const [state, formAction, isPending] = useActionState(saveProfileAction, initial);

  // Optimistic display name: updates the heading the instant the user submits.
  const [optimisticName, setOptimisticName] = useOptimistic(profile.displayName);

  return (
    <form
      action={formAction}
      onSubmit={(e) => {
        const fd = new FormData(e.currentTarget);
        setOptimisticName(String(fd.get("displayName") ?? ""));
      }}
      className="space-y-4"
    >
      <h2 className="text-xl font-bold text-brand">{optimisticName}</h2>

      <input
        name="displayName"
        defaultValue={profile.displayName}
        className="w-full rounded-xl border px-4 py-2"
        placeholder="Display name"
      />
      <textarea
        name="bio"
        defaultValue={profile.bio}
        className="w-full rounded-xl border px-4 py-2"
        placeholder="Your vibe in one line"
      />
      <input
        name="city"
        defaultValue={profile.city ?? ""}
        className="w-full rounded-xl border px-4 py-2"
```

```
        placeholder="City"
      />
      <input
        name="hobbies"
        defaultValue={profile.hobbies.join(", ")}
        className="w-full rounded-xl border px-4 py-2"
        placeholder="climbing, vinyl, matcha"
      />

      <motion.button
        type="submit"
        disabled={isPending}
        whileTap={{ scale: 0.96 }}
        className="w-full rounded-xl bg-brand py-3 font-semibold text-white disabled:opacity-60"
      >
        {isPending ? "Saving…" : "Save profile"}
      </motion.button>

      <AnimatePresence>
        {state.message && (
          <motion.p
            initial={{ opacity: 0, y: 4 }}
            animate={{ opacity: 1, y: 0 }}
            exit={{ opacity: 0 }}
            className={state.ok ? "text-green-600" : "text-red-600"}
          >
            {state.message}
          </motion.p>
        )}
      </AnimatePresence>
    </form>
  );
}
```

Because the form `action` is a Server Action, this works **even before JavaScript loads** — progressive enhancement for free. The optimistic name and pending state are the enhancement on top.

Architecture / diagram

```
sequenceDiagram
    participant B as Browser (Client Component)
    participant SA as Server Action ("use server")
    participant Q as Query Layer (server-only)
    participant DB as Supabase (RLS)

    Note over B: User edits & submits form
    B->>B: useOptimistic paints new name instantly
    B->>SA: formAction(formData) [secure POST]
    SA->>SA: getUser() — re-verify identity
    SA->>Q: saveProfile(user.id, patch)
    Q->>Q: profileToRow(patch)  // model -> row
    Q->>DB: UPDATE profiles ... (RLS checks owner)
    DB-->>Q: updated row
    Q->>Q: rowToProfile(row)   // row -> model
    Q-->>SA: Profile
    SA->>SA: revalidatePath("/profile/:id")
    SA-->>B: { ok, message } + fresh server render
    Note over B: Optimistic state confirmed or rolled back
```

The dependency rule, in one line:

```
Server Component / Server Action  ->  Query Layer  ->  Mapper  ->  Supabase
        (knows models only)         (knows tables)   (translates)   (raw rows)
```

Best practices & pitfalls

- **Do** re-authenticate inside every Server Action with `supabase.auth.getUser()`. The form fields are attacker-controlled; the session cookie is your source of truth.
- **Do** keep RLS as your last line of defense even with checks in the action. Defense in depth — never rely on app code alone.
- **Don't** return raw DB rows from the query layer. Always map to a model so components stay decoupled from your schema.
- **Don't** put secrets or the service-role key anywhere a Client Component can import. Use `import "server-only"` on query modules to enforce it.
- **Do** call `revalidatePath` (or `revalidateTag`) after a mutation, or your users will see stale cached data.
- **Don't** `revalidatePath` on a hot path you don't need to refresh — over-revalidating throws away good cache and hurts performance.
- **Do** validate and coerce `FormData` inside the action (it's all strings/ `File`s). Consider Zod for anything non-trivial.

- **Don't** swallow errors silently. Return a typed `{ ok, message }` state so the UI can react.
- **Pitfall:** forgetting that `params` and `searchParams` are Promises in Next.js 16 — `await` them (or `use()` them in a Client Component).
- **Pitfall:** mutating optimistic state outside a transition. `useOptimistic` updates must happen during the action dispatch (as in the `onSubmit` above), not in a random click handler.

Exercises

1. **Read-only warm-up.** Add `getProfileByUsername(username)` to the query layer and a `app/u/[username]/page.tsx` Server Component that renders it, calling `notFound()` when missing.
2. **Validate harder.** Extend `saveProfileAction` to reject a `bio` longer than 160 characters and a `hobbies` list longer than 8 items, returning a clear `message` for each.
3. **Optimistic hobbies.** Add a "remove" button next to each hobby chip that optimistically drops the chip while a Server Action persists the change. Roll back visually if the action returns `ok: false`.
4. **Tag-based revalidation.** Refactor reads to use `revalidateTag("profile:" + id)` instead of `revalidatePath`, and tag the fetch accordingly. Explain in a comment when tags beat paths.
5. **Stretch.** Add an avatar upload: a Server Action that puts a file in Supabase Storage, writes `avatar_url`, and revalidates. Map the public URL in `rowToProfile`.

Mini-project: "Edit My Vibe" profile editor

Ship a complete, production-shaped profile editor for the logged-in user.

Goal: `/profile/me/edit` lets the signed-in user update their display name, bio, city, and hobbies, with instant optimistic feedback and a graceful error state.

Steps:

1. Create `app/profile/me/edit/page.tsx` (Server Component). Read the user with `supabase.auth.getUser()`, then `getProfileById(user.id)`. Redirect to `/login` if there's no user.
2. Pass the loaded `Profile` into the `<EditProfileForm />` Client Component from this chapter.
3. Wire `saveProfileAction` exactly as shown, revalidating `/profile/${user.id}`.
4. Add inline validation messages and the optimistic display-name heading.
5. After a successful save, redirect to the public profile with `redirect(\`/profile/${user.id}\`)` inside the action (note: redirect` throws, so put it after the DB write).

Definition of done: disabling JavaScript still lets you save (progressive enhancement); with JS on, the heading updates instantly and a green confirmation animates in. No component imports Supabase directly — everything flows through the query layer.

Key takeaways

- **Server Components read data directly** — `await` your query, no client fetch, no spinner, credentials stay server-side.
- **Server Actions (`"use server"`) are your safe write path** — callable from forms, always re-verify auth, backed by RLS.
- `revalidatePath` **/** `revalidateTag` refresh the cache so users never see stale data after a mutation.
- **A typed query layer + mappers** (`getProfileById` / `saveProfile` , `rowToProfile` / `profileToRow`) keep components ignorant of your schema — the seam that makes refactors painless.
- **Optimistic UI** with `useOptimistic` + `useActionState` makes Viby feel instant while staying correct.

Up next — Chapter 18: Realtime with Supabase, where Viby comes alive: live presence, instant message delivery, and reactive feeds powered by Supabase Realtime channels.

Chapter 18 — Storage & Realtime Systems

> Until now Viby has been a place you *read*. In this chapter it starts to *breathe* — profile photos upload, and presence dots glow green the instant a friend opens the app.

What you'll learn

- How **Supabase Storage** works: buckets, object paths, and why per-user upload paths are non-negotiable.
- Building a reusable, typed **upload helper** for Viby avatars — including overwrite, public URLs, and old-file cleanup.
- How **Supabase Realtime** delivers three different things over one WebSocket: `postgres_changes`, **Broadcast**, and **Presence**.
- Subscribing to live database changes so a new "vibe" appears in the feed without a refresh.
- Building **online** and **typing** indicators with Presence and Broadcast.
- The security model that ties it all together: Storage RLS policies and Realtime authorization.

Prerequisites

- Chapter 14 (Supabase setup) and Chapter 15 (`@supabase/ssr` browser & server clients) — we reuse those clients verbatim.
- Chapter 16 (Row Level Security) — Storage policies are *just RLS on a hidden table*, so this will feel familiar.
- Chapter 17 (Server Actions & data fetching) — we trigger uploads from client components but persist URLs through actions.
- A `profiles` table and a `vibes` table (id, author_id, body, created_at) from earlier chapters.

Theory

Storage: a filing cabinet bolted onto your database

Supabase Storage is an S3-compatible object store with one superpower: **it speaks the same RLS language as your Postgres tables.** Think of it as a filing cabinet standing next to your database. The cabinet is divided into **buckets** (drawers). Inside a drawer, every file lives at a **path** — a string like `8f3.../avatar.png` that behaves like a folder tree even though storage is flat.

Two kinds of drawers:

- **Public buckets** — anyone with the URL can read the file. Perfect for avatars and cover photos on a discovery app, where the whole point is to be seen.
- **Private buckets** — reads require a short-lived **signed URL** you mint on the server. Use these for anything sensitive (ID verification, draft media).

The single most important rule: **scope every upload path by user id.** If everyone dumps files into a shared namespace, user A can overwrite user B's avatar, and your RLS policies have nothing to grab onto. By making the *first path segment* the uploader's `auth.uid()`, you get a clean folder-per-user structure that an RLS policy can check with one expression. The analogy: everyone gets their own labelled drawer, and the lock only opens for the name on the label.

Realtime: one socket, three jobs

Supabase Realtime is a single WebSocket connection multiplexed into **channels**. On a channel you can use three features, and beginners constantly confuse them:

Feature	Question it answers	Backed by	Survives reconnect?
`postgres_changes`	"What changed in the database?"	Postgres WAL (write-ahead log)	Yes — replays
Broadcast	"What did another client just *say*?"	Ephemeral message bus	No — fire & forget
Presence	"Who is here right now?"	In-memory state, synced	Diffs on join/leave

Analogy: a channel is a **group chat room**. `postgres_changes` is a bot that announces "the shared doc was edited." Broadcast is people typing messages that vanish if you weren't looking. Presence is the little "5 people online" list at the top that updates as people walk in and out.

Why three tools instead of one? Because durability has a cost. A new vibe is *data* — it must be persisted and reliably delivered, so it rides `postgres_changes`. A "typing…" indicator is *noise* — it should be cheap and disappear, so it rides Broadcast. "Who's online" is *state* — Presence keeps a synced snapshot and tells you the delta when it changes. Using the right one keeps your database write-free for things that shouldn't touch disk.

One gotcha: `postgres_changes` only fires for tables where **Realtime is enabled** (you add the table to the `supabase_realtime` publication), and the events you receive are still filtered by **RLS** — a user only hears about rows they're allowed to read.

In practice

1. Create the bucket and policies

Run this once in the Supabase SQL editor (or a migration). We make `avatars` public for reads but lock writes to the owning user.

```
-- Public bucket: reads are open, writes are guarded by policy
insert into storage.buckets (id, name, public)
values ('avatars', 'avatars', true)
on conflict (id) do nothing;

-- A user may upload/update/delete ONLY inside a folder named after their uid.
-- storage.foldername(name) returns the path segments as an array.
create policy "Avatar uploads are owner-scoped"
on storage.objects for insert to authenticated
with check (
  bucket_id = 'avatars'
  and (storage.foldername(name))[1] = (select auth.uid())::text
);

create policy "Avatar updates are owner-scoped"
on storage.objects for update to authenticated
using (
  bucket_id = 'avatars'
  and (storage.foldername(name))[1] = (select auth.uid())::text
);

create policy "Avatar deletes are owner-scoped"
on storage.objects for delete to authenticated
using (
  bucket_id = 'avatars'
  and (storage.foldername(name))[1] = (select auth.uid())::text
);
```

Reads need no policy because the bucket is public. Notice the path contract baked into `(storage.foldername(name))[1]`: the **first segment must equal the user's id.** Our helper will honor exactly that.

2. The upload helper

A typed helper that uploads an avatar, cleans up the old one, and returns a public URL. It runs in the browser using the SSR browser client from Chapter 15.

```ts
// lib/storage/avatars.ts
import { createBrowserClient } from "@supabase/ssr";

const supabase = createBrowserClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
);

const BUCKET = "avatars";
const MAX_BYTES = 2 * 1024 * 1024; // 2 MB
const ALLOWED = ["image/jpeg", "image/png", "image/webp"] as const;

export type UploadResult = { url: string; path: string };

export async function uploadAvatar(
  userId: string,
  file: File,
): Promise<UploadResult> {
  if (file.size > MAX_BYTES) throw new Error("Image must be under 2 MB.");
  if (!ALLOWED.includes(file.type as (typeof ALLOWED)[number])) {
    throw new Error("Use a JPEG, PNG, or WebP image.");
  }

  // Stable extension; cache-busting handled below so we can keep one path.
  const ext = file.name.split(".").pop()?.toLowerCase() ?? "png";
  const path = `${userId}/avatar.${ext}`; // <-- first segment = userId (RLS!)

  // Remove any previous avatar with a different extension so we don't orphan files.
  const { data: existing } = await supabase.storage.from(BUCKET).list(userId);
  const stale = (existing ?? [])
    .filter((f) => f.name.startsWith("avatar.") && f.name !== `avatar.${ext}`)
    .map((f) => `${userId}/${f.name}`);
  if (stale.length) await supabase.storage.from(BUCKET).remove(stale);

  const { error } = await supabase.storage
    .from(BUCKET)
    .upload(path, file, { upsert: true, contentType: file.type });
  if (error) throw new Error(error.message);

  const { data } = supabase.storage.from(BUCKET).getPublicUrl(path);
  // Append a version token so browsers/CDN refetch the new image at the same path.
```

```
  return { url: `${data.publicUrl}?v=${Date.now()}`, path };
}
```

Two real-world touches: `upsert: true` lets a user replace their photo at a stable path, and the `?v=` query string defeats CDN/browser caching so the new face shows up immediately.

3. Persisting the URL through a Server Action

The upload happens client-side (the file never needs to round-trip through your server), but the resulting URL belongs in the database. Save it with a Server Action.

```ts
// app/profile/actions.ts
"use server";

import { revalidatePath } from "next/cache";
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function saveAvatarUrl(url: string) {
  const cookieStore = await cookies(); // async in Next.js 16
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (all) =>
          all.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options),
          ),
      },
    },
  );

  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error("Not signed in.");

  const { error } = await supabase
    .from("profiles")
    .update({ avatar_url: url })
    .eq("id", user.id);
  if (error) throw new Error(error.message);

  revalidatePath("/profile");
}
```

4. The client uploader component

```tsx
// app/profile/avatar-uploader.tsx
"use client";

import { useState } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { uploadAvatar } from "@/lib/storage/avatars";
import { saveAvatarUrl } from "./actions";

export function AvatarUploader({ userId, initialUrl }: { userId: string; initialUrl: string }) {
  const [url, setUrl] = useState(initialUrl);
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);

  async function onPick(e: React.ChangeEvent<HTMLInputElement>) {
    const file = e.target.files?.[0];
    if (!file) return;
    setBusy(true);
    setError(null);
    try {
      const { url: newUrl } = await uploadAvatar(userId, file);
      await saveAvatarUrl(newUrl);
      setUrl(newUrl);
    } catch (err) {
      setError(err instanceof Error ? err.message : "Upload failed.");
    } finally {
      setBusy(false);
    }
  }

  return (
    <label className="group relative inline-block cursor-pointer">
      <AnimatePresence mode="wait">
        <motion.img
          key={url}
          src={url}
          alt="Your avatar"
          initial={{ opacity: 0, scale: 0.9 }}
          animate={{ opacity: 1, scale: 1 }}
          exit={{ opacity: 0 }}
          className="size-24 rounded-full object-cover ring-4 ring-[--color-brand]"
        />
      </AnimatePresence>
      <input type="file" accept="image/*" hidden onChange={onPick} disabled={busy} />
```

```
      {busy && <span className="absolute inset-0 grid place-items-center rounded-full bg-black/40 text-xs text-white">Uploading…</span>}
      {error && <p className="mt-2 text-xs text-red-500">{error}</p>}
    </label>
  );
}
```

The `--color-brand` token comes from your Tailwind v4 `@theme` block (`@theme { --color-brand: #FF6FB5; }`) — no `tailwind.config.js` in sight.

5. Realtime: live vibes in the feed

First enable Realtime for the table (run once):

```
alter publication supabase_realtime add table public.vibes;
```

Now a hook that subscribes to inserts and prepends them. RLS still applies, so users only receive vibes they're allowed to see.

```ts
// lib/realtime/use-live-vibes.ts
"use client";

import { useEffect } from "react";
import { createBrowserClient } from "@supabase/ssr";

type Vibe = { id: string; author_id: string; body: string; created_at: string };

const supabase = createBrowserClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
);

export function useLiveVibes(onInsert: (v: Vibe) => void) {
  useEffect(() => {
    const channel = supabase
      .channel("vibes-feed")
      .on(
        "postgres_changes",
        { event: "INSERT", schema: "public", table: "vibes" },
        (payload) => onInsert(payload.new as Vibe),
      )
      .subscribe();

    return () => {
      supabase.removeChannel(channel); // always tear down
    };
  }, [onInsert]);
}
```

6. Realtime: presence + typing

Presence answers "who's online," Broadcast carries the "typing…" pings. Both live on one channel.

```ts
// lib/realtime/use-presence.ts
"use client";

import { useEffect, useState } from "react";
import { createBrowserClient } from "@supabase/ssr";

const supabase = createBrowserClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
);

export function useRoomPresence(roomId: string, me: { id: string; name: string }) {
  const [online, setOnline] = useState<string[]>([]);
  const [typing, setTyping] = useState<string | null>(null);

  useEffect(() => {
    const channel = supabase.channel(`room:${roomId}`, {
      config: { presence: { key: me.id } },
    });

    channel
      .on("presence", { event: "sync" }, () => {
        const state = channel.presenceState<{ name: string }>();
        setOnline(Object.values(state).map((p) => p[0].name));
      })
      .on("broadcast", { event: "typing" }, ({ payload }) => {
        setTyping(payload.name);
        setTimeout(() => setTyping(null), 1500);
      })
      .subscribe(async (status) => {
        if (status === "SUBSCRIBED") {
          await channel.track({ name: me.name }); // announce myself
        }
      });

    return () => {
      supabase.removeChannel(channel);
    };
  }, [roomId, me.id, me.name]);

  function notifyTyping() {
    supabase.channel(`room:${roomId}`).send({
      type: "broadcast",
```

```
      event: "typing",
      payload: { name: me.name },
    });
  }

  return { online, typing, notifyTyping };
}
```

`channel.track()` registers you in Presence; the `sync` event fires whenever anyone joins or leaves. Typing pings go out as fire-and-forget Broadcast — no database writes, no history, exactly what an ephemeral hint should be.

Architecture / diagram

```
flowchart LR
  subgraph Browser["Viby Client (use client)"]
    UP[AvatarUploader] -->|upload File| ST
    UP -->|saveAvatarUrl| SA[Server Action]
    FEED[Feed] -. postgres_changes .- RT
    PRES[Presence dots] -. presence + broadcast .- RT
  end

  subgraph Supabase
    ST[(Storage: avatars bucket)]
    SA --> DB[(Postgres: profiles / vibes)]
    DB -->|WAL| RT[Realtime server]
    RT -->|RLS-filtered events| Browser
  end

  ST -->|public URL| FEED
```

The file goes straight to Storage; only the resulting *string* travels through your server. Database writes flow to Postgres, whose write-ahead log feeds Realtime, which fans RLS-filtered events back to every subscribed client.

Best practices & pitfalls

- **DO** make the first path segment the user id. It's the hook every Storage RLS policy relies on.
- **DO** validate file type and size *client-side for UX* and rely on bucket/RLS limits *server-side for security*. Never trust the client alone.

- **DON'T** forget `supabase.removeChannel()` in your cleanup. Leaked channels pile up across navigations and exhaust the connection.
- **DON'T** use `postgres_changes` for typing indicators — you'd hammer the database with throwaway writes. Use Broadcast.
- **DO** add the table to the `supabase_realtime` publication, or your `postgres_changes` subscription will silently receive nothing.
- **DON'T** assume Realtime bypasses RLS. It doesn't — if a user can't `select` a row, they won't get its change events. Test with a real signed-in client, not the service role.
- **DO** cache-bust public URLs (`?v=`) when you reuse a stable path, or users will keep seeing their old photo.
- **DON'T** store the service-role key in any `NEXT_PUBLIC_` variable. Uploads here use the anon key plus RLS — that's the point.
- **DO** clean up orphaned files on replace; storage costs and clutter are real at scale.

Exercises

1. **Cover photos.** Add a `covers` bucket and extend the helper with `uploadCover(userId, file)` reusing the per-user path pattern. Wire matching insert/update/delete policies.
2. **Drag-and-drop.** Enhance `AvatarUploader` with a Framer Motion drop zone using `onDragOver` / `onDrop`, animating a pink dashed border (`whileHover` / `animate`) when a file hovers.
3. **Live like counts.** Subscribe to `UPDATE` events on a `vibes` row's `like_count` and animate the number with a Framer Motion `key` -swap counter.
4. **Debounced typing.** The current `notifyTyping` could fire on every keystroke. Debounce it to at most one broadcast per 800 ms.
5. **Signed URLs (harder).** Create a *private* `verification` bucket and write a Server Action that returns a 60-second signed URL via `createSignedUrl` .

Mini-project: the live "Who's vibing now" rail

Build a sidebar rail on the Viby home feed that shows live presence and live vibes together.

Requirements 1. A `PresenceRail` client component that joins channel `room:home` , tracks the current user, and renders up to 8 online avatars with a green pulsing dot (Framer Motion `animate` loop on opacity/scale). 2. Use `useLiveVibes` so that when *anyone* posts, the new vibe slides into the top of the feed via `AnimatePresence` + `layout` . 3. The composer at the top calls `notifyTyping` (debounced from Exercise 4); other users see "Maya is vibing..." appear and fade. 4. Posting a vibe writes through a Server Action; the live insert — not a manual `setState` — is what adds it to the feed, proving the realtime loop end to end.

Acceptance: open two browser windows signed in as different users. Posting in one appears instantly in the other, both presence rails show two avatars, and typing in one shows the hint in the other. No page refresh, anywhere.

Key takeaways

- Supabase **Storage** is RLS-governed object storage; **per-user path prefixes** are what make policies enforceable.
- Public buckets serve open read URLs; private buckets need **signed URLs** minted server-side.
- Your upload helper should validate, upsert at a stable path, **clean up stale files**, and cache-bust the URL.
- Realtime multiplexes three tools on one channel: `postgres_changes` for durable data, **Broadcast** for ephemeral signals, **Presence** for who's-here state.
- `postgres_changes` requires the table in the `supabase_realtime` publication and **still respects RLS**.
- Always **remove channels** on cleanup to avoid leaks.

Up next — Chapter 19: Edge Functions & Webhooks, where Viby starts reacting to the outside world: image moderation on upload, and Stripe events arriving over the wire.

Chapter 19 — Project Setup & Foundation

> Every great app starts as an empty folder and a clear decision about where things go. Today we make those decisions for Viby — once — so the next forty chapters feel like flow instead of friction.

What you'll learn

- How to scaffold a **Next.js 16** project with the App Router, TypeScript, and Tailwind CSS v4
- The exact **stack install** for Viby (shadcn-style primitives, Framer Motion, Zustand, Supabase via `@supabase/ssr`)
- A **folder structure** that scales from demo to production without a single rename
- How to define **brand design tokens** in `globals.css` using Tailwind v4's `@theme` — no `tailwind.config.js`
- Building the `DeviceFrame`, the `Providers` tree, and the **Zustand store**
- The **demo / connected-mode seam**: one switch that runs Viby fully in-memory *or* against real Supabase

Prerequisites

- **Chapters 1–8** — JavaScript, TypeScript fundamentals, and the React 19 mental model (Server vs. Client Components).
- **Chapters 9–14** — Next.js App Router routing, Server Actions, and data fetching.
- **Chapters 15–18** — Tailwind v4, component composition, and an intro to Supabase Auth.
- Node.js **20.9+** and a package manager (we use `pnpm`, but `npm` / `yarn` work).

Theory

Why a foundation chapter at all?

Think of Viby like a house. You *could* start nailing up drywall immediately — but if the foundation isn't poured level and the plumbing rough-in is missing, every later step fights you. Our "foundation" is three things working together:

1. **Structure** — where files live, so you never hunt.
2. **Tokens** — the brand's colors, radii, and motion expressed *once*, so the whole app stays visually consistent automatically.
3. **A seam** — a clean line between *fake data* and *real backend* so you can build and demo the UI today and flip on Supabase tomorrow without rewriting components.

That third one is the secret weapon of professional teams. The "**demo / connected seam**" means your UI never imports Supabase directly. Instead it talks to a small **data layer**, and that layer decides — based on

one environment flag — whether to return in-memory mock data or hit the real database. The UI can't tell the difference. This is the same pattern behind feature flags, Storybook mocks, and offline-first apps.

Server-first, client-when-needed

Next.js 16 renders **Server Components by default**. They run on the server, never ship their JS to the browser, and can `await` data directly. You opt a component into the browser with the `"use client"` directive — and you only do that when you need state, effects, event handlers, or browser APIs. Viby's rule of thumb: *pages and layouts stay Server Components; anything interactive (a button with a Zustand store, a Framer Motion gesture) is a Client Component leaf.*

Tokens as a single source of truth

In Tailwind v4 there is no `tailwind.config.js`. You declare your design system **in CSS** with the `@theme` block. Define `--color-viby` once and Tailwind generates `bg-viby`, `text-viby`, `border-viby`, and friends for you. Change the hex in one place and the entire app re-skins. That's the analogy of a *style guide that the compiler actually enforces*.

Why a `DeviceFrame`?

Viby is **mobile-first**. On a laptop, a full-width mobile app looks lost in an ocean of whitespace. The `DeviceFrame` is a presentational shell — a phone-shaped container — so that on desktop you preview Viby exactly as it appears on a phone, and on an actual phone the frame melts away to full-screen. It's a teaching device *and* a real product decision used by many social apps' marketing previews.

In practice

1. Scaffold the project

```
pnpm create next-app@latest viby --typescript --eslint --app --tailwind --src-dir --use-pnpm
cd viby
```

Choices: **App Router**, **TypeScript**, **Tailwind**, and a `src/` **directory** (keeps config files at the root uncluttered). This gives you Next.js 16, React 19, and Tailwind v4 out of the box.

2. Install the Viby stack

```
# State + animation
pnpm add zustand framer-motion

# Supabase (SSR-aware client)
pnpm add @supabase/supabase-js @supabase/ssr

# UI utilities used by shadcn-style primitives
pnpm add class-variance-authority clsx tailwind-merge lucide-react
```

3. Folder structure

```
viby/
├─ src/
│  ├─ app/                    # App Router: routes, layouts, pages
│  │  ├─ layout.tsx           # Root layout — fonts, Providers, DeviceFrame
│  │  ├─ page.tsx             # Landing / discover feed
│  │  └─ globals.css          # Tailwind import + @theme tokens
│  ├─ components/
│  │  ├─ ui/                  # shadcn-style primitives (Button, Card…)
│  │  ├─ device-frame.tsx     # the phone shell
│  │  └─ providers.tsx        # client-side context providers
│  ├─ lib/
│  │  ├─ utils.ts             # cn() helper
│  │  ├─ env.ts               # mode detection (demo vs connected)
│  │  └─ supabase/
│  │     ├─ client.ts         # createBrowserClient
│  │     └─ server.ts         # createServerClient
│  ├─ data/                   # THE SEAM — UI imports from here, never Supabase
│  │  ├─ profiles.ts          # getProfiles() — demo or connected
│  │  └─ mock/profiles.ts     # in-memory seed data
│  ├─ store/
│  │  └─ use-viby-store.ts    # Zustand global store
│  └─ types/
│     └─ index.ts             # shared TypeScript types
├─ proxy.ts                   # middleware-style logic (Next.js 16)
└─ .env.local
```

> **Note on `proxy.ts`:** In Next.js 16, request-intercepting logic that older tutorials put in `middleware.ts` now lives in `proxy.ts` at the project root. We'll wire Supabase session refresh there in Chapter 22.

4. Design tokens in `globals.css`

```
@import "tailwindcss";

@theme {
  /* Brand */
  --color-viby: #ff6fb5;          /* bubblegum pink */
  --color-viby-ink: #1b1320;      /* near-black text */
  --color-viby-muted: #8b8194;    /* secondary text */
  --color-viby-bg: #ffffff;
  --color-viby-surface: #fff5fa;  /* soft pink wash */

  /* Radii — rounded, friendly, Gen-Z */
  --radius-viby: 1.25rem;

  /* Fonts (wired in layout.tsx) */
  --font-sans: var(--font-geist-sans), ui-sans-serif, system-ui, sans-serif;

  /* Motion */
  --ease-viby: cubic-bezier(0.22, 1, 0.36, 1);
}

:root {
  color-scheme: light;
}

body {
  background: var(--color-viby-bg);
  color: var(--color-viby-ink);
  -webkit-font-smoothing: antialiased;
}
```

Now `bg-viby`, `text-viby-muted`, `rounded-viby`, and `ease-[--ease-viby]` all exist as utilities — zero config file.

5. The `cn()` helper

```
// src/lib/utils.ts
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

6. The mode flag (the heart of the seam)

```
// src/lib/env.ts
// Connected mode turns on ONLY when real Supabase env vars are present.
export const isConnected =
  !!process.env.NEXT_PUBLIC_SUPABASE_URL &&
  !!process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

export const mode = isConnected ? "connected" : "demo";
```

```
# .env.local  — leave BLANK for demo mode; fill in for connected mode
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
```

7. Supabase clients (`@supabase/ssr`)

```
// src/lib/supabase/client.ts
import { createBrowserClient } from "@supabase/ssr";

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
  );
}
```

```
// src/lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies(); // async in Next.js 16

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options),
            );
          } catch {
            // Called from a Server Component — safe to ignore;
            // proxy.ts refreshes the session instead.
          }
        },
      },
    },
  );
}
```

8. The data layer — where the seam lives

```
// src/types/index.ts
export type Profile = {
  id: string;
  name: string;
  age: number;
  city: string;
  bio: string;
  hobbies: string[];
  avatarUrl: string;
};
```

```ts
// src/data/mock/profiles.ts
import type { Profile } from "@/types";

export const mockProfiles: Profile[] = [
  {
    id: "p1",
    name: "Maya",
    age: 24,
    city: "Austin",
    bio: "Bouldering, vinyl crates, and aggressively good coffee.",
    hobbies: ["climbing", "vinyl", "coffee"],
    avatarUrl: "/avatars/maya.jpg",
  },
  {
    id: "p2",
    name: "Devon",
    age: 27,
    city: "Austin",
    bio: "Run-club captain. I will drag you to a 6am 5k.",
    hobbies: ["running", "board games", "tacos"],
    avatarUrl: "/avatars/devon.jpg",
  },
];
```

```ts
// src/data/profiles.ts  — the UI imports THIS, never Supabase
import { isConnected } from "@/lib/env";
import { mockProfiles } from "@/data/mock/profiles";
import type { Profile } from "@/types";

export async function getProfiles(): Promise<Profile[]> {
  if (!isConnected) {
    // Demo mode: instant, in-memory, no network.
    return mockProfiles;
  }

  // Connected mode: real Supabase read.
  const { createClient } = await import("@/lib/supabase/server");
  const supabase = await createClient();
  const { data, error } = await supabase
    .from("profiles")
    .select("id, name, age, city, bio, hobbies, avatar_url");

  if (error) throw error;

  return (data ?? []).map((row) => ({
    id: row.id,
    name: row.name,
    age: row.age,
    city: row.city,
    bio: row.bio,
    hobbies: row.hobbies ?? [],
    avatarUrl: row.avatar_url,
  }));
}
```

The component that renders the feed only ever sees `getProfiles()`. It has no idea whether the data is fake or real — and that's the whole point.

9. The Zustand store

```
// src/store/use-viby-store.ts
"use client";

import { create } from "zustand";

type VibyState = {
  likedIds: string[];
  toggleLike: (id: string) => void;
  isLiked: (id: string) => boolean;
};

export const useVibyStore = create<VibyState>((set, get) => ({
  likedIds: [],
  toggleLike: (id) =>
    set((state) => ({
      likedIds: state.likedIds.includes(id)
        ? state.likedIds.filter((x) => x !== id)
        : [...state.likedIds, id],
    })),
  isLiked: (id) => get().likedIds.includes(id),
}));
```

10. Providers (client boundary)

```
// src/components/providers.tsx
"use client";

import { type ReactNode } from "react";

export function Providers({ children }: { children: ReactNode }) {
  // Today: a thin wrapper. Later: theme, toast, Supabase realtime context.
  return <>{children}</>;
}
```

11. The `DeviceFrame`

```tsx
// src/components/device-frame.tsx
import { type ReactNode } from "react";
import { cn } from "@/lib/utils";

export function DeviceFrame({ children }: { children: ReactNode }) {
  return (
    <div className="flex min-h-dvh items-center justify-center bg-viby-surface p-0 sm:p-6">
      <div
        className={cn(
          "relative w-full overflow-hidden bg-viby-bg shadow-2xl",
          // Full-screen on phones; framed phone shape on larger screens.
          "min-h-dvh sm:min-h-0 sm:h-[844px] sm:max-w-[390px] sm:rounded-[2.5rem]",
          "sm:ring-8 sm:ring-viby-ink/90",
        )}
      >
        {children}
      </div>
    </div>
  );
}
```

12. Wire it all in the root layout

```tsx
// src/app/layout.tsx
import type { Metadata } from "next";
import { Geist } from "next/font/google";
import { Providers } from "@/components/providers";
import { DeviceFrame } from "@/components/device-frame";
import "./globals.css";

const geist = Geist({ subsets: ["latin"], variable: "--font-geist-sans" });

export const metadata: Metadata = {
  title: "Viby — find your people",
  description: "Make friends through shared hobbies and lifestyle.",
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en" className={geist.variable}>
      <body className="font-sans">
        <Providers>
          <DeviceFrame>{children}</DeviceFrame>
        </Providers>
      </body>
    </html>
  );
}
```

A quick Server Component page that consumes the seam:

```
// src/app/page.tsx  (Server Component — note the await)
import { getProfiles } from "@/data/profiles";
import { mode } from "@/lib/env";

export default async function DiscoverPage() {
  const profiles = await getProfiles();

  return (
    <main className="p-5">
      <header className="mb-4 flex items-baseline justify-between">
        <h1 className="text-2xl font-bold text-viby">Viby</h1>
        <span className="text-xs text-viby-muted">{mode} mode</span>
      </header>

      <ul className="space-y-3">
        {profiles.map((p) => (
          <li key={p.id} className="rounded-viby bg-viby-surface p-4">
            <p className="font-semibold">
              {p.name}, {p.age}
            </p>
            <p className="text-sm text-viby-muted">{p.bio}</p>
          </li>
        ))}
      </ul>
    </main>
  );
}
```

Run it:

```
pnpm dev
```

With `.env.local` blank you'll see `demo mode` and Maya + Devon. Fill in real keys and the *same page* serves live data.

Architecture / diagram

```
flowchart TD
  UI["Server Component (page.tsx)"] -->|await getProfiles| DL["Data layer (src/data/profiles.ts)"]
  DL -->|isConnected?| FLAG{"env.ts<br/>demo or connected"}
  FLAG -->|demo| MOCK["mockProfiles (in-memory)"]
  FLAG -->|connected| SB["Supabase server client"]
  SB --> DB[("Postgres: profiles")]

  subgraph Client
    STORE["Zustand store"]
    FRAME["DeviceFrame + Providers"]
  end

  UI -.renders inside.-> FRAME
  FRAME -.interactive leaves use.-> STORE
```

The UI sits on the **left** and never reaches past the data layer. The **flag** in the middle is the only thing that decides demo vs. real — flip it and nothing else changes.

Best practices & pitfalls

- **Do** keep all backend access behind `src/data/*`. **Don't** import a Supabase client inside a component — that hard-wires you to the backend and breaks demo mode.
- **Do** define tokens once in `@theme`. **Don't** sprinkle raw hex like `#ff6fb5` in components — use `bg-viby`.
- **Do** keep `"use client"` on leaves (the store, animated buttons). **Don't** mark `layout.tsx` or `page.tsx` as client — you lose `async` data fetching and ship needless JS.
- **Do** `await cookies()` and `await params` in Next.js 16 — they're async now. Forgetting `await` is the #1 upgrade bug.
- **Do** use `proxy.ts` (not `middleware.ts`) for request interception in Next.js 16.
- **Don't** commit `.env.local`. Confirm it's in `.gitignore` (Next.js adds it by default).
- **Don't** put secrets behind the `NEXT_PUBLIC_` prefix unless they're truly public — that prefix exposes them to the browser. The anon key is safe; a service-role key never is.

Exercises

1. **Token tweak (easy).** Add a `--color-viby-accent` token (try a mint `#5ce0c2`) and use it for the "mode" badge text.
2. **Second entity (easy–medium).** Create a `Hobby` type, a `mock/hobbies.ts`, and a `getHobbies()` in the data layer following the exact seam pattern.
3. **Store growth (medium).** Add a `passedIds` array and a `pass(id)` action to the Zustand store, ensuring a profile can't be both liked and passed.

4. **Frame polish (medium).** Add a notch/status-bar strip to the top of `DeviceFrame` that only appears on `sm:` and up.
5. **Seam proof (challenge).** Without changing `page.tsx`, make `getProfiles()` log `"[demo]"` or `"[connected]"` to the server console — proving the UI is decoupled from the source.

Mini-project — The Discover skeleton

Build a working **Discover feed skeleton** end to end:

1. Scaffold and install the stack (steps 1–2).
2. Drop in the tokens, `cn()`, `env.ts`, the data layer, the store, `DeviceFrame`, `Providers`, and the layout.
3. Make `page.tsx` render the profiles from `getProfiles()` inside the `DeviceFrame`.
4. Add a small **Client Component** `LikeButton` that calls `useVibyStore().toggleLike(p.id)` and shows a filled heart when liked (use `lucide-react`'s `Heart`). This is your first taste of the Server-renders / Client-interacts split.
5. Confirm the app runs in **demo mode** with no Supabase keys, shows `demo mode`, and lets you like Maya and Devon.

Success looks like: a phone-framed pink feed, two profiles, a heart that toggles — all running with zero backend.

Key takeaways

- Next.js 16 is **server-first**; reach for `"use client"` only on interactive leaves.
- Tailwind v4 puts your design system in `@theme` **tokens** — no config file, one source of truth.
- The **data layer (`src/data/*`)** is a seam: the UI never imports Supabase, so demo and connected modes share identical components.
- A single env-derived flag (`isConnected`) decides the mode — making Viby buildable, demoable, and testable today.
- `DeviceFrame` + `Providers` + the **Zustand store** give you a mobile-shaped shell and clean client-state from day one.

Up next — Chapter 20: Design System & UI Primitives, where we turn these tokens into a full set of shadcn-style components (Button, Card, Avatar, Badge) with Framer Motion polish.

Chapter 20 — The Onboarding Flow

> First impressions are everything — and in Viby, onboarding *is* the product. A delightful five-minute wizard turns a stranger into someone your community wants to meet.

So far you've built auth, a database, and a profile schema. But a brand-new Viby account is a ghost: no hobbies, no photos, no vibe. Nobody discovers a ghost. In this chapter we build the **onboarding wizard** — the joyful, animated, friction-light flow that takes a freshly-signed-up user and turns them into a discoverable human being.

What you'll learn

- How to model a **multi-step wizard** as a single client component with a state machine, not five separate pages.
- Driving smooth **enter/exit step transitions** with Framer Motion's `AnimatePresence` and a directional slide.
- Building a **progress bar** that animates as the user advances.
- Collecting **hobbies, 3–4 prompt-based photos, a vibe, and location**, with per-step **validation** that blocks bad data without nagging.
- Uploading images to **Supabase Storage** and persisting everything atomically through a `saveOnboarding` **Server Action**.
- Redirecting completed users away from onboarding using `proxy.ts`.

Prerequisites

- **Chapter 8** — Tailwind v4 theme tokens (`@theme`) and the Viby color system.
- **Chapter 12** — Zustand stores for ephemeral, in-memory UI state.
- **Chapter 15** — Supabase Auth with `@supabase/ssr` (`createBrowserClient` / `createServerClient`).
- **Chapter 17** — Server Actions and the `"use server"` directive.
- **Chapter 18** — the `profiles` table and Row Level Security.

Theory

A wizard is a state machine, not a stack of pages

The naive instinct is to make each step its own route: `/onboarding/hobbies`, `/onboarding/photos`, and so on. Resist it. The moment you split steps across routes, you lose the half-filled form on every navigation, you can't animate between steps, and "go back" becomes a browser-history minefield.

Think of the wizard like a **photo carousel inside a single frame**. The frame (the card, the progress bar, the Next button) never moves. Only the *slide* inside it changes. That mental model maps perfectly to React: one

client component owns a `step` index and a single `data` object; each step is just a different child rendered into the same frame.

This gives us three superpowers:

1. **State survives** as the user moves back and forth — it lives in one place.
2. **We can animate** the swap, because the old slide and the new slide briefly coexist.
3. **Validation is local** — each step says "am I valid?" and the frame decides whether Next is allowed.

Why AnimatePresence

React normally unmounts a component the instant it leaves the tree — there's no time to play an exit animation. `AnimatePresence` is Framer Motion's answer: it *keeps a leaving element mounted* just long enough to run its `exit` animation, then removes it. For a wizard, that means the outgoing step can slide left while the incoming step slides in from the right. The analogy: AnimatePresence is a stage manager who lets an actor finish their exit line before pulling the curtain.

Prompt-based photos, not a dump

Viby is social *discovery*, not a dating clone, and our photo step reflects that. Instead of "upload 6 selfies," we use **prompts** — "A hobby in action," "Your happy place," "Caught laughing." Prompts produce richer, more honest profiles and give us captions for free. We require **3 to 4** filled prompts: enough to feel like a person, few enough to finish in a sitting.

Persist once, at the end

We hold everything in client state and commit it in a **single Server Action** at the finish line. Why not save per-step? Because a half-onboarded profile is worse than no profile — it shows up in discovery looking broken. One atomic `saveOnboarding` keeps the database honest: a profile is either fully onboarded or not at all.

In practice

1. The wizard state, in Zustand

We keep the in-progress answers in a Zustand store so any step can read and write without prop-drilling. This is ephemeral demo state (Chapter 12) — the source of truth becomes the database only after `saveOnboarding`.

```ts
// stores/use-onboarding.ts
import { create } from "zustand";

export type Vibe = "chill" | "adventurous" | "creative" | "social" | "nerdy";

export interface PromptPhoto {
  prompt: string;
  /** Object URL for preview; the File is what we upload. */
  file: File | null;
  previewUrl: string | null;
}

export interface OnboardingData {
  hobbies: string[];
  photos: PromptPhoto[];
  vibe: Vibe | null;
  location: { city: string; lat: number | null; lng: number | null };
}

interface OnboardingState extends OnboardingData {
  setHobbies: (hobbies: string[]) => void;
  setPhoto: (index: number, file: File) => void;
  setVibe: (vibe: Vibe) => void;
  setLocation: (loc: OnboardingData["location"]) => void;
}

const PROMPTS = ["A hobby in action", "Your happy place", "Caught laughing", "This is so me"];

export const useOnboarding = create<OnboardingState>((set) => ({
  hobbies: [],
  photos: PROMPTS.map((prompt) => ({ prompt, file: null, previewUrl: null })),
  vibe: null,
  location: { city: "", lat: null, lng: null },

  setHobbies: (hobbies) => set({ hobbies }),
  setPhoto: (index, file) =>
    set((state) => {
      const photos = [...state.photos];
      // Revoke the old preview URL to avoid leaking memory.
      if (photos[index].previewUrl) URL.revokeObjectURL(photos[index].previewUrl!);
      photos[index] = { ...photos[index], file, previewUrl: URL.createObjectURL(file) };
      return { photos };
    }),
```

```
  setVibe: (vibe) => set({ vibe }),
  setLocation: (location) => set({ location }),
}));
```

2. The frame: progress bar + AnimatePresence

This is the heart of the chapter. The `OnboardingWizard` is a single `"use client"` component. It owns the `step` index and a `direction` (so we know which way to slide), reads validity from a per-step validator, and only commits at the end.

```tsx
// components/onboarding/onboarding-wizard.tsx
"use client";

import { useState, useTransition } from "react";
import { AnimatePresence, motion } from "framer-motion";
import { useRouter } from "next/navigation";
import { useOnboarding } from "@/stores/use-onboarding";
import { saveOnboarding } from "@/app/onboarding/actions";
import { HobbiesStep } from "./steps/hobbies-step";
import { PhotosStep } from "./steps/photos-step";
import { VibeStep } from "./steps/vibe-step";
import { LocationStep } from "./steps/location-step";

const STEPS = [
  { key: "hobbies", title: "What are you into?", Component: HobbiesStep },
  { key: "photos", title: "Show us your world", Component: PhotosStep },
  { key: "vibe", title: "What's your vibe?", Component: VibeStep },
  { key: "location", title: "Where are you?", Component: LocationStep },
] as const;

const slide = {
  enter: (dir: number) => ({ x: dir > 0 ? 64 : -64, opacity: 0 }),
  center: { x: 0, opacity: 1 },
  exit: (dir: number) => ({ x: dir > 0 ? -64 : 64, opacity: 0 }),
};

export function OnboardingWizard() {
  const [step, setStep] = useState(0);
  const [direction, setDirection] = useState(1);
  const [pending, startTransition] = useTransition();
  const [error, setError] = useState<string | null>(null);
  const router = useRouter();
  const data = useOnboarding();

  const { title, Component } = STEPS[step];
  const isLast = step === STEPS.length - 1;
  const progress = ((step + 1) / STEPS.length) * 100;

  // Each step exposes whether it's complete. Kept here so the frame owns the gate.
  const stepValid = validateStep(step, data);

  function goNext() {
    if (!stepValid) return;
```

```
  setError(null);
  if (isLast) return finish();
  setDirection(1);
  setStep((s) => s + 1);
}

function goBack() {
  setError(null);
  setDirection(-1);
  setStep((s) => Math.max(0, s - 1));
}

function finish() {
  startTransition(async () => {
    // Upload happens client-side; the action receives public URLs + plain data.
    const photoUrls = await uploadPhotos(data.photos);
    const result = await saveOnboarding({
      hobbies: data.hobbies,
      photos: photoUrls,
      vibe: data.vibe!,
      location: data.location,
    });
    if (result?.error) {
      setError(result.error);
      return;
    }
    router.replace("/discover");
  });
}

return (
  <div className="mx-auto flex min-h-dvh w-full max-w-md flex-col px-5 py-8">
    {/* Progress bar */}
    <div className="mb-8 h-2 w-full overflow-hidden rounded-full bg-pink-100">
      <motion.div
        className="h-full rounded-full bg-brand"
        initial={false}
        animate={{ width: `${progress}%` }}
        transition={{ type: "spring", stiffness: 220, damping: 30 }}
      />
    </div>

    <h1 className="mb-6 text-2xl font-bold tracking-tight text-gray-900">{title}</h1>
```

```
      {/* Animated step area */}
      <div className="relative flex-1">
        <AnimatePresence mode="wait" custom={direction}>
          <motion.div
            key={STEPS[step].key}
            custom={direction}
            variants={slide}
            initial="enter"
            animate="center"
            exit="exit"
            transition={{ type: "spring", stiffness: 260, damping: 30 }}
          >
            <Component />
          </motion.div>
        </AnimatePresence>
      </div>

      {error && <p className="mt-4 text-sm text-red-600">{error}</p>}

      {/* Controls */}
      <div className="mt-6 flex items-center gap-3">
        {step > 0 && (
          <button
            type="button"
            onClick={goBack}
            disabled={pending}
            className="rounded-full px-5 py-3 text-sm font-semibold text-gray-600 hover:bg-gray-100"
          >
            Back
          </button>
        )}
        <button
          type="button"
          onClick={goNext}
          disabled={!stepValid || pending}
          className="ml-auto rounded-full bg-brand px-7 py-3 text-sm font-semibold text-white shadow-lg
shadow-pink-200 transition disabled:opacity-40"
        >
          {pending ? "Saving…" : isLast ? "Finish" : "Next"}
        </button>
      </div>
    </div>
```

```
  );
}

function validateStep(step: number, d: ReturnType<typeof useOnboarding.getState>) {
  switch (step) {
    case 0:
      return d.hobbies.length >= 3;
    case 1:
      return d.photos.filter((p) => p.file).length >= 3;
    case 2:
      return d.vibe !== null;
    case 3:
      return d.location.city.trim().length > 1;
    default:
      return false;
  }
}
```

> `mode="wait"` tells AnimatePresence to finish the exit animation before mounting the next step. Because we change the `key` on the `motion.div`, Framer Motion treats each step as a *new* element — that's what triggers enter/exit. The `custom` prop pipes `direction` into the variant functions so the slide direction follows the user.

3. A representative step: hobbies

Steps are dumb and focused. They read/write the store and nothing else — validation lives in the frame. Here's the hobbies chips selector.

```tsx
// components/onboarding/steps/hobbies-step.tsx
"use client";

import { motion } from "framer-motion";
import { useOnboarding } from "@/stores/use-onboarding";

const ALL_HOBBIES = [
  "Hiking", "Coffee", "Gaming", "Live music", "Cooking", "Photography",
  "Climbing", "Reading", "Cycling", "Painting", "Yoga", "Board games",
];

export function HobbiesStep() {
  const { hobbies, setHobbies } = useOnboarding();

  function toggle(h: string) {
    setHobbies(hobbies.includes(h) ? hobbies.filter((x) => x !== h) : [...hobbies, h]);
  }

  return (
    <div>
      <p className="mb-4 text-sm text-gray-500">Pick at least 3. This is how we find your people.</p>
      <div className="flex flex-wrap gap-2">
        {ALL_HOBBIES.map((h) => {
          const active = hobbies.includes(h);
          return (
            <motion.button
              key={h}
              type="button"
              onClick={() => toggle(h)}
              whileTap={{ scale: 0.94 }}
              className={`rounded-full border px-4 py-2 text-sm font-medium transition ${
                active
                  ? "border-brand bg-brand text-white"
                  : "border-gray-200 bg-white text-gray-700 hover:border-pink-300"
              }`}
            >
              {h}
            </motion.button>
          );
        })}
      </div>
    </div>
```

```
  );
}
```

The vibe and location steps follow the same shape: read from the store, write on interaction. Location can call the browser Geolocation API and reverse-geocode, or just accept a typed city — keep the demo honest and let the user type.

4. The photos step + Supabase Storage upload

The prompt-based grid: each cell is a labeled file input that previews the chosen image.

```tsx
// components/onboarding/steps/photos-step.tsx
"use client";

import Image from "next/image";
import { useOnboarding } from "@/stores/use-onboarding";

export function PhotosStep() {
  const { photos, setPhoto } = useOnboarding();

  return (
    <div>
      <p className="mb-4 text-sm text-gray-500">Add 3–4 photos answering these prompts.</p>
      <div className="grid grid-cols-2 gap-3">
        {photos.map((p, i) => (
          <label
            key={p.prompt}
            className="relative flex aspect-square cursor-pointer flex-col justify-end overflow-hidden rounded-2xl border-2 border-dashed border-pink-200 bg-pink-50/50 p-3"
          >
            {p.previewUrl && (
              <Image src={p.previewUrl} alt={p.prompt} fill className="object-cover" sizes="50vw" />
            )}
            <span className="relative z-10 rounded-md bg-black/45 px-2 py-1 text-xs font-medium text-white">
              {p.prompt}
            </span>
            <input
              type="file"
              accept="image/*"
              className="hidden"
              onChange={(e) => {
                const file = e.target.files?.[0];
                if (file) setPhoto(i, file);
              }}
            />
          </label>
        ))}
      </div>
    </div>
  );
}
```

Uploading happens in the wizard's `finish()` via this client helper. We upload to a per-user folder so RLS-style Storage policies can scope access.

```
// lib/upload-photos.ts
import { createBrowserClient } from "@supabase/ssr";
import type { PromptPhoto } from "@/stores/use-onboarding";

export async function uploadPhotos(photos: PromptPhoto[]) {
  const supabase = createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
  );

  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error("Not signed in");

  const filled = photos.filter((p) => p.file);
  const uploaded: { prompt: string; url: string }[] = [];

  for (const p of filled) {
    const ext = p.file!.name.split(".").pop() ?? "jpg";
    const path = `${user.id}/${crypto.randomUUID()}.${ext}`;
    const { error } = await supabase.storage
      .from("profile-photos")
      .upload(path, p.file!, { upsert: false, contentType: p.file!.type });
    if (error) throw error;

    const { data } = supabase.storage.from("profile-photos").getPublicUrl(path);
    uploaded.push({ prompt: p.prompt, url: data.publicUrl });
  }

  return uploaded;
}
```

5. The `saveOnboarding` Server Action

The action runs on the server with the user's session. It validates again (never trust the client), writes the profile, flips `onboarded` to `true`, and revalidates discovery.

```ts
// app/onboarding/actions.ts
"use server";

import { revalidatePath } from "next/cache";
import { cookies } from "next/headers";
import { createServerClient } from "@supabase/ssr";

interface SavePayload {
  hobbies: string[];
  photos: { prompt: string; url: string }[];
  vibe: string;
  location: { city: string; lat: number | null; lng: number | null };
}

export async function saveOnboarding(payload: SavePayload) {
  const cookieStore = await cookies();
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) =>
          toSet.forEach(({ name, value, options }) => cookieStore.set(name, value, options)),
      },
    },
  );

  const { data: { user } } = await supabase.auth.getUser();
  if (!user) return { error: "Your session expired. Please sign in again." };

  // Server-side validation — the gate of last resort.
  if (payload.hobbies.length < 3) return { error: "Pick at least 3 hobbies." };
  if (payload.photos.length < 3) return { error: "Add at least 3 photos." };
  if (!payload.vibe) return { error: "Choose a vibe." };
  if (payload.location.city.trim().length < 2) return { error: "Tell us your city." };

  const { error } = await supabase
    .from("profiles")
    .update({
      hobbies: payload.hobbies,
      photos: payload.photos,
      vibe: payload.vibe,
```

```
      city: payload.location.city,
      lat: payload.location.lat,
      lng: payload.location.lng,
      onboarded: true,
    })
    .eq("id", user.id);

  if (error) return { error: "Couldn't save your profile. Try again." };

  revalidatePath("/discover");
  return { success: true };
}
```

6. Gate onboarding with `proxy.ts`

Next.js 16 replaces `middleware.ts` with `proxy.ts`. We use it to (a) keep the Supabase session fresh and (b) bounce already-onboarded users out of `/onboarding`, and un-onboarded users *into* it.

```
// proxy.ts
import { NextResponse, type NextRequest } from "next/server";
import { createServerClient } from "@supabase/ssr";

export async function proxy(request: NextRequest) {
  const response = NextResponse.next({ request });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => request.cookies.getAll(),
        setAll: (toSet) =>
          toSet.forEach(({ name, value, options }) =>
            response.cookies.set(name, value, options),
          ),
      },
    },
  );

  const { data: { user } } = await supabase.auth.getUser();
  const { pathname } = request.nextUrl;

  if (user && pathname.startsWith("/onboarding")) {
    const { data: profile } = await supabase
      .from("profiles").select("onboarded").eq("id", user.id).single();
    if (profile?.onboarded) {
      return NextResponse.redirect(new URL("/discover", request.url));
    }
  }
  return response;
}

export const config = { matcher: ["/onboarding/:path*", "/discover/:path*"] };
```

Architecture / diagram

```
flowchart TD
    A[User signs up] --> B{proxy.ts: onboarded?}
    B -- yes --> Z[/discover/]
    B -- no --> C[OnboardingWizard]

    subgraph Wizard [Single client component]
      C --> D[Step 0: Hobbies]
      D --> E[Step 1: Photos]
      E --> F[Step 2: Vibe]
      F --> G[Step 3: Location]
      D -. Zustand store .-> S[(OnboardingData)]
      E -. .-> S
      F -. .-> S
      G -. .-> S
    end

    G --> H[finish: uploadPhotos]
    H --> I[Supabase Storage]
    H --> J[saveOnboarding Server Action]
    J --> K[(profiles: onboarded=true)]
    J --> Z
```

The frame stays put; only the slide changes. State funnels into one Zustand object, then commits once.

Best practices & pitfalls

- **Do** validate in *both* places — the client gate makes the UI feel responsive; the Server Action is the gate that actually matters.
- **Do** `URL.revokeObjectURL` previews when replacing a photo (shown in the store). Forgetting this leaks memory across a long session.
- **Don't** save per-step. A partially onboarded profile pollutes discovery. Commit atomically with `onboarded: true` as the final flip.
- **Don't** upload images *inside* the Server Action by serializing `File`s through it — large binaries don't belong in action payloads. Upload from the browser client, pass back URLs.
- **Do** use `AnimatePresence mode="wait"` for wizards. Without it, steps cross-fade on top of each other and look broken.
- **Don't** forget the `custom` prop. If you animate direction-aware slides but never pass `custom={direction}`, every step slides the same way regardless of Back/Next.
- **Do** make the Next button reflect state: disabled + dimmed when invalid, "Saving…" during the transition. Silent dead buttons feel broken.
- **Don't** trust client geolocation to always resolve. Always allow a typed city fallback.

- **Do** scope Storage uploads to `${user.id}/...` and back it with a Storage policy so users can't overwrite each other's photos.

Exercises

1. **Reorder guard.** Add a `?step=` URL query param synced to the wizard so a refresh keeps the user on the same step (but still blocks skipping ahead past invalid steps). Use `useSearchParams` and `router.replace`.
2. **Caption editing.** Let users override a prompt's text with their own caption, stored alongside the photo URL in `profiles.photos`.
3. **Drag to reorder photos.** Use Framer Motion's `Reorder.Group` so the first photo becomes the profile's hero image.
4. **Optimistic finish.** Wrap the redirect in `useOptimistic` so the UI shows a celebratory confetti screen immediately, rolling back if `saveOnboarding` returns an error.
5. **Resumable drafts.** Persist the Zustand store to `localStorage` (via `persist` middleware) so a user who closes the tab mid-onboarding picks up where they left off — but clear it on successful finish.

Mini-project: the "Almost there!" review step

Add a **fifth, read-only confirmation step** before Finish. It renders a compact summary card: the chosen hobbies as chips, a 2×2 grid of photo thumbnails with their prompts, the vibe as an emoji badge, and the city. Each section has an "Edit" link that jumps the wizard back to the relevant step (`setStep`) with the correct slide direction. The Finish button lives only on this step. This is the single highest-converting screen in most onboarding flows — seeing a polished preview of *themselves* makes users want to commit. Wire it so `saveOnboarding` is still only ever called once, from this review step.

Key takeaways

- A wizard is **one component, one state object, one commit** — never a stack of routes.
- `AnimatePresence` with `mode="wait"`, a `key` per step, and a `custom` direction gives professional slide transitions in ~20 lines.
- Hold answers in **Zustand**; commit atomically through a `saveOnboarding` **Server Action** that re-validates server-side and flips `onboarded`.
- **Upload images from the browser**, pass URLs to the action; keep binaries out of action payloads.
- Gate the route with `proxy.ts` so onboarded users skip the flow and newcomers can't escape it.

Up next — Chapter 21: The Discover Feed, where those freshly-minted profiles come alive in a swipeable, vibe-filtered discovery experience.

Chapter 21 — Profiles & the Identity System

> Your profile is the first thing another human sees on Viby — make it feel less like a database row and more like a person you'd want to grab coffee with.

In this chapter we leave behind the plumbing and build the part of Viby people will actually *fall in love with*: the identity layer. We'll craft the signature non-circular avatar tile, a cinematic profile header with a floating polaroid, a buttery `EditProfileSheet`, real photo uploads to Supabase Storage, and persistence that survives refreshes and devices.

What you'll learn

- How to model a `profiles` table that mirrors Supabase Auth users via a foreign key + RLS
- Building the signature **squircle identity tile** (our non-circular avatar) as a reusable component
- A cinematic header with a **floating polaroid** using Framer Motion layout + gesture animation
- An accessible, mobile-first **EditProfileSheet** with optimistic updates
- Uploading photos to **Supabase Storage** with signed-path conventions and image validation
- Persisting profile edits via **Server Actions** with `revalidatePath`

Prerequisites

- Chapter 7 — Tailwind v4 `@theme` tokens (we lean on our brand tokens here)
- Chapter 12 — Supabase Auth with `@supabase/ssr` (we reuse the server/browser clients)
- Chapter 15 — Server Actions and the `"use server"` directive
- Chapter 18 — Zustand for ephemeral UI state (the sheet's open/close + draft state)

Theory

A profile is a *projection* of an auth user, not the user itself

When someone signs up, Supabase Auth creates a row in the protected `auth.users` table. You don't own that table and shouldn't write app data into it. Instead we create our **own** `public.profiles` table whose primary key *is* the auth user's `id`. Think of `auth.users` as a passport (identity, immutable, government-issued) and `profiles` as the personality you show the world — your display name, bio, hobbies, the polaroid you picked. One passport, one public persona, linked by id.

This separation matters because:

1. **Security boundaries** — Row Level Security (RLS) on `profiles` lets anyone *read* a profile (discovery is the whole point of Viby) but only the owner can *write* it.
2. **Schema freedom** — you evolve `profiles` freely without touching auth.
3. **Realtime + joins** — every post, like, and message can join to `profiles` cheaply.

Identity is *visual*, not just data

Most apps default to a circular avatar. Circles are safe and forgettable. Viby's brand is a confident bubblegum-pink Gen-Z aesthetic, so our signature shape is a **squircle** — a superellipse rendered via CSS — wrapped in a soft pink ring. It's instantly recognizable and screenshots beautifully. The lesson: *identity components are brand surface area*. Treat the avatar like a logo.

The "floating polaroid" metaphor

The profile header is the cinematic moment. We layer a wide gradient cover, then a single **polaroid-style photo** that floats slightly above the surface with a subtle tilt and a spring on hover/drag. It reads as a physical object dropped onto a digital page — tangible, playful, premium. Framer Motion's spring physics is what sells the "weight."

Optimistic by default

When a user saves their bio, they shouldn't stare at a spinner. We update the UI immediately (optimistic), fire the Server Action, and only roll back if the server rejects. The mental model: *trust the user's intent, verify in the background.*

In practice

1. The database: a `profiles` table mirroring auth users

Run this in the Supabase SQL editor (or as a migration):

```
create table public.profiles (
  id          uuid primary key references auth.users (id) on delete cascade,
  username    text unique not null,
  display_name text not null default '',
  bio         text default '',
  hobbies     text[] default '{}',
  avatar_url  text,
  polaroid_url text,
  updated_at  timestamptz default now()
);

alter table public.profiles enable row level security;

-- Anyone can read profiles (discovery is public).
create policy "Profiles are viewable by everyone"
  on public.profiles for select
  using (true);

-- Only the owner can insert their own row.
create policy "Users can insert their own profile"
  on public.profiles for insert
  with check (auth.uid() = id);

-- Only the owner can update their own row.
create policy "Users can update their own profile"
  on public.profiles for update
  using (auth.uid() = id);
```

Auto-create a profile the moment a user signs up, using a trigger so you never have an authenticated user without a profile:

```
create function public.handle_new_user()
returns trigger language plpgsql security definer set search_path = '' as $$
begin
  insert into public.profiles (id, username, display_name)
  values (new.id, 'user_' || substr(new.id::text, 1, 8), '');
  return new;
end;
$$;

create trigger on_auth_user_created
  after insert on auth.users
  for each row execute function public.handle_new_user();
```

Create a Storage bucket for photos (Dashboard → Storage → New bucket → `avatars`, public read). Add a policy so users only write to their own folder:

```
create policy "Users manage own avatar folder"
  on storage.objects for all
  using ( bucket_id = 'avatars' and (storage.foldername(name))[1] = auth.uid()::text )
  with check ( bucket_id = 'avatars' and (storage.foldername(name))[1] = auth.uid()::text );
```

2. Typed Supabase clients (recap from Ch. 12)

```
// lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) =>
          toSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options)
          ),
      },
    }
  );
}
```

```
// lib/supabase/client.ts
import { createBrowserClient } from "@supabase/ssr";

export const createClient = () =>
  createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
```

3. The signature identity tile (squircle avatar)

A reusable Server Component — no client JS needed for the static case.

```tsx
// components/identity-tile.tsx
import Image from "next/image";
import { cn } from "@/lib/utils";

type Props = {
  src?: string | null;
  name: string;
  size?: number; // px
  className?: string;
};

export function IdentityTile({ src, name, size = 72, className }: Props) {
  const initials = name.trim().slice(0, 2).toUpperCase() || "VB";
  return (
    <div
      className={cn(
        "relative grid place-items-center overflow-hidden",
        "rounded-tile bg-brand/10 ring-2 ring-brand/60 shadow-tile",
        className
      )}
      style={{ width: size, height: size }}
    >
      {src ? (
        <Image src={src} alt={name} fill sizes={`${size}px`} className="object-cover" />
      ) : (
        <span className="font-semibold text-brand" style={{ fontSize: size * 0.32 }}>
          {initials}
        </span>
      )}
    </div>
  );
}
```

The squircle and brand tokens live in your global stylesheet (Tailwind v4 — **no** `tailwind.config.js`):

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5;
  --radius-tile: 28% / 38%; /* superellipse-ish squircle */
  --shadow-tile: 0 8px 24px -8px rgba(255, 111, 181, 0.45);
}
```

> The `28% / 38%` border-radius gives that distinctive non-circular squircle — softer than a rounded rect, edgier than a circle. That asymmetry *is* the brand.

4. The cinematic header with a floating polaroid

```tsx
// components/profile-header.tsx
"use client";

import { motion } from "framer-motion";
import Image from "next/image";
import { IdentityTile } from "./identity-tile";

type Props = {
  displayName: string;
  username: string;
  bio: string;
  avatarUrl?: string | null;
  polaroidUrl?: string | null;
};

export function ProfileHeader({ displayName, username, bio, avatarUrl, polaroidUrl }: Props) {
  return (
    <header className="relative">
      {/* Cinematic gradient cover */}
      <div className="h-40 w-full bg-gradient-to-br from-brand via-pink-400 to-fuchsia-300 sm:h-56" />

      {/* Floating polaroid */}
      {polaroidUrl && (
        <motion.div
          drag
          dragConstraints={{ left: -12, right: 12, top: -8, bottom: 8 }}
          dragElastic={0.25}
          initial={{ rotate: -4, y: 24, opacity: 0 }}
          animate={{ rotate: -4, y: 0, opacity: 1 }}
          whileHover={{ rotate: 0, scale: 1.04 }}
          transition={{ type: "spring", stiffness: 220, damping: 18 }}
          className="absolute right-5 top-20 w-28 cursor-grab rounded-sm bg-white p-2 pb-6 shadow-xl
active:cursor-grabbing sm:w-36"
        >
          <Image
            src={polaroidUrl}
            alt={`${displayName}'s polaroid`}
            width={160}
            height={160}
            className="aspect-square w-full object-cover"
          />
        </motion.div>
      )}
```

```
      {/* Identity row overlapping the cover */}
      <div className="-mt-10 flex items-end gap-4 px-5">
        <IdentityTile src={avatarUrl} name={displayName} size={88} className="shadow-lg" />
        <div className="pb-2">
          <h1 className="text-xl font-bold leading-tight">{displayName || "New Viber"}</h1>
          <p className="text-sm text-zinc-500">@{username}</p>
        </div>
      </div>

      <p className="mt-3 px-5 text-sm leading-relaxed text-zinc-700">{bio}</p>
    </header>
  );
}
```

5. The Server Action: persist + upload

```ts
// app/profile/actions.ts
"use server";

import { revalidatePath } from "next/cache";
import { createClient } from "@/lib/supabase/server";

const MAX_BYTES = 5 * 1024 * 1024; // 5 MB
const ALLOWED = ["image/jpeg", "image/png", "image/webp"];

export async function updateProfile(formData: FormData) {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) return { ok: false, error: "Not authenticated" };

  const patch: Record<string, unknown> = {
    display_name: String(formData.get("display_name") ?? "").slice(0, 80),
    bio: String(formData.get("bio") ?? "").slice(0, 280),
    updated_at: new Date().toISOString(),
  };

  // Optional photo upload (avatar). Folder is keyed by user id for RLS.
  const file = formData.get("avatar") as File | null;
  if (file && file.size > 0) {
    if (!ALLOWED.includes(file.type)) return { ok: false, error: "Unsupported file type" };
    if (file.size > MAX_BYTES) return { ok: false, error: "Image too large (max 5 MB)" };

    const ext = file.type.split("/")[1];
    const path = `${user.id}/avatar-${Date.now()}.${ext}`;
    const { error: upErr } = await supabase.storage
      .from("avatars")
      .upload(path, file, { upsert: true, contentType: file.type });
    if (upErr) return { ok: false, error: upErr.message };

    const { data: pub } = supabase.storage.from("avatars").getPublicUrl(path);
    patch.avatar_url = pub.publicUrl;
  }

  const { error } = await supabase.from("profiles").update(patch).eq("id", user.id);
  if (error) return { ok: false, error: error.message };

  revalidatePath("/profile");
```

```
  return { ok: true };
}
```

6. The EditProfileSheet (client, optimistic)

We use Zustand only for sheet open/close (ephemeral UI), and `useActionState` + `useOptimistic` for the form lifecycle.

```tsx
// components/edit-profile-sheet.tsx
"use client";

import { useActionState, useOptimistic, useRef, useState } from "react";
import { AnimatePresence, motion } from "framer-motion";
import { updateProfile } from "@/app/profile/actions";

type Profile = { display_name: string; bio: string; avatar_url?: string | null };

export function EditProfileSheet({ profile }: { profile: Profile }) {
  const [open, setOpen] = useState(false);
  const [preview, setPreview] = useState<string | null>(profile.avatar_url ?? null);
  const fileRef = useRef<HTMLInputElement>(null);

  const [optimistic, setOptimistic] = useOptimistic(profile);

  const [state, formAction, pending] = useActionState(
    async (_prev: unknown, formData: FormData) => {
      setOptimistic({
        display_name: String(formData.get("display_name") ?? ""),
        bio: String(formData.get("bio") ?? ""),
        avatar_url: optimistic.avatar_url,
      });
      const res = await updateProfile(formData);
      if (res.ok) setOpen(false);
      return res;
    },
    null
  );

  return (
    <>
      <button
        onClick={() => setOpen(true)}
        className="rounded-full bg-brand px-5 py-2 text-sm font-semibold text-white shadow-tile"
      >
        Edit profile
      </button>

      <AnimatePresence>
        {open && (
          <>
            <motion.div
```

```
            className="fixed inset-0 z-40 bg-black/40"
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            exit={{ opacity: 0 }}
            onClick={() => setOpen(false)}
          />
          <motion.div
            role="dialog"
            aria-modal="true"
            aria-label="Edit profile"
            className="fixed inset-x-0 bottom-0 z-50 rounded-t-3xl bg-white p-6"
            initial={{ y: "100%" }}
            animate={{ y: 0 }}
            exit={{ y: "100%" }}
            transition={{ type: "spring", stiffness: 300, damping: 32 }}
          >
            <div className="mx-auto mb-4 h-1.5 w-12 rounded-full bg-zinc-200" />
            <form action={formAction} className="space-y-4">
              <label className="block">
                <span className="text-sm font-medium">Display name</span>
                <input
                  name="display_name"
                  defaultValue={optimistic.display_name}
                  maxLength={80}
                  className="mt-1 w-full rounded-xl border border-zinc-200 px-3 py-2"
                />
              </label>

              <label className="block">
                <span className="text-sm font-medium">Bio</span>
                <textarea
                  name="bio"
                  defaultValue={optimistic.bio}
                  maxLength={280}
                  rows={3}
                  className="mt-1 w-full rounded-xl border border-zinc-200 px-3 py-2"
                />
              </label>

              <input
                ref={fileRef}
                type="file"
                name="avatar"
```

```
                  accept="image/jpeg,image/png,image/webp"
                  className="text-sm"
                  onChange={(e) => {
                    const f = e.target.files?.[0];
                    if (f) setPreview(URL.createObjectURL(f));
                  }}
                />
                {preview && (
                  <img src={preview} alt="" className="h-16 w-16 rounded-tile object-cover" />
                )}

                {state && !state.ok && (
                  <p className="text-sm text-red-600">{state.error}</p>
                )}

                <button
                  type="submit"
                  disabled={pending}
                  className="w-full rounded-full bg-brand py-3 font-semibold text-white
disabled:opacity-60"
                >
                  {pending ? "Saving…" : "Save changes"}
                </button>
              </form>
            </motion.div>
          </>
        )}
      </AnimatePresence>
    </>
  );
}
```

7. The profile page (Server Component, async params)

```tsx
// app/profile/[username]/page.tsx
import { notFound } from "next/navigation";
import { createClient } from "@/lib/supabase/server";
import { ProfileHeader } from "@/components/profile-header";
import { EditProfileSheet } from "@/components/edit-profile-sheet";

export default async function ProfilePage({
  params,
}: {
  params: Promise<{ username: string }>;
}) {
  const { username } = await params; // Next.js 16: params is async
  const supabase = await createClient();

  const { data: profile } = await supabase
    .from("profiles")
    .select("*")
    .eq("username", username)
    .single();

  if (!profile) notFound();

  const {
    data: { user },
  } = await supabase.auth.getUser();
  const isOwner = user?.id === profile.id;

  return (
    <main className="mx-auto max-w-xl pb-24">
      <ProfileHeader
        displayName={profile.display_name}
        username={profile.username}
        bio={profile.bio}
        avatarUrl={profile.avatar_url}
        polaroidUrl={profile.polaroid_url}
      />
      {isOwner && (
        <div className="mt-6 px-5">
          <EditProfileSheet profile={profile} />
        </div>
      )}
    </main>
```

```
  );
}
```

Architecture / diagram

```
flowchart TD
  A[auth.users<br/>Supabase Auth] -- "id FK + trigger" --> B[public.profiles<br/>RLS: public read, owner write]
  C[ProfilePage<br/>Server Component] -- "await params + getUser()" --> B
  C --> D[ProfileHeader<br/>floating polaroid]
  C --> E[EditProfileSheet<br/>'use client']
  E -- "FormData (fields + File)" --> F["updateProfile()<br/>'use server' action"]
  F -- "upload to {userId}/..." --> G[(Storage: avatars bucket)]
  F -- "update row" --> B
  F -- "revalidatePath('/profile')" --> C
  G -- "getPublicUrl" --> F
```

The flow: a request hits the Server Component, which reads the profile and decides ownership. The client sheet collects edits, hands a `FormData` (text + the `File`) to the Server Action, which validates, uploads to a user-scoped Storage folder, updates the row, and revalidates the cached page.

Best practices & pitfalls

- **Do** key Storage paths by `auth.uid()` (`${user.id}/...`) so your one RLS policy covers every file. **Don't** put files at the bucket root — you lose per-user isolation.
- **Do** validate `file.type` and `file.size` *on the server*. Client `accept` attributes are a UX hint, not security.
- **Do** call `supabase.auth.getUser()` (which re-validates with the auth server) in Server Actions — never trust `getSession()` for authorization decisions.
- **Don't** access `params` synchronously in Next.js 16 — it's a Promise. `await` it (or `use()` it in a Client Component).
- **Don't** forget cache invalidation: without `revalidatePath`, the saved profile won't reflect until a hard reload.
- **Do** add cache-busting to avatar URLs (timestamped filename, as we do) so updated photos don't serve from a stale CDN cache.
- **Don't** reach for `middleware.ts`. In our stack, middleware-style logic lives in `proxy.ts` — use it for session refresh, not profile writes.
- **Do** debounce/limit huge uploads client-side too, but treat the server limit as the real gate.
- **Don't** store the polaroid and avatar in the same field — they play different roles (identity vs. expression).

Exercises

1. **Username editing** — add a `username` field to the sheet with a uniqueness check. Surface the Postgres unique-violation error (`23505`) as a friendly "That handle's taken."
2. **Hobby chips** — render `profile.hobbies` as pink squircle chips under the bio, and add a multi-add input in the sheet that writes the `text[]` array.
3. **Drag-to-reorder polaroids** — extend to multiple polaroids using Framer Motion's `Reorder` group, persisting the new order.
4. **Skeleton loading** — add a `loading.tsx` for the profile route with a shimmering squircle placeholder.
5. **Crop before upload** — integrate a square-crop step (e.g. a canvas crop) so every avatar is a clean square before it reaches Storage.

Mini-project: "Your Viby Identity Card"

Build a self-contained `/me` route that:

1. Redirects unauthenticated users to `/login` (use the session check in `proxy.ts` from Ch. 12).
2. Fetches the current user's profile and renders the `ProfileHeader` with `isOwner` always true.
3. Shows the `EditProfileSheet` with **live optimistic preview** — typing in the display name updates the header instantly via shared state (lift state up or use a Zustand store).
4. On save, uploads a new avatar to Storage, persists via the Server Action, and shows a pink toast "Identity updated."
5. Bonus: render a tiny "share card" — the identity tile + name + first hobby chip — that you could later turn into an OG image.

Ship it mobile-first. When you can edit your name, drop a new photo, and watch the header animate in under a second, you've built the heart of a real social app.

Key takeaways

- A `profiles` table mirrors `auth.users` by id, with RLS for **public read / owner write**, auto-created via a trigger.
- The **squircle identity tile** is brand surface area — a reusable component, not an afterthought.
- The **floating polaroid** uses Framer Motion springs and drag to feel like a physical object.
- **Server Actions** handle validation, user-scoped **Storage** uploads, persistence, and `revalidatePath` in one place.
- `useOptimistic` + `useActionState` give instant, trustworthy UX without manual spinners.
- Authorize with `getUser()` , validate uploads server-side, and key files by user id.

Up next — Chapter 22: The Discovery Feed, where these profiles come alive in a swipeable, realtime grid of people you'll actually want to meet.

Chapter 22 — Discover & the Matching Algorithm

> The screen where Viby stops being a profile form and starts feeling like magic: "how did it know I'd vibe with these people?"

Every social-discovery app lives or dies by one screen — the one that hands a stranger to you and quietly says *trust me, you two will get along*. In this chapter we build Viby's **Discover** experience end to end: the browse → open → explore → decide flow, the query that fetches good candidates, and the **compatibility score** that ranks them. No black magic — just an honest, explainable function you could defend in a code review.

What you'll learn

- The mental model behind a *discovery* feed (not a dating swipe clone) and why ranking beats raw recency.
- How to fetch a sane **candidate pool** from Supabase without scanning your whole user table.
- A weighted **compatibility scoring** function over hobbies, vibe, lifestyle, proximity, and community overlap — with real TypeScript.
- Wiring it into a Next.js 16 Server Component, then layering a client card stack with Framer Motion gestures.
- The pitfalls that turn a "smart" feed into a slow, creepy, or boring one.

Prerequisites

- Chapter 9–10: the Supabase data model (`profiles`, `hobbies`, `profile_hobbies`, `communities`) and `@supabase/ssr` clients.
- Chapter 12: Server Components, Server Actions, and the `proxy.ts` auth gate.
- Chapter 17: the `geography` / lat-lng columns and the Haversine helper you wrote for "people near me."
- Chapter 19: Zustand demo state and the shadcn-style `Card`, `Button`, `Avatar` primitives.

Theory

A discovery feed answers a deceptively simple question: *of everyone on Viby, who should this user see first?* Recency ("newest profiles") is the lazy answer and it rots fast — your most active users see the same faces, newcomers drown, and nothing feels personal.

The professional answer is **ranking**: score every reasonable candidate against the viewer, sort descending, show the top slice. Think of it like a great party host. The host doesn't introduce you to all 200 guests alphabetically. They scan the room, remember you love bouldering and lo-fi and you're new in Lisbon, and walk you toward the three people most likely to become your people. That scan-and-introduce instinct is exactly what our scoring function encodes.

We split the work into two stages, and the split matters:

1. **Candidate fetching (recall).** Cheaply narrow thousands of users down to a few hundred *plausible* matches using coarse filters the database is good at — same city-ish region, active recently, not already connected or blocked. This is a SQL job.
2. **Scoring & ranking (precision).** Take that small pool into application code and compute a rich, weighted compatibility score. This is a TypeScript job.

Why not do it all in SQL? Because the interesting signal — overlapping hobbies, vibe affinity, lifestyle fit — is multi-dimensional and easier to reason about, test, and tune in code. And why not score *everyone* in code? Because pulling 50,000 rows to rank 12 cards is how you get a 4-second feed and an angry DBA. Recall is cheap and wide; precision is expensive and narrow. Keep them separate.

The score itself is a **weighted sum of normalized sub-scores**. Each dimension produces a number in `[0, 1]`, we multiply by a weight, and sum. Normalization is the part beginners skip and regret: if hobby overlap can be 0–8 and distance can be 0–500, the raw numbers aren't comparable and distance silently dominates everything. Squash each signal to `[0, 1]` *first*, weight *second*.

Our five dimensions for Viby:

Dimension	Signal	Weight
Hobbies	Shared tagged hobbies (Jaccard overlap)	0.35
Vibe	Cosine similarity of personality/vibe vector	0.20
Lifestyle	Match on schedule, social energy, going-out style	0.15
Proximity	Closer = better, with a soft falloff	0.20
Community	Shared communities/groups joined	0.10

Hobbies lead because Viby's whole premise is *shared interests build real friendships*. Weights are product decisions, not laws of physics — keep them in one config object so a product manager can tune them without touching logic.

In practice

1. The candidate query (recall)

We fetch only plausible people: roughly the same region, active in the last 30 days, and not the viewer, not blocked, not already connected. Do the heavy exclusion in the database.

```
-- supabase/migrations/0042_discover_candidates.sql
-- A SQL function keeps the recall logic in one place and lets PostgREST call it via rpc().
create or replace function public.discover_candidates(
  viewer uuid,
  region_code text,
  max_results int default 200
)
returns setof public.profiles
language sql
stable
as $$
  select p.*
  from public.profiles p
  where p.id <> viewer
    and p.region_code = region_code
    and p.last_active_at > now() - interval '30 days'
    and p.discoverable = true
    and not exists (
      select 1 from public.connections c
      where (c.a_id = viewer and c.b_id = p.id)
         or (c.b_id = viewer and c.a_id = p.id)
    )
    and not exists (
      select 1 from public.blocks b
      where (b.blocker_id = viewer and b.blocked_id = p.id)
         or (b.blocker_id = p.id and b.blocked_id = viewer)
    )
  order by p.last_active_at desc
  limit max_results;
$$;
```

> Region is a coarse bucket (e.g. metro area), not exact distance — distance is a *scoring* signal, not a *filter*. A great match 12 km away should still surface.

2. The scoring function

Pure, synchronous, framework-free. That makes it trivially unit-testable — no mocks, no database, just data in, number out.

```ts
// lib/discover/scoring.ts
export interface VibyProfile {
  id: string;
  hobbyIds: string[];        // e.g. ["bouldering", "lofi", "ramen"]
  vibe: number[];            // fixed-length vector, each value 0..1 (from onboarding quiz)
  lifestyle: {
    schedule: "early-bird" | "night-owl" | "flexible";
    socialEnergy: "introvert" | "ambivert" | "extrovert";
    goingOut: "homebody" | "balanced" | "always-out";
  };
  lat: number;
  lng: number;
  communityIds: string[];
}

// Product-tunable weights. Must sum to 1 for an interpretable 0..1 final score.
export const WEIGHTS = {
  hobbies: 0.35,
  vibe: 0.2,
  lifestyle: 0.15,
  proximity: 0.2,
  community: 0.1,
} as const;

/** Jaccard index: |A ∩ B| / |A ∪ B| — naturally normalized to [0,1]. */
function jaccard(a: string[], b: string[]): number {
  if (a.length === 0 && b.length === 0) return 0;
  const setB = new Set(b);
  const intersection = a.filter((x) => setB.has(x)).length;
  const union = new Set([...a, ...b]).size;
  return union === 0 ? 0 : intersection / union;
}

/** Cosine similarity of two equal-length vectors, clamped to [0,1]. */
function cosine(a: number[], b: number[]): number {
  if (a.length !== b.length || a.length === 0) return 0;
  let dot = 0;
  let magA = 0;
  let magB = 0;
  for (let i = 0; i < a.length; i++) {
    dot += a[i] * b[i];
    magA += a[i] * a[i];
    magB += b[i] * b[i];
```

```
  }
  if (magA === 0 || magB === 0) return 0;
  return Math.max(0, Math.min(1, dot / (Math.sqrt(magA) * Math.sqrt(magB))));
}

/** Fraction of categorical lifestyle fields that match. */
function lifestyleScore(a: VibyProfile["lifestyle"], b: VibyProfile["lifestyle"]): number {
  const keys = ["schedule", "socialEnergy", "goingOut"] as const;
  const matches = keys.filter((k) => a[k] === b[k]).length;
  return matches / keys.length;
}

/** Haversine distance in km (Chapter 17). */
function distanceKm(a: VibyProfile, b: VibyProfile): number {
  const R = 6371;
  const dLat = ((b.lat - a.lat) * Math.PI) / 180;
  const dLng = ((b.lng - a.lng) * Math.PI) / 180;
  const lat1 = (a.lat * Math.PI) / 180;
  const lat2 = (b.lat * Math.PI) / 180;
  const h =
    Math.sin(dLat / 2) ** 2 +
    Math.cos(lat1) * Math.cos(lat2) * Math.sin(dLng / 2) ** 2;
  return 2 * R * Math.asin(Math.sqrt(h));
}

/** Soft proximity score: 1 when co-located, decaying to ~0 by ~50km. */
function proximityScore(a: VibyProfile, b: VibyProfile): number {
  const km = distanceKm(a, b);
  // Exponential falloff feels natural: nearby matters a lot, far barely moves.
  return Math.exp(-km / 15); // 0km→1, 15km→0.37, 50km→0.04
}

export interface ScoredCandidate {
  profile: VibyProfile;
  score: number;
  breakdown: Record<keyof typeof WEIGHTS, number>;
}

export function scoreCandidate(viewer: VibyProfile, candidate: VibyProfile): ScoredCandidate {
  const breakdown = {
    hobbies: jaccard(viewer.hobbyIds, candidate.hobbyIds),
    vibe: cosine(viewer.vibe, candidate.vibe),
    lifestyle: lifestyleScore(viewer.lifestyle, candidate.lifestyle),
```

```
    proximity: proximityScore(viewer, candidate),
    community: jaccard(viewer.communityIds, candidate.communityIds),
  };

  const score = (Object.keys(WEIGHTS) as (keyof typeof WEIGHTS)[]).reduce(
    (sum, key) => sum + breakdown[key] * WEIGHTS[key],
    0,
  );

  return { profile: candidate, score, breakdown };
}

export function rankCandidates(viewer: VibyProfile, pool: VibyProfile[]): ScoredCandidate[] {
  return pool
    .map((c) => scoreCandidate(viewer, c))
    .sort((a, b) => b.score - a.score);
}
```

A quick sanity test — this is the kind of test that catches a weight typo before users do:

```
// lib/discover/scoring.test.ts
import { describe, it, expect } from "vitest";
import { scoreCandidate, type VibyProfile } from "./scoring";

const base: VibyProfile = {
  id: "viewer", hobbyIds: ["bouldering", "lofi"], vibe: [0.8, 0.2, 0.5],
  lifestyle: { schedule: "night-owl", socialEnergy: "ambivert", goingOut: "balanced" },
  lat: 38.72, lng: -9.14, communityIds: ["lisbon-climbers"],
};

describe("scoreCandidate", () => {
  it("returns 0..1 and ranks a twin above a stranger", () => {
    const twin = { ...base, id: "twin" };
    const stranger: VibyProfile = {
      id: "stranger", hobbyIds: ["golf"], vibe: [0, 1, 0],
      lifestyle: { schedule: "early-bird", socialEnergy: "introvert", goingOut: "homebody" },
      lat: 40.4, lng: -3.7, communityIds: [],
    };
    const t = scoreCandidate(base, twin).score;
    const s = scoreCandidate(base, stranger).score;
    expect(t).toBeGreaterThan(s);
    expect(t).toBeLessThanOrEqual(1);
    expect(s).toBeGreaterThanOrEqual(0);
  });
});
```

3. The Discover page (Server Component)

Fetch on the server, rank on the server, ship a ready-made list to the client. The viewer never waits on the browser to do math.

```tsx
// app/(app)/discover/page.tsx
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";
import { redirect } from "next/navigation";
import { rankCandidates, type VibyProfile } from "@/lib/discover/scoring";
import { DiscoverDeck } from "./discover-deck";

function toVibyProfile(row: any): VibyProfile {
  return {
    id: row.id,
    hobbyIds: row.profile_hobbies?.map((h: any) => h.hobby_id) ?? [],
    vibe: row.vibe_vector ?? [],
    lifestyle: row.lifestyle,
    lat: row.lat,
    lng: row.lng,
    communityIds: row.community_members?.map((c: any) => c.community_id) ?? [],
  };
}

export default async function DiscoverPage() {
  const cookieStore = await cookies();
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) =>
          toSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options),
          ),
      },
    },
  );

  const { data: { user } } = await supabase.auth.getUser();
  if (!user) redirect("/login");

  // Viewer profile (with the relations we score on).
  const { data: viewerRow } = await supabase
    .from("profiles")
    .select("*, profile_hobbies(hobby_id), community_members(community_id)")
    .eq("id", user.id)
```

```
    .single();

  if (!viewerRow) redirect("/onboarding");
  const viewer = toVibyProfile(viewerRow);

  // Candidate pool via the SQL recall function.
  const { data: candidateRows } = await supabase.rpc("discover_candidates", {
    viewer: user.id,
    region_code: viewerRow.region_code,
    max_results: 200,
  });

  // Hydrate relations for the pool in one batched query (avoid N+1).
  const ids = (candidateRows ?? []).map((r: any) => r.id);
  const { data: enriched } = await supabase
    .from("profiles")
    .select("*, profile_hobbies(hobby_id), community_members(community_id)")
    .in("id", ids);

  const pool = (enriched ?? []).map(toVibyProfile);
  const ranked = rankCandidates(viewer, pool).slice(0, 25);

  return (
    <main className="mx-auto min-h-dvh max-w-md px-4 py-6">
      <h1 className="mb-4 text-2xl font-bold text-fg">Discover</h1>
      <DiscoverDeck
        cards={ranked.map((r) => ({
          id: r.profile.id,
          score: r.score,
          breakdown: r.breakdown,
        }))}
      />
    </main>
  );
}
```

Tailwind v4 tokens (no `tailwind.config.js`) — the brand pink lives in CSS:

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5;
  --color-brand-fg: #ffffff;
  --color-fg: #1a1320;
  --color-muted: #8b8195;
  --radius-card: 1.5rem;
}
```

4. The card deck (Client Component)

`"use client"` only here, where we need gestures and local state. A swipe right is a Server Action call to record a "like"; the card animates out either way.

```tsx
// app/(app)/discover/discover-deck.tsx
"use client";

import { useState } from "react";
import { motion, AnimatePresence, type PanInfo } from "framer-motion";
import { decideOnCandidate } from "./actions";

type Card = { id: string; score: number; breakdown: Record<string, number> };

export function DiscoverDeck({ cards }: { cards: Card[] }) {
  const [stack, setStack] = useState(cards);
  const top = stack[0];

  async function decide(id: string, liked: boolean) {
    setStack((s) => s.slice(1)); // optimistic: pop immediately
    await decideOnCandidate(id, liked);
  }

  function onDragEnd(_: unknown, info: PanInfo, id: string) {
    if (info.offset.x > 120) decide(id, true);
    else if (info.offset.x < -120) decide(id, false);
  }

  if (!top) {
    return <p className="mt-20 text-center text-muted">You're all caught up. Check back soon.</p>;
  }

  return (
    <div className="relative h-[28rem]">
      <AnimatePresence>
        <motion.div
          key={top.id}
          drag="x"
          dragConstraints={{ left: 0, right: 0 }}
          onDragEnd={(e, info) => onDragEnd(e, info, top.id)}
          initial={{ scale: 0.95, opacity: 0 }}
          animate={{ scale: 1, opacity: 1 }}
          exit={{ x: 0, opacity: 0, transition: { duration: 0.2 } }}
          whileTap={{ scale: 0.98 }}
          className="absolute inset-0 flex flex-col justify-end rounded-[var(--radius-card)] bg-brand
p-6 text-brand-fg shadow-xl"
        >
          <span className="text-sm/relaxed opacity-90">
```

```
            {Math.round(top.score * 100)}% vibe match
          </span>
          <div className="mt-2 flex gap-3">
            <button
              onClick={() => decide(top.id, false)}
              className="rounded-full bg-white/20 px-5 py-2 font-semibold"
            >
              Skip
            </button>
            <button
              onClick={() => decide(top.id, true)}
              className="rounded-full bg-white px-5 py-2 font-semibold text-brand"
            >
              Connect
            </button>
          </div>
        </motion.div>
      </AnimatePresence>
    </div>
  );
}
```

```
// app/(app)/discover/actions.ts
"use server";

import { revalidatePath } from "next/cache";
import { createServerActionClient } from "@/lib/supabase/server";

export async function decideOnCandidate(candidateId: string, liked: boolean) {
  const supabase = await createServerActionClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) return;

  await supabase.from("decisions").insert({
    actor_id: user.id,
    target_id: candidateId,
    liked,
  });
  // A DB trigger creates a connection when both sides liked — covered in Ch. 23.
  revalidatePath("/discover");
}
```

Architecture / diagram

```
flowchart TD
  A[User opens /discover] --> B[Server Component]
  B --> C{Auth via getUser}
  C -->|no user| R[redirect /login]
  C -->|ok| D[Load viewer profile + relations]
  D --> E[rpc discover_candidates<br/>RECALL: region, active, not blocked]
  E --> F[Batch-hydrate hobbies + communities]
  F --> G[rankCandidates<br/>PRECISION: weighted score]
  G --> H[Top 25 -> Client Deck]
  H --> I[Swipe/Tap decide]
  I --> J[Server Action: insert decision]
  J --> K[(DB trigger: mutual like -> connection)]
  J --> H
```

The clean seam: the database does cheap wide **recall**, application code does rich narrow **precision**, the client does only presentation and intent capture.

Best practices & pitfalls

- **Do normalize every sub-score to `[0,1]` before weighting.** Mixing raw distance (0–500) with hobby counts (0–8) lets one signal silently bully the rest.
- **Do keep weights in one config object.** Tuning the feed should be a one-line diff, not an archaeology dig through branching logic.
- **Do filter blocked/connected users in SQL, not in the score.** A score of 0 still *renders* the person; exclusion must be a hard filter upstream.
- **Don't score your entire user table.** Recall first. Scoring 50k profiles to show 25 cards is a self-inflicted performance bug.
- **Don't make the score a black box.** Keep the `breakdown` — it powers "85% match because you both love bouldering," debugging, and trust.
- **Don't over-weight proximity.** This is discovery, not "who's literally closest." A soft exponential falloff beats a hard radius filter.
- **Avoid feedback loops.** If you only show the top-ranked people forever, newcomers never surface. Inject a little exploration (see exercises).
- **Respect privacy.** Never ship a viewer's raw lat/lng to the client; compute distance server-side and send only the score.

Exercises

1. **Tie-breaker.** When two candidates score within 0.01, break the tie by `last_active_at` (more recently active wins). Add it to `rankCandidates` without changing the score.

2. **Explain the match.** Write `topReason(breakdown)` that returns a human string like `"You both love bouldering"` for the highest-weighted contributing dimension, and render it on the card.
3. **Exploration slot.** After ranking, replace card #5 with a random candidate from the *lower half* of the pool, so newcomers get a chance. Keep the rest ranked.
4. **Cold start.** A brand-new user has no `vibe` vector and few hobbies. Make `scoreCandidate` gracefully fall back so they still get reasonable matches (hint: redistribute weight from missing dimensions).
5. **Weight tuning.** Add a `WEIGHTS_VARIANT_B` and a Zustand toggle so you can A/B two weight sets in the demo build, logging which gets more "Connect" taps.

Mini-project

Build the "Why we matched" expander. On each Discover card, add a tappable chevron that opens a Framer Motion `AnimatePresence` panel showing the five sub-scores as labeled progress bars (hobbies, vibe, lifestyle, proximity, community), each filled to `breakdown[dim] * 100`%, colored with `--color-brand`. Sort the bars descending so the strongest reason is on top. This forces you to thread the full `breakdown` from the server to the client and turns your scoring math into a trust-building feature users can actually see. Bonus: animate the bar fill on expand with `initial={{ width: 0 }}`.

Key takeaways

- Discovery is **ranking**, not recency — score plausible candidates and show the best.
- Split the work: **cheap wide recall in SQL**, **rich narrow precision in TypeScript**.
- A compatibility score is a **weighted sum of normalized sub-scores** — normalize first, weight second.
- Keep the scoring function **pure and testable**, and keep the **breakdown** for trust and debugging.
- Server Components do the fetching and math; the client handles only gestures and intent.

Up next — Chapter 23: Connections & Realtime Chat. Two people both tapped "Connect." Now we make the match real with a Postgres trigger and open a live conversation over Supabase Realtime.

Chapter 23 — Immersive Profiles & the Connect Flow

> A great profile doesn't *list* a person — it lets you *meet* them before you ever say hello.

In the last few chapters you built Viby's discovery feed: cards swipe by, people surface based on shared hobbies. But a feed card is a teaser. When a learner taps "View profile," they cross a threshold — from *browsing* to *deciding*. This chapter is about that threshold: the immersive, scrollable profile that earns a connection, and the Connect modal that turns a quiet impression into a real first message.

What you'll learn

- How to build a **scrollable immersive profile**: a swipeable photo carousel, prompt cards, a lifestyle grid, and an activity timeline.
- The **decision-after-scrolling** pattern — why Viby withholds the Connect action until someone has actually *experienced* the profile, and how to implement it without feeling gimmicky.
- How to derive and display **mutual interests** between the viewer and the profile owner.
- Building the **Connect modal** with auto-generated **icebreakers** and a personalized **intro message**, persisted through a Supabase **Server Action**.
- Composing Framer Motion gestures, `AnimatePresence`, and scroll-driven UI without jank.

Prerequisites

- **Chapter 9** (Tailwind v4 `@theme` tokens) — we lean on Viby's pink design tokens.
- **Chapter 14** (Server Components, async `params`, Server Actions).
- **Chapter 17** (Supabase via `@supabase/ssr`, `getAll` / `setAll` cookie handlers).
- **Chapter 21** (Framer Motion gestures and `AnimatePresence`).
- **Chapter 22** (the discovery feed — the profile is what a card links *into*).

Theory

A discovery feed answers *"who is out there?"* A profile answers *"do I want to know this person?"* Those are different jobs, and they need different shapes.

Think of the feed card as a **movie poster** and the profile as the **trailer**. A poster is glanceable and identical in size to every other poster. A trailer unfolds over time, controls its own pacing, and is designed to make you *feel* something before the credits. Viby's immersive profile is the trailer: full-bleed photos you swipe through, prompts that reveal personality ("My Sunday in three words…"), a lifestyle grid, and a timeline of recent activity. The viewer scrolls, and with each section they learn a little more.

This pacing is the whole point of **decision-after-scrolling**. On a dating clone you can hot-or-not from a thumbnail — but Viby is about *finding your people*, and snap judgments produce shallow connections and

regret. So we deliberately keep the primary action — **Connect** — out of reach until the viewer has scrolled through enough of the profile to have actually met the person. It's a gentle friction: not a paywall, not a nag, just an interaction design that says *slow down, this is a human*. Concretely, the floating Connect button stays dimmed and disabled until the viewer reaches (or nears) the bottom of the scroll, then springs in.

Two analogies make the mechanics click:

- **The carousel is a flipbook.** We don't load ten `<img>` tags stacked vertically; we render one active image and translate the strip horizontally on swipe. The browser only paints what's on screen.
- **Mutual interests are a Venn diagram.** The viewer has a set of hobby IDs; the profile owner has another. The intersection is gold — it's literally *why* these two should talk, and it powers the icebreakers.

The **Connect modal** is where intent becomes action. We don't drop the user onto a blank message box (intimidating, high-abandon). Instead we surface the mutual-interest Venn result, offer **icebreakers** generated from those shared hobbies ("You both love bouldering — what's your go-to gym?"), and let one tap pre-fill the intro message. The user can edit, then send. Lower friction, warmer first messages, better connections.

In practice

1. The route and data (Server Component)

Profiles live at `/profile/[username]`. In Next.js 16 the `params` prop is async, so we `await` it.

```tsx
// app/profile/[username]/page.tsx
import { notFound } from "next/navigation";
import { createClient } from "@/lib/supabase/server";
import { getCurrentUserId } from "@/lib/auth";
import { ImmersiveProfile } from "@/components/profile/immersive-profile";

export default async function ProfilePage({
  params,
}: {
  params: Promise<{ username: string }>;
}) {
  const { username } = await params;
  const supabase = await createClient();

  const { data: profile } = await supabase
    .from("profiles")
    .select(
      `id, username, display_name, age, city, bio,
       photos:profile_photos(url, position),
       prompts:profile_prompts(question, answer),
       lifestyle:profile_lifestyle(key, value),
       interests:profile_interests(hobby:hobbies(id, label, emoji)),
       activity:profile_activity(kind, label, created_at)`
    )
    .eq("username", username)
    .single();

  if (!profile) notFound();

  // The Venn intersection: which of the viewer's hobbies this person shares.
  const viewerId = await getCurrentUserId();
  const { data: mine } = await supabase
    .from("profile_interests")
    .select("hobby_id")
    .eq("profile_id", viewerId);

  const myHobbyIds = new Set((mine ?? []).map((r) => r.hobby_id));
  const mutual = profile.interests
    .map((i) => i.hobby)
    .filter((h) => myHobbyIds.has(h.id));
```

```
  return <ImmersiveProfile profile={profile} mutual={mutual} />;
}
```

The Supabase client uses the `@supabase/ssr` server helper you wired up in Chapter 17:

```
// lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) =>
          toSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options)
          ),
      },
    }
  );
}
```

2. The photo carousel (Client Component)

The carousel is interactive, so it's a Client Component. We render the whole strip but translate it horizontally; swipe distance and velocity decide whether we advance.

```tsx
"use client";

import { useState } from "react";
import { motion, AnimatePresence } from "framer-motion";

export function PhotoCarousel({ photos }: { photos: { url: string }[] }) {
  const [index, setIndex] = useState(0);
  const count = photos.length;

  const paginate = (dir: number) =>
    setIndex((i) => Math.min(Math.max(i + dir, 0), count - 1));

  return (
    <div className="relative h-[70vh] w-full overflow-hidden rounded-b-3xl bg-black">
      <AnimatePresence initial={false}>
        <motion.img
          key={index}
          src={photos[index].url}
          alt=""
          className="absolute inset-0 h-full w-full object-cover"
          initial={{ opacity: 0 }}
          animate={{ opacity: 1 }}
          exit={{ opacity: 0 }}
          transition={{ duration: 0.25 }}
          drag="x"
          dragConstraints={{ left: 0, right: 0 }}
          dragElastic={0.2}
          onDragEnd={(_, info) => {
            if (info.offset.x < -80 || info.velocity.x < -500) paginate(1);
            else if (info.offset.x > 80 || info.velocity.x > 500) paginate(-1);
          }}
        />
      </AnimatePresence>

      {/* Story-style progress segments */}
      <div className="absolute left-3 right-3 top-3 flex gap-1">
        {photos.map((_, i) => (
          <div
            key={i}
            className={`h-1 flex-1 rounded-full ${
              i <= index ? "bg-white" : "bg-white/30"
            }`}
          />
```

```
        ))}
      </div>
    </div>
  );
}
```

The `@theme` tokens (defined once in `app/globals.css`) give us Viby's brand pink everywhere:

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-viby: #ff6fb5;
  --color-viby-soft: #ffe1ef;
  --radius-card: 1.5rem;
}
```

3. Decision-after-scrolling: the gated Connect button

We track scroll progress and only enable Connect once the viewer reaches the end. This is the heart of the chapter — friction with a purpose.

```
"use client";

import { useState } from "react";
import { motion, useScroll, useMotionValueEvent } from "framer-motion";
import { PhotoCarousel } from "./photo-carousel";
import { ConnectModal } from "./connect-modal";

type Hobby = { id: string; label: string; emoji: string };

export function ImmersiveProfile({
  profile,
  mutual,
}: {
  profile: any;
  mutual: Hobby[];
}) {
  const { scrollYProgress } = useScroll();
  const [reachedEnd, setReachedEnd] = useState(false);
  const [open, setOpen] = useState(false);

  useMotionValueEvent(scrollYProgress, "change", (p) => {
    if (p > 0.85) setReachedEnd(true); // unlock near the bottom
  });

  return (
    <div className="mx-auto max-w-md pb-28">
      <PhotoCarousel photos={profile.photos} />

      <header className="px-5 pt-4">
        <h1 className="text-2xl font-bold">
          {profile.display_name}, {profile.age}
        </h1>
        <p className="text-sm text-neutral-500">{profile.city}</p>
        <p className="mt-3 leading-relaxed">{profile.bio}</p>
      </header>

      {/* Prompts */}
      <section className="mt-6 space-y-3 px-5">
        {profile.prompts.map((p: any) => (
          <div key={p.question} className="rounded-card bg-viby-soft p-4">
            <p className="text-xs font-semibold uppercase text-viby">
              {p.question}
            </p>
```

```
            <p className="mt-1 text-lg font-medium">{p.answer}</p>
          </div>
        ))}
      </section>

      {/* Lifestyle grid */}
      <section className="mt-6 grid grid-cols-2 gap-3 px-5">
        {profile.lifestyle.map((l: any) => (
          <div key={l.key} className="rounded-card border p-3">
            <p className="text-xs text-neutral-500">{l.key}</p>
            <p className="font-medium">{l.value}</p>
          </div>
        ))}
      </section>

      {/* Activity timeline */}
      <section className="mt-6 px-5">
        <h2 className="mb-3 font-semibold">Recently on Viby</h2>
        <ol className="relative border-l border-viby-soft pl-4">
          {profile.activity.map((a: any) => (
            <li key={a.created_at} className="mb-4">
              <span className="absolute -left-[5px] mt-1.5 h-2 w-2 rounded-full bg-viby" />
              <p className="text-sm">{a.label}</p>
            </li>
          ))}
        </ol>
      </section>

      {/* Floating gated Connect button */}
      <div className="fixed inset-x-0 bottom-4 mx-auto max-w-md px-5">
        <motion.button
          disabled={!reachedEnd}
          onClick={() => setOpen(true)}
          animate={{
            opacity: reachedEnd ? 1 : 0.4,
            scale: reachedEnd ? 1 : 0.98,
          }}
          whileTap={reachedEnd ? { scale: 0.96 } : undefined}
          className="w-full rounded-full bg-viby py-4 font-semibold text-white shadow-lg
disabled:cursor-not-allowed"
        >
          {reachedEnd ? `Connect with ${profile.display_name}` : "Keep scrolling to connect"}
        </motion.button>
```

```
      </div>

      <ConnectModal
        open={open}
        onClose={() => setOpen(false)}
        profile={profile}
        mutual={mutual}
      />
    </div>
  );
}
```

4. The Connect modal: mutual interests, icebreakers, intro message

We generate icebreakers from the mutual hobbies, let a tap pre-fill the message, then submit through a Server Action.

```
"use client";

import { useState } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { sendConnectRequest } from "@/app/actions/connect";

type Hobby = { id: string; label: string; emoji: string };

function icebreakersFor(name: string, mutual: Hobby[]) {
  return mutual.slice(0, 3).map(
    (h) => `You both love ${h.label.toLowerCase()} ${h.emoji} — what got ${name} into it?`
  );
}

export function ConnectModal({
  open,
  onClose,
  profile,
  mutual,
}: {
  open: boolean;
  onClose: () => void;
  profile: any;
  mutual: Hobby[];
}) {
  const [message, setMessage] = useState("");
  const [pending, setPending] = useState(false);
  const icebreakers = icebreakersFor(profile.display_name, mutual);

  async function handleSend() {
    setPending(true);
    await sendConnectRequest({ targetId: profile.id, message });
    setPending(false);
    onClose();
  }

  return (
    <AnimatePresence>
      {open && (
        <motion.div
          className="fixed inset-0 z-50 flex items-end bg-black/40"
          initial={{ opacity: 0 }}
          animate={{ opacity: 1 }}
```

```
      exit={{ opacity: 0 }}
      onClick={onClose}
    >
      <motion.div
        className="w-full rounded-t-3xl bg-white p-5"
        initial={{ y: "100%" }}
        animate={{ y: 0 }}
        exit={{ y: "100%" }}
        transition={{ type: "spring", damping: 30, stiffness: 300 }}
        onClick={(e) => e.stopPropagation()}
      >
        <h2 className="text-lg font-bold">
          Connect with {profile.display_name}
        </h2>

        {mutual.length > 0 ? (
          <div className="mt-3 flex flex-wrap gap-2">
            {mutual.map((h) => (
              <span
                key={h.id}
                className="rounded-full bg-viby-soft px-3 py-1 text-sm text-viby"
              >
                {h.emoji} {h.label}
              </span>
            ))}
          </div>
        ) : (
          <p className="mt-3 text-sm text-neutral-500">
            No shared hobbies yet — say hi and find out what clicks.
          </p>
        )}

        <p className="mt-4 text-xs font-semibold uppercase text-neutral-400">
          Icebreakers
        </p>
        <div className="mt-2 space-y-2">
          {icebreakers.map((ib) => (
            <button
              key={ib}
              onClick={() => setMessage(ib)}
              className="w-full rounded-card border border-viby-soft p-3 text-left text-sm"
            >
              {ib}
```

```
                </button>
              ))}
            </div>

            <textarea
              value={message}
              onChange={(e) => setMessage(e.target.value)}
              placeholder="Write your intro message..."
              rows={3}
              className="mt-4 w-full rounded-card border p-3 text-sm outline-none focus:border-viby"
            />

            <button
              onClick={handleSend}
              disabled={pending || message.trim().length === 0}
              className="mt-3 w-full rounded-full bg-viby py-3 font-semibold text-white
disabled:opacity-50"
            >
              {pending ? "Sending..." : "Send connect request"}
            </button>
          </motion.div>
        </motion.div>
      )}
    </AnimatePresence>
  );
}
```

5. The Server Action

```ts
// app/actions/connect.ts
"use server";

import { revalidatePath } from "next/cache";
import { createClient } from "@/lib/supabase/server";

export async function sendConnectRequest({
  targetId,
  message,
}: {
  targetId: string;
  message: string;
}) {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) throw new Error("Not authenticated");

  const clean = message.trim();
  if (clean.length === 0 || clean.length > 1000) {
    throw new Error("Message must be 1–1000 characters.");
  }

  const { error } = await supabase.from("connect_requests").insert({
    from_id: user.id,
    to_id: targetId,
    intro_message: clean,
    status: "pending",
  });

  if (error) throw new Error(error.message);
  revalidatePath("/connections");
}
```

Row-Level Security (Chapter 17) does the real enforcement on the server; never trust the client to set `from_id`. The Server Action derives it from the authenticated session.

Architecture / diagram

```
flowchart TD
  A["Feed card tap"] --> B["/profile/[username] - Server Component"]
  B --> C["Supabase select: photos, prompts, lifestyle, interests, activity"]
  B --> D["Compute mutual = viewer.hobbies ∩ owner.hobbies"]
  C --> E["ImmersiveProfile - Client"]
  D --> E
  E --> F["Carousel + Prompts + Lifestyle + Timeline"]
  F --> G{"scrollYProgress > 0.85?"}
  G -- "no" --> H["Connect button dimmed + disabled"]
  G -- "yes" --> I["Connect button springs in"]
  I --> J["ConnectModal: mutual chips + icebreakers + intro"]
  J --> K["Server Action sendConnectRequest"]
  K --> L["Supabase insert connect_requests - RLS enforced"]
```

Best practices & pitfalls

- **Do** keep data fetching in the Server Component and pass plain props down. Don't fetch the profile inside a `"use client"` component — you lose streaming and leak query logic to the browser.
- **Do** derive `from_id` server-side from `auth.getUser()`. **Don't** accept a `fromId` from the client; it's the classic impersonation hole.
- **Do** treat decision-after-scrolling as *encouragement*, not a wall. Unlock near the bottom (`> 0.85`), not at an exact pixel, and provide an accessible fallback — disabled buttons should still be reachable and explained by their label.
- **Don't** stack every photo as a tall column of `<img>` — it tanks LCP. Render the active image (or use `next/image` with `priority` on the first) and lazy-load the rest.
- **Do** stop click propagation on the modal sheet (`e.stopPropagation()`) so tapping inside doesn't dismiss it, while the backdrop tap still closes.
- **Don't** forget `key={index}` on the `AnimatePresence` child — without a changing key, the enter/exit animation won't fire.
- **Do** validate message length in the Server Action *and* rely on RLS. Client-side `disabled` is UX; the server is security.
- **Don't** auto-send an icebreaker. Pre-fill, then let the human edit — a verbatim template message reads as spam.

Exercises

1. **Read progress bar.** Add a thin pink progress bar fixed to the top that fills with `scrollYProgress` using `motion.div` and `scaleX` . (Hint: `style={{ scaleX: scrollYProgress }}` with `transformOrigin: "left"` .)
2. **Tap-to-advance carousel.** Add left/right tap zones over the photo so users can advance without dragging — match the story-style segments to the active index.

3. **Smarter icebreakers.** When the viewer and owner share a *lifestyle* value (e.g. both "Early bird"), generate an extra icebreaker referencing it, ranked above the hobby ones.
4. **Optimistic Connect.** Use `useOptimistic` to flip the button to "Request sent ✓" instantly while the Server Action runs, rolling back on error.
5. **Empty-state polish (harder).** If `mutual.length === 0`, fetch the owner's *most popular* hobby and craft a discovery icebreaker ("You haven't tried pottery yet — they're obsessed. Curious?").

Mini-project

Build the "Profile Preview" self-view. Add a `/profile/me` route that renders the *same* `ImmersiveProfile` component with the logged-in user's data, but with two differences: (1) the gated Connect button is replaced by an "Edit profile" button that's always enabled, and (2) a banner at the top reads "This is how others see you." Reuse the carousel, prompts, lifestyle grid, and timeline untouched. This proves your profile is a genuinely reusable component and gives users the confidence-building preview every great social app ships. Bonus: add a `motion` shimmer on the banner so it feels alive.

Key takeaways

- Profiles are **trailers, not posters** — they unfold over time to make the viewer *feel* the person.
- **Decision-after-scrolling** is intentional, gentle friction that produces deeper connections; gate the action near the bottom, never wall it off.
- Fetch in **Server Components**, animate in **Client Components**, and persist through **Server Actions** with server-derived identity + RLS.
- **Mutual interests** (a set intersection) are the engine behind both the Venn chips and the icebreakers.
- Pre-fill icebreakers, but always let the human edit — warmth beats automation.

Up next: Chapter 24 — *Realtime Chat & Presence*, where an accepted connect request opens a live conversation powered by Supabase Realtime, typing indicators, and read receipts.

Chapter 24 — The Activity Feed

> Every great social app has a heartbeat. In this chapter, you build Viby's.

So far, Viby has been about *who*: profiles, matching, the people you might click with. Now we build the *what they're doing* — a persistent, scrollable Activity Feed where members post a photo from their morning trail run, tag it `#hiking`, drop a location, and watch reactions roll in live. Think Strava's "I did a thing" energy crossed with Instagram's visual scroll, minus the vanity-metric pressure. This is the feature that turns a directory of strangers into a living community.

What you'll learn

- How to model a feed post in PostgreSQL with activity tags, geolocation, and timestamps
- Building a **composer** with client-side image upload to Supabase Storage
- Rendering an infinite-ish feed with Server Components, then hydrating interactivity
- **Reactions and comments** wired through Server Actions
- **Activity-type filtering** that keeps URL state shareable
- **Realtime inserts** so new posts appear without a refresh
- Row Level Security (RLS) so the feed is safe by default

Prerequisites

- Chapter 9 (Supabase Auth and `@supabase/ssr` clients)
- Chapter 12 (PostgreSQL schema design and RLS basics)
- Chapter 15 (Supabase Storage and signed/public URLs)
- Chapter 18 (Server Actions and revalidation)
- Chapter 21 (Zustand for ephemeral client state)
- Chapter 22 (Framer Motion: `motion`, `AnimatePresence`, gestures)

Theory

A feed looks simple — a list of cards — but it's secretly three hard problems stacked together: **storage**, **freshness**, and **interaction**.

Storage. A post is structured metadata (caption, tag, location, author, time) *plus* a heavy binary blob (the image). The golden rule: databases store **pointers**, not pixels. We push the image to Supabase Storage (object storage, served over a CDN) and save only its URL in PostgreSQL. Think of the database as a library card catalog and Storage as the shelves — the catalog card is tiny and fast to search; the book itself lives elsewhere.

Freshness. Traditional web apps are *pull* — you refresh, the server hands you the latest. A feed feels alive only when it's *push*: the server taps you on the shoulder and says "hey, Maya just posted." Supabase Realtime gives us this for free by streaming PostgreSQL's replication log (the same write-ahead log the database uses internally) over a WebSocket. When a row hits the `posts` table, every subscribed client hears about it in milliseconds. The analogy: instead of you walking to the mailbox every minute, the mail carrier rings your doorbell.

Interaction. Reactions and comments are *writes triggered by reading*. The instinct of a beginner is to fetch everything through client-side `useEffect`. The professional instinct in Next.js 16 is the opposite: **read on the server, write through Server Actions, and only reach for the client when you need interactivity or realtime**. Server Components fetch the initial feed (fast, SEO-friendly, no loading spinner), Server Actions mutate data with progressive enhancement, and a thin `"use client"` layer handles optimistic reactions and the live socket.

One more concept: **optimistic UI**. When you tap a heart, waiting 300ms for a round-trip feels broken. So we update the UI *immediately* (assume success), fire the mutation in the background, and only roll back if it fails. React 19's `useOptimistic` hook is built exactly for this.

In practice

1. The schema

Run this in the Supabase SQL editor. We model an `activity_type` enum so the tag set is constrained (no typo'd `#hikingg`), and we denormalize a `reaction_count` only if we measure a need — for now we count on read.

```
-- Constrained set of activity tags
create type activity_type as enum (
  'running', 'hiking', 'cycling', 'climbing',
  'yoga', 'gaming', 'cooking', 'music', 'art', 'other'
);

create table posts (
  id          uuid primary key default gen_random_uuid(),
  author_id   uuid not null references auth.users (id) on delete cascade,
  caption     text check (char_length(caption) <= 500),
  image_url   text,
  activity    activity_type not null default 'other',
  location    text,                          -- human label, e.g. "Griffith Park"
  created_at  timestamptz not null default now()
);

create table reactions (
  post_id     uuid not null references posts (id) on delete cascade,
  user_id     uuid not null references auth.users (id) on delete cascade,
  created_at  timestamptz not null default now(),
  primary key (post_id, user_id)            -- one reaction per user per post
);

create table comments (
  id          uuid primary key default gen_random_uuid(),
  post_id     uuid not null references posts (id) on delete cascade,
  author_id   uuid not null references auth.users (id) on delete cascade,
  body        text not null check (char_length(body) between 1 and 300),
  created_at  timestamptz not null default now()
);

-- Feed queries always sort by recency
create index posts_created_at_idx on posts (created_at desc);
create index comments_post_idx    on comments (post_id, created_at);
```

Now lock it down with **RLS**. The feed is public-read for signed-in members, but you can only write as yourself:

```
alter table posts     enable row level security;
alter table reactions enable row level security;
alter table comments  enable row level security;

create policy "feed is readable by authenticated users"
  on posts for select to authenticated using (true);

create policy "users insert their own posts"
  on posts for insert to authenticated
  with check (auth.uid() = author_id);

create policy "users delete their own posts"
  on posts for delete to authenticated
  using (auth.uid() = author_id);

create policy "reactions readable"
  on reactions for select to authenticated using (true);
create policy "users manage their own reactions"
  on reactions for all to authenticated
  using (auth.uid() = user_id) with check (auth.uid() = user_id);

create policy "comments readable"
  on comments for select to authenticated using (true);
create policy "users write their own comments"
  on comments for insert to authenticated
  with check (auth.uid() = author_id);
```

Then create a **public** storage bucket called `post-images` in the dashboard (Storage → New bucket → Public), and enable Realtime for the `posts` table (Database → Replication → toggle `posts`).

2. The feed (Server Component)

The page is a Server Component. It reads cookies, builds a server Supabase client, and fetches the first page. The `activity` filter comes from the URL search params so it's shareable and survives refresh. In Next.js 16, `searchParams` is async — you `await` it.

```tsx
// app/feed/page.tsx
import { createClient } from "@/lib/supabase/server";
import { Composer } from "@/components/feed/composer";
import { FilterBar } from "@/components/feed/filter-bar";
import { FeedStream } from "@/components/feed/feed-stream";

export default async function FeedPage({
  searchParams,
}: {
  searchParams: Promise<{ activity?: string }>;
}) {
  const { activity } = await searchParams;
  const supabase = await createClient();

  let query = supabase
    .from("posts")
    .select(
      `id, caption, image_url, activity, location, created_at, author_id,
       author:profiles!author_id (username, avatar_url),
       reactions (user_id),
       comments (id)`,
    )
    .order("created_at", { ascending: false })
    .limit(20);

  if (activity) query = query.eq("activity", activity);

  const { data: posts } = await query;

  return (
    <main className="mx-auto max-w-xl px-4 pb-24">
      <Composer />
      <FilterBar active={activity ?? null} />
      <FeedStream initialPosts={posts ?? []} activity={activity ?? null} />
    </main>
  );
}
```

Our server client from Chapter 9, for reference — note the `getAll` / `setAll` cookie handlers required by `@supabase/ssr`:

```
// lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) => {
          try {
            toSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options),
            );
          } catch {
            // Called from a Server Component — safe to ignore; proxy.ts refreshes the session.
          }
        },
      },
    },
  );
}
```

3. The composer (Server Action + client upload)

We upload the image straight from the browser to Storage (keeps heavy bytes off your server function), then call a Server Action with the resulting URL. The Action runs on the server, so RLS and `auth.uid()` are authoritative.

```
// app/feed/actions.ts
"use server";

import { createClient } from "@/lib/supabase/server";
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";

export async function createPost(formData: FormData) {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) redirect("/login");

  const caption = String(formData.get("caption") ?? "").slice(0, 500);
  const activity = String(formData.get("activity") ?? "other");
  const location = String(formData.get("location") ?? "") || null;
  const imageUrl = String(formData.get("imageUrl") ?? "") || null;

  const { error } = await supabase.from("posts").insert({
    author_id: user.id,
    caption,
    activity,
    location,
    image_url: imageUrl,
  });
  if (error) return { error: error.message };

  revalidatePath("/feed");
  return { ok: true };
}
```

The composer itself is a client component because it handles file selection and upload progress. Notice we use `useTransition` to keep the UI responsive while the Action runs:

```tsx
// components/feed/composer.tsx
"use client";

import { useRef, useState, useTransition } from "react";
import { createBrowserClient } from "@supabase/ssr";
import { motion, AnimatePresence } from "framer-motion";
import { createPost } from "@/app/feed/actions";

const ACTIVITIES = ["running", "hiking", "cycling", "yoga", "gaming", "art"];

export function Composer() {
  const [preview, setPreview] = useState<string | null>(null);
  const [uploading, setUploading] = useState(false);
  const [activity, setActivity] = useState("running");
  const fileRef = useRef<HTMLInputElement>(null);
  const [pending, startTransition] = useTransition();

  const supabase = createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
  );

  async function uploadImage(file: File): Promise<string> {
    setUploading(true);
    const ext = file.name.split(".").pop();
    const path = `${crypto.randomUUID()}.${ext}`;
    const { error } = await supabase.storage
      .from("post-images")
      .upload(path, file, { cacheControl: "3600", upsert: false });
    setUploading(false);
    if (error) throw error;
    const { data } = supabase.storage.from("post-images").getPublicUrl(path);
    return data.publicUrl;
  }

  async function onSubmit(formData: FormData) {
    const file = fileRef.current?.files?.[0];
    if (file) formData.set("imageUrl", await uploadImage(file));
    formData.set("activity", activity);
    startTransition(async () => {
      await createPost(formData);
      setPreview(null);
      if (fileRef.current) fileRef.current.value = "";
```

```
    });
  }

  return (
    <form
      action={onSubmit}
      className="mb-6 rounded-2xl border border-pink-100 bg-white p-4 shadow-sm"
    >
      <textarea
        name="caption"
        placeholder="What are you up to?"
        maxLength={500}
        className="w-full resize-none rounded-xl border-0 text-base outline-none placeholder:text-gray-
400"
        rows={2}
      />

      <AnimatePresence>
        {preview && (
          <motion.img
            src={preview}
            alt="preview"
            initial={{ opacity: 0, scale: 0.96 }}
            animate={{ opacity: 1, scale: 1 }}
            exit={{ opacity: 0 }}
            className="my-3 max-h-72 w-full rounded-xl object-cover"
          />
        )}
      </AnimatePresence>

      <div className="mt-2 flex items-center gap-2 overflow-x-auto pb-1">
        {ACTIVITIES.map((a) => (
          <button
            key={a}
            type="button"
            onClick={() => setActivity(a)}
            className={`shrink-0 rounded-full px-3 py-1 text-sm transition ${
              activity === a
                ? "bg-[var(--color-brand)] text-white"
                : "bg-pink-50 text-pink-600"
            }`}
          >
            #{a}
```

```
          </button>
        ))}
      </div>

      <input
        name="location"
        placeholder="Add a location"
        className="mt-2 w-full rounded-lg bg-gray-50 px-3 py-2 text-sm outline-none"
      />

      <div className="mt-3 flex items-center justify-between">
        <label className="cursor-pointer text-sm font-medium text-pink-600">
          + Photo
          <input
            ref={fileRef}
            type="file"
            accept="image/*"
            hidden
            onChange={(e) => {
              const f = e.target.files?.[0];
              if (f) setPreview(URL.createObjectURL(f));
            }}
          />
        </label>
        <button
          disabled={pending || uploading}
          className="rounded-full bg-[var(--color-brand)] px-5 py-2 text-sm font-semibold text-white
disabled:opacity-50"
        >
          {uploading ? "Uploading…" : pending ? "Posting…" : "Post"}
        </button>
      </div>
    </form>
  );
}
```

Our Tailwind v4 brand token, defined once in CSS — no `tailwind.config.js`:

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5;
  --color-brand-soft: #ffe3f1;
}
```

4. Reactions, comments, and realtime

The feed stream is a client component so it can own the live socket and optimistic state. It seeds from `initialPosts` (already fetched on the server — no spinner), then listens for inserts.

```tsx
// components/feed/feed-stream.tsx
"use client";

import { useEffect, useState } from "react";
import { createBrowserClient } from "@supabase/ssr";
import { motion, AnimatePresence } from "framer-motion";
import { PostCard, type Post } from "./post-card";

export function FeedStream({
  initialPosts,
  activity,
}: {
  initialPosts: Post[];
  activity: string | null;
}) {
  const [posts, setPosts] = useState<Post[]>(initialPosts);

  useEffect(() => setPosts(initialPosts), [initialPosts]);

  useEffect(() => {
    const supabase = createBrowserClient(
      process.env.NEXT_PUBLIC_SUPABASE_URL!,
      process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    );

    const channel = supabase
      .channel("feed-inserts")
      .on(
        "postgres_changes",
        { event: "INSERT", schema: "public", table: "posts" },
        (payload) => {
          const row = payload.new as Post;
          // Respect the active filter
          if (activity && row.activity !== activity) return;
          setPosts((prev) =>
            prev.some((p) => p.id === row.id) ? prev : [row, ...prev],
          );
        },
      )
      .subscribe();

    return () => {
      supabase.removeChannel(channel);
```

```
    };
  }, [activity]);

  return (
    <ul className="space-y-4">
      <AnimatePresence initial={false}>
        {posts.map((post) => (
          <motion.li
            key={post.id}
            layout
            initial={{ opacity: 0, y: -12 }}
            animate={{ opacity: 1, y: 0 }}
            exit={{ opacity: 0 }}
          >
            <PostCard post={post} />
          </motion.li>
        ))}
      </AnimatePresence>
    </ul>
  );
}
```

The reaction is the showcase for optimistic UI. We use React 19's `useOptimistic` so the heart fills instantly:

```tsx
// components/feed/like-button.tsx
"use client";

import { useOptimistic, useTransition } from "react";
import { motion } from "framer-motion";
import { toggleReaction } from "@/app/feed/actions";

export function LikeButton({
  postId,
  count,
  reacted,
}: {
  postId: string;
  count: number;
  reacted: boolean;
}) {
  const [, startTransition] = useTransition();
  const [optimistic, setOptimistic] = useOptimistic(
    { count, reacted },
    (_state, next: boolean) => ({
      reacted: next,
      count: count + (next ? 1 : 0) - (reacted ? 1 : 0),
    }),
  );

  return (
    <button
      onClick={() =>
        startTransition(async () => {
          setOptimistic(!optimistic.reacted);
          await toggleReaction(postId, !optimistic.reacted);
        })
      }
      className="flex items-center gap-1 text-sm"
    >
      <motion.span
        key={String(optimistic.reacted)}
        initial={{ scale: 0.6 }}
        animate={{ scale: 1 }}
        className={optimistic.reacted ? "text-[var(--color-brand)]" : "text-gray-400"}
      >
        {optimistic.reacted ? "♥" : "♡"}
      </motion.span>
```

```
      <span className="tabular-nums">{optimistic.count}</span>
    </button>
  );
}
```

The matching Server Actions:

```
// app/feed/actions.ts  (append)
export async function toggleReaction(postId: string, on: boolean) {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) return;

  if (on) {
    await supabase
      .from("reactions")
      .upsert({ post_id: postId, user_id: user.id });
  } else {
    await supabase
      .from("reactions")
      .delete()
      .match({ post_id: postId, user_id: user.id });
  }
}

export async function addComment(postId: string, body: string) {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user || !body.trim()) return;

  await supabase
    .from("comments")
    .insert({ post_id: postId, author_id: user.id, body: body.trim() });
  revalidatePath("/feed");
}
```

The `FilterBar` is just links that set the search param — no client JS needed, and the state lives in the URL:

```tsx
// components/feed/filter-bar.tsx
import Link from "next/link";

const TAGS = ["running", "hiking", "cycling", "yoga", "gaming", "art"];

export function FilterBar({ active }: { active: string | null }) {
  return (
    <nav className="mb-4 flex gap-2 overflow-x-auto pb-1">
      <Link
        href="/feed"
        className={`shrink-0 rounded-full px-3 py-1 text-sm ${
          !active ? "bg-[var(--color-brand)] text-white" : "bg-gray-100"
        }`}
      >
        All
      </Link>
      {TAGS.map((t) => (
        <Link
          key={t}
          href={`/feed?activity=${t}`}
          className={`shrink-0 rounded-full px-3 py-1 text-sm ${
            active === t ? "bg-[var(--color-brand)] text-white" : "bg-gray-100"
          }`}
        >
          #{t}
        </Link>
      ))}
    </nav>
  );
}
```

Architecture / diagram

```
flowchart TD
  U[User in browser] -->|1. select image| ST[(Supabase Storage<br/>post-images bucket)]
  ST -->|2. public URL| C[Composer client component]
  C -->|3. createPost Server Action| SA[Next.js server]
  SA -->|4. INSERT row| DB[(PostgreSQL: posts)]
  DB -->|5. replication log| RT{{Supabase Realtime}}
  RT -->|6. WebSocket INSERT event| FS[FeedStream client]
  FS -->|7. prepend card| V[Live feed UI]
  SA -->|revalidatePath /feed| V
```

The two paths to the screen: the **author** sees their post via `revalidatePath`, while **everyone else** sees it pushed through Realtime. Same data, two delivery mechanisms.

Best practices & pitfalls

- **Do** upload images from the client straight to Storage. **Don't** pipe binary through a Server Action — you'll hit body-size limits and waste serverless compute.
- **Do** rely on RLS as your real authorization boundary. **Don't** trust `author_id` sent from the client; always derive identity from `auth.getUser()` server-side.
- **Do** seed the client feed from Server-Component data. **Don't** fetch the first page in a `useEffect` — that gives users a blank flash and tanks perceived performance.
- **Do** clean up your Realtime channel in the `useEffect` return. **Don't** leave channels open on unmount — you'll leak subscriptions and double-render new posts.
- **Pitfall:** Realtime `INSERT` payloads contain only the raw row — no joined `author` or counts. Either fetch the full row by id when an event arrives, or render gracefully with the data you have and let the next refresh fill gaps.
- **Pitfall:** forgetting to toggle Realtime replication on the table in the dashboard — the socket connects but never fires. Check the Replication tab first when "nothing happens."
- **Do** constrain tags with an enum and caption length with a `check`. Validate at the database edge, not just in React.
- **Do** add `loading: "lazy"` and proper `sizes` to feed images, or use `next/image`, so a long scroll doesn't download every photo at once.

Exercises

1. **Comment composer.** Build a `CommentSheet` client component that lists a post's comments and posts new ones via `addComment`, with optimistic insert.
2. **Relative timestamps.** Render `created_at` as "3m ago" / "2h ago" using `Intl.RelativeTimeFormat`, updating on an interval — without causing hydration mismatches.

3. **Hydrate realtime rows.** When a Realtime `INSERT` arrives, fetch the full joined row (author, counts) by id before prepending, so live posts look identical to server-rendered ones.
4. **Infinite scroll.** Add cursor pagination keyed on `created_at`, loading 20 more posts when an `IntersectionObserver` sentinel enters view.
5. **Delete with undo.** Let authors delete their own post with a 5-second "Undo" toast (Framer Motion `AnimatePresence`) before the Server Action actually runs.

Mini-project — "Trail Drop"

Build a focused, shareable feed page for a single activity: `/feed/hiking`.

Requirements: - A dedicated route (`app/feed/[activity]/page.tsx`) that reads the dynamic `params` (remember: `await params` in Next.js 16) and filters server-side. - The composer pre-selects that activity and is locked to it. - Realtime is scoped so only matching posts stream in (use the `filter` option on the channel: `filter: 'activity=eq.hiking'`). - A header showing the live member count reacting today, animated with a Framer Motion count-up. - Empty state: a friendly "Be the first to drop a trail 🥾" card with a motion entrance.

Ship it, open two browser windows side by side, post from one, and watch it land in the other. That moment — the post appearing without a refresh — is the whole magic of realtime, and now you own it.

Key takeaways

- A feed = **structured metadata in Postgres + binary in Storage**; the DB holds pointers, never pixels.
- **Read on the server** (Server Components), **write through Server Actions**, reach for `"use client"` only for interactivity and realtime.
- **RLS is your real auth** — identity comes from `auth.getUser()`, never the client payload.
- `useOptimistic` makes reactions feel instant; roll back only on failure.
- **Supabase Realtime** streams the Postgres replication log over WebSocket — remember to enable replication and clean up channels.
- Keep filter state in the **URL** so feeds are shareable and refresh-safe.

Up next — Chapter 25: Direct Messages, where we turn one-to-many broadcasting into private, one-to-one realtime chat with read receipts and presence.

Chapter 25 — The Connections System

> The moment two people both tap "Connect" is where Viby stops being an app and starts being a place people belong.

So far, Viby can *show* people to each other. In this chapter we build the beating heart of social discovery: the swipe, the mutual match, and the Circle of humans a user actually gets to talk to. This is the feature that turns browsing into belonging.

What you'll learn

- How a **swipe / connect / skip** model works under the hood, and why we persist intent rather than UI state
- Designing a `connections` table and a **mutual-match** rule in PostgreSQL (with a clean, race-safe way to create a `match` on a reciprocal connect)
- Writing **Server Actions** to record connect/skip decisions safely
- Building the **Circle screen** — the grid of confirmed matches — as a Server Component
- **Surfacing connections inside chat**, so a new match instantly becomes a conversation
- Using **Framer Motion gestures** for a tactile swipe deck without a heavyweight library

Prerequisites

- Chapter 9 — Supabase Auth and the `@supabase/ssr` client helpers
- Chapter 12 — Row Level Security (RLS) and policies
- Chapter 18 — Server Actions and `revalidatePath`
- Chapter 21 — The discovery feed (`profiles`, recommendation ordering)
- Chapter 23 — Realtime chat and the `messages` table

If you can read and write a row with RLS enabled, you're ready.

Theory

A connection is **directional intent**. When Maya taps "Connect" on Theo, she is saying: *"I'd like to be in Theo's Circle."* That is a one-way arrow — `Maya → Theo`. It is meaningless to Theo until he reciprocates.

A **match** is the *symmetric* relationship that exists only when both arrows point at each other: `Maya → Theo` **and** `Theo → Maya`. Think of it like a handshake. One hand extended is an offer; the connection only "completes" when the other hand reaches back. Until then, nobody gets notified and no chat opens. This respects consent and keeps the experience pressure-free — a core promise of Viby being a discovery app, *not* a dating clone.

This gives us a clean mental model:

- **Connect** = insert a directional `connect` row.

- **Skip** = insert a directional `skip` row (we remember it so we never show that person again).
- **Match** = the derived fact that two reciprocal `connect` rows exist.

Why persist *skips*? Two reasons. First, the feed must not resurface someone you've already passed on — that feels broken and a little insulting. Second, "intent data" is gold for recommendations later (Chapter 27). We store the decision, not the swipe animation. The UI is disposable; the data is forever.

The subtle part is **when and how a match is born**. Maya connects with Theo while Theo has already connected with Maya — at that instant a match should appear in *both* Circles, atomically, exactly once. Do it naively with "read, check, write" and two simultaneous requests can both think they're first and create duplicate matches. We'll solve this the boring, bulletproof way: let the *database* decide, with a unique constraint and an `ON CONFLICT` upsert.

In practice

1. The schema

We use one table for directional decisions and one for the symmetric match. The match table stores a **canonical ordered pair** (`user_a < user_b`) so a pair can only ever produce one match row.

```
-- supabase/migrations/0025_connections.sql

create type connection_kind as enum ('connect', 'skip');

create table public.connections (
  id          uuid primary key default gen_random_uuid(),
  actor_id    uuid not null references auth.users (id) on delete cascade,
  target_id   uuid not null references auth.users (id) on delete cascade,
  kind        connection_kind not null,
  created_at  timestamptz not null default now(),
  -- one decision per (actor -> target); re-deciding updates the row
  unique (actor_id, target_id),
  check (actor_id <> target_id)
);

create table public.matches (
  id          uuid primary key default gen_random_uuid(),
  user_a      uuid not null references auth.users (id) on delete cascade,
  user_b      uuid not null references auth.users (id) on delete cascade,
  created_at  timestamptz not null default now(),
  -- canonical order guarantees one row per pair
  check (user_a < user_b),
  unique (user_a, user_b)
);

create index on public.connections (actor_id, kind);
create index on public.matches (user_a);
create index on public.matches (user_b);
```

2. The match-on-mutual-connect function

We push the "is this mutual?" logic into a single `SECURITY DEFINER` function so it runs atomically. The `ON CONFLICT DO NOTHING` makes it idempotent: even if two reciprocal connects land in the same millisecond, exactly one match row survives.

```
-- still in 0025_connections.sql

create or replace function public.record_connection(p_target uuid, p_kind connection_kind)
returns table (matched boolean, match_id uuid)
language plpgsql
security definer
set search_path = public
as $$
declare
  v_actor uuid := auth.uid();
  v_a uuid;
  v_b uuid;
  v_match_id uuid;
  v_reciprocal boolean := false;
begin
  if v_actor is null then
    raise exception 'not authenticated';
  end if;
  if v_actor = p_target then
    raise exception 'cannot connect with yourself';
  end if;

  -- 1. persist this decision (idempotent: re-deciding overwrites)
  insert into public.connections (actor_id, target_id, kind)
  values (v_actor, p_target, p_kind)
  on conflict (actor_id, target_id)
  do update set kind = excluded.kind, created_at = now();

  -- skips never create matches
  if p_kind = 'skip' then
    return query select false, null::uuid;
    return;
  end if;

  -- 2. did the target already connect with us?
  select exists (
    select 1 from public.connections
    where actor_id = p_target and target_id = v_actor and kind = 'connect'
  ) into v_reciprocal;

  if not v_reciprocal then
    return query select false, null::uuid;
    return;
```

```
  end if;

  -- 3. mutual! create the canonical match row, idempotently
  v_a := least(v_actor, p_target);
  v_b := greatest(v_actor, p_target);

  insert into public.matches (user_a, user_b)
  values (v_a, v_b)
  on conflict (user_a, user_b) do nothing
  returning id into v_match_id;

  if v_match_id is null then
    select id into v_match_id from public.matches
    where user_a = v_a and user_b = v_b;
  end if;

  return query select true, v_match_id;
end;
$$;
```

3. Row Level Security

Users may only read their own decisions and the matches they're part of. Writes go exclusively through the function above, so we keep direct insert policies tight.

```
alter table public.connections enable row level security;
alter table public.matches enable row level security;

create policy "read own connections"
  on public.connections for select
  using (auth.uid() = actor_id);

create policy "read own matches"
  on public.matches for select
  using (auth.uid() = user_a or auth.uid() = user_b);
```

> Note: because `record_connection` is `SECURITY DEFINER`, it bypasses RLS for its inserts — exactly what we want, since the user is acting on their own behalf and we validate `auth.uid()` inside.

4. The Server Action

Thin wrapper that calls the RPC, revalidates the feed and Circle, and tells the client whether a match just happened.

```ts
// app/(app)/discover/actions.ts
"use server";

import { revalidatePath } from "next/cache";
import { createServerClient } from "@/lib/supabase/server";

export type ConnectResult =
  | { ok: true; matched: boolean; matchId: string | null }
  | { ok: false; error: string };

export async function recordConnection(
  targetId: string,
  kind: "connect" | "skip",
): Promise<ConnectResult> {
  const supabase = await createServerClient();

  const { data, error } = await supabase
    .rpc("record_connection", { p_target: targetId, p_kind: kind })
    .single();

  if (error) return { ok: false, error: error.message };

  revalidatePath("/discover");
  if (kind === "connect" && data?.matched) revalidatePath("/circle");

  return { ok: true, matched: data?.matched ?? false, matchId: data?.match_id ?? null };
}
```

Here's the `createServerClient` helper from Chapter 9, for reference — note the `getAll` / `setAll` cookie handlers required by `@supabase/ssr`:

```
// lib/supabase/server.ts
import { cookies } from "next/headers";
import { createServerClient as createClient } from "@supabase/ssr";

export async function createServerClient() {
  const cookieStore = await cookies();
  return createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) => {
          try {
            toSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options),
            );
          } catch {
            // called from a Server Component — safe to ignore; proxy.ts refreshes the session
          }
        },
      },
    },
  );
}
```

5. The swipe deck (client gesture)

A focused `"use client"` component handles the drag gesture and optimistic removal. We keep the data fetching on the server and hand the deck a plain array.

```tsx
// app/(app)/discover/SwipeDeck.tsx
"use client";

import { useState, useTransition } from "react";
import { motion, AnimatePresence, type PanInfo } from "framer-motion";
import { recordConnection } from "./actions";
import { MatchCelebration } from "./MatchCelebration";

type Candidate = { id: string; displayName: string; avatarUrl: string; bio: string; vibes: string[] };

export function SwipeDeck({ initial }: { initial: Candidate[] }) {
  const [deck, setDeck] = useState(initial);
  const [matched, setMatched] = useState<Candidate | null>(null);
  const [, startTransition] = useTransition();

  function decide(card: Candidate, kind: "connect" | "skip") {
    setDeck((d) => d.filter((c) => c.id !== card.id)); // optimistic
    startTransition(async () => {
      const res = await recordConnection(card.id, kind);
      if (res.ok && res.matched) setMatched(card);
    });
  }

  const onDragEnd = (card: Candidate) => (_: unknown, info: PanInfo) => {
    if (info.offset.x > 120) decide(card, "connect");
    else if (info.offset.x < -120) decide(card, "skip");
  };

  return (
    <div className="relative mx-auto h-[520px] w-full max-w-sm">
      <AnimatePresence>
        {deck.slice(-3).map((card, i, arr) => {
          const top = i === arr.length - 1;
          return (
            <motion.div
              key={card.id}
              drag={top ? "x" : false}
              dragSnapToOrigin
              onDragEnd={top ? onDragEnd(card) : undefined}
              initial={{ scale: 0.95, y: 12 }}
              animate={{ scale: 1 - (arr.length - 1 - i) * 0.04, y: (arr.length - 1 - i) * -10 }}
              exit={{ x: 0, opacity: 0, scale: 0.8, transition: { duration: 0.2 } }}
              whileDrag={{ rotate: 0 }}
```

```
              className="absolute inset-0 rounded-3xl border border-pink-100 bg-white shadow-xl
overflow-hidden"
            >
              <img src={card.avatarUrl} alt="" className="h-72 w-full object-cover" />
              <div className="p-5">
                <h3 className="text-xl font-bold text-gray-900">{card.displayName}</h3>
                <p className="mt-1 text-sm text-gray-500">{card.bio}</p>
                <div className="mt-3 flex flex-wrap gap-2">
                  {card.vibes.map((v) => (
                    <span key={v} className="rounded-full bg-pink-50 px-3 py-1 text-xs font-medium
text-[--color-brand]">
                      {v}
                    </span>
                  ))}
                </div>
              </div>
              {top && (
                <div className="absolute inset-x-0 bottom-0 flex justify-center gap-6 p-4">
                  <button onClick={() => decide(card, "skip")} aria-label="Skip"
                    className="grid h-14 w-14 place-items-center rounded-full bg-gray-100 text-
2xl">✕</button>
                  <button onClick={() => decide(card, "connect")} aria-label="Connect"
                    className="grid h-14 w-14 place-items-center rounded-full bg-[--color-brand] text-
2xl text-white">♥</button>
                </div>
              )}
            </motion.div>
          );
        })}
      </AnimatePresence>

      {deck.length === 0 && (
        <p className="grid h-full place-items-center text-center text-gray-400">
          That's everyone for now — check back soon!
        </p>
      )}

      <AnimatePresence>
        {matched && <MatchCelebration person={matched} onClose={() => setMatched(null)} />}
      </AnimatePresence>
    </div>
  );
}
```

The brand color comes from our Tailwind v4 theme tokens — no config file:

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5;
  --color-brand-soft: #fff0f7;
}
```

6. The Circle screen (Server Component)

The Circle is the user's confirmed matches. Because it's a Server Component, we fetch with the user's session and RLS does the filtering for free.

```
// app/(app)/circle/page.tsx
import Link from "next/link";
import { createServerClient } from "@/lib/supabase/server";

export default async function CirclePage() {
  const supabase = await createServerClient();
  const { data: { user } } = await supabase.auth.getUser();

  const { data: matches } = await supabase
    .from("matches")
    .select(`
      id, created_at,
      a:profiles!matches_user_a_fkey ( id, display_name, avatar_url ),
      b:profiles!matches_user_b_fkey ( id, display_name, avatar_url )
    `)
    .order("created_at", { ascending: false });

  const people = (matches ?? []).map((m) => (m.a.id === user!.id ? m.b : m.a));

  return (
    <main className="mx-auto max-w-2xl p-4">
      <h1 className="text-2xl font-bold text-gray-900">Your Circle</h1>
      <p className="mt-1 text-sm text-gray-500">{people.length} people you both said yes to.</p>

      <ul className="mt-6 grid grid-cols-2 gap-4 sm:grid-cols-3">
        {people.map((p) => (
          <li key={p.id}>
            <Link href={`/chat/${p.id}`} className="block rounded-2xl border border-pink-100 p-3
transition hover:shadow-md">
              <img src={p.avatar_url} alt="" className="aspect-square w-full rounded-xl object-cover"
/>
              <span className="mt-2 block text-center text-sm font-semibold text-gray-800">
{p.display_name}</span>
            </Link>
          </li>
        ))}
      </ul>

      {people.length === 0 && (
        <p className="mt-16 text-center text-gray-400">No matches yet. Head to Discover and find your
people!</p>
      )}
    </main>
```

```
  );
}
```

> The `profiles!matches_user_a_fkey` syntax disambiguates the two foreign keys both pointing at `auth.users` / `profiles`. Without it, PostgREST can't tell which join you mean.

7. Surfacing connections in chat

A match should *open a door*. We don't pre-create empty conversations — instead, the chat list is **derived from matches**. Tapping a Circle member routes to `/chat/[peerId]`, and the messages query (Chapter 23) loads on demand. The chat list page reuses the same match query:

```
// app/(app)/chat/page.tsx (excerpt)
const people = (matches ?? []).map((m) => (m.a.id === user!.id ? m.b : m.a));
// render each as a chat row → Link href={`/chat/${person.id}`}
```

This keeps a single source of truth: **you can only chat with someone in your Circle.** Enforce it at the data layer too — your `messages` RLS policy should check that a `matches` row exists for the sender/recipient pair before allowing an insert.

8. The match celebration

```tsx
// app/(app)/discover/MatchCelebration.tsx
"use client";
import Link from "next/link";
import { motion } from "framer-motion";

export function MatchCelebration({
  person, onClose,
}: { person: { id: string; displayName: string; avatarUrl: string }; onClose: () => void }) {
  return (
    <motion.div
      initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }}
      className="fixed inset-0 z-50 grid place-items-center bg-[--color-brand]/90 p-6 text-white"
      onClick={onClose}
    >
      <motion.div
        initial={{ scale: 0.6, y: 30 }} animate={{ scale: 1, y: 0 }}
        transition={{ type: "spring", stiffness: 260, damping: 18 }}
        className="text-center"
      >
        <p className="text-sm font-semibold uppercase tracking-widest">It's a match</p>
        <h2 className="mt-2 text-4xl font-black">You & {person.displayName}</h2>
        <img src={person.avatarUrl} alt="" className="mx-auto mt-6 h-32 w-32 rounded-full border-4
border-white object-cover" />
        <Link href={`/chat/${person.id}`}
          className="mt-8 inline-block rounded-full bg-white px-8 py-3 font-bold text-[--color-brand]">
          Say hi 👋
        </Link>
      </motion.div>
    </motion.div>
  );
}
```

Architecture / diagram

```
sequenceDiagram
    participant U as Maya (client)
    participant SA as Server Action
    participant DB as record_connection()
    participant T as connections / matches

    U->>SA: recordConnection(theoId, "connect")
    SA->>DB: rpc record_connection
    DB->>T: upsert connection (Maya→Theo)
    DB->>T: SELECT reciprocal? (Theo→Maya, connect)
    alt reciprocal exists
        DB->>T: INSERT match (least, greatest) ON CONFLICT DO NOTHING
        DB-->>SA: { matched: true, match_id }
        SA-->>U: revalidate /circle → show MatchCelebration
    else not yet
        DB-->>SA: { matched: false }
        SA-->>U: next card
    end
```

```
connections (directional intent)        matches (symmetric, canonical)
+----------+-----------+--------+        +--------+--------+
| actor_id | target_id | kind   |        | user_a | user_b |
+----------+-----------+--------+        +--------+--------+
| Maya     | Theo      | connect|  --->  | Maya   | Theo   |  (user_a < user_b)
| Theo     | Maya      | connect|  ---^  +--------+--------+
| Maya     | Ravi      | skip   |        (created only on mutual connect)
+----------+-----------+--------+
```

Best practices & pitfalls

- **DO let the database enforce uniqueness.** The `matches` unique constraint + `ON CONFLICT DO NOTHING` is what makes simultaneous reciprocal connects safe. Never "SELECT then INSERT" in app code for this.
- **DO store the canonical pair** (`least` / `greatest`) so a pair maps to exactly one match row regardless of who connected first.
- **DON'T trust the client for `actor_id`.** Always derive it from `auth.uid()` inside the `SECURITY DEFINER` function — never accept it as a parameter.
- **DO make the action idempotent.** Re-tapping connect (double-click, retry) must not create duplicate intent or duplicate matches. The `unique (actor_id, target_id)` upsert handles this.
- **DON'T pre-create chat rooms** for every match — derive conversations from matches and lazy-load messages. Empty rooms are clutter and storage you'll regret.

- **DO gate `messages` inserts with RLS** that requires an existing match. A connection feature isn't done until people *can't* message strangers.
- **PITFALL: optimistic UI desync.** If the Server Action fails, reconcile — re-add the card or toast an error. Don't leave the deck silently out of sync with the DB.
- **PITFALL: forgetting `set search_path` in `SECURITY DEFINER` functions** — it's a real privilege-escalation vector. Always pin it.
- **DON'T resurface skipped users.** Your discover query must `LEFT JOIN` / `NOT IN` against `connections` for the current actor.

Exercises

1. **Filter the feed.** Update the Discover server query so candidates exclude anyone the current user has already connected with *or* skipped, and exclude existing matches. (Hint: `not in (select target_id from connections where actor_id = auth.uid())`.)
2. **Undo a skip.** Add a "Bring back" button that deletes the most recent `skip` row for the user and revalidates `/discover`. Keep it RLS-safe.
3. **Match count badge.** Add a live badge to the Circle nav item showing the number of matches created in the last 24 hours. Try it first with a server query, then upgrade to Supabase Realtime on the `matches` table.
4. **Pending connections (intermediate).** Build a "Liked you" screen that shows people who connected with the current user but whom the user hasn't decided on yet — without leaking it before mutual consent. Think carefully about which RLS policy makes this safe.
5. **Race-condition test (advanced).** Write a SQL or pgTAP test that fires two reciprocal `record_connection` calls concurrently and asserts exactly one `matches` row exists afterward.

Mini-project

Ship the full connect → match → chat loop.

1. Run the `0025_connections.sql` migration (tables, function, RLS).
2. Wire `app/(app)/discover/page.tsx` (Server Component) to fetch candidates excluding prior decisions, and render `<SwipeDeck initial={candidates} />`.
3. Implement the `recordConnection` Server Action and confirm both button taps and drag gestures persist rows.
4. Build the Circle screen and verify two test accounts that mutually connect both see each other (and that `MatchCelebration` fires for the second connector).
5. From a Circle card, navigate into `/chat/[peerId]` and send the first message — then add the `messages` RLS policy that *requires* a match, and prove a non-matched user gets rejected.

Success criteria: a brand-new pair can go from two swipes to a live conversation, with zero duplicate matches even under rapid double-taps.

Key takeaways

- A **connection is directional intent**; a **match is the symmetric handshake** derived from two reciprocal connects.
- Persist **both connects and skips** — the UI animation is throwaway, the decision data is permanent and powers recommendations later.
- Make match creation **atomic and idempotent** with a canonical ordered pair, a unique constraint, and `ON CONFLICT DO NOTHING` — never read-then-write in app code.
- Keep writes inside a validated `SECURITY DEFINER` RPC and reads behind tight **RLS**; derive the Circle and chat list from `matches` as a single source of truth.
- Optimistic UI + Server Actions gives a snappy swipe deck, but always **reconcile on failure**.

Up next — Chapter 26: Realtime Presence & Notifications, where we light up "online now" dots, typing indicators, and push a "You have a new match!" notification the instant that handshake completes.

Chapter 26 — Realtime Chat

> The moment two strangers who love the same obscure board game start typing to each other — *that's* Viby working. Let's build the room where it happens.

What you'll learn

- How to model `conversations` and `messages` so 1-on-1 chat scales cleanly
- Sending messages with **optimistic UI** so the bubble appears the instant you hit send
- Subscribing to **Supabase Realtime** so new messages stream in live
- **Typing indicators** and **online presence** using Realtime broadcast & presence channels
- **Unread counters** that survive refreshes and stay accurate
- A polished **conversation list** that re-sorts itself as messages arrive

Prerequisites

- Chapter 9 (Supabase Auth + `@supabase/ssr` clients)
- Chapter 12 (Postgres schema design + Row Level Security)
- Chapter 18 (Zustand for client state)
- Chapter 22 (Server Actions and the `proxy.ts` auth gate)
- Comfort with `motion` / `AnimatePresence` from Chapter 20

Theory

Chat *feels* simple — text goes in a box, appears on a screen. Under the hood it's the hardest UI in the whole app, because three different forces fight each other:

1. **Persistence** — messages must survive a refresh, so they live in Postgres.
2. **Liveness** — the other person's message must appear *now*, not after a poll.
3. **Perceived speed** — *your own* message must appear before the server even confirms it.

Think of it like a **restaurant kitchen with a pass**. When you order (send a message), the waiter writes it on a ticket and *immediately* tells you "got it" — that's **optimistic UI**, the bubble shows up grey before the kitchen confirms. The ticket goes to the kitchen (Postgres `insert`). When *any* dish is plated, a bell rings on the pass and every waiter hears it — that's **Realtime**, a broadcast everyone subscribed to the channel receives. You don't keep walking back to ask "is it ready yet?" (that's polling, and it's wasteful) — you *subscribe to the bell*.

Three Realtime primitives matter here, and they're easy to confuse:

- **Postgres Changes** — Supabase tails the database's write-ahead log and pushes `INSERT` / `UPDATE` / `DELETE` events. This is the source of truth: it fires *after* a row is committed. Use it for messages, because messages are durable.

- **Broadcast** — a lightweight, ephemeral message bus. Nothing is stored. Perfect for "Alex is typing…", which nobody needs to remember.
- **Presence** — a synchronized roster of who's currently on a channel, with automatic join/leave events. Perfect for the green "online" dot.

The rule of thumb: **if losing it on refresh is fine, use Broadcast/Presence; if it must persist, write to Postgres and listen to Postgres Changes.** Typing and online status are gloriously disposable. Messages and unread counts are not.

One more concept: the **optimistic reconciliation problem**. You insert a fake message locally with a temporary id, then the *real* row arrives via Realtime with a real id. If you're not careful you'll show it twice. We solve this with a stable `client_id` so the optimistic row and the database row are the same row.

In practice

1. The schema

Two tables, plus a `read_state` table for unread counts. Run this in the Supabase SQL editor (this builds on the migrations pattern from Chapter 12).

```
-- A conversation is a 1-on-1 thread between two users.
create table conversations (
  id uuid primary key default gen_random_uuid(),
  user_a uuid not null references auth.users (id) on delete cascade,
  user_b uuid not null references auth.users (id) on delete cascade,
  last_message text,
  last_message_at timestamptz not null default now(),
  created_at timestamptz not null default now(),
  -- order the pair so (a,b) and (b,a) can't both exist
  constraint ordered_pair check (user_a < user_b),
  constraint unique_pair unique (user_a, user_b)
);

create table messages (
  id uuid primary key default gen_random_uuid(),
  conversation_id uuid not null references conversations (id) on delete cascade,
  sender_id uuid not null references auth.users (id) on delete cascade,
  body text not null check (char_length(body) between 1 and 4000),
  client_id text not null,           -- stable id from the client for dedupe
  created_at timestamptz not null default now()
);

-- Per-user, per-conversation read cursor.
create table read_state (
  conversation_id uuid not null references conversations (id) on delete cascade,
  user_id uuid not null references auth.users (id) on delete cascade,
  last_read_at timestamptz not null default now(),
  primary key (conversation_id, user_id)
);

create index messages_conversation_idx on messages (conversation_id, created_at desc);

-- Realtime needs the table in the publication.
alter publication supabase_realtime add table messages;
```

Now lock it down with RLS so users only touch their own conversations:

```
alter table conversations enable row level security;
alter table messages enable row level security;
alter table read_state enable row level security;

create policy "see own conversations" on conversations for select
  using (auth.uid() in (user_a, user_b));

create policy "read messages in own conversations" on messages for select
  using (exists (
    select 1 from conversations c
    where c.id = conversation_id and auth.uid() in (c.user_a, c.user_b)
  ));

create policy "send as self in own conversations" on messages for insert
  with check (
    sender_id = auth.uid() and exists (
      select 1 from conversations c
      where c.id = conversation_id and auth.uid() in (c.user_a, c.user_b)
    )
  );

create policy "manage own read state" on read_state for all
  using (user_id = auth.uid()) with check (user_id = auth.uid());
```

> Why `user_a < user_b` ? It guarantees one canonical row per pair. Without it, Alex→Sam and Sam→Alex become two separate threads — a classic duplicate-conversation bug.

2. Server Action: send a message

We send through a Server Action so the write happens with the user's authenticated server client and the conversation's `last_message` updates atomically.

```ts
// app/chat/actions.ts
"use server";

import { createServerClient } from "@/lib/supabase/server";
import { revalidatePath } from "next/cache";

export async function sendMessage(
  conversationId: string,
  body: string,
  clientId: string
) {
  const supabase = await createServerClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) return { error: "Not authenticated" };

  const text = body.trim();
  if (!text) return { error: "Empty message" };

  const { error } = await supabase.from("messages").insert({
    conversation_id: conversationId,
    sender_id: user.id,
    body: text,
    client_id: clientId,
  });
  if (error) return { error: error.message };

  // Bump the conversation so the list re-sorts.
  await supabase
    .from("conversations")
    .update({ last_message: text, last_message_at: new Date().toISOString() })
    .eq("id", conversationId);

  revalidatePath("/chat");
  return { ok: true };
}
```

3. The optimistic message store (Zustand)

```ts
// stores/useChatStore.ts
import { create } from "zustand";

export type ChatMessage = {
  id: string;            // db id, or temp id while pending
  client_id: string;     // stable dedupe key
  body: string;
  sender_id: string;
  created_at: string;
  pending?: boolean;
};

type ChatState = {
  messages: ChatMessage[];
  setInitial: (m: ChatMessage[]) => void;
  addOptimistic: (m: ChatMessage) => void;
  upsertFromServer: (m: ChatMessage) => void;
};

export const useChatStore = create<ChatState>((set) => ({
  messages: [],
  setInitial: (m) => set({ messages: m }),
  addOptimistic: (m) => set((s) => ({ messages: [...s.messages, m] })),
  upsertFromServer: (incoming) =>
    set((s) => {
      // Reconcile by client_id: replace the optimistic row if present.
      const idx = s.messages.findIndex((x) => x.client_id === incoming.client_id);
      if (idx === -1) return { messages: [...s.messages, incoming] };
      const next = [...s.messages];
      next[idx] = incoming;
      return { messages: next };
    }),
}));
```

The `upsertFromServer` reconciliation is the whole trick: the optimistic row and the realtime row share a `client_id`, so the server version *replaces* the grey pending bubble instead of stacking on top of it.

4. The chat thread: realtime + optimistic send

```tsx
// app/chat/[id]/Thread.tsx
"use client";

import { use, useEffect, useRef, useState, useTransition } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { createBrowserClient } from "@/lib/supabase/client";
import { useChatStore, type ChatMessage } from "@/stores/useChatStore";
import { sendMessage } from "../actions";

export function Thread({
  conversationId,
  meId,
  initial,
}: {
  conversationId: string;
  meId: string;
  initial: ChatMessage[];
}) {
  const supabase = createBrowserClient();
  const { messages, setInitial, addOptimistic, upsertFromServer } = useChatStore();
  const [text, setText] = useState("");
  const [, startTransition] = useTransition();
  const bottomRef = useRef<HTMLDivElement>(null);

  useEffect(() => setInitial(initial), [initial, setInitial]);

  // Subscribe to new messages for this conversation.
  useEffect(() => {
    const channel = supabase
      .channel(`messages:${conversationId}`)
      .on(
        "postgres_changes",
        {
          event: "INSERT",
          schema: "public",
          table: "messages",
          filter: `conversation_id=eq.${conversationId}`,
        },
        (payload) => upsertFromServer(payload.new as ChatMessage)
      )
      .subscribe();

    return () => {
```

```
    supabase.removeChannel(channel);
  };
}, [conversationId, supabase, upsertFromServer]);

useEffect(() => {
  bottomRef.current?.scrollIntoView({ behavior: "smooth" });
}, [messages.length]);

function handleSend() {
  const body = text.trim();
  if (!body) return;
  const clientId = crypto.randomUUID();

  // 1. Show it instantly.
  addOptimistic({
    id: `temp-${clientId}`,
    client_id: clientId,
    body,
    sender_id: meId,
    created_at: new Date().toISOString(),
    pending: true,
  });
  setText("");

  // 2. Persist. Realtime will deliver the canonical row and reconcile.
  startTransition(async () => {
    const res = await sendMessage(conversationId, body, clientId);
    if (res?.error) {
      // mark the bubble as failed (left as an exercise)
      console.error(res.error);
    }
  });
}

return (
  <div className="flex h-full flex-col">
    <div className="flex-1 space-y-2 overflow-y-auto p-4">
      <AnimatePresence initial={false}>
        {messages.map((m) => {
          const mine = m.sender_id === meId;
          return (
            <motion.div
              key={m.client_id}
```

```
                initial={{ opacity: 0, y: 8, scale: 0.97 }}
                animate={{ opacity: m.pending ? 0.6 : 1, y: 0, scale: 1 }}
                className={`flex ${mine ? "justify-end" : "justify-start"}`}
              >
                <span
                  className={`max-w-[75%] rounded-2xl px-4 py-2 text-sm ${
                    mine
                      ? "bg-brand text-white"
                      : "bg-gray-100 text-gray-900"
                  }`}
                >
                  {m.body}
                </span>
              </motion.div>
            );
          })}
        </AnimatePresence>
        <div ref={bottomRef} />
      </div>

      <div className="flex gap-2 border-t p-3">
        <input
          value={text}
          onChange={(e) => setText(e.target.value)}
          onKeyDown={(e) => e.key === "Enter" && handleSend()}
          placeholder="Say hi…"
          className="flex-1 rounded-full border px-4 py-2 text-sm outline-none focus:border-brand"
        />
        <button
          onClick={handleSend}
          className="rounded-full bg-brand px-5 py-2 text-sm font-semibold text-white"
        >
          Send
        </button>
      </div>
    </div>
  );
}
```

The `bg-brand` token comes from our Tailwind v4 `@theme` block (Chapter 6):

```
/* app/globals.css */
@theme {
  --color-brand: #ff6fb5;
}
```

5. Typing indicators (Broadcast) + presence (Presence)

Both ride on a single channel. Typing is ephemeral broadcast; online status is presence.

```ts
// hooks/usePresenceAndTyping.ts
"use client";

import { useEffect, useRef, useState } from "react";
import { createBrowserClient } from "@/lib/supabase/client";

export function usePresenceAndTyping(conversationId: string, meId: string) {
  const supabase = createBrowserClient();
  const [peerTyping, setPeerTyping] = useState(false);
  const [peerOnline, setPeerOnline] = useState(false);
  const channelRef = useRef<ReturnType<typeof supabase.channel>>(null);
  const stopTimer = useRef<ReturnType<typeof setTimeout>>(null);

  useEffect(() => {
    const channel = supabase.channel(`room:${conversationId}`, {
      config: { presence: { key: meId } },
    });
    channelRef.current = channel;

    channel
      .on("broadcast", { event: "typing" }, ({ payload }) => {
        if (payload.userId === meId) return;
        setPeerTyping(true);
        clearTimeout(stopTimer.current!);
        stopTimer.current = setTimeout(() => setPeerTyping(false), 2500);
      })
      .on("presence", { event: "sync" }, () => {
        const state = channel.presenceState();
        // anyone present who isn't me => peer online
        setPeerOnline(Object.keys(state).some((k) => k !== meId));
      })
      .subscribe(async (status) => {
        if (status === "SUBSCRIBED") {
          await channel.track({ online_at: new Date().toISOString() });
        }
      });

    return () => {
      supabase.removeChannel(channel);
    };
  }, [conversationId, meId, supabase]);

  // Call this on each keystroke; it's a fire-and-forget broadcast.
```

```
  function emitTyping() {
    channelRef.current?.send({
      type: "broadcast",
      event: "typing",
      payload: { userId: meId },
    });
  }

  return { peerTyping, peerOnline, emitTyping };
}
```

Wire `emitTyping()` into the input's `onChange`, and render `peerTyping` as an animated three-dot bubble. The 2.5s timer auto-clears the indicator if keystrokes stop — broadcast has no "stopped typing" event, so we infer it.

6. Unread counters

Unread = messages newer than my `last_read_at` that I didn't send. We compute it server-side for accuracy, and reset it when the thread opens.

```
// app/chat/actions.ts (append)
export async function markRead(conversationId: string) {
  const supabase = await createServerClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) return;
  await supabase.from("read_state").upsert({
    conversation_id: conversationId,
    user_id: user.id,
    last_read_at: new Date().toISOString(),
  });
}
```

```
// lib/chat/unread.ts — used by the conversation list (Server Component)
import { createServerClient } from "@/lib/supabase/server";

export async function getUnreadCount(conversationId: string, meId: string) {
  const supabase = await createServerClient();
  const { data: rs } = await supabase
    .from("read_state")
    .select("last_read_at")
    .eq("conversation_id", conversationId)
    .eq("user_id", meId)
    .maybeSingle();

  const since = rs?.last_read_at ?? "1970-01-01";
  const { count } = await supabase
    .from("messages")
    .select("*", { count: "exact", head: true })
    .eq("conversation_id", conversationId)
    .neq("sender_id", meId)
    .gt("created_at", since);

  return count ?? 0;
}
```

Call `markRead(conversationId)` from the `Thread` once it mounts (in a `useEffect` via a transition). The badge in the list then reflects only genuinely-unseen messages.

Architecture / diagram

```
sequenceDiagram
    participant A as Alex (browser)
    participant DB as Supabase Postgres
    participant RT as Supabase Realtime
    participant S as Sam (browser)

    Note over A: types "hey!"
    A->>A: addOptimistic() — grey bubble appears
    A->>DB: sendMessage() Server Action → INSERT
    A-->>S: broadcast "typing" (ephemeral, no DB)
    DB->>RT: WAL change (committed row)
    RT-->>A: postgres_changes INSERT
    A->>A: upsertFromServer() — reconcile by client_id
    RT-->>S: postgres_changes INSERT → bubble appears
    S->>DB: markRead() on open → unread resets
```

```
Channels per conversation
─────────────────────────────────────────────

 messages:<id>   ── postgres_changes ──> durable message stream
 room:<id>       ── broadcast(typing) ──> ephemeral "typing…"
 room:<id>       ── presence ──────────> who's online right now
```

Best practices & pitfalls

- **Do** dedupe by `client_id`, not by array index or db `id`. The optimistic row has no db id yet.
- **Do** scope channels per conversation (`messages:<id>`) and always `removeChannel` in cleanup — leaked subscriptions silently multiply and you'll process the same insert five times.
- **Don't** trust the `filter` on `postgres_changes` for security. RLS is what actually stops a user reading another thread; the filter is a performance convenience.
- **Don't** store typing state in Postgres. A row write per keystroke will hammer your database and your bill. Broadcast is free-flowing and ephemeral for a reason.
- **Do** compute unread counts on the server. A purely client-side counter drifts the moment a tab was closed when a message arrived.
- **Don't** forget `alter publication supabase_realtime add table messages;` — the single most common "why isn't realtime firing?" cause.
- **Do** debounce or throttle `emitTyping()` if you broadcast on every keystroke at scale; once per ~300ms is plenty.
- **Do** handle the failed-send case — flip `pending` to a `failed` flag and offer a retry, rather than leaving a ghost bubble.

Exercises

1. **Failed-send retry.** Add a `status: "sending" | "sent" | "failed"` to `ChatMessage`. When `sendMessage` returns an error, mark the bubble failed and render a tap-to-retry icon.
2. **Typing dots component.** Build a `<TypingBubble />` with three dots that bounce using Framer Motion's `animate` and staggered `transition.delay`. Show it only when `peerTyping` is true, wrapped in `AnimatePresence`.
3. **Read receipts.** Using the peer's `read_state.last_read_at`, render a subtle "Seen" under the last message you sent if its `created_at` is older than their read cursor.
4. **Live unread badge.** Make the conversation-list badge update without a refresh by subscribing to `messages` inserts at the list level and incrementing the count for non-active threads.
5. **Pagination.** The thread loads only the latest 50 messages. Add "load older" that fetches the previous page by `created_at` cursor and prepends without breaking scroll position.

Mini-project — "Viby DMs" end to end

Build a working two-pane chat at `/chat`:

- **Left pane:** a `ConversationList` Server Component that fetches the user's conversations ordered by `last_message_at desc`, renders each with the peer's avatar, the `last_message` preview, a relative timestamp, and an unread badge from `getUnreadCount`. The bubblegum-pink badge only shows when count > 0.
- **Right pane:** the `Thread` from this chapter, with optimistic send, realtime delivery, a presence dot in the header ("Online" in pink when `peerOnline`), and a typing bubble.
- On open, call `markRead`. On send, watch the optimistic bubble reconcile.

Acceptance test: open two browsers (one incognito) signed in as two users who matched in an earlier chapter. Type in one — the dots appear in the other within a beat, the message lands live, the unread badge clears on open, and your own bubble never duplicates. Ship it to Vercel and DM a classmate.

Key takeaways

- **Persist messages in Postgres; stream them with `postgres_changes`.** That's your durable backbone.
- **Optimistic UI + `client_id` reconciliation** gives instant-feeling sends with zero duplicates.
- **Broadcast for typing, Presence for online** — ephemeral signals never touch the database.
- **Unread counts belong on the server**, derived from a per-user read cursor.
- **RLS is the real security boundary;** realtime filters are only for efficiency.

Up next — Chapter 27: Push Notifications & Background Sync, so the conversation pulls people back in even when Viby is closed.

Chapter 27 — Groups & Notifications

> A social app stops being a feed and starts being a *community* the moment people can gather in rooms and feel the tap on the shoulder when something happens.

So far Viby connects people one-to-one. In this chapter we give them places to belong — **Groups** (hobby communities people create and join) — and the nervous system that keeps everyone in the loop: a **realtime notifications** system that fires on new connections, messages, comments, reactions, and group invites, surfacing them as live toasts.

What you'll learn

- How to model many-to-many membership (`groups` , `group_members`) and a group feed in PostgreSQL with Row Level Security.
- How to build create/join flows with **Server Actions** and optimistic UI.
- The architecture of a **notifications** table and why notifications are *materialized rows*, not computed on the fly.
- How to subscribe to Supabase **Realtime** changes from a Client Component and pop **Framer Motion** toasts.
- How to keep an unread badge accurate without hammering the database.

Prerequisites

- Chapter 9–11: Supabase Auth, the `@supabase/ssr` client helpers, and RLS basics.
- Chapter 18: Server Actions and `revalidatePath` .
- Chapter 22: the connections/friendship model (`connections` table).
- Chapter 24: Realtime fundamentals (you built the live chat there — we reuse the same `postgres_changes` subscription pattern).
- Chapter 26: the Zustand store and our toast primitive.

Theory

Groups are a join table wearing a costume

A "community" feels like a big, fuzzy concept, but underneath it's the most classic relational pattern there is: **many-to-many**. A user can be in many groups; a group has many users. You never store an array of member IDs on the group row (that path leads to race conditions and unqueryable data). Instead you put a thin **join table** between them — `group_members` — where each row says "user X is in group Y, with role R."

Think of it like a coat-check: the group is the cloakroom, each `group_members` row is a numbered ticket. The ticket carries a little metadata (your role: `member` , `admin` , `owner`) and lets either side look the other up

instantly.

The **group feed** is then just posts scoped to a `group_id` — the same posts table you've used, with one extra foreign key.

Notifications are *events you saved*, not events you compute

Here's the single most important idea in this chapter, and the one beginners get wrong: **a notification is a stored row, not a query.**

It's tempting to think "the user's notifications are: all unread messages + all new comments on their posts + …" and compute that with a giant join on every page load. Don't. That's slow, it's impossible to mark individual items read, and it can't represent "Sara reacted then un-reacted."

Instead, treat a notification as a **fact that happened at a point in time** — like a stamped letter dropped in a mailbox. When an event occurs (someone comments, reacts, sends an invite), you *write a row* into a `notifications` table addressed to the recipient. That row is immutable history. The mailbox (the bell icon) just lists rows and tracks which envelopes have been opened (`read_at`).

This "write on event" model is called a **materialized** or **fan-out-on-write** notification system. It costs a tiny write when the event happens and makes every read cheap.

Realtime is a phone line you leave off the hook

Polling ("any new notifications yet? …yet? …yet?") is the rude way. Supabase Realtime lets the client **subscribe** to inserts on the `notifications` table filtered to *its own* rows. Postgres streams the new row down a WebSocket the instant it's committed. Your React component receives it and shows a toast. No polling, no refresh — the tap on the shoulder.

In practice

1. The schema

```sql
-- Groups (communities)
create table public.groups (
  id          uuid primary key default gen_random_uuid(),
  slug        text unique not null,
  name        text not null,
  description text,
  cover_url   text,
  created_by  uuid not null references auth.users (id) on delete cascade,
  created_at  timestamptz not null default now()
);

create table public.group_members (
  group_id  uuid not null references public.groups (id) on delete cascade,
  user_id   uuid not null references auth.users (id) on delete cascade,
  role      text not null default 'member' check (role in ('member','admin','owner')),
  joined_at timestamptz not null default now(),
  primary key (group_id, user_id)
);

-- Group feed: posts already exist; add an optional group scope
alter table public.posts
  add column group_id uuid references public.groups (id) on delete cascade;

-- Notifications: one row per event, addressed to a recipient
create table public.notifications (
  id           uuid primary key default gen_random_uuid(),
  recipient_id uuid not null references auth.users (id) on delete cascade,
  actor_id     uuid references auth.users (id) on delete set null,
  type         text not null
                 check (type in ('connection','message','comment','reaction','group_invite')),
  entity_id    uuid,            -- the post/message/group this is about
  metadata     jsonb not null default '{}',
  read_at      timestamptz,
  created_at   timestamptz not null default now()
);

create index notifications_recipient_idx
  on public.notifications (recipient_id, created_at desc);
```

A few deliberate choices: `actor_id` is **nullable with** `on delete set null` so a deleted account doesn't erase your history (the notification just becomes "someone"). `entity_id` + `type` together tell the UI where to deep-link. `read_at IS NULL` is our definition of *unread* — cleaner than a boolean because it records *when* it was read.

2. Row Level Security

```
alter table public.groups          enable row level security;
alter table public.group_members   enable row level security;
alter table public.notifications   enable row level security;

-- Anyone signed in can discover groups; only members see member lists
create policy "groups are public to read"
  on public.groups for select using (true);

create policy "members read their memberships"
  on public.group_members for select
  using (
    exists (
      select 1 from public.group_members gm
      where gm.group_id = group_members.group_id
        and gm.user_id = auth.uid()
    )
  );

create policy "users join groups as themselves"
  on public.group_members for insert
  with check (user_id = auth.uid());

-- Notifications: you can ONLY ever read your own
create policy "read own notifications"
  on public.notifications for select
  using (recipient_id = auth.uid());

create policy "update own notifications"
  on public.notifications for update
  using (recipient_id = auth.uid());
```

Notice there's **no client-side INSERT policy on `notifications`**. Notifications are written by *Server Actions* using the server client (or, even better, a Postgres trigger / service role) so a malicious client can't forge a notification addressed to someone else.

3. A reusable "notify" helper (Server)

```ts
// lib/notifications.ts
import "server-only";
import { createServerSupabase } from "@/lib/supabase/server";

type NotifyInput = {
  recipientId: string;
  actorId: string;
  type: "connection" | "message" | "comment" | "reaction" | "group_invite";
  entityId?: string;
  metadata?: Record<string, unknown>;
};

export async function notify(input: NotifyInput) {
  // Never notify yourself about your own action.
  if (input.recipientId === input.actorId) return;

  const supabase = await createServerSupabase();
  const { error } = await supabase.from("notifications").insert({
    recipient_id: input.recipientId,
    actor_id: input.actorId,
    type: input.type,
    entity_id: input.entityId ?? null,
    metadata: input.metadata ?? {},
  });
  if (error) console.error("[notify] failed:", error.message);
}
```

> Recall from Chapter 11 that `createServerSupabase` wraps `createServerClient` from `@supabase/ssr` with the `getAll` / `setAll` cookie handlers. We `await` it because Next.js 16 `cookies()` is async.

4. Server Actions: join a group, comment, and fan out

```ts
// app/groups/[slug]/actions.ts
"use server";

import { revalidatePath } from "next/cache";
import { createServerSupabase } from "@/lib/supabase/server";
import { notify } from "@/lib/notifications";

export async function joinGroup(groupId: string) {
  const supabase = await createServerSupabase();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error("Not authenticated");

  const { error } = await supabase
    .from("group_members")
    .insert({ group_id: groupId, user_id: user.id });
  if (error && error.code !== "23505") throw error; // ignore "already a member"

  revalidatePath(`/groups`);
}

export async function commentOnPost(postId: string, body: string) {
  const supabase = await createServerSupabase();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error("Not authenticated");

  const { data: post } = await supabase
    .from("posts")
    .select("id, author_id, group_id")
    .eq("id", postId)
    .single();
  if (!post) throw new Error("Post not found");

  await supabase.from("comments").insert({
    post_id: postId,
    author_id: user.id,
    body,
  });

  // Fan out: tell the post's author someone commented.
  await notify({
    recipientId: post.author_id,
    actorId: user.id,
    type: "comment",
```

```
    entityId: postId,
    metadata: { groupId: post.group_id },
  });

  revalidatePath(`/posts/${postId}`);
}
```

5. The Join button — optimistic with `useTransition`

```tsx
// components/groups/join-button.tsx
"use client";

import { useState, useTransition } from "react";
import { motion } from "framer-motion";
import { joinGroup } from "@/app/groups/[slug]/actions";

export function JoinButton({ groupId, joined }: { groupId: string; joined: boolean }) {
  const [isMember, setIsMember] = useState(joined);
  const [isPending, startTransition] = useTransition();

  function handleJoin() {
    setIsMember(true); // optimistic
    startTransition(async () => {
      try {
        await joinGroup(groupId);
      } catch {
        setIsMember(false); // roll back on failure
      }
    });
  }

  return (
    <motion.button
      whileTap={{ scale: 0.94 }}
      onClick={handleJoin}
      disabled={isMember || isPending}
      className="rounded-full px-5 py-2 text-sm font-semibold text-white
                 bg-[var(--color-brand)] disabled:opacity-60 transition"
    >
      {isMember ? "Joined" : "Join group"}
    </motion.button>
  );
}
```

Our brand pink lives in a Tailwind v4 theme token (defined once in `app/globals.css`):

```
@import "tailwindcss";

@theme {
  --color-brand: #FF6FB5;
  --color-brand-ink: #B11D6E;
}
```

6. The Notification Bell — subscribe + toast

This is the payoff. A Client Component opens a Realtime subscription scoped to the current user and renders a live unread badge. New rows trigger a Framer Motion toast.

```tsx
// components/notifications/notification-bell.tsx
"use client";

import { useEffect, useState } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { createBrowserSupabase } from "@/lib/supabase/client";

type Notification = {
  id: string;
  type: string;
  actor_id: string | null;
  metadata: Record<string, unknown>;
  read_at: string | null;
};

const COPY: Record<string, string> = {
  connection: "wants to connect with you",
  message: "sent you a message",
  comment: "commented on your post",
  reaction: "reacted to your post",
  group_invite: "invited you to a group",
};

export function NotificationBell({
  userId,
  initialUnread,
}: {
  userId: string;
  initialUnread: number;
}) {
  const [unread, setUnread] = useState(initialUnread);
  const [toast, setToast] = useState<Notification | null>(null);

  useEffect(() => {
    const supabase = createBrowserSupabase();

    const channel = supabase
      .channel(`notifications:${userId}`)
      .on(
        "postgres_changes",
        {
          event: "INSERT",
          schema: "public",
```

```
          table: "notifications",
          filter: `recipient_id=eq.${userId}`,
        },
        (payload) => {
          const n = payload.new as Notification;
          setUnread((c) => c + 1);
          setToast(n);
          setTimeout(() => setToast(null), 4000);
        }
      )
      .subscribe();

    return () => {
      supabase.removeChannel(channel);
    };
  }, [userId]);

  return (
    <>
      <button className="relative" aria-label={`Notifications, ${unread} unread`}>
        <BellIcon />
        {unread > 0 && (
          <span
            className="absolute -top-1 -right-1 min-w-5 h-5 px-1 grid place-items-center
                       rounded-full bg-[var(--color-brand)] text-white text-xs font-bold"
          >
            {unread > 9 ? "9+" : unread}
          </span>
        )}
      </button>

      <AnimatePresence>
        {toast && (
          <motion.div
            initial={{ opacity: 0, y: 24, scale: 0.96 }}
            animate={{ opacity: 1, y: 0, scale: 1 }}
            exit={{ opacity: 0, y: 12, scale: 0.98 }}
            transition={{ type: "spring", stiffness: 380, damping: 28 }}
            className="fixed bottom-6 right-6 z-50 max-w-xs rounded-2xl bg-white
                       shadow-lg ring-1 ring-black/5 px-4 py-3 text-sm"
          >
            <span className="font-semibold text-[var(--color-brand-ink)]">Someone </span>
            {COPY[toast.type] ?? "did something"}
```

```
          </motion.div>
        )}
      </AnimatePresence>
    </>
  );
}

function BellIcon() {
  return (
    <svg width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="currentColor"
      strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
      <path d="M6 8a6 6 0 0 1 12 0c0 7 3 9 3 9H3s3-2 3-9" />
      <path d="M10.3 21a1.94 1.94 0 0 0 3.4 0" />
    </svg>
  );
}
```

The bell receives its `initialUnread` count from the Server Component that renders the layout, so the badge is correct on first paint and Realtime keeps it live afterward:

```
// app/(app)/layout.tsx  (excerpt — Server Component)
import { createServerSupabase } from "@/lib/supabase/server";
import { NotificationBell } from "@/components/notifications/notification-bell";

export default async function AppLayout({ children }: { children: React.ReactNode }) {
  const supabase = await createServerSupabase();
  const { data: { user } } = await supabase.auth.getUser();

  const { count } = await supabase
    .from("notifications")
    .select("id", { count: "exact", head: true })
    .is("read_at", null);

  return (
    <div>
      <header className="flex items-center justify-end p-4">
        {user && <NotificationBell userId={user.id} initialUnread={count ?? 0} />}
      </header>
      {children}
    </div>
  );
}
```

> One setup step: in the Supabase dashboard, enable Realtime for the `notifications` table (Database → Replication → add the table to the `supabase_realtime` publication). RLS still applies to Realtime, so a user only ever receives *their own* rows even though everyone subscribes to the same table.

7. Marking read

```
// app/(app)/notifications/actions.ts
"use server";
import { createServerSupabase } from "@/lib/supabase/server";
import { revalidatePath } from "next/cache";

export async function markAllRead() {
  const supabase = await createServerSupabase();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) return;

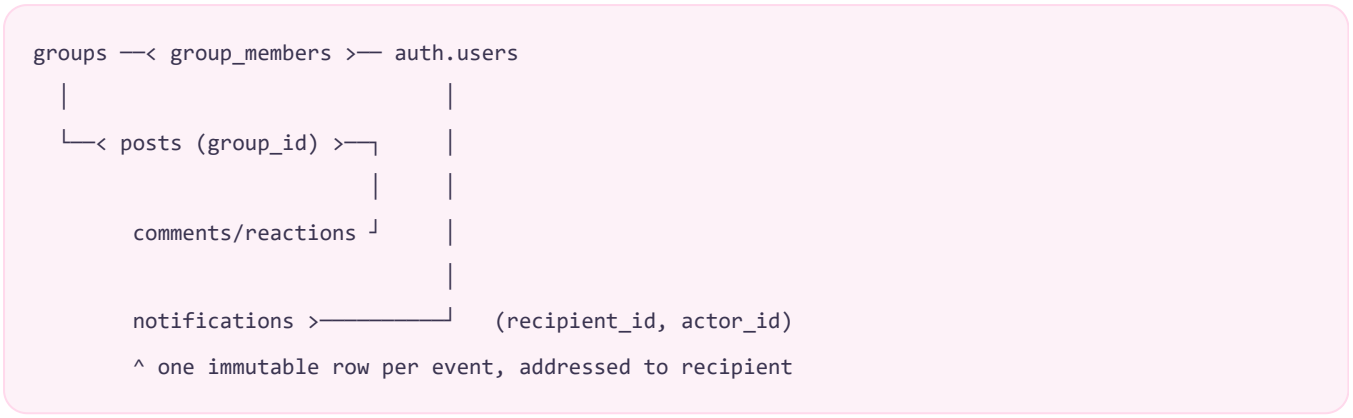
  await supabase
    .from("notifications")
    .update({ read_at: new Date().toISOString() })
    .is("read_at", null); // RLS already scopes this to the current user

  revalidatePath("/notifications");
}
```

Architecture / diagram

```
sequenceDiagram
    participant A as Actor (Sara)
    participant SA as Server Action
    participant DB as Postgres (notifications)
    participant RT as Supabase Realtime
    participant B as Recipient browser (Max)

    A->>SA: commentOnPost(postId, "love this!")
    SA->>DB: insert comment
    SA->>DB: notify(recipient=Max, type=comment)
    DB-->>RT: WAL change (INSERT)
    RT-->>B: postgres_changes (filtered recipient_id=Max)
    B->>B: setUnread(+1) + spring toast
```



Best practices & pitfalls

- **Do** write notifications server-side only. Never give clients INSERT rights — they'd forge notifications for other users.
- **Do** guard against self-notification (`recipientId === actorId`). Nobody wants "You commented on your own post."
- **Don't** compute notifications with a giant join at read time. Materialize one row per event.
- **Do** rely on RLS for Realtime security — the `filter` in `.on()` is a convenience, not a security boundary. The policy is what actually protects rows.
- **Don't** forget to `removeChannel` in the `useEffect` cleanup, or you'll leak WebSocket subscriptions and double-fire toasts in React Strict Mode.
- **Do** collapse noisy events (e.g., 12 reactions → "Sara and 11 others reacted") by updating `metadata` on an existing unread row instead of inserting 12 rows. Add this once you feel the volume.
- **Don't** store member IDs as an array column on `groups` . Use the join table — it's queryable, RLS-able, and race-free.
- **Do** index `notifications(recipient_id, created_at desc)` — the bell query sorts on exactly this.
- **Don't** trust `count` from a stale client; seed from the server on load, then let Realtime adjust deltas.

Exercises

1. **Member avatars.** On the group page, render a stacked avatar row of the first 5 members plus a "+N" bubble. Query `group_members` joined to profiles, ordered by `joined_at` .
2. **Leave a group.** Add a `leaveGroup` Server Action and toggle the `JoinButton` between Join/Leave with optimistic state and a roll-back on error.
3. **Deep links.** Make each notification clickable: build a `hrefFor(notification)` helper that routes `comment` / `reaction` to `/posts/{entity_id}` , `group_invite` to `/groups/{slug}` , and `connection` to the actor's profile.
4. **Group invites.** Add an `inviteToGroup(groupId, userId)` action that inserts a `group_invite` notification, and accept/decline buttons in the toast that call back into Server Actions.

5. **Reaction collapsing (hard).** Before inserting a `reaction` notification, check for an existing *unread* reaction notification on the same `entity_id`; if found, bump a `count` in its `metadata` and update `created_at` instead of inserting a new row.

Mini-project — "Viby Circles"

Build a complete community page at `/groups/[slug]`:

1. **Server Component** loads the group by `slug`, its member count, and the latest 20 posts where `group_id = group.id` (newest first).
2. A **header** with the cover image, name, description, member count, and the optimistic `JoinButton`.
3. A **group feed** of posts; each post has a comment box wired to `commentOnPost`. Posting a comment fans out a `comment` notification to the post's author.
4. The shared **NotificationBell** in the layout: open two browser profiles, comment as one user, and watch the spring toast and badge increment live for the other — with **zero page refresh**.
5. Bonus: add an "Invite a friend" button that writes a `group_invite` notification, and confirm it toasts on the recipient's screen in realtime.

Ship it, then DM the link to a friend — that loop (join → post → get notified) is the heartbeat of every social product you'll ever build.

Key takeaways

- Communities are a **many-to-many** relationship: `groups` + a `group_members` join table, never an array column.
- A notification is a **stored, immutable row addressed to a recipient** — fan-out-on-write, not computed-on-read.
- `read_at IS NULL` is a cleaner "unread" definition than a boolean, because it records *when*.
- **Server Actions** (or triggers) write notifications; clients never get INSERT rights.
- **Supabase Realtime** + RLS streams each user only their own new rows; Framer Motion turns them into delightful toasts.
- Seed counts from the server for a correct first paint, then let Realtime apply live deltas.

Up next — Chapter 28: Search & Discovery, where we make Viby findable: full-text search on people and groups, trending hobbies, and a "people you might vibe with" recommendation feed.

Chapter 28 — Performance & Optimization

> A fast app feels like magic; a slow one feels broken — even when every feature works perfectly.

By now your Viby app *does* a lot: profiles, hobby feeds, realtime chat, image uploads. This chapter is where you make it *feel* premium. Performance isn't a final polish step — it's a product feature your users feel in their thumbs.

What you'll learn

- What **Core Web Vitals** (LCP, INP, CLS) actually measure and why Google ranks you on them
- Shipping **optimized images** with `next/image` and Supabase Storage
- **Code-splitting & lazy loading** heavy client components with `next/dynamic`
- **Memoization** and how to stop React from re-rendering the whole feed
- **Analyzing your bundle** to find the 200KB library you forgot about
- **Virtualizing** Viby's infinite hobby feed so 5,000 cards don't crater the page
- Keeping **Framer Motion** animations buttery at 60fps

Prerequisites

- Chapter 11 (Server vs. Client Components) — we lean hard on the Server Component default
- Chapter 17 (Supabase Storage & uploads) — we optimize the avatars you stored there
- Chapter 22 (Framer Motion) — we revisit those animations through a performance lens
- Chapter 25 (the infinite feed) — the list we're about to virtualize

---

Theory

Why "fast" is not one thing

Performance is really three separate questions a user asks subconsciously:

1. **Did it show up?** (loading)
2. **Did it respond when I tapped?** (interactivity)
3. **Did it stop jumping around?** (visual stability)

Google formalized these as **Core Web Vitals**:

- **LCP — Largest Contentful Paint**: when the biggest thing in the viewport (usually a hero image or heading) finishes rendering. Target: **under 2.5s**. *Analogy: how long until the restaurant actually puts food on your table.*
- **INP — Interaction to Next Paint**: how quickly the UI visibly reacts to taps/clicks across the whole session. Target: **under 200ms**. (INP replaced the old FID metric in 2024.) *Analogy: how fast the waiter*

looks up when you wave.

- **CLS — Cumulative Layout Shift**: how much content jumps around as things load. Target: **under 0.1**. *Analogy: the table not wobbling every time a dish arrives.*

These aren't vanity numbers. They affect SEO ranking, and they correlate directly with bounce rate. A Gen-Z user on a mid-range Android over spotty 4G is unforgiving.

Where time actually goes

The two biggest performance sins in a React app are almost always:

- **Shipping too much JavaScript** — every KB must be downloaded, parsed, and executed on a phone CPU far slower than your MacBook.
- **Re-rendering too often** — React re-running components that didn't need to change.

The good news: Next.js 16's App Router defaults fight both for you. **Server Components ship zero JS to the browser.** A component is server-rendered unless you opt out with `"use client"`. So the cheapest optimization is simply *not* reaching for `"use client"` out of habit.

The mental model for re-renders

React re-renders a component when its state changes, its props change, or its parent re-renders. That last one surprises people: a parent re-rendering re-renders *all* its children by default, even children whose props are identical. Memoization (`React.memo`, `useMemo`, `useCallback`) is how you tell React "skip this — nothing it depends on changed."

But memoization isn't free, and premature memoization is clutter. **Measure first.** React 19's compiler (when enabled) auto-memoizes a lot of this, but understanding the manual tools makes you a better engineer.

In practice

1. Images: the single biggest LCP win

Viby is image-heavy — avatars, hobby cover photos, profile galleries. Unoptimized images are the #1 cause of bad LCP. Always use `next/image`: it serves modern formats (AVIF/WebP), generates responsive `srcset`s, lazy-loads off-screen images, and *reserves space* so nothing shifts (helping CLS).

First, allow your Supabase Storage domain in `next.config.ts`:

```
// next.config.ts
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  images: {
    remotePatterns: [
      {
        protocol: "https",
        hostname: "YOUR_PROJECT.supabase.co",
        pathname: "/storage/v1/object/public/**",
      },
    ],
  },
};

export default nextConfig;
```

Now a hobby card. Note `priority` on above-the-fold images (skips lazy-loading so LCP fires sooner), explicit `sizes` so the browser downloads the right resolution, and `fill` with a sized parent to prevent layout shift:

```tsx
// components/feed/HobbyCard.tsx  (Server Component — no "use client")
import Image from "next/image";

type Hobby = { id: string; title: string; coverUrl: string };

export function HobbyCard({ hobby, priority = false }: { hobby: Hobby; priority?: boolean }) {
  return (
    <article className="overflow-hidden rounded-2xl bg-white shadow-sm">
      <div className="relative aspect-[4/3] w-full">
        <Image
          src={hobby.coverUrl}
          alt={hobby.title}
          fill
          priority={priority}
          sizes="(max-width: 640px) 100vw, (max-width: 1024px) 50vw, 33vw"
          className="object-cover"
        />
      </div>
      <h3 className="p-4 font-semibold text-zinc-900">{hobby.title}</h3>
    </article>
  );
}
```

Mark only the *first* few cards `priority`; lazy-load the rest (the default).

2. Code-splitting heavy client components

Viby's emoji picker for chat is ~150KB and most users never open it. Don't make everyone pay for it on first load. Split it with `next/dynamic`, and skip SSR for browser-only widgets:

```tsx
// components/chat/MessageComposer.tsx
"use client";

import { useState } from "react";
import dynamic from "next/dynamic";

// Loaded only when needed; never server-rendered.
const EmojiPicker = dynamic(() => import("./EmojiPicker"), {
  ssr: false,
  loading: () => <div className="h-72 w-72 animate-pulse rounded-xl bg-pink-50" />,
});

export function MessageComposer() {
  const [showEmoji, setShowEmoji] = useState(false);

  return (
    <div className="relative">
      <button
        onClick={() => setShowEmoji((v) => !v)}
        className="rounded-full px-3 py-2 text-[#FF6FB5]"
        aria-label="Add emoji"
      >
        😊
      </button>
      {showEmoji && <EmojiPicker />}
    </div>
  );
}
```

The emoji bundle is now a separate chunk fetched on first click — not in your initial page load.

3. Memoization: stop the feed from re-rendering

Suppose Viby's feed has a like counter in a parent. Every time the count changes, the parent re-renders — and so does every `HobbyCard`, even though their data didn't change. Wrap the card in `React.memo`, and stabilize the callback with `useCallback`:

```tsx
// components/feed/LikeableCard.tsx
"use client";

import { memo, useCallback, useState } from "react";

type Props = { id: string; title: string; onLike: (id: string) => void };

// Re-renders only when its own props change by reference.
export const LikeableCard = memo(function LikeableCard({ id, title, onLike }: Props) {
  console.log("render", id); // watch this stop firing for unchanged cards
  return (
    <button onClick={() => onLike(id)} className="rounded-2xl bg-white p-4 shadow-sm">
      {title} 💖
    </button>
  );
});

export function Feed({ hobbies }: { hobbies: { id: string; title: string }[] }) {
  const [likes, setLikes] = useState(0);

  // Stable reference across renders → memo'd children don't re-render.
  const handleLike = useCallback((id: string) => {
    setLikes((n) => n + 1);
    console.log("liked", id);
  }, []);

  return (
    <>
      <p className="mb-3 text-sm text-zinc-500">{likes} likes</p>
      {hobbies.map((h) => (
        <LikeableCard key={h.id} id={h.id} title={h.title} onLike={handleLike} />
      ))}
    </>
  );
}
```

Without `useCallback`, `handleLike` would be a *new* function every render, breaking `memo`. Use `useMemo` similarly for expensive computed values (e.g., filtering/sorting a big hobby list) — not for cheap ones.

4. List virtualization for the infinite feed

When Viby's feed grows to thousands of cards, the browser chokes on DOM nodes — even memoized ones, because they all exist. **Virtualization** renders only the rows currently in (and near) the viewport. We'll use

`@tanstack/react-virtual`:

```
npm install @tanstack/react-virtual
```

```tsx
// components/feed/VirtualFeed.tsx
"use client";

import { useRef } from "react";
import { useVirtualizer } from "@tanstack/react-virtual";
import { HobbyCard } from "./HobbyCard";

type Hobby = { id: string; title: string; coverUrl: string };

export function VirtualFeed({ hobbies }: { hobbies: Hobby[] }) {
  const parentRef = useRef<HTMLDivElement>(null);

  const virtualizer = useVirtualizer({
    count: hobbies.length,
    getScrollElement: () => parentRef.current,
    estimateSize: () => 320, // estimated card height in px
    overscan: 5, // render a few extra above/below for smooth scroll
  });

  return (
    <div ref={parentRef} className="h-[100dvh] overflow-auto">
      <div style={{ height: virtualizer.getTotalSize(), position: "relative" }}>
        {virtualizer.getVirtualItems().map((item) => (
          <div
            key={hobbies[item.index].id}
            style={{
              position: "absolute",
              top: 0,
              left: 0,
              width: "100%",
              transform: `translateY(${item.start}px)`,
            }}
          >
            <HobbyCard hobby={hobbies[item.index]} priority={item.index < 3} />
          </div>
        ))}
      </div>
    </div>
  );
}
```

5,000 hobbies, ~12 DOM cards at a time. The scrollbar still feels full because the spacer div has the *total* height.

5. Analyzing your bundle

You can't optimize what you can't see. Add the analyzer:

```
npm install --save-dev @next/bundle-analyzer
```

```
// next.config.ts
import type { NextConfig } from "next";
import withBundleAnalyzer from "@next/bundle-analyzer";

const nextConfig: NextConfig = {
  images: { /* ...as above... */ },
};

const analyzer = withBundleAnalyzer({ enabled: process.env.ANALYZE === "true" });
export default analyzer(nextConfig);
```

```
# package.json script: "analyze": "ANALYZE=true next build"
ANALYZE=true npm run build
```

A treemap opens in your browser. Look for surprises — a date library, an icon set imported whole, a `"use client"` component that dragged a server-only dependency into the client. Fix by importing narrowly (`import { format } from "date-fns/format"`) or moving logic server-side.

6. Animation performance

Framer Motion is fast *if* you animate the right properties. Animating `transform` (`x`, `y`, `scale`, `rotate`) and `opacity` is GPU-accelerated and never triggers layout. Animating `width`, `height`, `top`, or `margin` forces layout recalculation on every frame and causes jank.

```
// components/feed/CardReveal.tsx
"use client";

import { motion } from "framer-motion";

export function CardReveal({ children }: { children: React.ReactNode }) {
  return (
    <motion.div
      initial={{ opacity: 0, y: 16 }}   // ✅ transform + opacity = 60fps
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.3, ease: "easeOut" }}
      // Respect users who get motion sick:
      style={{ willChange: "transform, opacity" }}
    >
      {children}
    </motion.div>
  );
}
```

Avoid animating `height: 0 → auto`; animate `scaleY` or use a fixed/measured height instead. And honor `prefers-reduced-motion` — Framer Motion's `MotionConfig` with `reducedMotion="user"` does this app-wide.

Architecture / diagram

How a request becomes a fast, stable paint:

```
flowchart TD
    A[Request /feed] --> B[Server Component renders on server]
    B --> C{Component needs browser APIs / interactivity?}
    C -- No --> D[Ship HTML only — 0 KB JS]
    C -- Yes --> E[Mark 'use client' → JS chunk]
    E --> F{Heavy & rarely used?}
    F -- Yes --> G[next/dynamic → separate lazy chunk]
    F -- No --> H[Bundled in route chunk]
    D --> I[next/image streams AVIF/WebP, reserves space]
    G --> I
    H --> I
    I --> J[Virtualizer renders only visible rows]
    J --> K[Good LCP / low CLS / fast INP ✅]
```

The throughline: **default to the server, opt into the client deliberately, and only ship what the user can actually see.**

Best practices & pitfalls

- **Do** keep components as Server Components until you genuinely need state, effects, or browser APIs. `"use client"` is a cost, not a default.
- **Do** set `priority` on the one or two largest above-the-fold images — and *only* those.
- **Do** always provide `sizes` with `fill` images, or you'll download desktop-sized images to phones.
- **Don't** memoize everything reflexively. `memo` on a component whose props change every render just adds an extra comparison.
- **Don't** put `useCallback` / `useMemo` around trivial values — the bookkeeping can cost more than the work.
- **Don't** animate layout-affecting properties (`width` , `top` , `margin`). Use `transform` .
- **Do** virtualize any list that can exceed a few hundred items. Pagination + virtualization compound nicely.
- **Don't** import whole libraries (`import _ from "lodash"`). Import the function (`import debounce from "lodash/debounce"`).
- **Do** measure with the Network tab throttled to "Slow 4G" and CPU 4x slowdown — that's your real user.
- **Pitfall:** `ssr: false` in `next/dynamic` only works inside Client Components. Calling it from a Server Component throws.

Exercises

1. **Audit one route.** Run `ANALYZE=true npm run build` on the `/feed` route. Screenshot the treemap and name the three largest chunks. Replace one whole-library import with a narrow one and re-measure the delta.
2. **Kill a layout shift.** Find an image in Viby rendered with a plain `<img>` and no dimensions. Convert it to `next/image` with `fill` + aspect-ratio parent. Confirm CLS drops in Lighthouse.
3. **Lazy-load a modal.** Pick a rarely-used component (settings panel, report dialog) and convert it to `next/dynamic` with `ssr: false` and a skeleton `loading` state.
4. **Catch a rogue re-render.** Add a `console.log` to a feed card. Trigger a parent state change and watch every card log. Fix it with `memo` + `useCallback` so only relevant cards re-render.
5. **Virtualize & profile.** Render 2,000 mock hobbies without virtualization, record the React Profiler commit time, then add `VirtualFeed` and compare.

Mini-project: The Buttery Feed

Build a production-grade `/discover` page that ties the whole chapter together.

Requirements:

1. A **Server Component** page that fetches the first page of hobbies from Supabase (`createServerClient` with `getAll` / `setAll` cookie handlers from Chapter 14).
2. Cards rendered with `next/image` , correct `sizes` , and `priority` on the first three.

3. The list passed into a `VirtualFeed` client component using `@tanstack/react-virtual`.
4. Each card wrapped in a `memo`'d `CardReveal` using Framer Motion (`opacity` + `y` only).
5. A lazy-loaded **filter sheet** (`next/dynamic`, `ssr: false`) that opens on tap.
6. Wrap the app in `<MotionConfig reducedMotion="user">` to respect accessibility.

Definition of done: Lighthouse mobile run scores **LCP < 2.5s, CLS < 0.1, INP < 200ms** on a throttled profile, and the bundle analyzer shows the filter sheet in its *own* chunk — not the initial load.

Starter for the page shell:

```
// app/discover/page.tsx  (Server Component)
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";
import { VirtualFeed } from "@/components/feed/VirtualFeed";

export default async function DiscoverPage() {
  const cookieStore = await cookies();
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) => toSet.forEach(({ name, value, options }) =>
          cookieStore.set(name, value, options)),
      },
    },
  );

  const { data: hobbies } = await supabase
    .from("hobbies")
    .select("id, title, coverUrl:cover_url")
    .limit(200);

  return (
    <main className="mx-auto max-w-3xl">
      <h1 className="px-4 py-6 text-2xl font-bold text-[#FF6FB5]">Discover your people</h1>
      <VirtualFeed hobbies={hobbies ?? []} />
    </main>
  );
}
```

Key takeaways

- **Core Web Vitals** — LCP (loading), INP (interactivity), CLS (stability) — are the metrics that define "fast" and affect your ranking.
- **Server Components ship zero JS.** The cheapest optimization is not opting into `"use client"`.
- `next/image` with `priority`, `sizes`, and reserved space is your biggest single LCP + CLS win.
- `next/dynamic` keeps rarely-used heavy code out of the initial bundle.
- `memo` + `useCallback` / `useMemo` stop needless re-renders — but measure before reaching for them.
- **Virtualize** long lists; **analyze** your bundle; **animate only** `transform` **and** `opacity`**.**

Up next — Chapter 29: Testing & Quality Assurance, where we make sure all this speed never ships a regression: unit tests, component tests, and end-to-end flows for Viby.

Chapter 29 — Caching, Server Components & Scalable Architecture

> The difference between an app that feels instant and one that feels broken is almost never the database — it's *where* you decided to do the work.

By now Viby works. People sign up, build hobby profiles, discover their people, and chat in realtime. But "works on my laptop with three users" and "works on Vercel with three thousand" are different sports. This chapter is where you stop being someone who *makes features* and start being someone who *makes systems*.

What you'll learn

- The four caching layers in Next.js 16 and how `revalidate`, tags, and `dynamic` actually behave
- Streaming UI with `<Suspense>` so slow data never blocks a fast page
- A decision framework for **where logic lives**: client, Server Component, Server Action, or data layer
- A scalable **feature-first folder architecture** that survives 50+ screens
- A clean **data-access layer (DAL)** that centralizes every query and auth check
- How the **demo (Zustand) vs. connected (Supabase)** seam holds up as the app grows

Prerequisites

- Chapter 7 (Server vs. Client Components) and Chapter 11 (Server Actions & forms)
- Chapter 18 (Supabase Auth with `@supabase/ssr`) and Chapter 21 (Realtime chat)
- Chapter 24 (the demo/connected pattern with Zustand) — we extend that seam here

---

Theory

Caching is just "remembering so you don't redo work"

Imagine a barista who memorizes your regular order. The first time, they ask everything. After that, you say "the usual" and coffee appears. Caching is that — but you have to decide *how long* "the usual" stays valid before they re-ask. Too short and you're slow; too long and you serve a stale order.

Next.js 16 gives you **four places** to remember things:

1. **Request Memoization** — within a *single* render pass, identical `fetch()` calls (or React `cache()`-wrapped functions) run once. Free, automatic, lasts one request. This is why you can call your "get current user" helper in five components without five queries.
2. **Data Cache** — persists fetched data *across* requests and deployments until you invalidate it. This is the big one for performance.
3. **Full Route Cache** — caches the rendered HTML/RSC payload of static routes at build time.

4. **Router Cache** — a client-side, in-memory cache of visited routes for snappy back/forward navigation.

The mental shift in the App Router: **rendering is dynamic by default unless you opt into caching.** In Next.js 16, `fetch` is **not** cached unless you say so, and reading dynamic inputs (cookies, headers, `searchParams`) makes a route dynamic. You choose static behavior deliberately.

Static, dynamic, and revalidation

Three knobs control a route segment:

- **Static** — rendered once, reused. Perfect for a marketing page or a public hobby directory.
- **Dynamic** — rendered per request. Required when output depends on the logged-in user.
- **Revalidate (ISR)** — static, but refreshed on a timer or on demand. "Cache this, but re-bake it every 60 seconds" or "re-bake it the moment someone edits their profile."

On-demand revalidation uses **tags**. You tag cached data, then call `revalidateTag('hobbies')` from a Server Action after a mutation. The barista forgets only *that* order, not the whole menu.

Streaming and Suspense: don't let the slow guest hold the door

Picture a dinner party. Some dishes are ready instantly; the soufflé takes ten minutes. You don't make every guest stand outside until the soufflé is done — you seat them, serve bread, and bring the soufflé when it's ready.

`<Suspense>` is that. You send the fast shell (header, nav, profile card) immediately, show a skeleton where the slow part will go, and **stream** the slow component in when its data resolves. The user sees a meaningful page in ~100ms instead of staring at a white screen for a slow query.

Where does logic go?

This is the question that separates juniors from seniors. The rule of thumb:

Concern	Lives in
Reading data for first paint	Server Component (calls the DAL)
Mutations (create/update/delete)	Server Action (`"use server"`)
Interactivity, animation, local UI state	Client Component (`"use client"`)
Database queries + auth checks	**Data-access layer** (one place)
Cross-cutting routing rules (auth redirects)	`proxy.ts`

Push data fetching **down** to the leaf that needs it (so Suspense can isolate it) and push secrets/queries **into** the DAL (so nothing leaks to the client and auth is never forgotten).

In practice

The data-access layer (one front door for your data)

Everything that touches Supabase goes through `src/data/`. No component imports the Supabase client directly. This is your security boundary and your caching boundary.

```
// src/data/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options),
            );
          } catch {
            // Called from a Server Component render — safe to ignore;
            // proxy.ts refreshes the session cookie instead.
          }
        },
      },
    },
  );
}
```

```
// src/data/profiles.ts
import "server-only";
import { cache } from "react";
import { createClient } from "@/data/supabase/server";

// React cache(): dedupes within a single request (Request Memoization).
export const getCurrentUser = cache(async () => {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();
  return user;
});

export async function getProfileByHandle(handle: string) {
  const supabase = await createClient();
  const { data, error } = await supabase
    .from("profiles")
    .select("id, handle, display_name, bio, avatar_url, hobbies(name, emoji)")
    .eq("handle", handle)
    .single();

  if (error) throw new Error("Profile not found");
  return data;
}
```

The `import "server-only"` line is a tripwire: if anyone ever imports this into a Client Component, the build fails loudly instead of shipping your service logic to the browser.

Discover feed: static directory, fresh on demand

Viby's public Discover page lists hobby communities. It rarely changes, so cache it — but refresh instantly when an admin adds a hobby.

```
// src/data/hobbies.ts
import "server-only";
import { unstable_cache } from "next/cache";
import { createClient } from "@/data/supabase/server";

export const getHobbyCommunities = unstable_cache(
  async () => {
    const supabase = await createClient();
    const { data } = await supabase
      .from("hobbies")
      .select("id, name, emoji, member_count")
      .order("member_count", { ascending: false });
    return data ?? [];
  },
  ["hobby-communities"], // cache key parts
  { tags: ["hobbies"], revalidate: 3600 }, // refresh hourly OR on demand
);
```

```
// src/app/discover/page.tsx  (Server Component — no "use client")
import { getHobbyCommunities } from "@/data/hobbies";
import { HobbyCard } from "@/features/discover/components/hobby-card";

export default async function DiscoverPage() {
  const communities = await getHobbyCommunities();

  return (
    <main className="mx-auto max-w-2xl px-4 py-8">
      <h1 className="text-2xl font-bold text-brand">Find your people</h1>
      <div className="mt-6 grid grid-cols-2 gap-3">
        {communities.map((c) => (
          <HobbyCard key={c.id} community={c} />
        ))}
      </div>
    </main>
  );
}
```

When an admin creates a hobby, the Server Action busts only the hobby cache:

```
// src/features/discover/actions.ts
"use server";

import { revalidateTag } from "next/cache";
import { createHobby } from "@/data/hobbies";
import { getCurrentUser } from "@/data/profiles";

export async function addHobbyAction(formData: FormData) {
  const user = await getCurrentUser();
  if (!user) throw new Error("Unauthorized");

  await createHobby({
    name: String(formData.get("name")),
    emoji: String(formData.get("emoji")),
  });

  revalidateTag("hobbies"); // every reader of "hobbies" re-bakes
}
```

Streaming a slow profile section

A profile page should paint instantly. The "people you might vibe with" matcher is slow (it scores shared hobbies), so we stream it.

```tsx
// src/app/u/[handle]/page.tsx
import { Suspense } from "react";
import { getProfileByHandle } from "@/data/profiles";
import { ProfileHeader } from "@/features/profile/components/profile-header";
import { VibeMatches } from "@/features/profile/components/vibe-matches";
import { MatchesSkeleton } from "@/features/profile/components/matches-skeleton";

// Next.js 16: params is async — await it.
export default async function ProfilePage({
  params,
}: {
  params: Promise<{ handle: string }>;
}) {
  const { handle } = await params;
  const profile = await getProfileByHandle(handle); // fast, blocks paint

  return (
    <main className="mx-auto max-w-2xl px-4 py-8">
      <ProfileHeader profile={profile} />

      {/* Fast shell renders now; matches stream in when ready */}
      <Suspense fallback={<MatchesSkeleton />}>
        <VibeMatches profileId={profile.id} />
      </Suspense>
    </main>
  );
}
```

```
// src/features/profile/components/vibe-matches.tsx  (async Server Component)
import { getVibeMatches } from "@/data/matches";
import { MatchRow } from "./match-row";

export async function VibeMatches({ profileId }: { profileId: string }) {
  const matches = await getVibeMatches(profileId); // the slow query

  return (
    <section className="mt-8">
      <h2 className="text-lg font-semibold">People you might vibe with</h2>
      <ul className="mt-3 space-y-2">
        {matches.map((m) => (
          <MatchRow key={m.id} match={m} />
        ))}
      </ul>
    </section>
  );
}
```

The skeleton is a Client Component only if it animates; otherwise keep it on the server:

```
// src/features/profile/components/matches-skeleton.tsx
"use client";
import { motion } from "framer-motion";

export function MatchesSkeleton() {
  return (
    <div className="mt-8 space-y-2">
      {[0, 1, 2].map((i) => (
        <motion.div
          key={i}
          className="h-14 rounded-xl bg-pink-100"
          animate={{ opacity: [0.5, 1, 0.5] }}
          transition={{ duration: 1.2, repeat: Infinity, delay: i * 0.15 }}
        />
      ))}
    </div>
  );
}
```

Cross-cutting auth in `proxy.ts`

Middleware-style logic lives in `proxy.ts` (not `middleware.ts`). It refreshes the Supabase session cookie and guards private routes.

```
// proxy.ts
import { type NextRequest, NextResponse } from "next/server";
import { createServerClient } from "@supabase/ssr";

export async function proxy(request: NextRequest) {
  let response = NextResponse.next({ request });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value, options }) =>
            response.cookies.set(name, value, options),
          );
        },
      },
    },
  );

  const {
    data: { user },
  } = await supabase.auth.getUser();

  const isPrivate = request.nextUrl.pathname.startsWith("/app");
  if (!user && isPrivate) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  return response;
}

export const config = {
  matcher: ["/((?!_next/static|_next/image|favicon.ico).*)"],
};
```

The demo/connected seam at scale

From Chapter 24, Viby runs in **demo mode** (Zustand, in-memory) or **connected mode** (Supabase). At scale you don't `if (demo)` inside components — you choose the implementation at the DAL boundary:

```
// src/data/matches.ts
import "server-only";
import { isDemo } from "@/lib/runtime";
import { demoMatches } from "@/data/demo/matches";
import { fetchVibeMatches } from "@/data/supabase/matches";

export async function getVibeMatches(profileId: string) {
  return isDemo() ? demoMatches(profileId) : fetchVibeMatches(profileId);
}
```

Components stay identical in both modes; only the data source swaps. One seam, enforced in one folder.

Architecture / diagram

```
flowchart TD
  Browser["Browser (Client Components: motion, Zustand UI state)"]
  Proxy["proxy.ts — session refresh + route guards"]
  RSC["Server Components (render, await DAL)"]
  Action["Server Actions ('use server') — mutations"]
  DAL["Data-Access Layer (src/data, 'server-only')"]
  Cache["Caching: cache() · unstable_cache(tags) · revalidateTag"]
  SB[("Supabase: Postgres · Auth · Storage")]
  Demo[("Demo store (in-memory)")]

  Browser -->|request| Proxy --> RSC
  Browser -->|form submit| Action
  RSC --> DAL
  Action --> DAL
  Action -.revalidateTag.-> Cache
  DAL --> Cache
  Cache --> SB
  DAL --> Demo
```

Feature-first folder layout — group by domain, not by file type, so a feature is self-contained and deletable:

```
src/
  app/                      # routes only (thin: compose features)
    discover/page.tsx
    u/[handle]/page.tsx
  features/                 # one folder per domain
    discover/
      components/
      actions.ts
      schema.ts
    profile/
      components/
      actions.ts
    chat/
  data/                     # the ONLY place that talks to data
    supabase/               # client factories + raw queries
    demo/                   # in-memory equivalents
    profiles.ts hobbies.ts matches.ts
  components/ui/            # shadcn-style primitives (Button, Card)
  lib/                      # pure helpers (runtime flags, formatters)
  stores/                   # Zustand stores (client UI state)
```

Best practices & pitfalls

- **Do** keep all queries + auth checks in `src/data/`. **Don't** call `createClient()` inside page components.
- **Do** add `import "server-only"` to every DAL module. It turns an accidental client leak into a build error.
- **Do** wrap per-request reads (like `getCurrentUser`) in React `cache()`. **Don't** assume `fetch` is cached — in Next.js 16 it isn't unless configured.
- **Do** use **tags +** `revalidateTag` for surgical invalidation after mutations. **Don't** reach for `revalidatePath("/")` as a sledgehammer — it nukes far more than you mean to.
- **Do** stream slow, independent sections with `<Suspense>`. **Don't** wrap your *whole* page in one Suspense boundary — you lose the fast shell.
- **Don't** cache anything user-specific with a shared key. Either scope the cache key to the user or keep the route dynamic.
- **Do** `await params` and `await cookies()` — they're async in Next.js 16. **Don't** add `"use client"` just to silence an error; understand which boundary you actually need.
- **Don't** scatter `if (demo)` through the UI. Resolve it once at the DAL.

Exercises

1. **Tag a cache.** Convert Viby's events list to `unstable_cache` with the tag `"events"`, then add a `createEventAction` that calls `revalidateTag("events")`. Confirm the list updates without a full reload.
2. **Stream a second section.** Add a `<Suspense>`-wrapped "Recent activity" feed to the profile page with its own skeleton. Verify the header paints before the feed.
3. **Find the leak.** Import a `src/data/` module into a Client Component and watch the build fail. Explain in a comment *why* `server-only` caught it.
4. **Scope a cache key.** Cache a user's saved-hobbies list correctly so two different users never see each other's data. (Hint: include the user id in the key array.)
5. **Refactor a seam.** Take one component still doing `if (isDemo())` inline and move that branch into its DAL function.

Mini-project — "Trending Communities" with on-demand revalidation

Build a `/discover/trending` page that ranks hobby communities by weekly join count.

1. In `src/data/hobbies.ts`, add `getTrendingCommunities()` wrapped in `unstable_cache` with key `["trending"]`, tag `"hobbies"`, and `revalidate: 600`.
2. Build `src/app/discover/trending/page.tsx` as a Server Component that maps results into animated `HobbyCard`s.
3. Wrap the list in `<Suspense>` with a `TrendingSkeleton` (Framer Motion pulse).
4. Add a "Join community" Server Action that inserts a membership row **and** calls `revalidateTag("hobbies")`, so trending re-ranks within the same session.
5. Wire it for both modes: a `demoTrending()` branch and a Supabase branch, selected in the DAL — zero `if (demo)` in the page.

Ship it to Vercel and watch the page serve from cache, then re-bake the instant you tap "Join."

Key takeaways

- Next.js 16 renders **dynamically by default**; caching is an intentional opt-in across four layers (request, data, route, router).
- Use **tags +** `revalidateTag` for precise, on-demand cache busting after mutations.
- **Stream** slow, independent sections with `<Suspense>` so the fast shell never waits.
- Logic has a home: reads in **Server Components**, mutations in **Server Actions**, queries + auth in the **DAL**, routing rules in **proxy.ts**.
- A **feature-first** structure plus a single data-access boundary keeps Viby scalable — and keeps the demo/connected seam to exactly one place.

Up next — Chapter 30: Observability & Performance — measuring Core Web Vitals, tracing slow queries, and proving Viby is fast under real load.

Chapter 30 — Security, RLS & Rate Limiting

> The moment Viby has one real user, it has one real attacker — and the only code you can trust runs on a machine you control.

What you'll learn

- Why **the client is never a security boundary**, and where the real boundary lives
- How **Row-Level Security (RLS)** turns Postgres itself into your authorization layer
- Validating input with **Zod** in Server Actions, and trusting *nothing* from the request
- Handling **secrets** correctly (and the difference between `NEXT_PUBLIC_` and server-only env)
- The mechanics of **XSS** and **CSRF**, and why the App Router defends you by default
- Adding **rate limiting** to protect signups, messages, and uploads

Prerequisites

- Supabase Auth and `@supabase/ssr` clients (Chapter 12)
- Server Actions and the `"use server"` directive (Chapter 15)
- `proxy.ts` for edge/middleware-style logic (Chapter 18)
- The Viby `profiles`, `posts`, and `messages` tables (Chapters 10–14)

Theory

The client is enemy territory

Here's the mental model that prevents 90% of security bugs: **everything that runs in the browser can be edited, replayed, and lied to.** The JavaScript you ship, the `disabled` attribute on a button, the `if (user.isAdmin)` check in a React component — all of it is *suggestion*, not enforcement. A determined user opens DevTools, edits your Zustand store, replays your `fetch` with a different `user_id`, and your "hidden" admin action fires.

Think of it like a nightclub. The styling on the door — the velvet rope, the sign saying "members only" — is the client. It sets expectations. But the **bouncer** is the server, and the **guest list** is the database. If the bouncer waves anyone through because the rope *looked* closed, the rope was decoration. Security lives at the door you control, every single time, for every single request.

So the rule: **validate and authorize on the server, ideally as close to the data as possible.** The closer the check sits to the data, the harder it is to bypass.

RLS: authorization that lives *in* the data

Most apps put authorization in application code: "before returning posts, filter by the logged-in user." That works until you have three API routes, two Server Actions, and a cron job — and one of them forgets the filter. Now any user can read anyone's private messages.

Row-Level Security moves the rule into Postgres. You write a *policy* — a SQL predicate — that the database evaluates on **every** query touching that table, no matter which client, route, or action issued it. Forget to filter in your code? The database filters for you. It's the difference between asking every employee to remember the safe combination versus installing a lock that physically can't open for the wrong key.

With Supabase, the magic is that the user's JWT flows into Postgres, and `auth.uid()` returns the authenticated user's id *inside the policy*. The policy becomes: "you may only see rows where `user_id = auth.uid()`." Even if your Server Action has a bug, the database refuses.

> **Critical:** RLS only protects you when queries run with the **anon** or **authenticated** role (the keys exposed to users). The **service role** key *bypasses RLS entirely*. Treat it like the master key to the building — server-only, never in the browser, used sparingly.

Validation, secrets, XSS, CSRF — the supporting cast

- **Input validation:** even authenticated users send garbage — a 50,000-character bio, a `role: "admin"` field you never asked for. Parse incoming data against a strict schema and reject everything else.
- **Secrets:** API keys belong in server-only environment variables. Anything prefixed `NEXT_PUBLIC_` is **baked into the browser bundle** — public forever.
- **XSS** (cross-site scripting): an attacker gets their script to run in *another user's* browser, usually by smuggling `<script>` into content you later render as raw HTML. React escapes by default; you only get burned via `dangerouslySetInnerHTML`.
- **CSRF** (cross-site request forgery): a malicious site silently submits a request *as* the logged-in victim, riding their cookies. The App Router's Server Actions are POST-only with Origin checks, which neutralizes the classic attack.
- **Rate limiting:** without it, one script can attempt 10,000 logins or flood your DMs in a minute. Limits cap the blast radius of abuse and runaway bots.

In practice

1. Turn on RLS for every user-facing table

In the Supabase SQL editor, enable RLS and write explicit policies for the Viby `posts` table. **Enabling RLS with no policy denies all access** — that's the safe default.

```
-- Enable RLS (default-deny once on)
alter table public.posts enable row level security;

-- Anyone authenticated can read PUBLIC posts; you always see your own.
create policy "read public or own posts"
on public.posts for select
to authenticated
using (
  visibility = 'public' or author_id = (select auth.uid())
);

-- You may only INSERT a post authored by yourself.
create policy "insert own posts"
on public.posts for insert
to authenticated
with check ( author_id = (select auth.uid()) );

-- You may only UPDATE / DELETE your own posts.
create policy "modify own posts"
on public.posts for update
to authenticated
using ( author_id = (select auth.uid()) )
with check ( author_id = (select auth.uid()) );

create policy "delete own posts"
on public.posts for delete
to authenticated
using ( author_id = (select auth.uid()) );
```

Two things worth burning into memory:

- `using` filters which existing rows the operation can *see/touch*; `with check` validates the row's *new* values. Inserts need `with check`; deletes need `using`; updates need both.
- Wrapping `auth.uid()` in `(select auth.uid())` lets Postgres evaluate it once per query instead of per row — a real performance win on large tables.

2. Read data with the SSR client — RLS does the rest

Because RLS enforces ownership, your Server Component query stays refreshingly dumb. No manual `.eq('author_id', user.id)` needed for security — though you may still add filters for *correctness*.

```
// app/feed/page.tsx — Server Component (no "use client")
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

async function getSupabase() {
  const cookieStore = await cookies(); // Next.js 16: async
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!, // anon key → RLS applies
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) =>
          toSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options)
          ),
      },
    }
  );
}

export default async function FeedPage() {
  const supabase = await getSupabase();
  // RLS guarantees we only get public posts + our own private ones.
  const { data: posts } = await supabase
    .from("posts")
    .select("id, body, visibility, author_id")
    .order("created_at", { ascending: false });

  return (
    <ul className="space-y-3">
      {posts?.map((p) => (
        <li key={p.id} className="rounded-2xl bg-white p-4 shadow-sm">
          {p.body}
        </li>
      ))}
    </ul>
  );
}
```

3. Validate in a Server Action — never trust the payload

A Server Action runs on the server, but the *arguments still come from the client*. Parse them. Here we validate a new Viby post with Zod, then let RLS confirm ownership at write time.

```ts
// app/feed/actions.ts
"use server";

import { z } from "zod";
import { revalidatePath } from "next/cache";
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

const PostInput = z.object({
  body: z.string().trim().min(1, "Say something!").max(500),
  visibility: z.enum(["public", "friends", "private"]),
});

export async function createPost(formData: FormData) {
  // 1. Validate shape + bounds. Reject anything extra/oversized.
  const parsed = PostInput.safeParse({
    body: formData.get("body"),
    visibility: formData.get("visibility"),
  });
  if (!parsed.success) {
    return { ok: false, error: parsed.error.issues[0].message };
  }

  const cookieStore = await cookies();
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) =>
          toSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options)
          ),
      },
    }
  );

  // 2. Get the trusted server-side identity. NEVER take author_id from the form.
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) return { ok: false, error: "Not signed in." };

  // 3. Insert. RLS's with-check will also reject a mismatched author_id.
```

```
  const { error } = await supabase.from("posts").insert({
    body: parsed.data.body,
    visibility: parsed.data.visibility,
    author_id: user.id, // derived from the session, not the request body
  });

  if (error) return { ok: false, error: "Could not post." };
  revalidatePath("/feed");
  return { ok: true };
}
```

> Use `supabase.auth.getUser()` for trust decisions, **not** `getSession()`. `getUser()` re-validates the token with the Auth server; `getSession()` reads the (potentially spoofable) cookie.

4. Secrets: public vs. server-only

```
# .env.local
# Safe to expose — shipped to the browser bundle:
NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGci...   # anon key is meant to be public

# SERVER ONLY — bypasses RLS. Never NEXT_PUBLIC_, never imported in a client file:
SUPABASE_SERVICE_ROLE_KEY=eyJhbGci...
RESEND_API_KEY=re_...
```

A service-role client must only ever be constructed in server code (Server Actions, Route Handlers, `proxy.ts`):

```
// lib/supabase-admin.ts — import ONLY from server code
import { createClient } from "@supabase/supabase-js";

export const supabaseAdmin = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!, // bypasses RLS — handle with care
  { auth: { persistSession: false } }
);
```

5. XSS: render content as text, never as HTML

React already escapes `{post.body}`. The danger is opting out:

```
// ❌ DANGER: if body contains "<img src=x onerror=stealCookies()>", it runs.
<div dangerouslySetInnerHTML={{ __html: post.body }} />

// ✅ Safe: React escapes this. A user typing "<script>" sees literal text.
<p>{post.body}</p>
```

If you genuinely must render user-authored rich text (e.g. a Markdown bio), **sanitize first** with a library like `isomorphic-dompurify`, allowlisting tags — never trust raw input.

6. Rate limiting in `proxy.ts`

We protect the auth routes with a sliding-window limiter (Upstash Redis here; any shared store works). Putting it in `proxy.ts` means it runs *before* the request ever reaches your action.

```
// proxy.ts (project root) — runs before matched routes
import { NextResponse, type NextRequest } from "next/server";
import { Ratelimit } from "@upstash/ratelimit";
import { Redis } from "@upstash/redis";

const ratelimit = new Ratelimit({
  redis: Redis.fromEnv(), // UPSTASH_REDIS_REST_URL / _TOKEN (server-only)
  limiter: Ratelimit.slidingWindow(5, "60 s"), // 5 attempts / minute / IP
  prefix: "viby:auth",
});

export async function proxy(request: NextRequest) {
  const ip =
    request.headers.get("x-forwarded-for")?.split(",")[0]?.trim() ?? "anon";
  const { success, limit, remaining, reset } = await ratelimit.limit(ip);

  if (!success) {
    return NextResponse.json(
      { error: "Too many attempts. Slow down a sec." },
      {
        status: 429,
        headers: {
          "Retry-After": String(Math.ceil((reset - Date.now()) / 1000)),
          "X-RateLimit-Remaining": String(remaining),
        },
      }
    );
  }
  return NextResponse.next();
}

export const config = {
  matcher: ["/login", "/signup", "/api/auth/:path*"],
};
```

> IP-based limits are a first line of defense, not a fortress (IPs are shared and spoofable on some hops). For sensitive actions, *also* limit per authenticated `user.id`.

Architecture / diagram

```
flowchart TD
    A[Browser / Untrusted] -->|request + cookies| B[proxy.ts<br/>rate limit + origin]
    B -->|429 if over limit| A
    B --> C[Server Component / Server Action<br/>'use server']
    C --> D{Zod validation}
    D -->|invalid| E[Reject 4xx]
    D -->|valid| F[supabase.auth.getUser<br/>trusted identity]
    F --> G[(Postgres + RLS)]
    G -->|policy: author_id = auth.uid| H{Allowed?}
    H -->|no| I[Row denied by DB]
    H -->|yes| J[Data returned]

    subgraph TRUST["Server / Trusted boundary"]
        B
        C
        D
        F
        G
    end
```

The takeaway the diagram encodes: **a request passes through three independent gates** — rate limit, validation, and RLS. Each is enforced server-side, so a bypass of one is still caught by the next.

Best practices & pitfalls

Do

- Enable RLS on **every** table exposed to the anon/authenticated roles, before launch.
- Derive identity from `auth.getUser()` server-side; never accept `user_id` from the body.
- Validate every Server Action input with a strict schema (`.max()` , `.enum()` , no passthrough).
- Keep the service-role key server-only; construct admin clients in `lib/` files imported solely by server code.
- Rate-limit auth, messaging, and upload endpoints — by IP *and* by user id for sensitive ones.

Don't

- Don't rely on hidden/disabled UI, client-side role checks, or Zustand state for authorization.
- Don't use `getSession()` for trust decisions — it reads spoofable cookies.
- Don't `dangerouslySetInnerHTML` raw user content. Sanitize, or render as text.
- Don't enable RLS and forget the policies — you'll lock yourself out (default-deny). And don't write a `using (true)` policy "to fix it."
- Don't put any secret behind a `NEXT_PUBLIC_` prefix; it ships to the browser.

Exercises

1. **Lock down profiles.** Add RLS policies to `profiles`: anyone authenticated can `select` any profile, but only the owner can `update` their own row. Verify by trying to update another user's row from the SQL editor with a different `auth.uid()`.
2. **Schema-harden a Server Action.** Take the Viby "edit bio" action and add a Zod schema capping the bio at 280 chars and stripping a sneaky injected `role` field. Confirm extra fields are ignored.
3. **Find the leak.** Given a Route Handler that builds its Supabase client with `SUPABASE_SERVICE_ROLE_KEY`, explain in two sentences why RLS no longer protects it, then refactor it to use the anon key + user session.
4. **Per-user limiting.** Extend the limiter so sending a DM is capped at 20 messages/minute *per authenticated user id*, not per IP. (Hint: build the limiter key from `user.id`.)
5. **(Stretch) Sanitize rich text.** Allow Markdown bios: render them, but pipe the HTML through `isomorphic-dompurify` with an allowlist of `b, i, em, strong, a, p, br`. Prove that `<script>` and `onerror=` are stripped.

Mini-project — "Vibe Reports": a secure abuse-report flow

Build a feature letting users report a post. It must be safe end to end:

1. **Table + RLS.** Create `reports (id, post_id, reporter_id, reason, created_at)`. Policy: a user may `insert` only with `reporter_id = (select auth.uid())`, and may `select` only their *own* reports. No `update` / `delete` policies (reports are immutable to users).
2. **Server Action.** `submitReport(formData)` validates `reason` with Zod (`z.enum(["spam","harassment","other"])` plus an optional `note` capped at 300 chars), derives `reporter_id` from `getUser()`, and inserts.
3. **Rate limit.** In `proxy.ts`, cap report submissions to 10/hour per user id, returning a friendly 429.
4. **UI.** A `"use client"` modal (Framer Motion `AnimatePresence` for the fade/scale) with a reason dropdown. On success, animate a pink ✓ and a "Thanks, our team will review this" toast. Render the confirmation text with `{}` — never `dangerouslySetInnerHTML`.
5. **Admin view (optional).** A server-only page using `supabaseAdmin` (service role) that lists all reports — demonstrating the *one* legitimate place the master key belongs, and why that page must itself check for an admin role.

Ship it and try to break it: open DevTools, change the `reporter_id` in the network payload, and confirm RLS rejects the write.

Key takeaways

- The client is never a security boundary — **enforce on the server, close to the data.**
- **RLS is your real authorization layer**: a default-deny lock the database evaluates on every query.
- Derive identity from `getUser()`; **never trust** `user_id` / `role` sent from the browser.

- Validate every input with a strict schema; the service-role key bypasses RLS and stays server-only.
- React escapes by default (beware `dangerouslySetInnerHTML`); Server Actions blunt CSRF; **rate limit** to cap abuse.

Up next — Chapter 31: Realtime & Presence, where we'll wire up Supabase Realtime so Viby feeds, typing indicators, and "who's online" update live — securely, with RLS still standing guard over every subscription.

Chapter 31 — Testing, Debugging & Error Handling

> Shipping a feature is the easy part. Shipping a feature that *stays* shipped — that survives refactors, weird inputs, and 2 a.m. production traffic — is what separates a hobbyist from a professional. This chapter is where Viby grows a spine.

What you'll learn

- How to **debug the App Router** like a surgeon: where code actually runs (server vs. client), and how to read the right logs
- How **error boundaries** (`error.tsx` , `global-error.tsx`) and **loading boundaries** (`loading.tsx` , Suspense) catch failures and stalls gracefully
- What a **hydration mismatch** really is — and the exact bug Viby's `TimeAgo` component hit, plus how we fixed it
- A realistic **testing pyramid**: unit (Vitest), integration (Testing Library), and end-to-end (Playwright) — and which to use when
- **Structured logging** that survives the leap from `console.log` to production

Prerequisites

- Chapter 6 (Next.js App Router fundamentals) and Chapter 8 (Server vs. Client Components) — non-negotiable here
- Chapter 16 (Auth, sessions, RLS) and Chapter 17 (Server Actions) — we'll test and harden them
- Chapter 24 (Activity Feed) — the `TimeAgo` and `PostCard` components live here
- Comfort running terminal commands and reading a stack trace

---

Theory

Where does my code even run?

The single biggest source of App Router confusion is forgetting that your code runs in **two different universes**. A Server Component executes once, on a machine in a data center, and streams HTML to the browser. A Client Component (`"use client"`) executes there *too* (for the initial HTML), and *then again* in the browser to become interactive.

Think of it like a **theater production**. The Server Component is the *script and the printed program* — written once, handed to the audience. The Client Component is the *live actor* — they perform from that same script, but they breathe, move, and react. Debugging means first asking: **am I looking at the script or the actor?** A

`console.log` in a Server Component prints to your **terminal**. The same log in a Client Component prints to the **browser console**. Confuse the two and you'll stare at an empty console for an hour.

Errors and the boundary metaphor

A React app is a tree. When a component throws during render, React unwinds the whole subtree above it — by default, all the way to a blank white screen. An **error boundary** is a circuit breaker: it stops the cascade at a known point and shows a fallback instead of taking down the building. The App Router gives you this declaratively. Drop an `error.tsx` into a route folder and Next wraps that segment in a boundary automatically. One bad post in the feed no longer nukes the whole tab bar.

Loading boundaries are the same idea for *time* instead of *failure*. When a Server Component awaits slow data, a `loading.tsx` (which is just a Suspense fallback) shows instantly while the real content streams in behind it. The mental model: error boundaries catch *what broke*, Suspense boundaries cover *what's not ready yet*.

Hydration: the script must match the performance

Hydration is React attaching event listeners to server-rendered HTML. For it to work, **the HTML React produces on the client during the first render must byte-for-byte match what the server sent**. If they differ, React panics — it can't tell which version is "right" — and you get the dreaded *"Hydration failed because the server rendered HTML didn't match the client."*

The classic trigger is **anything non-deterministic**: `Date.now()`, `Math.random()`, `window`, locale-dependent formatting, or reading `localStorage` during render. The server renders at 10:00:00 and says "2m ago"; the client hydrates at 10:00:01 and computes "3m ago" — mismatch. We'll see Viby's real version of this below.

The testing pyramid

You can't manually click every flow on every commit. Tests do it for you. The industry model is a **pyramid**: a wide base of fast **unit tests** (pure functions, one thing each), a thinner band of **integration tests** (a component plus its real children), and a few precious **end-to-end tests** (a real browser walking a full user journey). The ratio matters: e2e tests are slow and flaky, so you want *few but high-value* ones — the signup → onboarding → first connection path, not every button.

In practice

We're on Next.js 16, React 19, Tailwind v4, Supabase via `@supabase/ssr`. All code below targets that stack.

1. The hydration bug Viby actually hit

Early on, the feed rendered relative timestamps with a naive component:

```
// ❌ The buggy version — DO NOT ship this
export function TimeAgo({ iso }: { iso: string }) {
  return <span>{timeAgo(iso)}</span>; // timeAgo() calls Date.now() internally
}
```

`timeAgo` uses `Date.now()`. The server computed the label at request time; the browser recomputed it milliseconds (or minutes, behind a CDN cache) later. Result: a console full of hydration errors on every feed load. Here's the fix we actually shipped (`components/common/TimeAgo.tsx`):

```
"use client";

import { useEffect, useState } from "react";
import { timeAgo } from "@/lib/utils";

/**
 * Relative timestamps depend on the current time, which differs between the
 * server render and client hydration — a guaranteed mismatch. We render the
 * initial value with suppressHydrationWarning and refresh it on the client.
 */
export function TimeAgo({ iso, className }: { iso: string; className?: string }) {
  const [label, setLabel] = useState(() => timeAgo(iso));

  useEffect(() => {
    setLabel(timeAgo(iso));
    const id = setInterval(() => setLabel(timeAgo(iso)), 60_000);
    return () => clearInterval(id);
  }, [iso]);

  return (
    <span className={className} suppressHydrationWarning>
      {label}
    </span>
  );
}
```

Two things make this correct:

1. `suppressHydrationWarning` tells React, for *this element only*, "I know the text may differ across server and client — don't warn." This is the *sanctioned* escape hatch, not a hack. Use it surgically on the element whose content is intentionally time-dependent.
2. The `useEffect` runs **only on the client**, after hydration, so it can freely call `Date.now()` and even tick every 60 seconds without ever causing a mismatch.

The general escape hatch for "client-only" values is the mounted flag:

```
const [mounted, setMounted] = useState(false);
useEffect(() => setMounted(true), []);
if (!mounted) return null; // or a stable skeleton
```

Use `suppressHydrationWarning` when the *shape* matches but the *text* differs (timestamps); use the mounted flag when the element shouldn't render server-side at all (e.g. something reading `localStorage`).

2. Error boundaries per route segment

`error.tsx` must be a Client Component — it receives the error and a `reset()` function to retry the segment. Here's one for the feed (`app/(app)/feed/error.tsx`):

```
"use client";

import { useEffect } from "react";
import { Button } from "@/components/ui/button";

export default function FeedError({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  useEffect(() => {
    // Send to your logger; `digest` correlates with the server-side stack.
    console.error("[feed] render error", error.digest, error.message);
  }, [error]);

  return (
    <div className="flex flex-col items-center gap-4 py-20 text-center">
      <p className="text-2xl">🤓</p>
      <p className="text-sm text-neutral-500">
        Your feed hit a snag. It&apos;s not you, it&apos;s us.
      </p>
      <Button onClick={reset}>Try again</Button>
    </div>
  );
}
```

`error.tsx` only catches errors *below* it in the tree, and **not** in the layout at the same level. To catch a crash in the root layout itself, use `app/global-error.tsx` (it must render its own `<html>` and `<body>`):

```
"use client";

export default function GlobalError({ reset }: { reset: () => void }) {
  return (
    <html lang="en">
      <body className="grid min-h-dvh place-items-center bg-white">
        <button onClick={reset} className="text-[#FF6FB5] underline">
          Reload Viby
        </button>
      </body>
    </html>
  );
}
```

3. Loading boundaries with Suspense

A `loading.tsx` is the zero-config Suspense fallback for the whole segment. For the feed, show skeleton cards in Viby's pink-on-white aesthetic (`app/(app)/feed/loading.tsx`):

```
export default function FeedLoading() {
  return (
    <div className="space-y-4 p-4">
      {Array.from({ length: 4 }).map((_, i) => (
        <div key={i} className="animate-pulse rounded-2xl bg-neutral-100 p-4">
          <div className="mb-3 h-4 w-1/3 rounded bg-neutral-200" />
          <div className="h-24 rounded-xl bg-neutral-200" />
        </div>
      ))}
    </div>
  );
}
```

For finer control — say, streaming the feed while the profile header renders instantly — wrap just the slow part in `<Suspense>` :

```
import { Suspense } from "react";

export default async function FeedPage() {
  return (
    <>
      <FeedHeader /> {/* instant */}
      <Suspense fallback={<FeedLoading />}>
        <FeedStream /> {/* async Server Component, streams in */}
      </Suspense>
    </>
  );
}
```

4. Defensive Server Actions

Server Actions are public HTTP endpoints in disguise — never trust the input. Notice how Viby's `saveProfile` already returns a typed result instead of throwing, so the client can show a friendly toast (`app/actions/profile.ts`):

```
"use server";

import { createClient } from "@/lib/supabase/server";

export async function saveProfile(input: ProfileEdit) {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) return { ok: false as const, error: "Not signed in" };

  const { error } = await supabase
    .from("profiles")
    .update({ display_name: input.displayName /* ...rest */ })
    .eq("id", user.id);

  if (error) return { ok: false as const, error: error.message };
  return { ok: true as const };
}
```

The pattern — **return a discriminated union (`{ ok: true } | { ok: false, error }`) rather than throwing** — means the error boundary is your *last* line of defense, not your first. Predictable failures get handled inline; only the truly unexpected ones bubble up to `error.tsx`.

5. Unit tests with Vitest

Install the toolchain:

```
npm i -D vitest @vitejs/plugin-react jsdom \
  @testing-library/react @testing-library/jest-dom @testing-library/user-event
```

```
// vitest.config.ts
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";
import { resolve } from "node:path";

export default defineConfig({
  plugins: [react()],
  test: { environment: "jsdom", globals: true, setupFiles: ["./vitest.setup.ts"] },
  resolve: { alias: { "@": resolve(__dirname, ".") } },
});
```

```
// vitest.setup.ts
import "@testing-library/jest-dom/vitest";
```

Pure functions are the cheapest, highest-leverage tests. Lock down `timeAgo` (`lib/utils.test.ts`):

```
import { describe, it, expect, vi, afterEach } from "vitest";
import { timeAgo } from "@/lib/utils";

describe("timeAgo", () => {
  afterEach(() => vi.useRealTimers());

  it("says 'just now' under a minute", () => {
    vi.setSystemTime(new Date("2026-05-30T12:00:00Z"));
    expect(timeAgo("2026-05-30T11:59:30Z")).toBe("just now");
  });

  it("formats hours and days", () => {
    vi.setSystemTime(new Date("2026-05-30T12:00:00Z"));
    expect(timeAgo("2026-05-30T09:00:00Z")).toBe("3h");
    expect(timeAgo("2026-05-28T12:00:00Z")).toBe("2d");
  });
});
```

`vi.setSystemTime` freezes `Date.now()`, making a time-dependent function deterministic — the same trick that *prevents* the hydration class of bug also makes it *testable*.

6. Integration test of a Client Component

Render `PostCard` with its real children and assert behavior (`components/feed/PostCard.test.tsx`):

```
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { PostCard } from "@/components/feed/PostCard";

it("toggles the like count when tapped", async () => {
  const post = { id: "p1", author: "Mia", body: "anyone climb?", likes: 3, createdAt: new
Date().toISOString() };
  render(<PostCard post={post} />);

  await userEvent.click(screen.getByRole("button", { name: /like/i }));
  expect(screen.getByText("4")).toBeInTheDocument();
});
```

Test what the *user* sees (`getByRole` , `getByText`), never internal state. If a refactor keeps the behavior, the test keeps passing — that's the whole point.

7. End-to-end with Playwright

```
npm i -D @playwright/test && npx playwright install
```

```
// e2e/connect.spec.ts
import { test, expect } from "@playwright/test";

test("a new user discovers and connects with someone", async ({ page }) => {
  await page.goto("/discover");
  await expect(page.getByRole("heading", { name: /discover/i })).toBeVisible();

  await page.getByRole("button", { name: /connect/i }).first().click();
  await expect(page.getByText(/request sent/i)).toBeVisible();
});
```

One golden-path e2e test per critical journey. They're slow — run them in CI and before release, not on every save.

8. Structured logging

`console.log` is fine in dev but useless in production (no context, no levels, no correlation). Wrap it once so you can swap in a real sink (Vercel Logs, Sentry, Axiom) later without touching call sites (`lib/logger.ts`):

```
type Level = "debug" | "info" | "warn" | "error";

function log(level: Level, msg: string, meta?: Record<string, unknown>) {
  const entry = { level, msg, at: new Date().toISOString(), ...meta };
  // In prod, JSON lines are machine-parseable; in dev, readable.
  console[level === "debug" ? "log" : level](
    process.env.NODE_ENV === "production" ? JSON.stringify(entry) : `[${level}] ${msg}`,
    process.env.NODE_ENV === "production" ? "" : meta ?? "",
  );
}

export const logger = {
  info: (m: string, meta?: Record<string, unknown>) => log("info", m, meta),
  warn: (m: string, meta?: Record<string, unknown>) => log("warn", m, meta),
  error: (m: string, meta?: Record<string, unknown>) => log("error", m, meta),
};
```

Now `logger.error("saveProfile failed", { userId: user.id, error: error.message })` gives you searchable, structured events instead of a lonely string.

Architecture / diagram

How a request flows through Viby's safety nets — from proxy auth refresh down to the boundaries that catch trouble:

```
flowchart TD
  A[Request] --> B[proxy.ts: refresh Supabase session]
  B --> C[Layout renders]
  C --> D{Suspense boundary}
  D -- data not ready --> E[loading.tsx skeleton]
  D -- data ready --> F[Server Component renders]
  F -->|throws| G[error.tsx fallback + reset]
  F -->|ok| H[HTML streamed to browser]
  H --> I[Hydration: client re-renders]
  I -- text matches --> J[Interactive ✅]
  I -- mismatch --> K[Hydration error ⚠️]
  K -. fix .-> L[suppressHydrationWarning / useEffect / mounted flag]
  L --> J
```

Best practices & pitfalls

- **Do** ask "server or client?" *first* when debugging — check the terminal for server logs, the browser console for client logs. Most lost hours are looking in the wrong place.
- **Do** return typed `{ ok }` results from Server Actions; reserve `throw` for the genuinely exceptional. Boundaries are a safety net, not flow control.
- **Don't** slap `suppressHydrationWarning` on whole subtrees to silence errors — that hides real bugs. Apply it to the single element whose content is *intentionally* non-deterministic.
- **Don't** read `window`, `localStorage`, or `Date.now()` during render in a component that runs on the server. Move it into `useEffect`.
- **Do** put `error.tsx` and `loading.tsx` at *meaningful* segments (feed, chat, profile) so one failure doesn't blank the whole shell — but remember `error.tsx` can't catch its sibling `layout.tsx`.
- **Don't** test implementation details (state, props, internals). Test what the user sees and does.
- **Do** keep e2e tests few and golden-path; lean on unit and integration for coverage. An inverted pyramid (lots of e2e) is slow and flaky.
- **Don't** ship `console.log`. Route everything through one `logger` so production has structure and you have one place to add a real sink.

Exercises

1. **Reproduce the bug.** Temporarily replace `TimeAgo` with the naive `<span>{timeAgo(iso)}</span>` version, load `/feed`, and read the hydration error in the console. Then revert and confirm it's gone. Understanding the failure cements the fix.
2. **Add a chat error boundary.** Create `app/(app)/chat/[id]/error.tsx` with a "Couldn't load this conversation" fallback and a working `reset()` button styled in brand pink.
3. **Cover the matching logic.** Write a Vitest suite for `lib/matching.ts` (the discover algorithm from Chapter 22). Include an edge case: two profiles with zero shared interests should score 0, not `NaN`.
4. **Stream the feed.** Wrap the async feed list in `<Suspense>` with the skeleton fallback, and prove the header paints before the posts by throttling the network in DevTools.
5. **Harden a Server Action.** Add input validation (reject empty `displayName`, cap `bio` length) to `saveProfile` and write an integration test asserting it returns `{ ok: false }` for bad input.

Mini-project — A typed `<ErrorState>` + tested logger pipeline

Build a small, reusable resilience kit for Viby:

1. Create `components/common/ErrorState.tsx` — a Client Component taking `{ title, message, onRetry }`, styled with the bubblegum-pink retry button and a friendly emoji. Use it as the body of *both* `app/(app)/feed/error.tsx` and `app/(app)/chat/[id]/error.tsx` so every boundary looks consistent.
2. Wire `lib/logger.ts` (from §8) into each `error.tsx`'s `useEffect`, logging `error.digest` plus the segment name.

3. Add a Vitest test for `logger` that spies on `console.error` (via `vi.spyOn`) and asserts that `logger.error("boom", { userId: "u1" })` produces valid JSON containing `userId` when `NODE_ENV === "production"`.
4. Add one Playwright test that forces the feed action to fail (e.g. point it at a bad route) and asserts your `ErrorState` title appears and the retry button is clickable.

When you're done, you'll have a failure path that's *designed*, *consistent*, *logged*, and *tested* — the exact difference between a demo and a product.

Key takeaways

- App Router code runs in two places; **always determine server vs. client before debugging**, and look in the matching console.
- `error.tsx` / `global-error.tsx` are circuit breakers; `loading.tsx` / `Suspense` cover not-yet-ready data. Place them at meaningful segments.
- **Hydration mismatches** come from non-deterministic render values. Viby's `TimeAgo` fix — `suppressHydrationWarning` plus a client-side `useEffect` — is the canonical pattern.
- Favor the **testing pyramid**: many fast unit tests, fewer integration tests, a handful of golden-path e2e tests. Freeze time with `vi.setSystemTime` to make time-dependent code deterministic.
- Server Actions should **return typed results**, not throw, and everything should flow through one **structured logger**.

Up next — Chapter 32: Performance & Optimization, where we make Viby not just correct, but *fast*: bundle splitting, image optimization, caching, and squeezing the App Router for buttery-smooth Gen-Z scroll.

Chapter 32 — Environment, Config & Deploying to Vercel

> Your app finally leaves your laptop and meets the world — let's make that moment boring, repeatable, and safe.

You've built Viby. It runs beautifully at `localhost:3000`. But a social-discovery app that only *you* can open isn't a product — it's a rehearsal. This chapter is the dress rehearsal becoming opening night: we'll wire up configuration the professional way, produce a real production build, and ship Viby to Vercel with both preview and production environments plus a custom domain.

What you'll learn

- The difference between **public** and **server-only** environment variables, and why mixing them up leaks secrets
- How `.env.local`, `.env`, and Vercel's environment system fit together
- How to run and read a real **production build** (`next build`) and what changes vs. dev
- Deploying Viby to Vercel two ways: the **CLI** and **GitHub** integration
- How **preview** deployments differ from **production**, and how to use them in a team
- Attaching a **custom domain** with HTTPS

Prerequisites

- Chapter 8 (Next.js App Router & Server Components) — you'll lean on the server/client boundary here
- Chapter 19 (Supabase Auth with `@supabase/ssr`) — we configure its keys for production
- Chapter 27 (`proxy.ts` for edge logic) — we'll confirm it ships correctly
- A GitHub account and the Vercel free (Hobby) tier — both free

---

Theory

Why environment variables exist

Imagine your app's *recipe* (the code) and its *ingredients list for this kitchen* (the configuration). The recipe is identical whether you cook at home or in a restaurant. The ingredients — which Supabase project, which database URL, which API keys — change per kitchen. **Environment variables** are how we keep the recipe fixed while swapping ingredients between your laptop, a teammate's laptop, a preview deploy, and production.

Hard-coding your Supabase keys into a `.ts` file is like writing your home address on every copy of the recipe and then handing the recipe to strangers. Config belongs *outside* the code.

The one rule that prevents most disasters: public vs. server

This is the single most important idea in the chapter, so let's be precise.

In Next.js, an environment variable is exposed to the **browser** only if its name begins with `NEXT_PUBLIC_`. Everything else stays **server-side** and is never shipped in the JavaScript bundle.

- `NEXT_PUBLIC_SUPABASE_URL` → safe to expose. The browser needs it to talk to Supabase.
- `NEXT_PUBLIC_SUPABASE_ANON_KEY` → safe to expose. It's the *anonymous* key, designed for the client and protected by Row-Level Security (you set this up in Chapter 20).
- `SUPABASE_SERVICE_ROLE_KEY` → **never** prefix this with `NEXT_PUBLIC_`. It bypasses RLS and can read/write any row. If it lands in the browser bundle, anyone can pull your entire users table.

The analogy: `NEXT_PUBLIC_` is the lobby of an apartment building — public, fine for strangers to see. Everything without the prefix is behind the locked front door of an apartment. The prefix is the *only* thing deciding which side of that door a value lives on. Get it wrong and you either break the app (a secret the browser needed wasn't exposed) or leak a master key.

A subtle gotcha: `NEXT_PUBLIC_` values are **inlined at build time**. They're baked into the bundle when `next build` runs, not read fresh at runtime. Change one, and you must rebuild.

The `.env` file family

Next.js loads several files automatically. Order of precedence (highest wins):

```
.env.local        ← your machine's secrets, NEVER committed
.env.production    ← committed, non-secret production defaults
.env.development   ← committed, non-secret dev defaults
.env               ← committed, shared non-secret defaults
```

The golden rule: **`.env.local` is git-ignored and holds anything secret.** Everything committed should be safe for the whole world to read.

In practice

1. Define Viby's local environment

Create `.env.local` at the project root:

```
# .env.local — git-ignored, never commit this file

# Public: shipped to the browser, protected by RLS
NEXT_PUBLIC_SUPABASE_URL="https://abcdefgh.supabase.co"
NEXT_PUBLIC_SUPABASE_ANON_KEY="eyJhbGciOiJIUzI1NiIs..."
NEXT_PUBLIC_SITE_URL="http://localhost:3000"

# Server-only: NEVER prefixed with NEXT_PUBLIC_
SUPABASE_SERVICE_ROLE_KEY="eyJhbGciOiJIUzI1NiIs...service..."
```

Confirm `.gitignore` already ignores it (Next.js scaffolds this for you):

```
# .gitignore
.env*.local
```

2. Validate config at startup (fail loud, fail early)

A missing variable in production is a 3am pager incident. Catch it the moment the server boots instead. Create a typed, validated config module:

```
// lib/env.ts
import { z } from "zod";

const serverSchema = z.object({
  SUPABASE_SERVICE_ROLE_KEY: z.string().min(1),
});

const publicSchema = z.object({
  NEXT_PUBLIC_SUPABASE_URL: z.string().url(),
  NEXT_PUBLIC_SUPABASE_ANON_KEY: z.string().min(1),
  NEXT_PUBLIC_SITE_URL: z.string().url(),
});

// Public vars must be referenced statically so Next.js can inline them.
export const publicEnv = publicSchema.parse({
  NEXT_PUBLIC_SUPABASE_URL: process.env.NEXT_PUBLIC_SUPABASE_URL,
  NEXT_PUBLIC_SUPABASE_ANON_KEY: process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY,
  NEXT_PUBLIC_SITE_URL: process.env.NEXT_PUBLIC_SITE_URL,
});

// Only parse server vars on the server — guard so the browser never touches them.
export const serverEnv =
  typeof window === "undefined"
    ? serverSchema.parse({
        SUPABASE_SERVICE_ROLE_KEY: process.env.SUPABASE_SERVICE_ROLE_KEY,
      })
    : ({} as z.infer<typeof serverSchema>);
```

Now your Supabase browser client (from Chapter 19) reads from validated config:

```
// lib/supabase/client.ts
import { createBrowserClient } from "@supabase/ssr";
import { publicEnv } from "@/lib/env";

export function createClient() {
  return createBrowserClient(
    publicEnv.NEXT_PUBLIC_SUPABASE_URL,
    publicEnv.NEXT_PUBLIC_SUPABASE_ANON_KEY,
  );
}
```

And the server client uses `getAll` / `setAll` cookie handlers — the only correct pattern with `@supabase/ssr`:

```
// lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";
import { publicEnv } from "@/lib/env";

export async function createClient() {
  const cookieStore = await cookies(); // async in Next.js 16

  return createServerClient(
    publicEnv.NEXT_PUBLIC_SUPABASE_URL,
    publicEnv.NEXT_PUBLIC_SUPABASE_ANON_KEY,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options),
            );
          } catch {
            // Called from a Server Component — safe to ignore; proxy.ts refreshes.
          }
        },
      },
    },
  );
}
```

The service-role key only ever appears in trusted server code — for example a Server Action that needs to bypass RLS for an admin task:

```
// app/admin/actions.ts
"use server";

import { createClient as createAdminClient } from "@supabase/supabase-js";
import { publicEnv, serverEnv } from "@/lib/env";

export async function banUser(userId: string) {
  // service_role bypasses RLS — server-only, never in a "use client" file
  const admin = createAdminClient(
    publicEnv.NEXT_PUBLIC_SUPABASE_URL,
    serverEnv.SUPABASE_SERVICE_ROLE_KEY,
  );

  const { error } = await admin
    .from("profiles")
    .update({ banned: true })
    .eq("id", userId);

  if (error) throw new Error("Could not ban user");
}
```

3. Run a real production build

Dev mode (`next dev`) is forgiving: hot reload, unminified, extra warnings. Production is the truth. Run it locally before you ever deploy:

```
npm run build   # runs `next build`
npm run start   # serves the production build on :3000
```

`next build` type-checks, lints, statically pre-renders what it can, and inlines every `NEXT_PUBLIC_` value. If a public env var is missing, **the build is where you'll find out** — exactly what you want. Read the output: it tells you which routes are static (`○`), dynamic (`ƒ`), and how big each bundle is. A surprise dynamic route often means you accidentally read a cookie or header in a page you expected to be static.

4. Deploy via the Vercel CLI

The fastest path from laptop to live URL:

```
npm i -g vercel
vercel login

# First run links the folder to a Vercel project (answer the prompts)
vercel              # creates a PREVIEW deployment

vercel --prod       # promotes to PRODUCTION
```

Push your environment variables to Vercel so the cloud build has them. You can do this per environment:

```
vercel env add SUPABASE_SERVICE_ROLE_KEY production
vercel env add NEXT_PUBLIC_SUPABASE_URL production
# ...repeat for preview and development as needed

# Pull cloud env vars down into a local .env.local for parity:
vercel env pull .env.local
```

5. Deploy via GitHub (the way real teams ship)

The CLI is great for solo experiments, but the professional default is **git-based deploys**. Connect once and every push becomes a deployment:

1. Push Viby to a GitHub repo.
2. In the Vercel dashboard: **Add New** → **Project** → **Import** your repo.
3. Vercel auto-detects Next.js — no build config needed.
4. Add your environment variables in **Settings** → **Environment Variables**, choosing the **Production**, **Preview**, and **Development** scopes per variable.
5. Deploy.

From now on:

- **Push to** `main` → Production deployment.
- **Open a Pull Request** → unique Preview deployment with its own URL.

Vercel even comments the preview URL on the PR so reviewers can click and test the actual running app.

Architecture / diagram

```
flowchart TD
    Dev["Developer laptop\n.env.local (secrets)"] -->|git push| GH["GitHub repo"]

    GH -->|push to feature branch / PR| PrevBuild["Vercel Build\n(Preview env vars)"]
    GH -->|push to main| ProdBuild["Vercel Build\n(Production env vars)"]

    PrevBuild --> PrevURL["viby-git-feat-x.vercel.app\nephemeral preview"]
    ProdBuild --> ProdURL["viby.app\nproduction + custom domain"]

    subgraph Build["Inside every build"]
      direction LR
      Inline["NEXT_PUBLIC_* inlined\ninto browser bundle"]
      Secret["server-only vars kept\nin Node runtime only"]
    end

    PrevBuild -.-> Build
    ProdBuild -.-> Build

    ProdURL --> Supabase[("Supabase\nAuth · Postgres · Storage")]
    PrevURL --> Supabase
```

The key insight the diagram encodes: **the same code** flows down both paths; only the **environment variables injected at build time** differ. Public vars get inlined into the bundle; secrets stay in the Node runtime and never reach the browser.

Best practices & pitfalls

- **Do** keep `.env.local` git-ignored. **Don't** ever commit a service-role key — if you do, rotate it immediately in the Supabase dashboard.
- **Do** prefix browser-needed values with `NEXT_PUBLIC_`. **Don't** prefix secrets with it "just to make the error go away."
- **Do** set `NEXT_PUBLIC_SITE_URL` per environment so OAuth redirects and email links point to the right host. A preview deploy redirecting to `localhost` is a classic broken-auth bug.
- **Do** run `npm run build` locally before pushing. The cloud build will fail on the same TypeScript and lint errors — find them on your machine first.
- **Don't** assume changing a `NEXT_PUBLIC_` value in the dashboard updates the live site. It's inlined at build time — you must **redeploy**.
- **Do** add your production and preview domains to **Supabase → Authentication → URL Configuration** (Site URL + Redirect URLs). Auth silently fails otherwise.
- **Don't** read secrets in a `"use client"` component or pass them as props from a Server Component — props are serialized to the browser.

- **Do** trust `proxy.ts` to deploy automatically; Vercel runs it at the edge with no extra config. **Don't** rename it to `middleware.ts` — Next.js 16's convention here is `proxy.ts`.
- **Do** use Preview deployments as your QA environment for every PR. **Don't** test risky changes by pushing straight to `main`.

Exercises

1. **Audit the boundary.** Add a `console.log(process.env.SUPABASE_SERVICE_ROLE_KEY)` inside a `"use client"` component, run `npm run build && npm run start`, open the browser, and confirm it logs `undefined`. Then move it to a Server Action and confirm it's present on the server. Delete both logs when done.
2. **Break the build on purpose.** Remove `NEXT_PUBLIC_SUPABASE_URL` from `.env.local` and run `npm run build`. Read the Zod error from `lib/env.ts`. Restore it. You've just proven your fail-loud guard works.
3. **Two environments, two values.** In the Vercel dashboard, set `NEXT_PUBLIC_SITE_URL` to different values for Preview and Production. Deploy both and confirm a page that renders `publicEnv.NEXT_PUBLIC_SITE_URL` shows the correct host in each.
4. **Ship a preview from a PR.** Create a branch, change Viby's hero headline, open a PR, and click the Vercel preview link in the PR comment to see your change live before merging.
5. **(Stretch) Custom domain.** Buy or reuse a domain, add it under **Vercel** → **Settings** → **Domains**, set the DNS records Vercel shows you, and confirm HTTPS provisions automatically. Update `NEXT_PUBLIC_SITE_URL` and the Supabase redirect URLs to match.

Mini-project — A "Build Info" badge for Viby

Ship a tiny diagnostics widget that proves your environment wiring is correct in every deployment. It shows which environment the running build belongs to and the git commit it was built from — invaluable when a teammate asks "is the fix live yet?"

Vercel injects system environment variables automatically — including `VERCEL_ENV` (`production` | `preview` | `development`) and `VERCEL_GIT_COMMIT_SHA`. Expose just what the badge needs by re-mapping them to public vars in `next.config.ts`:

```
// next.config.ts
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  env: {
    NEXT_PUBLIC_VERCEL_ENV: process.env.VERCEL_ENV ?? "development",
    NEXT_PUBLIC_COMMIT_SHA: process.env.VERCEL_GIT_COMMIT_SHA ?? "local",
  },
};

export default nextConfig;
```

Then build the badge as a client component with a subtle Framer Motion entrance:

```
// components/build-badge.tsx
"use client";

import { motion, AnimatePresence } from "framer-motion";
import { useState } from "react";

const env = process.env.NEXT_PUBLIC_VERCEL_ENV;
const sha = (process.env.NEXT_PUBLIC_COMMIT_SHA ?? "local").slice(0, 7);

export function BuildBadge() {
  const [open, setOpen] = useState(true);

  // Hide in production so end users never see it.
  if (env === "production") return null;

  return (
    <AnimatePresence>
      {open && (
        <motion.button
          onClick={() => setOpen(false)}
          initial={{ opacity: 0, y: 12 }}
          animate={{ opacity: 1, y: 0 }}
          exit={{ opacity: 0, y: 12 }}
          whileTap={{ scale: 0.95 }}
          className="fixed bottom-4 left-4 z-50 rounded-full bg-[--color-brand] px-3 py-1.5 text-xs
font-semibold text-white shadow-lg"
        >
          {env} · {sha}
        </motion.button>
      )}
    </AnimatePresence>
  );
}
```

Define the brand token in your Tailwind v4 theme (no `tailwind.config.js`):

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5;
}
```

Drop it into your root layout so it appears on previews and locally, but auto-hides in production:

```
// app/layout.tsx
import { BuildBadge } from "@/components/build-badge";

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body>
        {children}
        <BuildBadge />
      </body>
    </html>
  );
}
```

Open a PR, click the preview URL, and you'll see a bubblegum-pink badge reading `preview · a1b2c3d`. Merge to `main`, visit production, and it's gone. You've now got a visible, deploy-aware signal — and proof that your public/server boundary and environment scoping all work end to end.

Key takeaways

- Config lives **outside** code in environment variables; the recipe stays fixed while ingredients change per environment.
- `NEXT_PUBLIC_` is the *only* switch deciding browser vs. server exposure — and public values are **inlined at build time**, so changing them means redeploying.
- `.env.local` is git-ignored and holds secrets; everything committed must be safe for the world.
- Validate env vars at startup so missing config fails the build, not production at 3am.
- Vercel ships the **same code** to preview and production, differing only by injected env vars; PRs get free preview URLs and `main` is production.
- Add your deploy domains to Supabase's auth URL config, and remember `proxy.ts` deploys to the edge automatically.

Up next — Chapter 33: Observability & Error Tracking, where we wire Viby up with logging, performance monitoring, and Sentry so you *know* when something breaks before your users tell you.

Chapter 33 — Supabase in Production

Your localhost Supabase has been a cozy sandbox — now we wire Viby to a real, hardened backend that thousands of strangers will hammer at 2am.

What you'll learn

- How to stand up a dedicated **production** Supabase project (separate from dev)
- Shipping your schema with **migrations** instead of clicking around the dashboard
- Creating **Storage buckets** and writing RLS-style **storage policies** for avatars and post photos
- Configuring **auth providers**, the **Site URL**, and **redirect allow-lists** so OAuth and magic links land where they should
- Why **connection pooling** matters on serverless (Vercel) and which connection string to use
- Setting up **backups** and a sane disaster-recovery posture

Prerequisites

- Chapter 9 (Supabase fundamentals & the `@supabase/ssr` clients)
- Chapter 12 (Auth flows: OAuth, magic links, sessions)
- Chapter 18 (Row Level Security and policies)
- Chapter 24 (Storage: uploading avatars and post images)
- Chapter 32 (Deploying Viby to Vercel and environment variables)

You should already have Viby running locally against a dev Supabase project. We're now creating its grown-up twin.

Theory

Think of your dev Supabase project as a **kitchen at home**: if you spill flour, nobody cares. Production is a **restaurant kitchen during a Saturday dinner rush** — health inspectors, paying customers, and zero tolerance for "oops I dropped the users table." The skills are the same; the discipline is not.

Three ideas separate hobby backends from production ones.

1. Environments are isolated, not shared. You never point dev and prod at the same database. A bad migration or a `delete from profiles` you ran "just to test" must never be able to touch real users. So we keep at least two Supabase projects: `viby-dev` and `viby-prod`. They have different URLs, different keys, different data. This is the single most important habit in this chapter.

2. Schema is code, not clicks. In dev it's tempting to add a column in the dashboard table editor. That works once. But production schema must be **reproducible and reviewable** — you want the exact same tables to exist whether you rebuild from scratch tomorrow or onboard a teammate next year. The answer is **migrations**: ordered SQL files, committed to git, applied in sequence. The dashboard is for *looking*, not *changing*.

3. Serverless changes how you connect. Traditional servers open a handful of long-lived database connections and reuse them. Vercel runs your code as **functions that spin up and die constantly** — each cold start could try to open a brand-new Postgres connection. Postgres has a hard connection limit (often ~60 on small plans), so a traffic spike can exhaust it and your app starts throwing `too many connections`. The fix is a **connection pooler** (Supabase ships PgBouncer/Supavisor in front of Postgres): your functions talk to the pooler, the pooler multiplexes a small set of real connections to the database. Analogy: instead of every customer getting their own dedicated waiter forever, a small pool of waiters serves everyone, table by table.

A subtlety that trips everyone up: for the Viby app's *normal* runtime requests we use the Supabase JS client over HTTP (PostgREST) — that goes through Supabase's API layer, not a raw Postgres socket, so pooling is handled for you. You only think hard about pooling/connection strings when you connect to Postgres **directly** (an ORM like Prisma/Drizzle, a migration tool, or a cron worker). Keep that distinction in your head.

In practice

1. Create the production project

In the Supabase dashboard: **New project** → name it `viby-prod`, pick a region **close to your Vercel deployment region** (latency between your functions and the DB is real), and set a strong database password (store it in your password manager — you'll need it for the direct connection string).

Once it's provisioning, grab three things from **Project Settings** → **API**:

- Project URL → `https://<ref>.supabase.co`
- `anon` public key
- `service_role` secret key (server-only — never ships to the browser)

2. Wire Viby's environment variables

On Vercel (Chapter 32), set these for the **Production** environment only. Your `.env.local` keeps the dev project's values.

```
# .env.production (managed in Vercel → Settings → Environment Variables)
NEXT_PUBLIC_SUPABASE_URL=https://YOUR_PROD_REF.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGci...prod-anon...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGci...prod-service-role...   # server only, NOT NEXT_PUBLIC
```

Our `@supabase/ssr` clients from Chapter 9 don't change — they read those env vars. As a reminder, the server client uses the modern `getAll` / `setAll` cookie handlers:

```
// lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies(); // Next.js 16: cookies() is async

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            );
          } catch {
            // Called from a Server Component — safe to ignore; proxy refreshes the session.
          }
        },
      },
    }
  );
}
```

3. Ship the schema with migrations

Install and link the Supabase CLI, then keep all SQL in versioned files.

```
npm install -D supabase
npx supabase login

# Link your local repo to the PROD project (ref is in the dashboard URL)
npx supabase link --project-ref YOUR_PROD_REF

# Create a migration file
npx supabase migration new init_viby_schema
```

That creates `supabase/migrations/<timestamp>_init_viby_schema.sql`. Put real Viby schema in it:

```
-- supabase/migrations/20260530120000_init_viby_schema.sql

create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  username text unique not null,
  display_name text,
  bio text,
  avatar_url text,
  hobbies text[] default '{}',
  created_at timestamptz not null default now()
);

create table public.posts (
  id uuid primary key default gen_random_uuid(),
  author_id uuid not null references public.profiles(id) on delete cascade,
  caption text,
  image_url text,
  created_at timestamptz not null default now()
);

-- RLS is OFF by default on new tables — turn it ON or your data is public.
alter table public.profiles enable row level security;
alter table public.posts enable row level security;

create policy "Profiles are viewable by everyone"
  on public.profiles for select using (true);

create policy "Users can update their own profile"
  on public.profiles for update using (auth.uid() = id);

create policy "Posts are viewable by everyone"
  on public.posts for select using (true);

create policy "Users can insert their own posts"
  on public.posts for insert with check (auth.uid() = author_id);

create policy "Users can delete their own posts"
  on public.posts for delete using (auth.uid() = author_id);
```

Apply it to production:

```
npx supabase db push
```

The CLI runs only migrations the prod DB hasn't seen yet, in order. From now on, every schema change is a new migration file in a PR — never a dashboard edit. To pull an out-of-band change someone made in the dashboard back into code, run `npx supabase db pull` and commit the generated migration.

4. Storage buckets & policies

Avatars and post images live in Storage. Create buckets and lock them down with policies — also as migration SQL so they're reproducible:

```
-- supabase/migrations/20260530121000_storage.sql

-- Public bucket for avatars (readable by anyone, writable only by the owner)
insert into storage.buckets (id, name, public)
values ('avatars', 'avatars', true)
on conflict (id) do nothing;

insert into storage.buckets (id, name, public)
values ('posts', 'posts', true)
on conflict (id) do nothing;

-- Anyone can read avatar files
create policy "Avatar images are publicly readable"
  on storage.objects for select
  using (bucket_id = 'avatars');

-- A user may only write into a folder named after their own uid: avatars/<uid>/...
create policy "Users upload their own avatar"
  on storage.objects for insert
  with check (
    bucket_id = 'avatars'
    and (storage.foldername(name))[1] = auth.uid()::text
  );

create policy "Users update their own avatar"
  on storage.objects for update
  using (
    bucket_id = 'avatars'
    and (storage.foldername(name))[1] = auth.uid()::text
  );
```

The folder-prefix trick (`avatars/<uid>/avatar.png`) is the standard pattern: the policy checks that the first path segment equals the caller's `auth.uid()`, so a user can never overwrite someone else's file.

Uploading from Viby's profile editor (a Server Action) looks like:

```
// app/(app)/settings/actions.ts
"use server";

import { createClient } from "@/lib/supabase/server";
import { revalidatePath } from "next/cache";

export async function uploadAvatar(formData: FormData) {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) throw new Error("Not authenticated");

  const file = formData.get("avatar") as File;
  const path = `${user.id}/avatar-${Date.now()}.${file.name.split(".").pop()}`;

  const { error } = await supabase.storage
    .from("avatars")
    .upload(path, file, { upsert: true, cacheControl: "3600" });
  if (error) throw error;

  const {
    data: { publicUrl },
  } = supabase.storage.from("avatars").getPublicUrl(path);

  await supabase.from("profiles").update({ avatar_url: publicUrl }).eq("id", user.id);
  revalidatePath("/settings");
}
```

5. Auth providers & redirect URLs

This is where most "it worked locally but breaks in prod" bugs live. In **Authentication → URL Configuration**:

- **Site URL**: `https://viby.app` (your real production domain — used as the default redirect)
- **Redirect URLs** (allow-list — Supabase rejects any redirect not listed): `https://viby.app/** https://*-yourteam.vercel.app/** # Vercel preview deployments http://localhost:3000/** # local dev`

For OAuth (say Google), enable the provider under **Authentication → Providers**, paste the Client ID/Secret from Google Cloud Console, and add Supabase's callback URL — `https://YOUR_PROD_REF.supabase.co/auth/v1/callback` — to Google's **Authorized redirect URIs**.

Your sign-in Server Action passes a `redirectTo` that must match the allow-list:

```
// app/(auth)/login/actions.ts
"use server";

import { createClient } from "@/lib/supabase/server";
import { redirect } from "next/navigation";
import { headers } from "next/headers";

export async function signInWithGoogle() {
  const supabase = await createClient();
  const origin = (await headers()).get("origin")!; // works in prod, preview, and local

  const { data, error } = await supabase.auth.signInWithOAuth({
    provider: "google",
    options: { redirectTo: `${origin}/auth/callback` },
  });
  if (error) throw error;
  if (data.url) redirect(data.url);
}
```

And the session refresh lives in `proxy.ts` (Next.js 16's middleware-style entry — not a `middleware.ts`):

```
// proxy.ts
import { createServerClient } from "@supabase/ssr";
import { NextResponse, type NextRequest } from "next/server";

export async function proxy(request: NextRequest) {
  let response = NextResponse.next({ request });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) => request.cookies.set(name, value));
          response = NextResponse.next({ request });
          cookiesToSet.forEach(({ name, value, options }) =>
            response.cookies.set(name, value, options)
          );
        },
      },
    }
  );

  await supabase.auth.getUser(); // refreshes the token cookie on every request
  return response;
}

export const config = {
  matcher: ["/((?!_next/static|_next/image|favicon.ico).*)"],
};
```

6. Connection pooling (only for direct Postgres access)

If you add Drizzle, Prisma, or a cron worker that opens raw Postgres connections, use the **pooler** connection string from **Project Settings** → **Database** → **Connection string**, not the direct `5432` one:

```
# Transaction-mode pooler (port 6543) — best for serverless / Vercel functions
DATABASE_URL="postgresql://postgres.YOUR_REF:PASSWORD@aws-0-REGION.pooler.supabase.com:6543/postgres?pgbouncer=true"

# Direct connection (port 5432) — use ONLY for migrations / long-lived workers
DIRECT_URL="postgresql://postgres.YOUR_REF:PASSWORD@aws-0-REGION.pooler.supabase.com:5432/postgres"
```

Transaction mode (6543) doesn't support prepared statements or session features, which is why the `?pgbouncer=true` flag (and Prisma's separate `directUrl` for migrations) exists. The Viby app itself, using `@supabase/ssr` over HTTP, **needs none of this** — it's already pooled at the API layer.

7. Backups

On paid plans Supabase takes **daily automated backups** with **Point-in-Time Recovery (PITR)** available as an add-on — PITR lets you restore to any second, which is what you want the day someone fat-fingers a delete. Free tier projects can be **paused** and have limited backups, so production should be on at least the Pro plan. Verify under **Project Settings** → **Database** → **Backups**. Belt-and-suspenders: schedule a periodic `pg_dump` of critical tables to object storage you control, and **test a restore** before you need one — an untested backup is a rumor.

Architecture / diagram

```
flowchart TD
  subgraph Vercel["Vercel (Production)"]
    SC[Server Components / Server Actions]
    PX[proxy.ts session refresh]
  end

  subgraph SB["Supabase viby-prod"]
    API[PostgREST + Auth API<br/>pooled at HTTP layer]
    POOL[Supavisor / PgBouncer<br/>:6543 transaction mode]
    PG[(PostgreSQL)]
    ST[Storage buckets<br/>avatars / posts]
  end

  Browser -->|cookies| PX --> SC
  SC -->|anon key over HTTPS| API
  SC -->|signed uploads| ST
  API --> PG
  ST --> PG

  Worker[Cron / ORM worker] -->|raw Postgres| POOL --> PG

  GOOGLE[Google OAuth] -->|callback URL| API
```

Best practices & pitfalls

- **Do** keep `viby-dev` and `viby-prod` as separate projects with separate keys. **Don't** ever run experiments against prod.
- **Do** treat every schema change as a committed migration. **Don't** edit tables in the prod dashboard — it desyncs your migration history.
- **Do** enable RLS on every table immediately (new tables ship with it *off*). A forgotten `enable row level security` is the #1 Supabase data leak.
- **Don't** expose the `service_role` key to the browser — never prefix it with `NEXT_PUBLIC_`. It bypasses RLS entirely.
- **Do** include preview-deployment URLs (`https://*-yourteam.vercel.app/**`) in the redirect allow-list, or OAuth silently fails on PR previews.
- **Do** use the **6543 transaction pooler** for any serverless direct-Postgres access; reserve **5432** for migrations. **Don't** run app traffic on 5432 — you'll exhaust connections.
- **Do** put production on a plan with PITR and **test a restore**. **Don't** trust a backup you've never restored.
- **Do** call `supabase.auth.getUser()` (verifies with the auth server) for authorization decisions, not `getSession()` (reads the cookie, spoofable).

Exercises

1. **Two-project hygiene.** Create `viby-prod`, copy its three API values into Vercel's Production env scope, and confirm `vercel env pull` shows prod values differ from your `.env.local`.
2. **Migrate, don't click.** Move Viby's full schema into `supabase/migrations/`, run `db push` against prod, then `supabase db reset` locally and confirm the app rebuilds identically from migrations alone.
3. **Storage lockdown.** Write a policy so a user can `delete` only their own files in the `posts` bucket. Verify by trying (and failing) to delete another user's object with the anon key.
4. **Redirect allow-list.** Deploy a PR to a Vercel preview URL, attempt Google sign-in, watch it fail, then add the wildcard preview pattern and confirm it works.
5. **Pooling proof (stretch).** Add Drizzle to a cron route, wire it to the 6543 pooler, and load-test it; then switch to 5432 and observe `too many connections` under concurrency.

Mini-project — Production launch checklist for Viby

Build a one-time, admin-only `/admin/health` Server Component that verifies your production wiring at a glance. It should, on the server:

- Confirm `NEXT_PUBLIC_SUPABASE_URL` points at the **prod** ref (not dev).
- Query `select count(*) from profiles` to prove DB connectivity and RLS-as-service-role works.
- List Storage buckets via `supabase.storage.listBuckets()` and assert `avatars` and `posts` exist and are public.
- Render a green/red checklist (Tailwind v4 tokens, a little Framer Motion `AnimatePresence` for rows fading in) covering: env scope correct, RLS enabled on `profiles` / `posts`, redirect URLs configured (manual confirm checkbox), backups/PITR enabled (manual confirm).

Ship it behind an `is_admin` check in your profiles table. The goal: a single page you load right after deploy that tells you Viby's backend is truly production-ready — no guessing.

Key takeaways

- Production = a **separate, isolated** Supabase project; never share keys or data with dev.
- **Migrations are the source of truth** for schema — dashboard edits desync your history.
- New tables ship with **RLS off**; turn it on everywhere, and never leak the `service_role` key.
- **Storage policies** use the folder-prefix-equals-`auth.uid()` pattern to scope writes per user.
- Auth breaks in prod when **Site URL** and the **redirect allow-list** (including preview URLs) aren't configured.
- Use the **6543 transaction pooler** for serverless direct-Postgres access; the `@supabase/ssr` HTTP client is already pooled.
- Put prod on a plan with **PITR backups** — and test a restore before you need one.

Up next: Chapter 34 — Observability & Error Tracking: wiring Sentry, structured logs, and uptime alerts so you *know* the moment Viby misbehaves in production.

Chapter 34 — Monitoring, Logging & Analytics

> You can't fix what you can't see — once Viby is live, your real users become your best (and most brutal) QA team.

What you'll learn

- The three pillars of observability — **logs, metrics, traces** — and how they differ
- Wiring **Sentry** into Next.js 16 for error tracking and performance monitoring
- Reading Vercel's built-in **Web Analytics** and **Speed Insights** (Core Web Vitals)
- Capturing meaningful **product events** (not vanity metrics) with PostHog
- Doing privacy-respectful **server-side event tracking** from Server Actions
- Building a lightweight ops dashboard from Supabase data
- Closing the loop: turning a funnel drop-off into a shipped fix

Prerequisites

- Chapter 9 (Server Components & Server Actions) — we'll instrument actions
- Chapter 12 (`proxy.ts` request handling) — we'll tag requests there
- Chapter 31 (Deploying to Vercel) — analytics ride on the deployment
- Chapter 33 (Environment variables & secrets) — DSNs and keys live in env

---

Theory

When Viby ran on `localhost`, you *were* the monitoring system: you saw every error in the terminal, every slow render in DevTools. In production, thousands of strangers on flaky phones across the world hit your app and you see... nothing. Unless you build the nerves to feel it.

Observability is the property of a system that lets you ask new questions about its behavior *without shipping new code*. It rests on three pillars:

- **Logs** — discrete, timestamped events. "User `a1b2` failed to upload an avatar at 14:03." Like a diary: great for *what exactly happened* in one case.
- **Metrics** — numbers aggregated over time. "Avatar uploads: p95 latency 1.2s, error rate 0.4%." Like a car dashboard: great for *how healthy is the whole system right now*.
- **Traces** — the path of a single request across functions and services. "This signup took 900ms: 120ms in `proxy.ts`, 600ms in Supabase auth, 180ms rendering." Like a flight recorder: great for *where the time went*.

Sitting on top of these are two business-facing layers:

- **Error tracking** turns raw exceptions into grouped, deduplicated, alertable *issues* with stack traces and user context. The difference between "something broke 4,000 times" and "*this one null check* broke for 4,000 users on iOS Safari."
- **Product analytics** answers *why people behave the way they do*: Do new users find their first community? Where does the onboarding funnel leak?

A useful mental model: **operational telemetry** (errors, latency, uptime) tells you whether the *machine* is healthy; **product analytics** tells you whether the *experience* is working. You need both. A perfectly stable app that nobody completes onboarding in is still failing.

One rule above all: **measure what changes a decision.** "Total signups: 41,300" feels good but changes nothing. "Onboarding completion dropped from 68% to 51% after Tuesday's release" sends you to a specific PR. The first is a vanity metric; the second closes the loop.

In practice

1. Error tracking & performance with Sentry

Sentry is the industry standard for error tracking and has first-class Next.js support. Install and run the wizard, then we'll look at the files it expects.

```
npm install @sentry/nextjs
npx @sentry/wizard@latest -i nextjs
```

In Next.js 16's App Router, client and edge config live in `instrumentation-client.ts` and `instrumentation.ts` (the wizard wires `next.config.ts` for you). Here's a hand-tuned client config for Viby:

```
// instrumentation-client.ts
import * as Sentry from "@sentry/nextjs";

Sentry.init({
  dsn: process.env.NEXT_PUBLIC_SENTRY_DSN,
  // Sample 10% of transactions for perf monitoring in prod; 100% locally.
  tracesSampleRate: process.env.NODE_ENV === "production" ? 0.1 : 1.0,
  // Session Replay: record 10% of sessions, 100% of sessions WITH an error.
  replaysSessionSampleRate: 0.1,
  replaysOnErrorSampleRate: 1.0,
  integrations: [
    Sentry.replayIntegration({
      // Privacy first: never leak DMs or emails into a replay.
      maskAllText: true,
      blockAllMedia: true,
    }),
  ],
  environment: process.env.VERCEL_ENV ?? "development",
});

// Required in Next.js 16 so navigation transactions are captured.
export const onRouterTransitionStart = Sentry.captureRouterTransitionStart;
```

```
// instrumentation.ts — runs on the server & edge runtimes
import * as Sentry from "@sentry/nextjs";

export async function register() {
  if (process.env.NEXT_RUNTIME === "nodejs") {
    await import("./sentry.server.config");
  }
  if (process.env.NEXT_RUNTIME === "edge") {
    await import("./sentry.edge.config");
  }
}

export const onRequestError = Sentry.captureRequestError;
```

The `onRequestError` export is the magic hook: Next.js 16 calls it for every uncaught Server Component / Server Action / route handler error, so you get server stack traces automatically.

Now, **context is everything**. A naked stack trace is a riddle; a stack trace tagged with the user and the community they were viewing is a bug report. Identify the user once you know who they are:

```
// lib/observability/identify.ts
import * as Sentry from "@sentry/nextjs";
import type { User } from "@supabase/supabase-js";

export function identifyUser(user: User | null) {
  if (!user) {
    Sentry.setUser(null);
    return;
  }
  // Send the id — NEVER the email or phone. Privacy by default.
  Sentry.setUser({ id: user.id });
}
```

When you *expect* an error might happen (a third-party image host timing out, say), capture it explicitly instead of swallowing it:

```
// In a Viby avatar-upload Server Action
"use server";

import * as Sentry from "@sentry/nextjs";
import { createServerClient } from "@supabase/ssr";

export async function uploadAvatar(formData: FormData) {
  try {
    // ...supabase storage upload...
  } catch (error) {
    Sentry.captureException(error, {
      tags: { feature: "profile", action: "avatar_upload" },
    });
    return { ok: false, message: "Couldn't save your photo — try again." };
  }
}
```

2. Vercel Web Analytics & Speed Insights

Vercel gives you privacy-friendly traffic analytics and real-user **Core Web Vitals** (LCP, INP, CLS) with two small packages and zero cookies.

```
npm install @vercel/analytics @vercel/speed-insights
```

Drop both components into the root layout — they're Client Components that Vercel renders as tiny scripts:

```tsx
// app/layout.tsx
import { Analytics } from "@vercel/analytics/next";
import { SpeedInsights } from "@vercel/speed-insights/next";

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body>
        {children}
        <Analytics />
        <SpeedInsights />
      </body>
    </html>
  );
}
```

That's it — deploy, and the Vercel dashboard fills with pageviews and field-data Web Vitals from your *actual* users (far more honest than a Lighthouse score on your fast laptop).

3. Product analytics with PostHog

For *behavioral* events we'll use PostHog. Initialize it client-side and expose a typed `track` helper so event names stay consistent — drifting event names ("signup", "Sign Up", "signed_up") are the silent killer of analytics.

```tsx
// app/providers.tsx
"use client";

import posthog from "posthog-js";
import { PostHogProvider } from "posthog-js/react";
import { useEffect } from "react";

export function AnalyticsProvider({ children }: { children: React.ReactNode }) {
  useEffect(() => {
    posthog.init(process.env.NEXT_PUBLIC_POSTHOG_KEY!, {
      api_host: "/ingest", // proxy through our own domain (see proxy.ts)
      person_profiles: "identified_only",
      capture_pageview: false, // App Router needs manual pageviews
    });
  }, []);

  return <PostHogProvider client={posthog}>{children}</PostHogProvider>;
}
```

```tsx
// lib/analytics/events.ts — the single source of truth for event names
import posthog from "posthog-js";

// A closed union means a typo becomes a compile error, not a lost insight.
type VibyEvent =
  | { name: "onboarding_started"; props?: Record<string, never> }
  | { name: "interests_selected"; props: { count: number } }
  | { name: "community_joined"; props: { communityId: string; source: "search" | "suggested" } }
  | { name: "first_post_created"; props: { communityId: string } };

export function track<E extends VibyEvent>(event: E["name"], props?: E["props"]) {
  posthog.capture(event, props);
}
```

```
// Using it in a "Join community" button
"use client";

import { track } from "@/lib/analytics/events";

export function JoinButton({ communityId }: { communityId: string }) {
  return (
    <button
      className="rounded-full bg-brand px-5 py-2 font-semibold text-white"
      onClick={() => {
        track("community_joined", { communityId, source: "suggested" });
        // ...call the join Server Action...
      }}
    >
      Join
    </button>
  );
}
```

The `bg-brand` utility comes from our Tailwind v4 theme token defined in CSS:

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5;
}
```

4. Proxying analytics through your own domain

Ad-blockers eat requests to `posthog.com`, silently destroying 20–40% of your data. Route them through a first-party path using Viby's `proxy.ts` (Next.js 16's middleware-style file) so requests go to `viby.app/ingest`:

```
// proxy.ts
import { NextResponse, type NextRequest } from "next/server";

export function proxy(request: NextRequest) {
  if (request.nextUrl.pathname.startsWith("/ingest")) {
    const url = new URL(request.url);
    const target = "https://us.i.posthog.com";
    return NextResponse.rewrite(
      new URL(url.pathname.replace("/ingest", "") + url.search, target),
    );
  }
  return NextResponse.next();
}

export const config = {
  matcher: ["/ingest/:path*"],
};
```

5. Server-side events & structured logging

Some events should *never* depend on a browser — money, signups, and anything you'd hate to lose to an ad-blocker. Fire those from the server with `posthog-node`, and emit a structured JSON log line that Vercel's log drains can parse:

```
// lib/analytics/server.ts
import { PostHog } from "posthog-node";

const client = new PostHog(process.env.NEXT_PUBLIC_POSTHOG_KEY!, {
  host: "https://us.i.posthog.com",
  flushAt: 1, // serverless is short-lived — don't batch
  flushInterval: 0,
});

export async function trackServer(
  distinctId: string,
  event: string,
  properties?: Record<string, unknown>,
) {
  client.capture({ distinctId, event, properties });
  await client.flush(); // ensure it sends before the function freezes
}
```

```ts
// lib/observability/log.ts — structured logs beat console.log("here")
type Level = "info" | "warn" | "error";

export function log(level: Level, message: string, meta: Record<string, unknown> = {}) {
  // One JSON object per line = queryable in Vercel/Datadog/Logflare.
  console[level === "error" ? "error" : "log"](
    JSON.stringify({ level, message, ts: new Date().toISOString(), ...meta }),
  );
}
```

```ts
// app/(auth)/actions.ts
"use server";

import { trackServer } from "@/lib/analytics/server";
import { log } from "@/lib/observability/log";

export async function completeOnboarding(userId: string, interestCount: number) {
  // ...persist to Supabase...
  await trackServer(userId, "onboarding_completed", { interestCount });
  log("info", "onboarding_completed", { userId, interestCount });
  return { ok: true };
}
```

Architecture / diagram

```
flowchart TD
  subgraph Client["Browser (React 19)"]
    UI["Viby UI<br/>track() + Sentry"]
  end
  subgraph Edge["Vercel Edge / proxy.ts"]
    PX["/ingest rewrite"]
  end
  subgraph Server["Server Actions / Route Handlers"]
    SA["completeOnboarding()<br/>trackServer() + log()"]
  end

  UI -->|behavioral events| PX --> PH[(PostHog<br/>product analytics)]
  UI -->|exceptions + replays| SN[(Sentry<br/>errors + perf)]
  UI -->|pageviews + Web Vitals| VA[(Vercel<br/>Analytics)]
  SA -->|server events| PH
  SA -->|JSON logs| LD[(Vercel Log Drain)]
  SA -->|onRequestError| SN

  PH --> DASH["Dashboards & Funnels"]
  SN --> ALERT["Alerts → Slack"]
  DASH --> DECIDE{"Decision:<br/>ship a fix"}
  ALERT --> DECIDE
```

The loop is the point: telemetry flows up from client and server into tools, tools surface dashboards and alerts, and those drive the *next* code change — which you then measure again.

Best practices & pitfalls

- **Do** sample performance traces in production (`tracesSampleRate: 0.1`). 100% sampling at scale is expensive and rarely more insightful.
- **Do** define event names in one typed file. **Don't** sprinkle raw string literals across components — typos silently fragment your funnels.
- **Don't** log PII. No emails, phone numbers, DM contents, or full auth tokens in Sentry, PostHog, or logs. Send IDs and let your DB resolve the rest.
- **Do** mask text and media in Session Replay. A replay that captures someone's private messages is a breach waiting to happen.
- **Don't** rely on client-only events for critical funnels — ad-blockers and crashed tabs lose them. Mirror the load-bearing ones server-side.
- **Do** proxy analytics through a first-party route, or accept losing a chunk of data to blockers.
- **Don't** alert on everything — alert fatigue means real fires get ignored. Page a human only for user-facing breakage (error-rate spikes, p95 latency cliffs).

- **Do** set `flushAt: 1` for `posthog-node` in serverless; otherwise events vanish when the function freezes.
- **Don't** confuse a Lighthouse score (lab) with Speed Insights (field). Optimize for the real-user numbers.

Exercises

1. **Tag the request.** In `proxy.ts`, read the `user-agent` and attach an `is_mobile` boolean as a request header, then surface it as a Sentry tag. (Easy)
2. **Funnel it.** Instrument the full onboarding flow — `onboarding_started` → `interests_selected` → `community_joined` → `first_post_created` — and build a PostHog funnel. Where's the biggest drop? (Medium)
3. **Custom Web Vital.** Use the `useReportWebVitals` hook to send INP to PostHog as a custom event, so you can correlate poor responsiveness with specific pages. (Medium)
4. **Synthetic check.** Write a Vercel Cron route handler that pings `/api/health`, and have it `Sentry.captureMessage` if Supabase is unreachable. (Hard)
5. **Cohort retention.** Build a PostHog retention report answering: of users who `community_joined`, what fraction return and post within 7 days? (Hard)

Mini-project — The Viby Onboarding Health dashboard

Build an internal `/admin/health` page (gated to admins) that closes the loop end-to-end.

Requirements:

1. Instrument the onboarding funnel with the four typed events from Exercise 2, mirroring `onboarding_completed` server-side via `trackServer`.
2. Create a Supabase view `onboarding_daily` that aggregates, per day: signups, completed onboardings, and completion rate.
3. Build a Server Component that reads the view and renders a simple bar chart of the last 14 days' completion rate.
4. Wrap each bar in a Framer Motion `motion.div` with a staggered `initial={{ scaleY: 0 }}` → `animate={{ scaleY: 1 }}` entrance so the dashboard feels alive.
5. Add a Sentry alert: if the onboarding Server Action's error rate exceeds 2% over 5 minutes, post to a Slack channel.

Stretch: annotate the chart with deploy timestamps (pull from the Vercel API) so a completion-rate dip lines up visibly with the release that caused it — the literal picture of closing the loop.

You'll know it works when you can answer, in one glance, "Is onboarding healthier or worse than last week, and did our last deploy help?"

Key takeaways

- Observability stands on **logs, metrics, and traces**; error tracking and product analytics build the business view on top.
- **Sentry** (`instrumentation-client.ts` + `onRequestError`) gives you grouped errors, performance, and replays across client and server.
- **Vercel Analytics + Speed Insights** deliver cookieless traffic data and real-user Core Web Vitals for free.
- **PostHog** with a single typed `track()` helper keeps event names clean; mirror critical events **server-side** and **proxy** through your own domain to survive ad-blockers.
- Never log PII; sample traces; alert only on user-facing pain.
- The whole point is the **loop**: measure → surface → decide → ship → measure again.

Up next — Chapter 35: Scaling & Performance Optimization, where we take everything these dashboards reveal and make Viby fast and cheap under real load.

Chapter 35 — MVP Strategy & Product Thinking

> You don't ship a product. You ship a hypothesis — and then you go find out if you were right.

What you'll learn

- What an MVP actually is (and the three things people mistake it for)
- How to scope ruthlessly so you ship in weeks, not quarters
- Problem/solution fit vs. product/market fit — and why the order matters
- The Build–Measure–Learn loop, made concrete with instrumentation
- Two prioritization frameworks you'll reach for constantly: RICE and MoSCoW
- Exactly how **Viby** was scoped from a wishlist of 40 features down to a shippable core

Prerequisites

- You've built Viby's core flows: auth (Chapter 11), profiles, the discovery feed, and Realtime chat (Chapters 22–28)
- You're comfortable with Server Components, Server Actions, and the `@supabase/ssr` clients
- Nice to have: the analytics groundwork from Chapter 31 — we'll lean on event tracking here

---

Theory

What an MVP really is

MVP stands for **Minimum Viable Product**. Every word is load-bearing, and most beginners get two of the three wrong.

- **Minimum** — the *smallest* thing you can build. Not "small but polished everywhere." Smallest.
- **Viable** — it must actually deliver the core value. A car with no engine is minimum, but it's not viable. A skateboard is.
- **Product** — a real person uses it and gets a real outcome. A landing page that collects emails is a *test*, not yet a product.

The famous analogy (from Henrik Kniberg): if the goal is *transportation*, don't ship a wheel, then a chassis, then a half-car, then a car. Each intermediate step is useless — nobody can ride a chassis. Instead ship a **skateboard**, then a scooter, then a bike, then a motorbike, then a car. Every step is *usable* and teaches you something about how people actually want to move.

```
WRONG: wheel -> axle -> chassis -> car      (nothing works until the end)
RIGHT: skateboard -> scooter -> bike -> car (every step is rideable & teaches you)
```

An MVP is **not**:

- A prototype (throwaway, internal, proves a technical risk)
- A beta (a near-complete product, just earlier)
- "Version 1 with fewer features" (that's just a smaller waterfall)

An MVP **is**: the smallest experiment that lets you learn whether real users get real value — *before* you've spent six months building the wrong thing.

Problem/solution fit comes before product/market fit

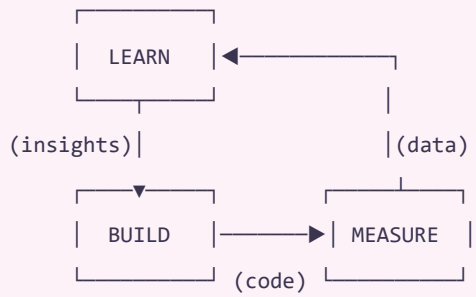
There are two milestones, in order:

1. **Problem/solution fit** — "Do people actually have this problem, and does my approach plausibly solve it?" This is qualitative. You learn it from interviews, a clickable prototype, a concierge MVP.
2. **Product/market fit (PMF)** — "Is there a market that pulls the product out of my hands?" This is quantitative: retention curves that flatten instead of decaying to zero, organic growth, people upset if you took it away.

The number-one startup killer is building a beautiful solution to a problem nobody has. For Viby, the problem hypothesis is sharp: "*Adults past college find it genuinely hard to make new platonic friends who share their hobbies.*" Notice it's **not** dating — that's a deliberate scoping decision that shapes every feature.

The Build–Measure–Learn loop

Eric Ries's *Lean Startup* loop is the engine of product thinking:



The trick everyone misses: you go around the loop **backwards** when planning. Start from what you want to **learn**, decide what to **measure** to learn it, then build the *minimum* needed to generate that measurement. Building first and asking "what should we measure?" afterward is how you ship features that move nothing.

Each loop should produce a **validated learning** — a sentence like: "*Users who join a Circle in their first session retain at 3x the rate of those who don't.*" That sentence changes your roadmap. A loop that produces only "we shipped it" produced nothing.

Prioritization: turning opinions into decisions

When everyone in the room has a favorite feature, you need a shared scoring rubric so the loudest voice doesn't win.

RICE scores each idea: `(Reach × Impact × Confidence) / Effort`.

- **Reach** — how many users per quarter does it touch?
- **Impact** — per-user effect (3 = massive, 1 = medium, 0.25 = minimal)
- **Confidence** — how sure are you? (100% / 80% / 50%)
- **Effort** — person-weeks

MoSCoW buckets scope for a single release: **M**ust / **S**hould / **C**ould / **W**on't. The discipline is the **Won't** column — writing down what you're explicitly *not* doing this cycle is the whole point.

In practice

Theory is worthless without instrumentation. An MVP you can't measure is just a guess with a deploy URL. Let's wire the **Measure** half of the loop into Viby.

1. A typed, minimal analytics event

Keep your event vocabulary tiny and intentional. For the MVP you might track only a handful of events — the ones tied to your core learning question.

```
// lib/analytics/events.ts
export type VibyEvent =
  | { name: "signup_completed"; props: { method: "email" | "oauth" } }
  | { name: "circle_joined"; props: { circleId: string; hobby: string } }
  | { name: "match_messaged"; props: { matchId: string } }
  | { name: "discovery_card_swiped"; props: { direction: "save" | "skip" } };

export type EventName = VibyEvent["name"];
```

2. Record events with a Server Action

Write events to Supabase from a Server Action. Note the `@supabase/ssr` `createServerClient` with `getAll` / `setAll` cookie handlers — the modern, correct API.

```ts
// app/actions/track.ts
"use server";

import { cookies } from "next/headers";
import { createServerClient } from "@supabase/ssr";
import type { VibyEvent } from "@/lib/analytics/events";

export async function trackEvent(event: VibyEvent) {
  // Next.js 16: cookies() is async — await it.
  const cookieStore = await cookies();

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (cookiesToSet) => {
          cookiesToSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options)
          );
        },
      },
    }
  );

  const {
    data: { user },
  } = await supabase.auth.getUser();

  await supabase.from("analytics_events").insert({
    user_id: user?.id ?? null,
    name: event.name,
    props: event.props,
  });
}
```

3. Fire it from a client component

The discovery feed is a client component (it needs gestures via Framer Motion). When a user saves or skips a card, we record it — that swipe ratio is a *measurement* feeding the loop.

```tsx
// components/discovery/swipe-card.tsx
"use client";

import { motion, useMotionValue, useTransform } from "framer-motion";
import { trackEvent } from "@/app/actions/track";

export function SwipeCard({ profile }: { profile: { id: string; name: string } }) {
  const x = useMotionValue(0);
  const rotate = useTransform(x, [-200, 200], [-12, 12]);
  const opacity = useTransform(x, [-200, 0, 200], [0.4, 1, 0.4]);

  function onDragEnd(_: unknown, info: { offset: { x: number } }) {
    if (Math.abs(info.offset.x) < 120) return; // not a decisive swipe
    const direction = info.offset.x > 0 ? "save" : "skip";
    // Fire-and-forget: measurement must never block the interaction.
    void trackEvent({ name: "discovery_card_swiped", props: { direction } });
  }

  return (
    <motion.div
      style={{ x, rotate, opacity }}
      drag="x"
      dragConstraints={{ left: 0, right: 0 }}
      onDragEnd={onDragEnd}
      className="rounded-3xl bg-white p-6 shadow-lg"
    >
      <h3 className="text-xl font-semibold text-[--color-brand]">{profile.name}</h3>
    </motion.div>
  );
}
```

4. The brand token, defined the Tailwind v4 way

That `[--color-brand]` resolves to a token defined in CSS — **no** `tailwind.config.js` in Tailwind v4.

```css
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-brand: #ff6fb5; /* Viby bubblegum pink */
  --color-brand-ink: #1f2937;
}
```

5. Read the funnel back

The point of measuring is to *learn*. A simple Server Component dashboard turns raw events into the activation funnel that tells you whether the MVP works.

```
// app/(admin)/funnel/page.tsx  — Server Component, no "use client"
import { cookies } from "next/headers";
import { createServerClient } from "@supabase/ssr";

export default async function FunnelPage() {
  const cookieStore = await cookies();
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: () => {}, // read-only page; no cookies to write
      },
    }
  );

  // One RPC that returns counts per funnel step (defined in SQL).
  const { data: funnel } = await supabase.rpc("activation_funnel");

  return (
    <div className="mx-auto max-w-md space-y-3 p-6">
      <h1 className="text-2xl font-bold text-[--color-brand]">Activation funnel</h1>
      {funnel?.map((step: { label: string; count: number }) => (
        <div key={step.label} className="flex justify-between rounded-xl bg-pink-50 p-4">
          <span>{step.label}</span>
          <span className="font-mono">{step.count}</span>
        </div>
      ))}
    </div>
  );
}
```

> Gate this admin route in `proxy.ts` (Viby's middleware-style file from Chapter 18) so only staff session cookies reach `/funnel`. Don't ship a public dashboard of your raw metrics.

Architecture / diagram

How Viby's MVP scope maps onto the Build–Measure–Learn loop:

```
flowchart TD
    H[Hypothesis: hobby-based Circles drive friendship] --> B[BUILD: auth + profile + Circles + 1:1
chat]
    B --> U[Real users use Viby]
    U --> M[MEASURE: signup -> circle_joined -> match_messaged]
    M --> D[(analytics_events in Supabase)]
    D --> F[activation_funnel RPC]
    F --> L{LEARN: does joining a Circle lift retention?}
    L -->|Yes, double down| H
    L -->|No, pivot scope| H
    style H fill:#ff6fb5,color:#fff
    style L fill:#ff6fb5,color:#fff
```

The loop is a *cycle*, not a line. Each pass tightens the hypothesis or kills it.

Best practices & pitfalls

Do

- Write your learning goal *before* you write code. "We'll learn X by measuring Y."
- Keep an explicit **Won't-do-this-cycle** list. Saying no is the job.
- Instrument the core funnel on day one — retroactive analytics is guesswork.
- Ship to *real* users fast, even 10 of them. Ten engaged users beat a thousand imaginary ones.
- Measure **retention**, not just signups. A flattening retention curve is the only real signal of PMF.

Don't

- Don't confuse "minimum" with "low quality." The slice you ship must feel premium — just *narrow*.
- Don't build features because a competitor has them. Build for *your* hypothesis.
- Don't gold-plate the parts no one has validated (Viby's video calls, gamified badges — all v2).
- Don't let analytics block the UI. Track fire-and-forget (`void trackEvent(...)`).
- Don't ship a metrics dashboard publicly or skip auth on `/funnel` .
- Don't mistake *activity* (we shipped 8 features!) for *progress* (we learned 1 true thing).

Exercises

1. **Scope the cut.** List 10 features you'd *want* in Viby. Sort them into MoSCoW buckets for a 3-week MVP. Defend your "Won't" column in two sentences each.

2. **RICE a feature.** Pick "voice notes in chat." Estimate Reach, Impact, Confidence, Effort and compute its RICE score. Compare it to "profile photo cropping." Which ships first?
3. **Add an event.** Extend the `VibyEvent` union with `profile_completed` and fire it from the profile form's Server Action. Keep the type exhaustive.
4. **Write the learning sentence.** For the discovery swipe data, write the one-sentence validated learning you'd hope to extract, and the SQL query that would confirm or deny it.
5. **(Stretch)** Build a `retention_cohort` Supabase RPC that buckets users by signup week and reports week-1 return rate. Render it in a Server Component.

Mini-project: The Viby "One-Metric Dashboard"

Build a single Server Component page, `/insights`, that answers **one** question: *Do users who join a Circle in their first session come back?*

Steps

1. Create the `analytics_events` table in Supabase with columns `id`, `user_id`, `name`, `props jsonb`, `created_at default now()`, and Row Level Security allowing inserts for authenticated users.
2. Wire `trackEvent` (above) into your signup action (`signup_completed`) and Circle join action (`circle_joined`).
3. Write a SQL function `circle_retention()` returning two numbers: 7-day return rate for users who fired `circle_joined` in session one vs. those who didn't.
4. Render both numbers side by side with the Viby pink token, plus a one-line verdict: *"Joining a Circle lifts week-1 retention by N%."*
5. Protect `/insights` in `proxy.ts`.

Definition of done: the page renders a single comparison and a plain-English verdict. If the verdict surprises you, congratulations — you just closed a Build–Measure–Learn loop.

Key takeaways

- An MVP is the **smallest experiment** that delivers real value and produces validated learning — not a small waterfall.
- Chase **problem/solution fit first**, then product/market fit; retention is your truest PMF signal.
- Plan the **Build–Measure–Learn** loop *backwards*: learn → measure → build the minimum.
- Use **RICE** to rank ideas and **MoSCoW** to scope a release; the **Won't** column is sacred.
- Viby was scoped to one hypothesis — *hobby-based Circles create friendships* — and everything off that path (dating, video, badges) was deliberately deferred.
- Instrument from day one; activity is not progress.

Up next — Chapter 36: Growth, Analytics & Retention Engineering, where we turn these raw events into cohort retention curves, activation metrics, and the growth loops that make Viby spread.

Chapter 36 — Growth, Retention, Scaling & Engineering Workflows

> You shipped a beautiful app. Now let's make it *grow*, *keep* the people who join, and survive the day 50,000 of them show up at once.

This is the chapter where Viby stops being a project and starts behaving like a *startup*. Code quality got you to launch. Growth loops, retention discipline, and a calm engineering workflow get you to traction.

What you'll learn

- **Growth loops & network effects** — why some products compound and most don't, and how to design loops into Viby.
- **Activation & retention metrics** — the North Star, the AARRR funnel, cohort retention curves, and how to instrument them.
- **Virality** — the viral coefficient (k-factor), invite loops, and why "share" buttons usually fail.
- **Scaling** — the product (database, realtime, caching) and the team (ownership, on-call).
- **Modern startup engineering workflows** — trunk-based git, fast reviews, CI/CD on Vercel, and feature flags so you can ship daily without fear.

Prerequisites

- Chapters 1–10: the Next.js 16 App Router, Server Components, and TypeScript foundations.
- Chapter 14–18: Supabase Auth, PostgreSQL, RLS, and Realtime.
- Chapter 28: Viby's "Find your people" discovery feed and the hobby-match graph.
- Chapter 33–35: deployment to Vercel and basic observability.

Theory

Growth loops beat funnels

A **funnel** is linear: an ad sends a stranger in one end, a fraction trickles out the other as a user. You pay for every new person. A **growth loop** is circular: the *output* of using the product becomes the *input* that brings the next person.

> Analogy: a funnel is a bucket you keep pouring water into. A loop is a sourdough starter — feed it once and it keeps making more of itself.

Viby's natural loop:

```
A new user joins → finds a hobby circle (e.g. "Bouldering Berlin")
→ posts a meetup → invites 2 friends who also boulder
→ those friends join, see the circle is alive → they post → invite more
```

Every active user *manufactures the reason* for the next user to join. That's a loop. Funnels you buy; loops you build.

Network effects (the moat)

A **network effect** means the product gets *more valuable* as more people use it. Viby's value is "find people near you who share your hobby." With 3 users in your city, that's worthless. With 3,000, it's magic. This is why social-discovery is hard to start (the *cold-start problem*) but defensible once warm — a competitor with better code but no users still loses.

Beat cold-start by going **narrow and dense**: don't launch "everyone, everywhere." Launch *one hobby in one city* until it's saturated, then expand. Density, not breadth, creates the magic moment.

Activation, retention, and the North Star

- **Activation** is the moment a new user first feels the value — the "aha." For Viby it's not signup; it's *joining a circle and seeing it's active*. Define it precisely, because everything optimizes toward it.
- **Retention** is whether they come back. It's the single best predictor of a real business. A leaky bucket can't be filled by more marketing.
- **North Star Metric (NSM)**: the one number that captures delivered value. For Viby: **Weekly Connected Users** — users who had a real interaction (joined a circle, RSVP'd, or messaged) in the last 7 days. Not signups, not page views — *value delivered*.

The classic frame is **AARRR** ("pirate metrics"): Acquisition → Activation → Retention → Referral → Revenue.

Virality and the k-factor

The **viral coefficient** k = (invites sent per user) × (conversion rate per invite).

- $k > 1$: each user brings more than one new user → exponential growth (rare, magical).
- $k < 1$: invites *amplify* other channels but don't sustain growth alone — still hugely valuable.

A "Share to Instagram" button is almost always $k \approx 0$. What works is an invite that is **load-bearing**: the recipient gets something real (a pre-joined circle, a friend already inside). Make the invite *part of using the product*, not a bolt-on.

Scaling — product and team

- **Product**: add database indexes before you need them, push reads to caches, keep Realtime channels scoped, and move slow work to background jobs.
- **Team**: as you grow past ~3 engineers, informal coordination breaks. You need **clear ownership**, small PRs, automated checks, and a deploy process anyone can run safely at 5pm on a Friday.

In practice

1. Instrument activation with a typed event helper

Track events from a single, typed module so naming stays consistent. We log to a Supabase `events` table (cheap, queryable) and could fan out to a product-analytics tool later.

```
-- supabase/migrations/0042_events.sql
create table public.events (
  id          bigint generated always as identity primary key,
  user_id     uuid references auth.users (id) on delete set null,
  name        text not null,
  props       jsonb not null default '{}'::jsonb,
  created_at  timestamptz not null default now()
);

create index events_name_created_idx on public.events (name, created_at desc);
create index events_user_idx on public.events (user_id, created_at desc);

alter table public.events enable row level security;

-- Users may insert their own events; reads are server/service-role only.
create policy "insert own events" on public.events
  for insert to authenticated
  with check (auth.uid() = user_id);
```

A Server Action is the clean way to record events — it runs on the server, so the service context and cookies are already available.

```ts
// app/_actions/track.ts
"use server";

import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export type VibyEvent =
  | "signup_completed"
  | "circle_joined"      // our activation event
  | "meetup_rsvp"
  | "message_sent"
  | "invite_sent"
  | "invite_accepted";

export async function track(name: VibyEvent, props: Record<string, unknown> = {}) {
  const cookieStore = await cookies();

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (toSet) =>
          toSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options),
          ),
      },
    },
  );

  const {
    data: { user },
  } = await supabase.auth.getUser();

  await supabase.from("events").insert({ user_id: user?.id ?? null, name, props });
}
```

Fire activation right where it happens — when a user joins a circle:

```
// app/circles/[id]/_actions/join.ts
"use server";

import { revalidatePath } from "next/cache";
import { track } from "@/app/_actions/track";
import { getServerClient } from "@/lib/supabase/server";

export async function joinCircle(circleId: string) {
  const supabase = await getServerClient();
  const { error } = await supabase
    .from("circle_members")
    .insert({ circle_id: circleId });

  if (error) return { ok: false as const, error: error.message };

  await track("circle_joined", { circleId }); // ← activation moment
  revalidatePath(`/circles/${circleId}`);
  return { ok: true as const };
}
```

2. Measure retention with a cohort query

Retention is best read as a **cohort curve**: group users by the week they signed up, then see what fraction were "connected" in each later week.

```
-- Weekly retention by signup cohort
with cohorts as (
  select id as user_id, date_trunc('week', created_at) as cohort_week
  from auth.users
),
activity as (
  select distinct user_id, date_trunc('week', created_at) as active_week
  from public.events
  where name in ('circle_joined', 'meetup_rsvp', 'message_sent')
)
select
  c.cohort_week,
  count(distinct c.user_id)                                    as cohort_size,
  (extract(epoch from (a.active_week - c.cohort_week)) / 604800)::int as week_offset,
  count(distinct a.user_id)                                    as retained
from cohorts c
join activity a on a.user_id = c.user_id and a.active_week >= c.cohort_week
group by 1, 3
order by 1, 3;
```

A healthy social product shows a retention curve that *flattens* (a stable plateau of returning users), not one that decays to zero. The plateau is your real business.

3. A load-bearing invite loop

Make the invite deliver value. The link pre-loads the friend's name *and* the circle, so the recipient lands on a warm, half-finished action — not a cold marketing page.

```ts
// app/_actions/invite.ts
"use server";

import { track } from "@/app/_actions/track";
import { getServerClient } from "@/lib/supabase/server";

export async function createInvite(circleId: string) {
  const supabase = await getServerClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();
  if (!user) return { ok: false as const };

  const { data, error } = await supabase
    .from("invites")
    .insert({ circle_id: circleId, inviter_id: user.id })
    .select("token")
    .single();

  if (error) return { ok: false as const };

  await track("invite_sent", { circleId });
  const url = `${process.env.NEXT_PUBLIC_SITE_URL}/i/${data.token}`;
  return { ok: true as const, url };
}
```

The landing route is a Server Component. In Next.js 16 the route `params` are async — you `await` them:

```tsx
// app/i/[token]/page.tsx
import { notFound } from "next/navigation";
import { getServerClient } from "@/lib/supabase/server";
import { AcceptInvite } from "./accept-invite";

export default async function InvitePage({
  params,
}: {
  params: Promise<{ token: string }>;
}) {
  const { token } = await params; // ← async params
  const supabase = await getServerClient();

  const { data: invite } = await supabase
    .from("invites")
    .select("circle:circles(id, name, member_count), inviter:profiles(display_name)")
    .eq("token", token)
    .single();

  if (!invite) notFound();

  return (
    <main className="mx-auto max-w-md p-6 text-center">
      <p className="text-muted-foreground text-sm">
        {invite.inviter.display_name} invited you to
      </p>
      <h1 className="text-2xl font-bold text-pink">{invite.circle.name}</h1>
      <p className="text-muted-foreground mt-1 text-sm">
        {invite.circle.member_count} people already inside
      </p>
      <AcceptInvite token={token} circleId={invite.circle.id} />
    </main>
  );
}
```

```
// app/i/[token]/accept-invite.tsx
"use client";

import { useTransition } from "react";
import { motion } from "framer-motion";
import { acceptInvite } from "@/app/_actions/accept-invite";

export function AcceptInvite({ token, circleId }: { token: string; circleId: string }) {
  const [pending, start] = useTransition();

  return (
    <motion.button
      whileTap={{ scale: 0.96 }}
      disabled={pending}
      onClick={() => start(() => acceptInvite(token, circleId))}
      className="bg-pink mt-6 w-full rounded-full px-6 py-3 font-semibold text-white disabled:opacity-
60"
    >
      {pending ? "Joining…" : "Join the circle"}
    </motion.button>
  );
}
```

The brand token comes from Tailwind v4's CSS-first `@theme` — no `tailwind.config.js`:

```
/* app/globals.css */
@import "tailwindcss";

@theme {
  --color-pink: #ff6fb5;
  --color-muted-foreground: oklch(0.55 0.02 320);
}
```

4. Ship safely with feature flags + a proxy

A simple env/cookie flag lets you merge unfinished growth experiments to `main` without exposing them. In Next.js 16 we use **`proxy.ts`** (not `middleware.ts`) for edge-level logic like cohort bucketing:

```
// proxy.ts  (project root)
import { NextResponse, type NextRequest } from "next/server";

export function proxy(request: NextRequest) {
  const response = NextResponse.next();

  // Sticky 50/50 experiment bucket for an invite-banner test.
  if (!request.cookies.has("exp_invite_banner")) {
    const bucket = Math.random() < 0.5 ? "control" : "variant";
    response.cookies.set("exp_invite_banner", bucket, {
      maxAge: 60 * 60 * 24 * 90,
      httpOnly: false,
      sameSite: "lax",
    });
  }
  return response;
}

export const config = { matcher: ["/((?!_next|api|.*\\..*).*)"] };
```

```
// lib/flags.ts
export function isEnabled(flag: string): boolean {
  // Comma-separated list in the Vercel env, e.g. FLAGS="new_feed,invite_banner"
  return (process.env.FLAGS ?? "").split(",").includes(flag);
}
```

5. CI/CD that runs on every PR

```yaml
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
  push:
    branches: [main]

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npx tsc --noEmit
      - run: npm test -- --run
      - run: npm run build
```

Vercel builds a **preview deployment** for every PR automatically. The flow: open PR → CI runs the checks above *and* Vercel posts a live preview URL → reviewer clicks it → merge to `main` → Vercel promotes to production. Green checks plus a working preview is your safety net.

Architecture / diagram

```mermaid
flowchart LR
  subgraph Loop["Viby Growth Loop"]
    A[New user signs up] --> B[Joins a hobby circle\nACTIVATION]
    B --> C[Sees active meetups\n+ posts/RSVPs]
    C --> D[Sends load-bearing invite]
    D --> E[Friend lands on warm page]
    E --> A
  end

  B -. track 'circle_joined' .-> EV[(events table)]
  D -. track 'invite_sent' .-> EV
  E -. track 'invite_accepted' .-> EV
  EV --> M[Cohort / NSM dashboard]
  M -->|insight| EXP[Experiment via flag + proxy.ts]
  EXP -->|tune the loop| Loop
```

The loop creates the data; the data drives the experiment; the experiment tunes the loop. That cycle — not any single feature — is what compounds.

Best practices & pitfalls

- **Do** define one activation event and one North Star, and write them on the wall. **Don't** chase vanity metrics (signups, page views) — they go up even as the product dies.
- **Do** read retention as a *cohort curve*. **Don't** trust a single "active users" number; it hides whether you're retaining or just acquiring faster than you churn.
- **Do** make invites deliver value to *both* sides. **Don't** ship a generic "Share" button and call it virality — `k ≈ 0`.
- **Do** go narrow and dense to beat cold-start. **Don't** launch a social-discovery app to "everyone" at once.
- **Do** keep PRs small (< ~400 lines) and merge daily behind flags. **Don't** sit on a 3,000-line branch for two weeks — it will rot and the review will be useless.
- **Do** add DB indexes for the queries you actually run (see the `events` indexes). **Don't** premature-optimize architecture you have no traffic for.
- **Do** put events behind RLS and never log secrets/PII into `props`. **Don't** trust client-reported user IDs — resolve the user server-side as `track` does.
- **Do** let CI gate merges and Vercel previews catch visual regressions. **Don't** deploy straight from your laptop.

Exercises

1. **Define & log.** Pick Viby's activation event (defend your choice in a comment) and ensure `track("circle_joined", …)` fires exactly once per user's first join — guard against double-counting on re-join.
2. **Cohort chart.** Run the retention SQL, export to CSV, and chart the curve. Does it flatten or decay? Write one sentence on what that implies.
3. **k-factor.** From the `events` table, compute `k = invites_sent_per_user × accept_rate` over the last 30 days using `invite_sent` and `invite_accepted`. Is Viby's loop sustaining, amplifying, or dead?
4. **Flagged experiment.** Use the `exp_invite_banner` cookie from `proxy.ts` to show an invite banner only to the `variant` bucket, and compare `invite_sent` rates between buckets.
5. **Harden the pipeline.** Add a CI step that fails the build if `npx tsc --noEmit` finds errors *or* if any new file lacks a test, and require the check before merge in branch protection.

Mini-project — The Viby Growth Dashboard

Build a `/admin/growth` Server Component page (gate it to admin users via RLS/role) that shows three live numbers and one chart, all from the `events` table:

1. **North Star** — Weekly Connected Users (distinct `user_id` with any of `circle_joined` / `meetup_rsvp` / `message_sent` in the last 7 days).
2. **Activation rate** — of users who signed up in the last 14 days, the % that fired `circle_joined` within 24h.
3. **k-factor** — `invite_sent` and `invite_accepted` over 30 days.
4. **Retention curve** — render the cohort query from section 2 as a small line chart (a `<motion.path>` is a fun, on-brand way to animate it in).

Requirements: all data fetched in Server Components (no client secrets), a Server Action to refresh, and the page protected so only admins can load it. When you can watch these four numbers move after shipping a change, you're running Viby like a startup — not a side project.

Key takeaways

- **Loops compound; funnels leak.** Design Viby so using it produces the next user.
- **Network effects are the moat** — win cold-start by going narrow and dense.
- **One activation event, one North Star, cohort retention.** That trio tells you the truth.
- **Virality is load-bearing invites,** not share buttons; measure with the k-factor.
- **Scale the product** with indexes, caches, and scoped Realtime — **scale the team** with small PRs, ownership, and automation.
- **Ship daily, safely:** trunk-based git, fast small reviews, CI gates, Vercel previews, and feature flags behind `proxy.ts`.

Up next — Chapter 37: Monetization & Sustainable Business Models, where we turn Viby's engaged, retained network into revenue without breaking the trust that made it grow.